CSCI 566: Deep Learning and its Applications

Yue Zhao
Thomas Lord Department of Computer Science
University of Southern California

Credits to previous versions of USC CSCI566, CMU 10601/701, Stanford CS 229, 231n

USC Viterbi School of Engineering

Neural Networks

Basics

It’s a matter of One Function



Really…?

Let’s take a look

One Function, Many Behaviors

Diagram. f(X) looks like one fixed map, one input type to one output.

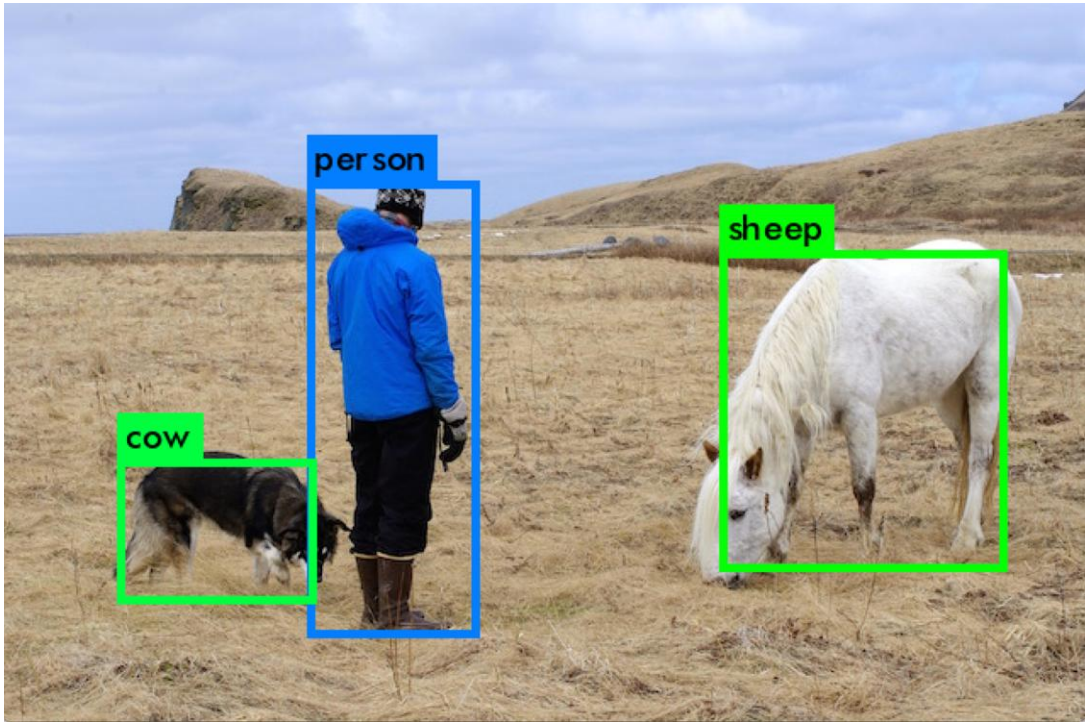
2026. One parameter set can serve chat, math, instruction following, many languages, and image plus text input.

How. The task description now rides inside X, along with context and data. The model still computes f(x; W).

Limit. That is conditional reuse, not a universal solver. Strong on one instruction, weak on another.

Today we open f and build it out of small weighted nonlinear pieces.

Object Detection



Object Detection

X:
Image

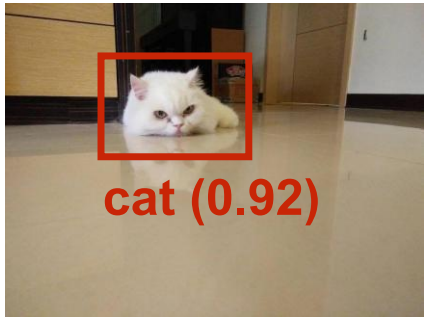


f(X)



Y:
Object label
&
Bounding box





Types of Learning



- Supervised Learning — desired output (**Y**) in training data
- Unsupervised Learning — **Y** not in training data
- Weakly / Semi-supervised Learning — some of **Y** in training data
- Reinforcement Learning — rewards based on a set of actions

Our goal is “to approximate”



There may exist an exact function (f*) mapping from X to Y.

Our goal is not to find this exact function.

Rather, we are happy as long as f(X) can **approximate** f*(x). f does NOT have to be exactly f*.

Our goal is “to approximate”



For f(X) to approximate any f*(X), f is better to be highly capable.

Deep learning is an effective method for this

- Non-linear (high capacity)
- Hierarchical
- End-to-End learning

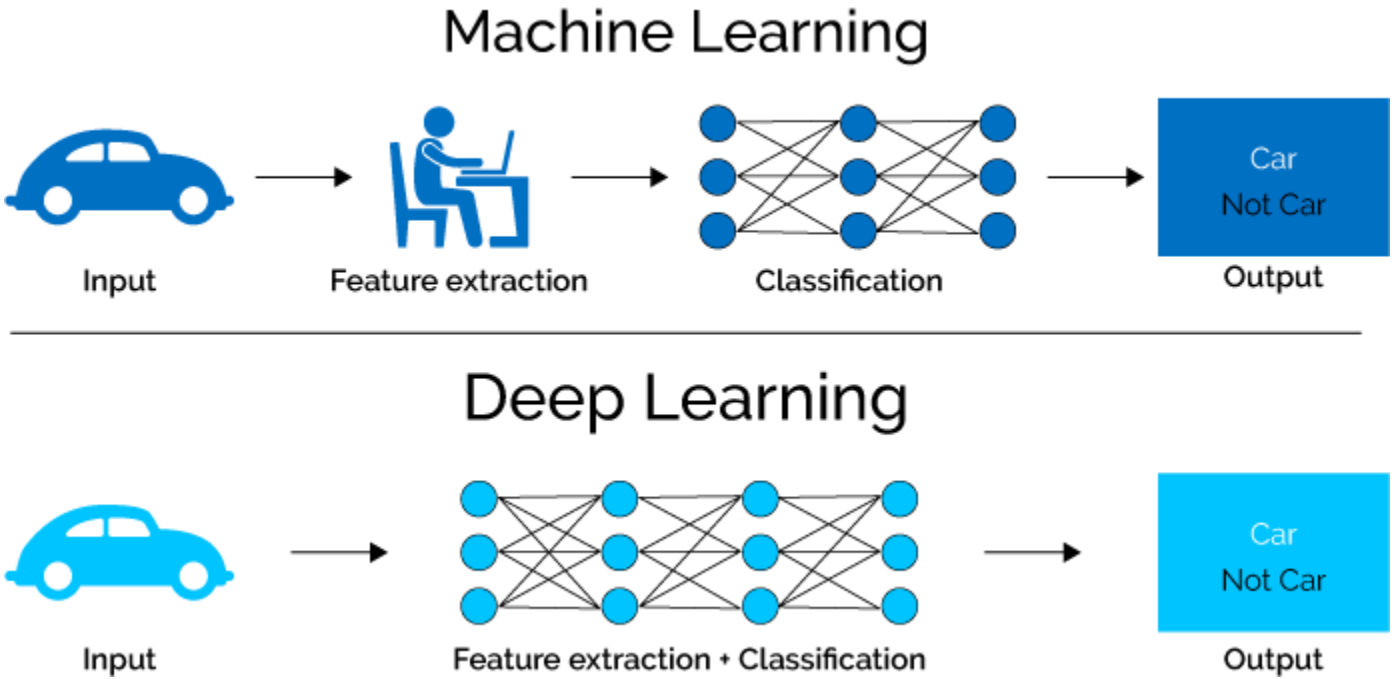
Recap: Our course



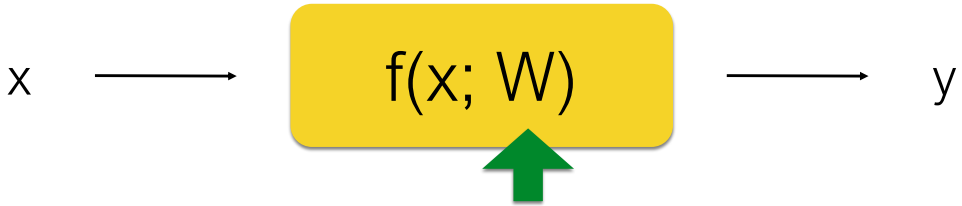


(1) How do we learn this function (using ML and DL)?
(2) How to formulate a problem into this?

Recap: Classical ML vs. DL (Better Representations)



Loss function & Optimization



- **Loss function (L)** measures how well learned W can map X to Y (compared to y*).
- **Optimization** finds the best W given a loss function L (i.e. finding W that minimizes L).

Optimization

How do we find W minimizing L?

* Random search
* Analytic solution
* Numerical approach (gradient descent)

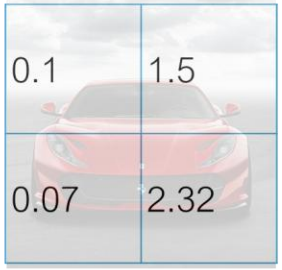
f(x; W) → y*

L(W; f,x,y*)

As an example, let's pick a simple loss function.

$L(W) = || f(x; W) - y^* ||_2$

Numerical Solution (gradient descent)



x

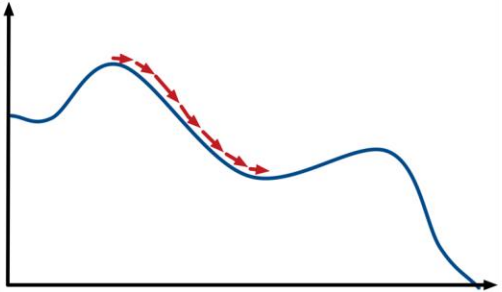
?	?	?	?
?	?	?	?
?	?	?	?

W

0
1
0

y* (ideal output)

$L(W) = || f(x; W) - y^* ||_2$



The **derivative** of a function in one-dimension

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

USC Viterbi School of Engineering

Numerical Solution (gradient descent)

-0.1	0.2	0.15	0.05
0.25	0.05	0.2	-0.1
0.25	0.15	-0.15	0.4

W

-0.1+ 0.001	**0.2+ 0.001**	0.15	0.05
0.25	0.05	0.2	-0.1
0.25	0.15	-0.15	0.4

W+h

0.5	**0.9**	?	?
?	?	?	?
?	?	?	?

gradient (dW)

L(W) = 1.6688 L(W+h) = 1.6697

Gradient = (1.6697-1.6688)/0.001 = 0.9

$$\frac{df(x)}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

USC Viterbi School of Engineering

Gradient Descent Grew Up

Baseline. Plain SGD applies one global step rule to every coordinate: W <- W - eta * G.

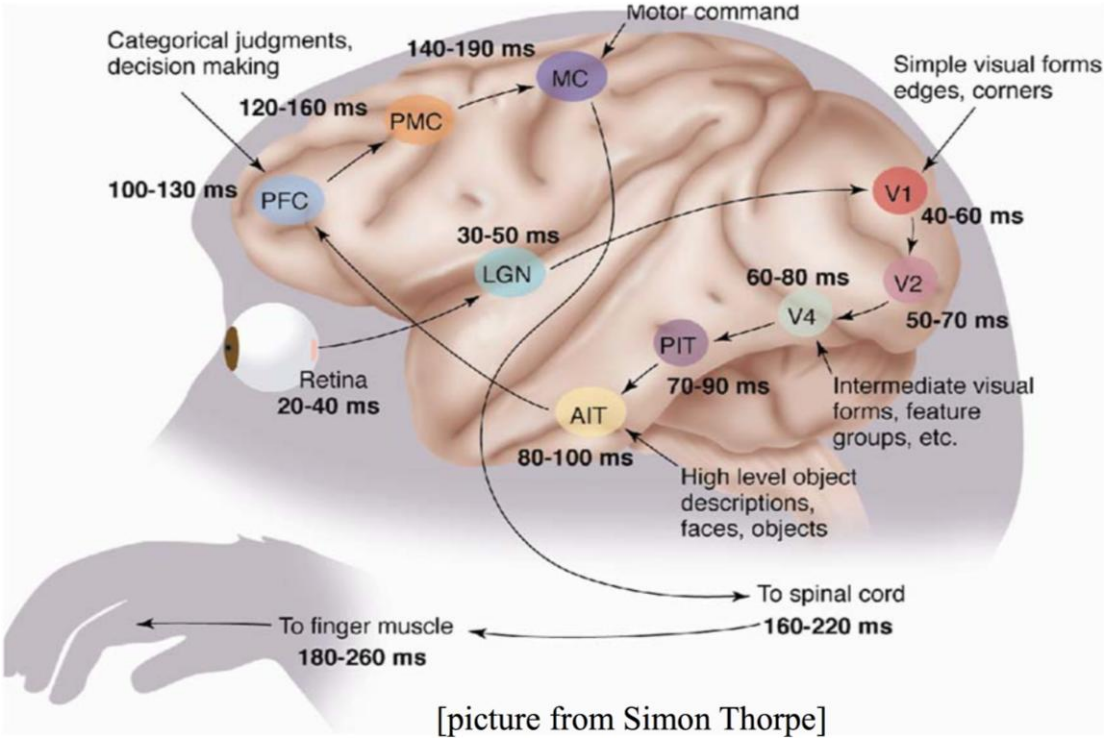
AdamW. It rescales each coordinate by running gradient statistics, and decays weights separately.

Why. One raw step size is too small for some parameters and unstable for others.

2026. Muon and SOAP precondition whole weight matrices. An NVIDIA study reports gains over AdamW at the scales they tested.

Optimizers consume gradients; they do not produce them. Where G comes from is next week.

Deep Learning may be Motivated by Human Brain



[picture from Simon Thorpe]

USC Viterbi School of Engineering

Be Careful with Brain Analogies!

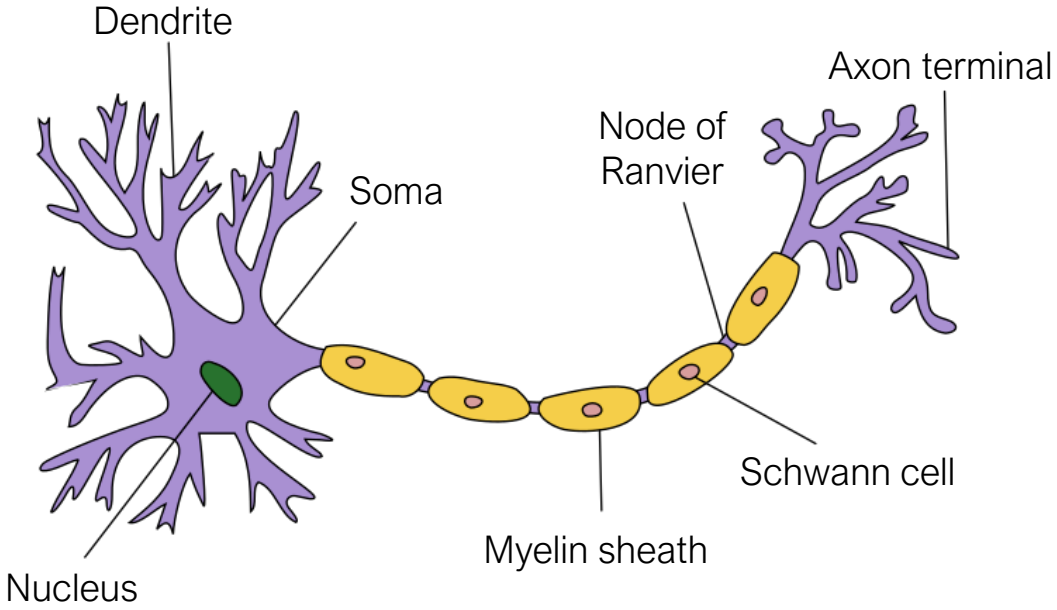
Biological Neurons:

- Many different types
- Dendrites can perform complex non-linear computations
- Synapses are not a single weight but a complex non-linear dynamical system

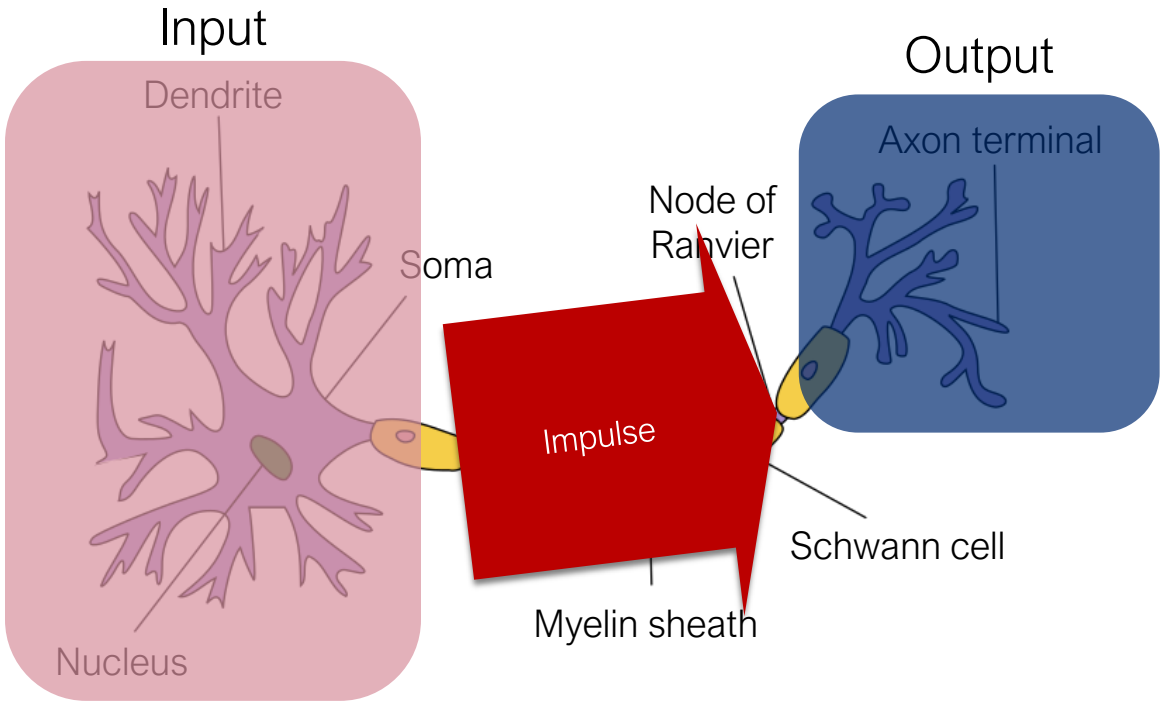
These are good motivations, while the actual relationship is weak.

[Dendritic Computation. London and Hausser]

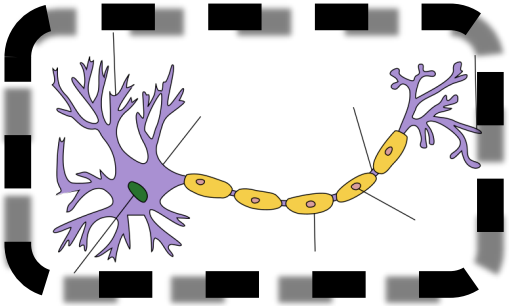
Neuron


Dendrite
Axon terminal
Node of
Ranvier
Soma
Schwann cell
Myelin sheath
Nucleus

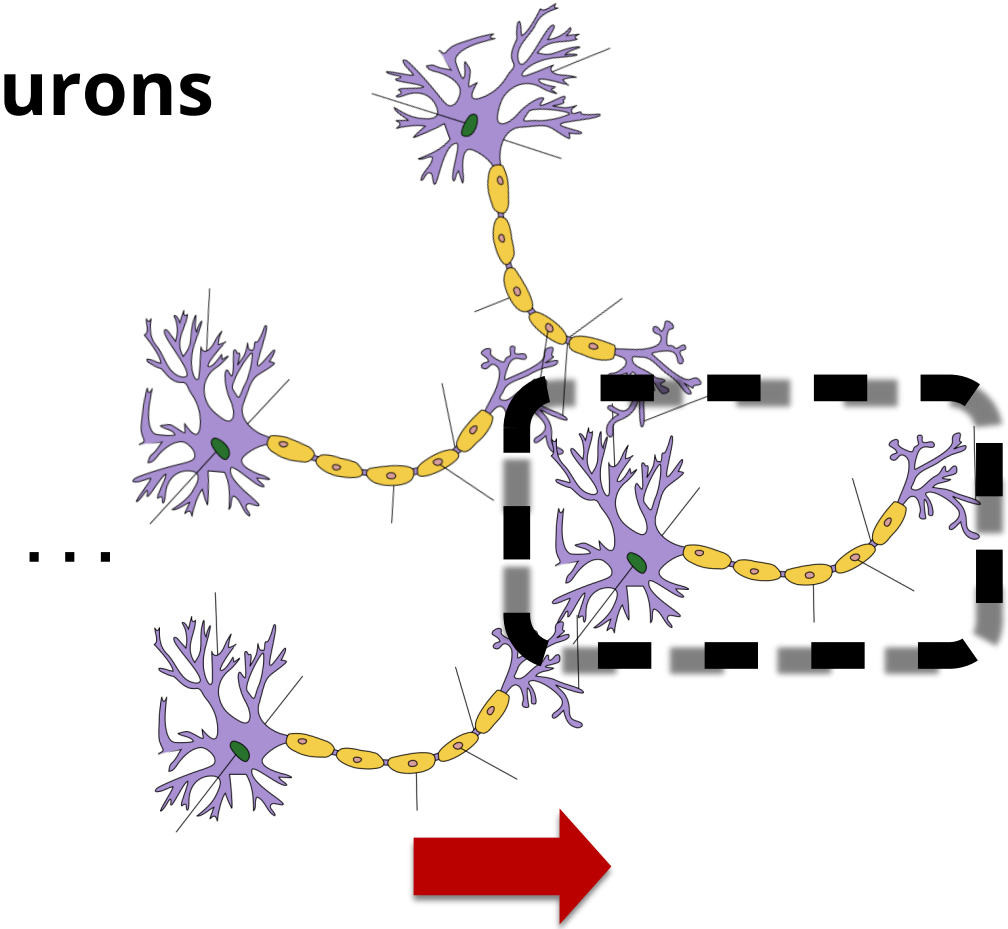
Neuron


Input
Dendrite
Soma
Nucleus
Output
Axon terminal
Node of Ranvier
Impulse
Schwann cell
Myelin sheath

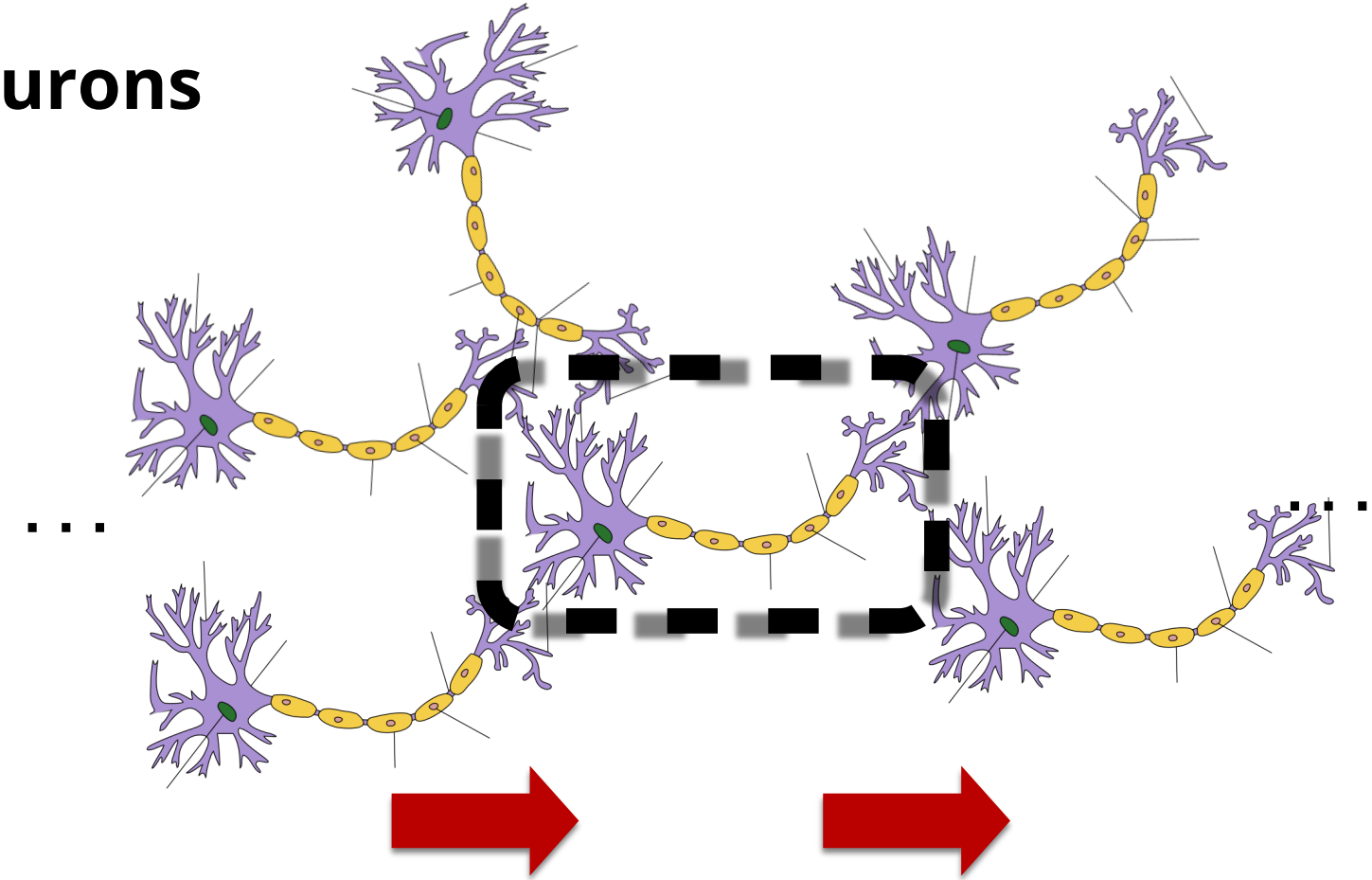
Neuron



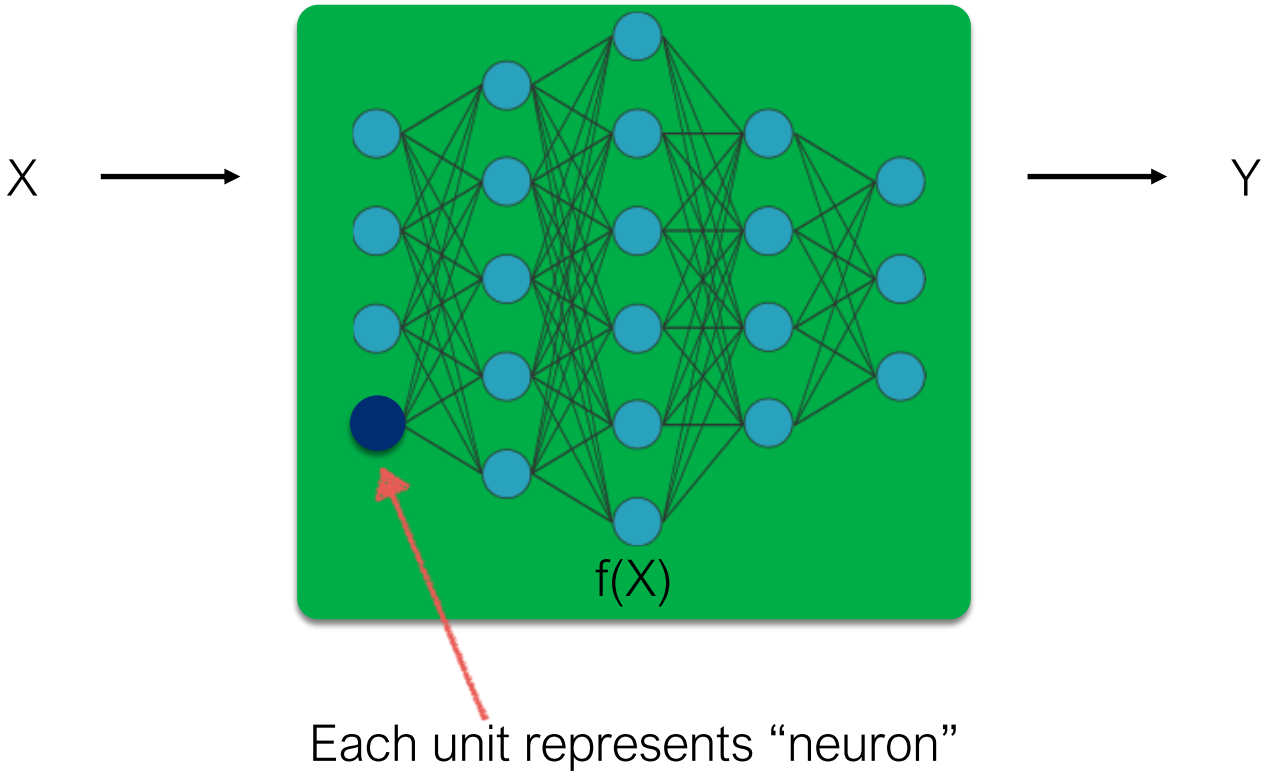
Neurons



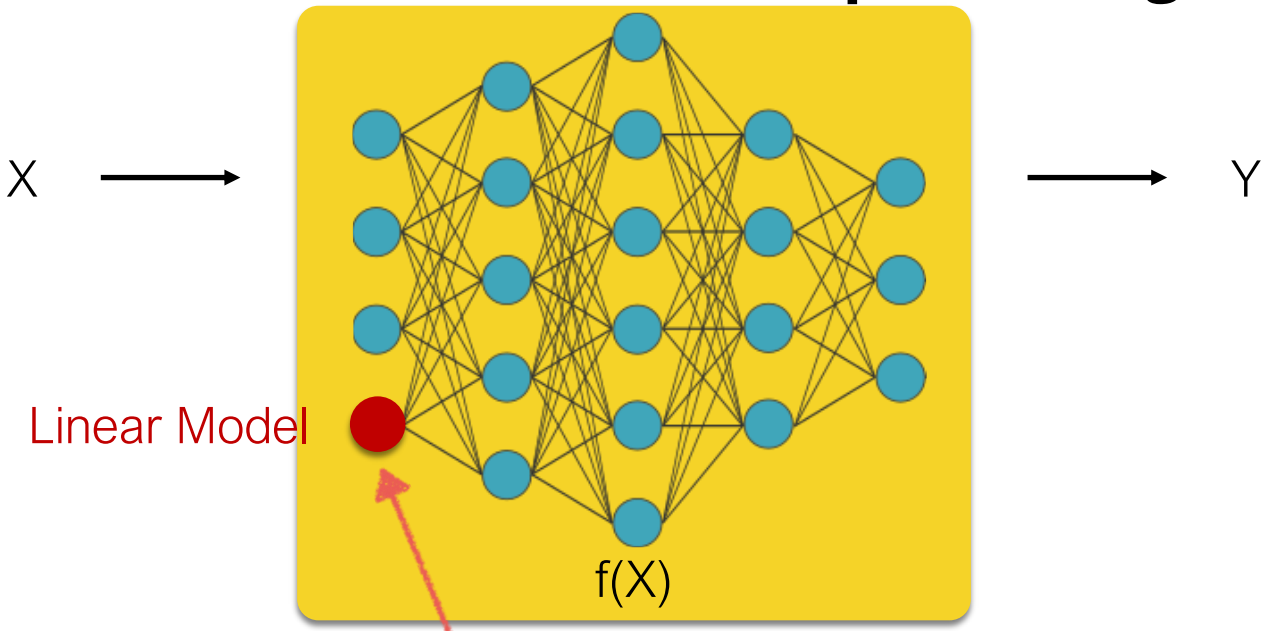
Neurons



From Neuron to Neural Networks



From Linear Classification to Deep Learning



Apparently, this is a unit used in deep neural networks too.

Image classification by Linear Classification

5x1 5x7500 7500x1

$f(x; W) = Wx + b$ 5x1

X → f(X; W) → Y



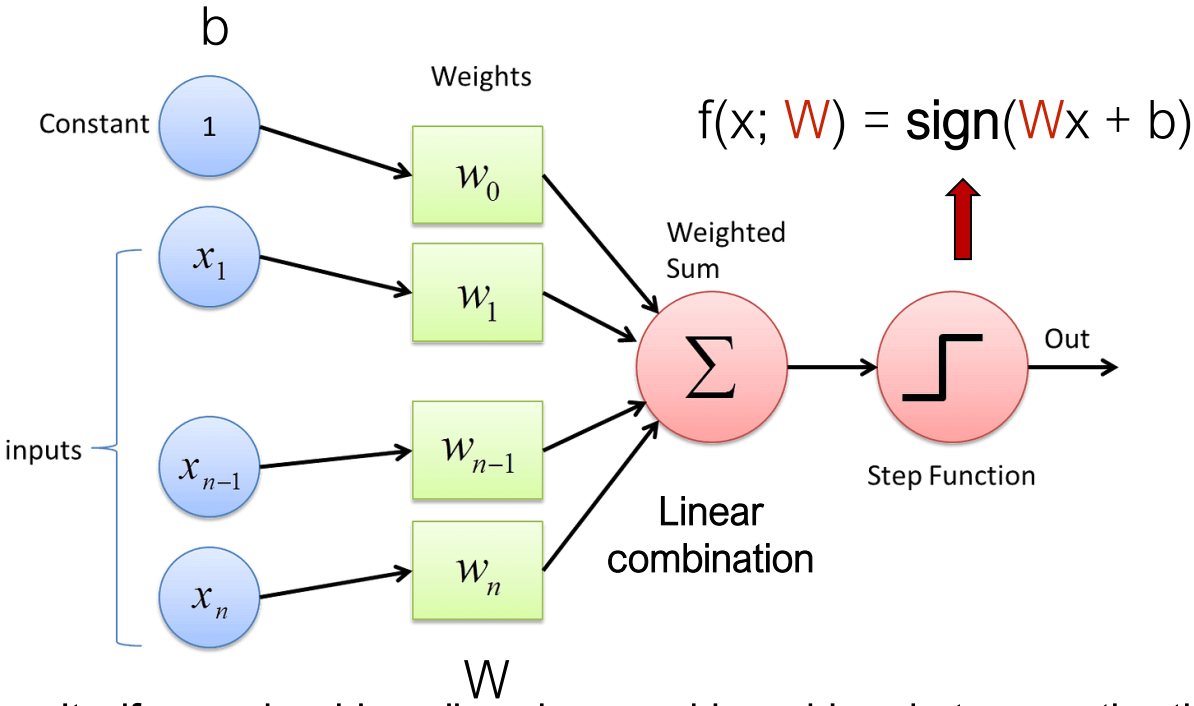
50x50x3 numbers
(7500 numbers)

Weights
(that we want to learn)

One-hot class vector
(e.g. <0,0,0,1,0>)

Y: output
X: input
W, b: **learned weight**

Each Neuron can be Simple, but connecting them?



Itself can only address linearly separable problem; but connecting them together makes a huge difference

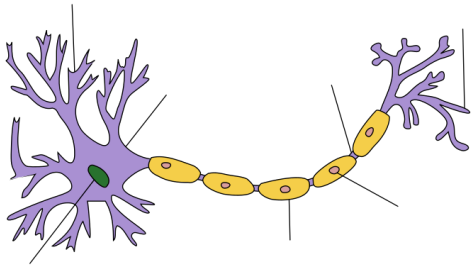
What is Neural Networks?

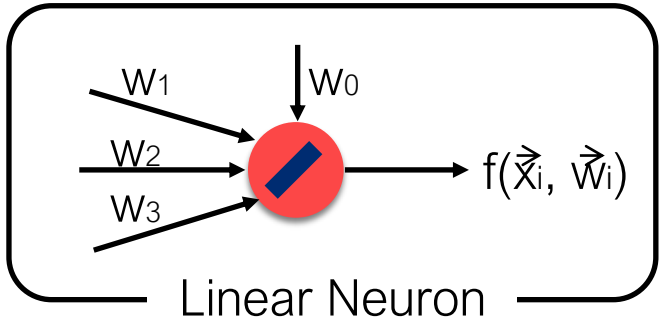
In simple terms,

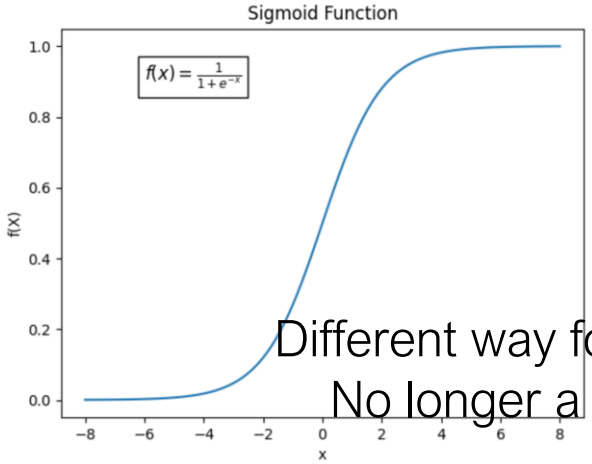
a set of **neurons (atomic functions)**

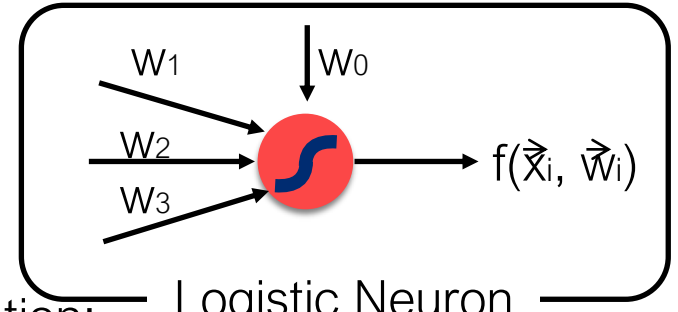
connected in a **non-linear** way (**how?**)

Neurons



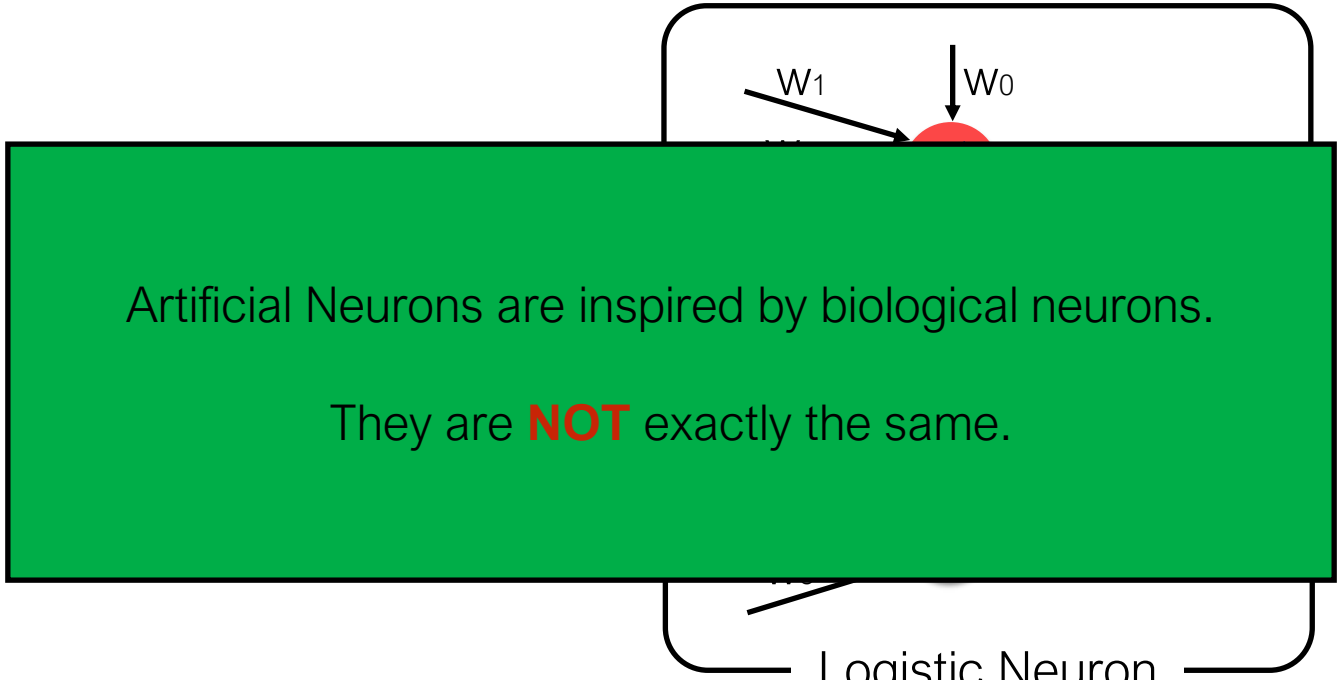






Different way for activation;
No longer a linear one

Neurons



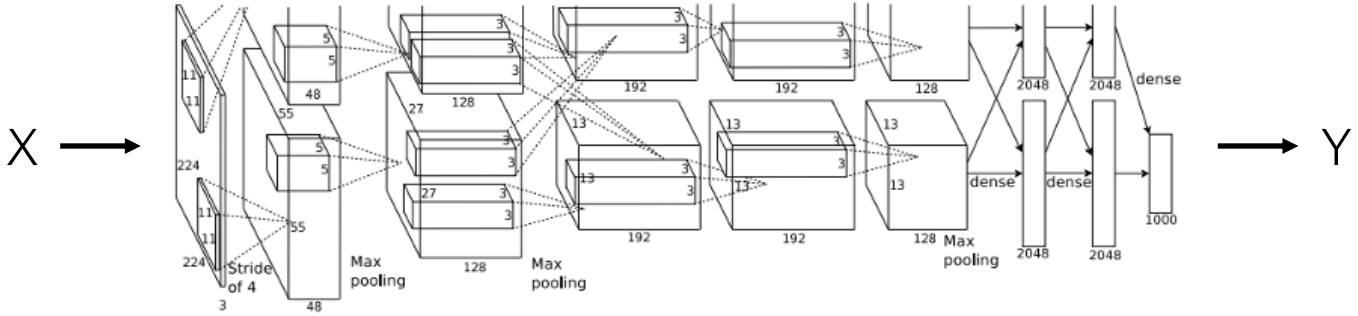
Artificial Neurons are inspired by biological neurons.

They are **NOT** exactly the same.

More neurons!

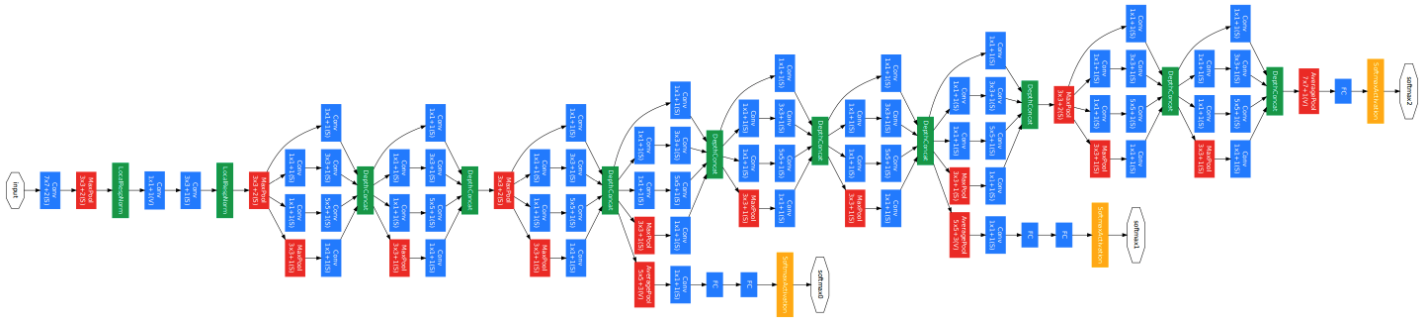
USC Viterbi School of Engineering

Convolutional Neural Networks



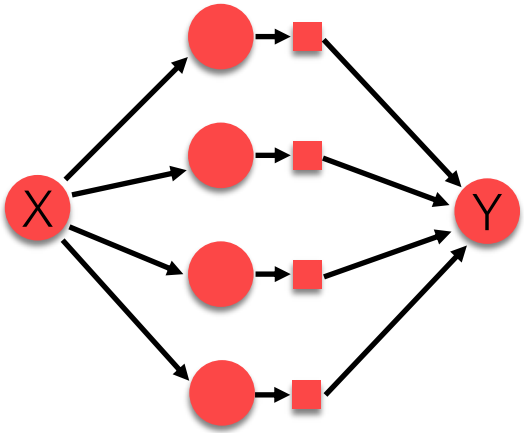
A Krizhevsky, et. al. ImageNet Classification with Deep Convolutional Neural Networks, NIPS 2012

Deep Neural Networks

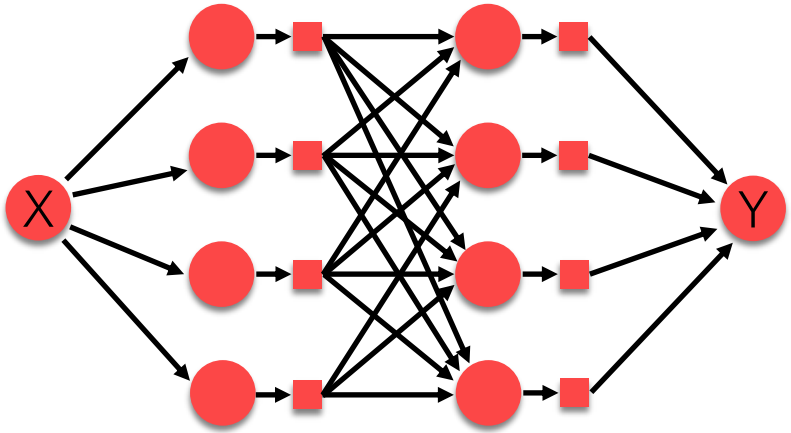


Szegedy et. al., Going deeper with convolutions, CVPR 2015

Why Deep Neural Networks?



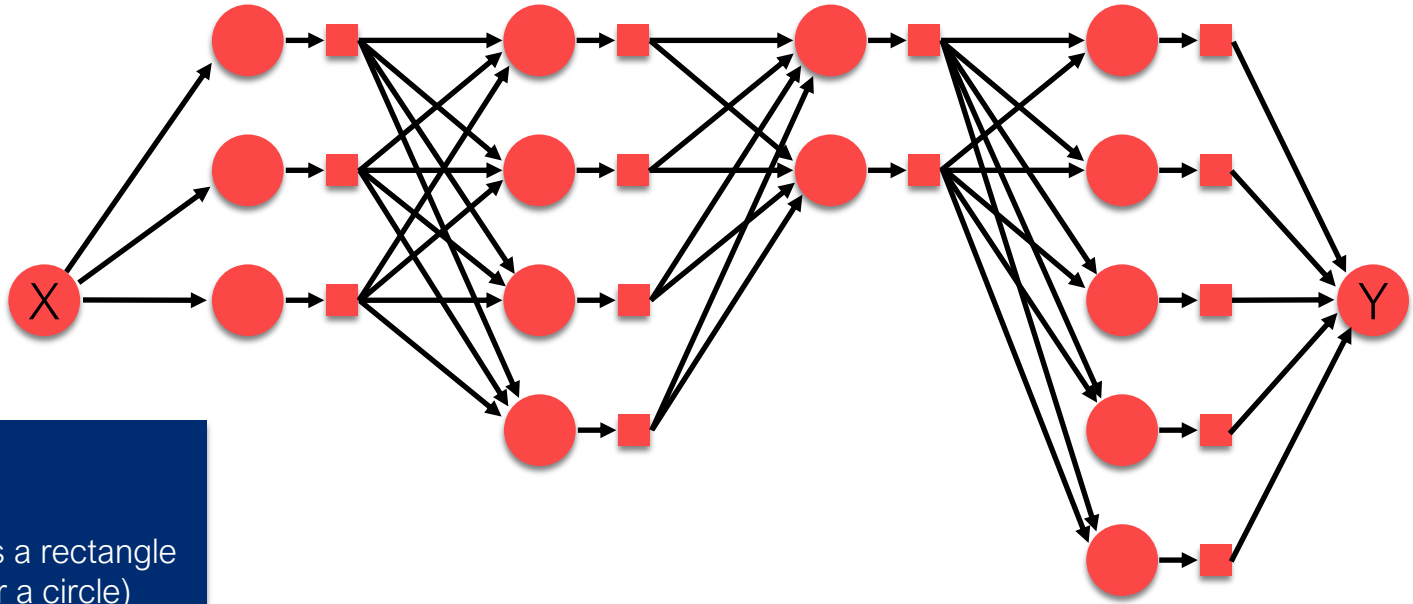
Shallow network



Deep network

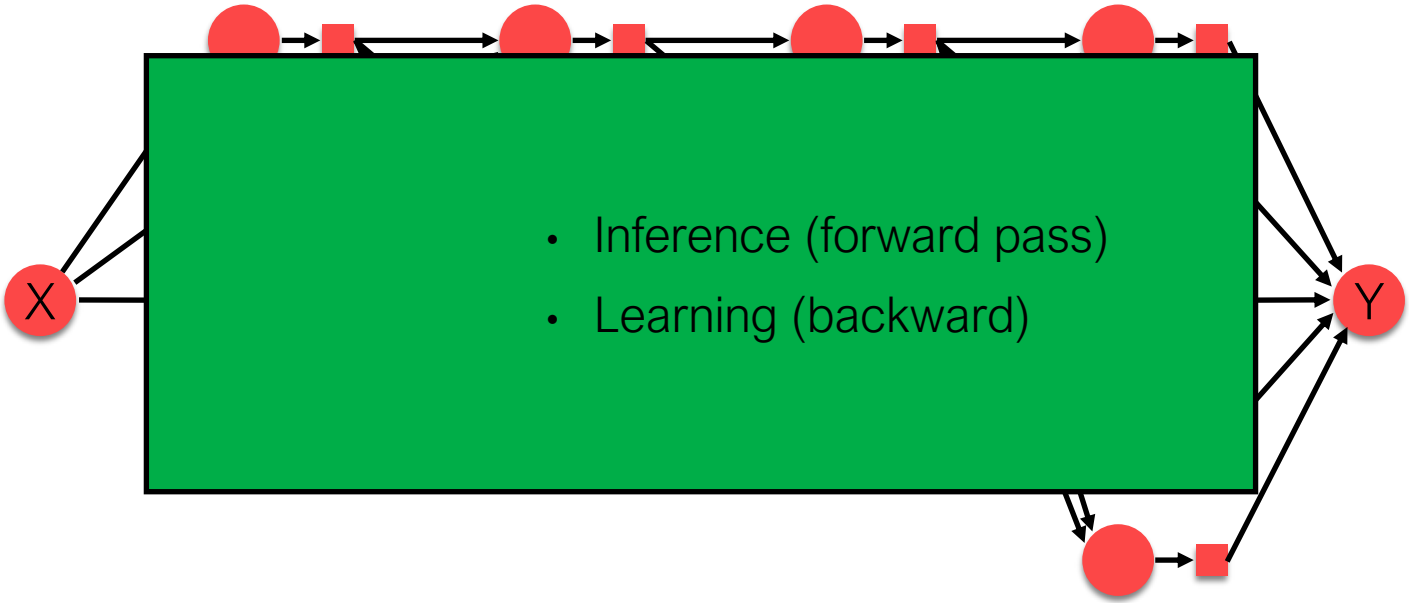
The algorithm for training a **deep** network

Neural Networks Examples

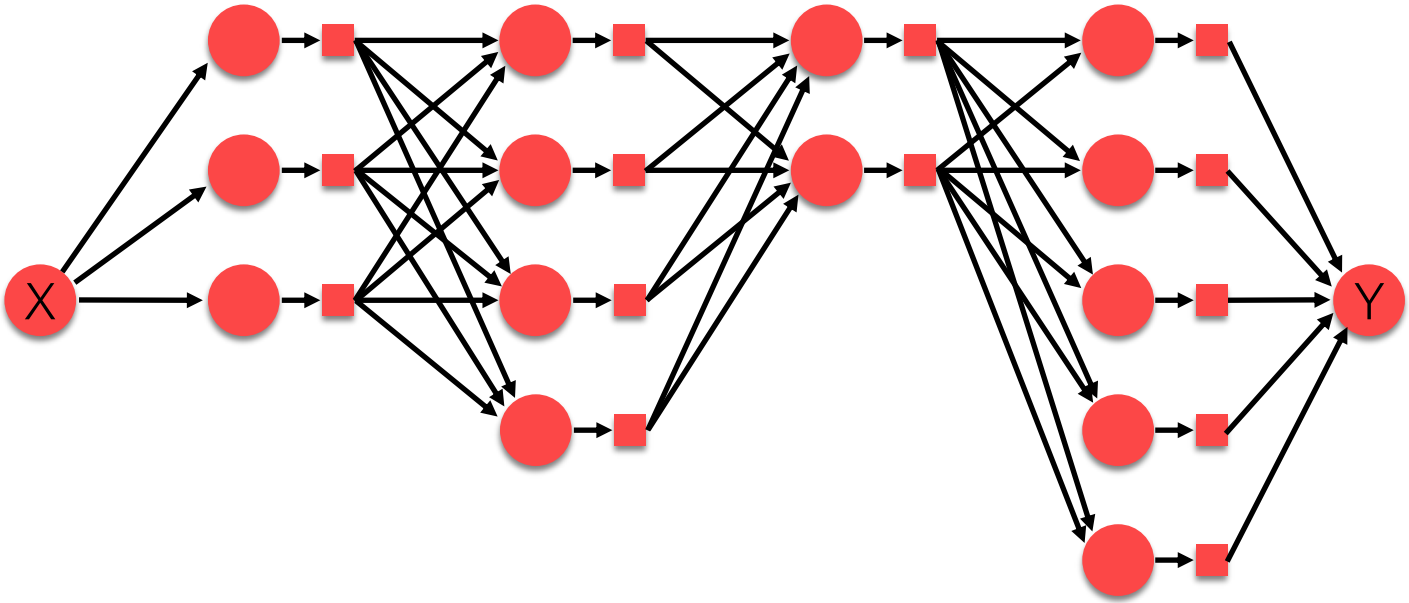


Put Y as a rectangle
(after a circle)

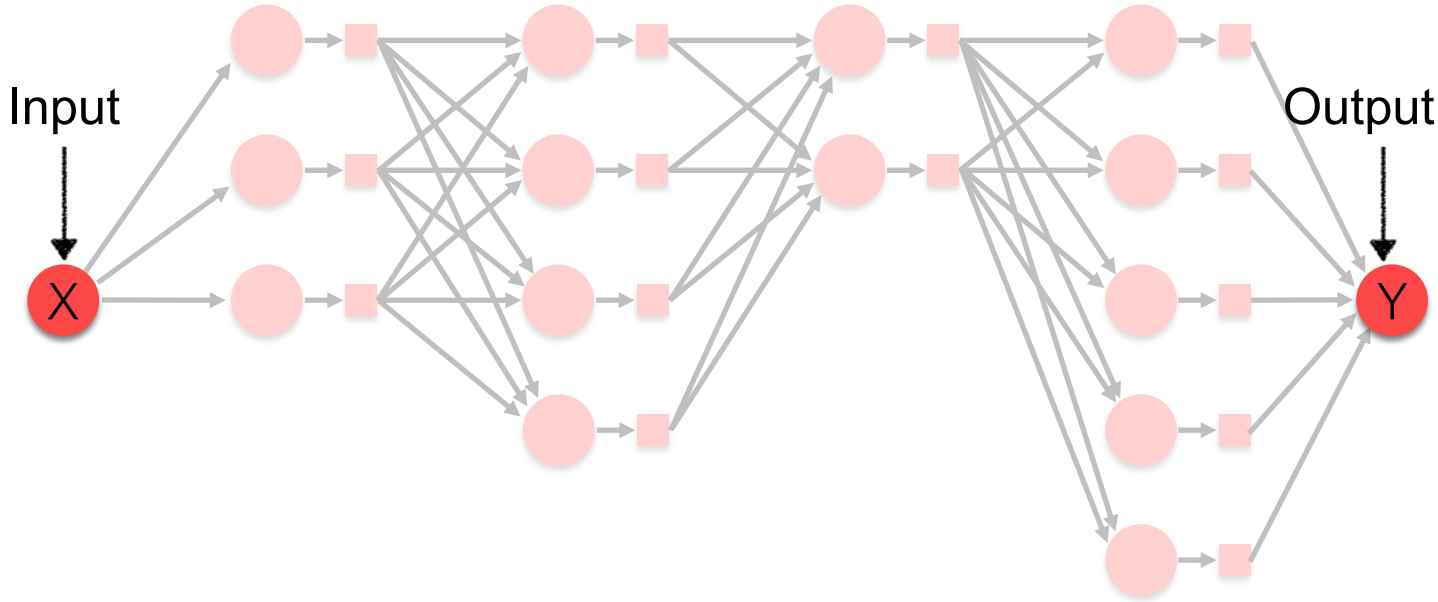
Neural Networks Examples



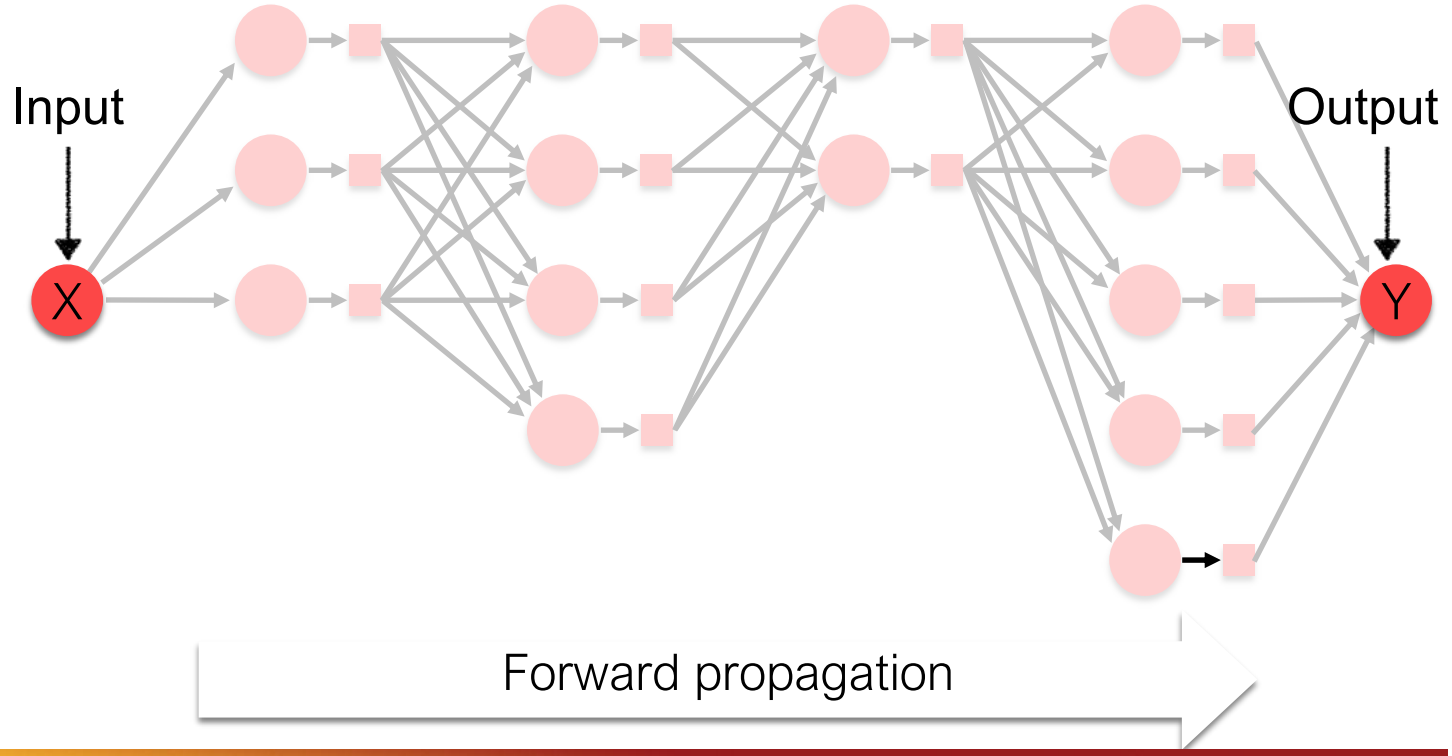
Forward Propagation



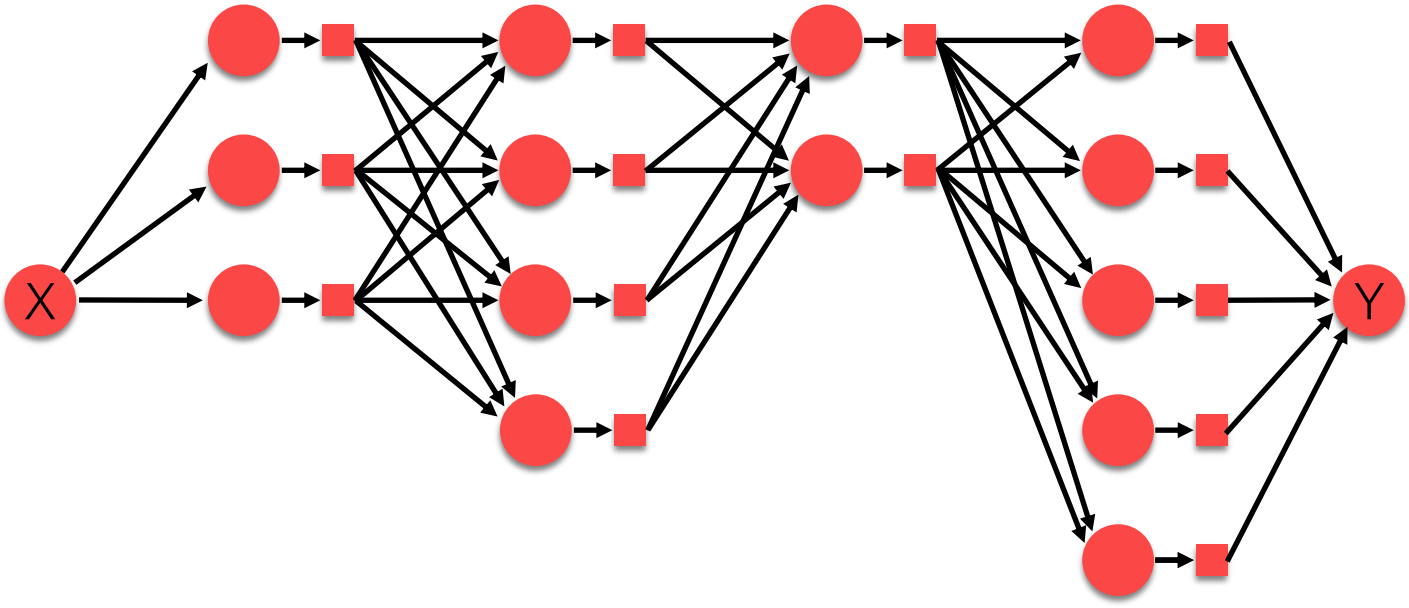
Forward Propagation



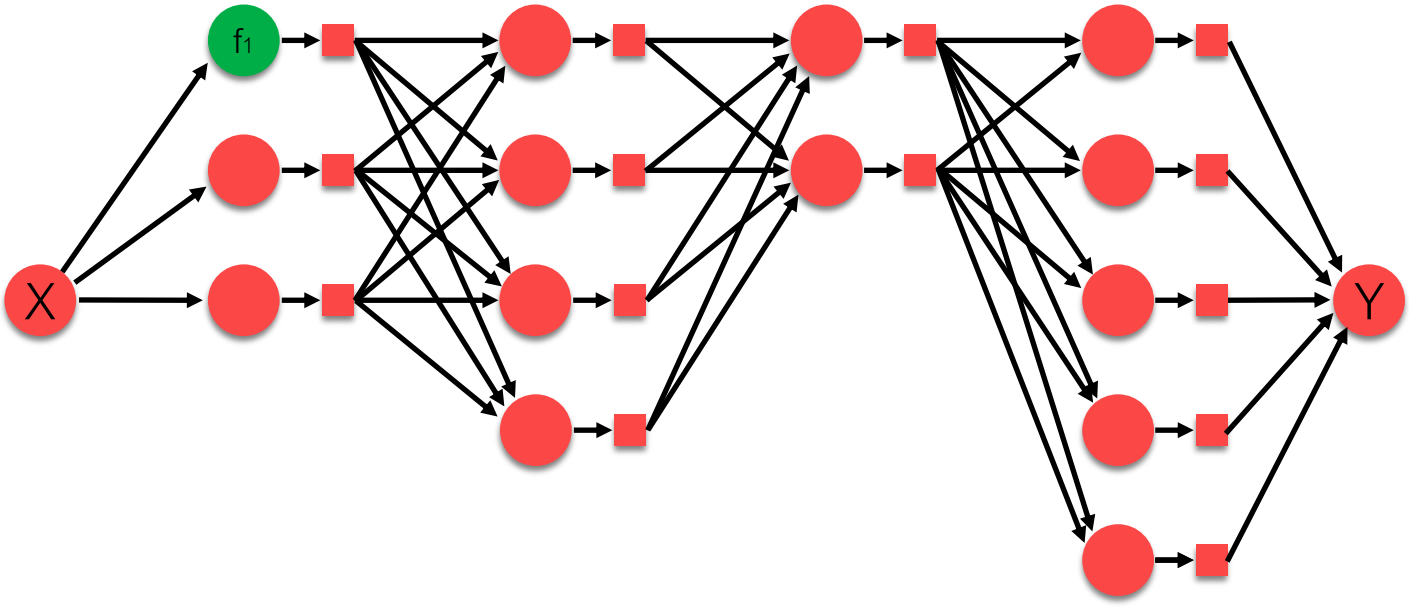
Forward Propagation



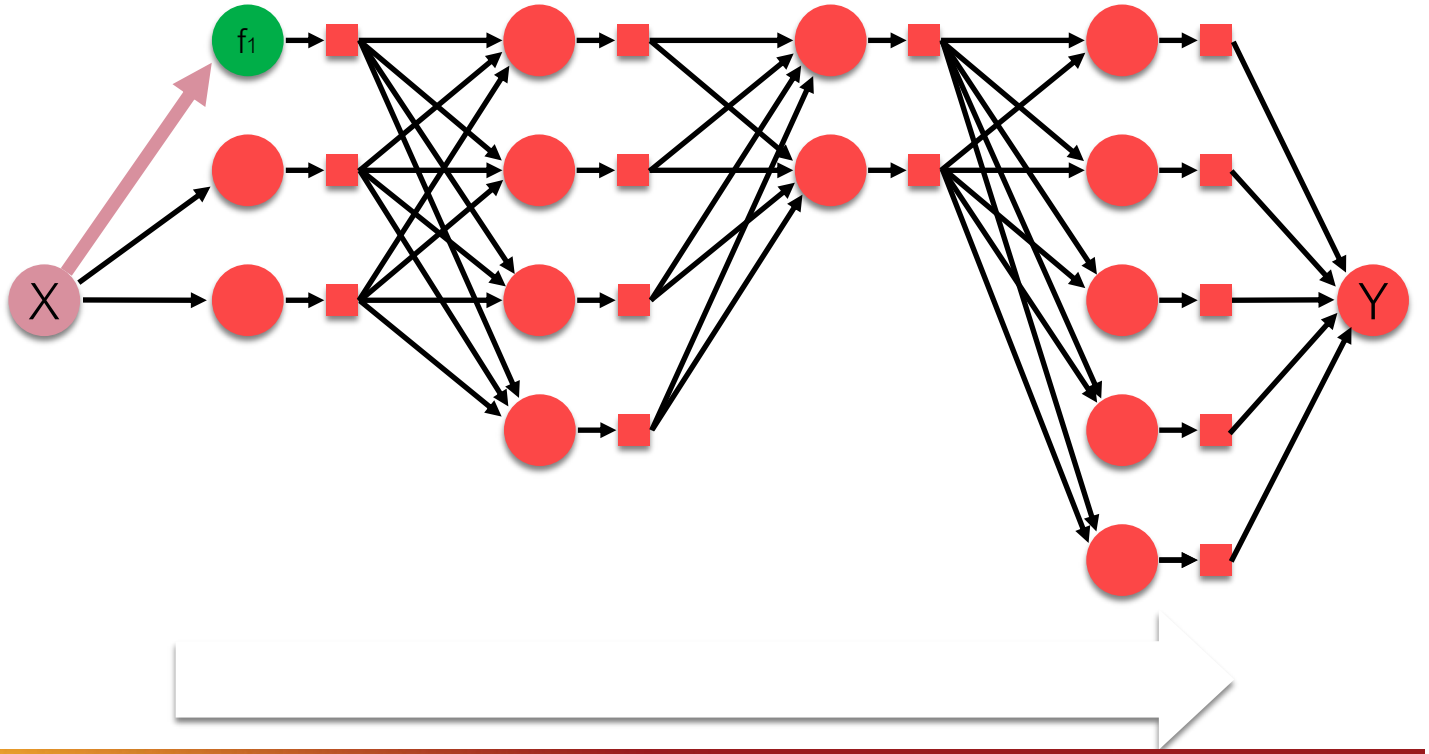
Forward Propagation



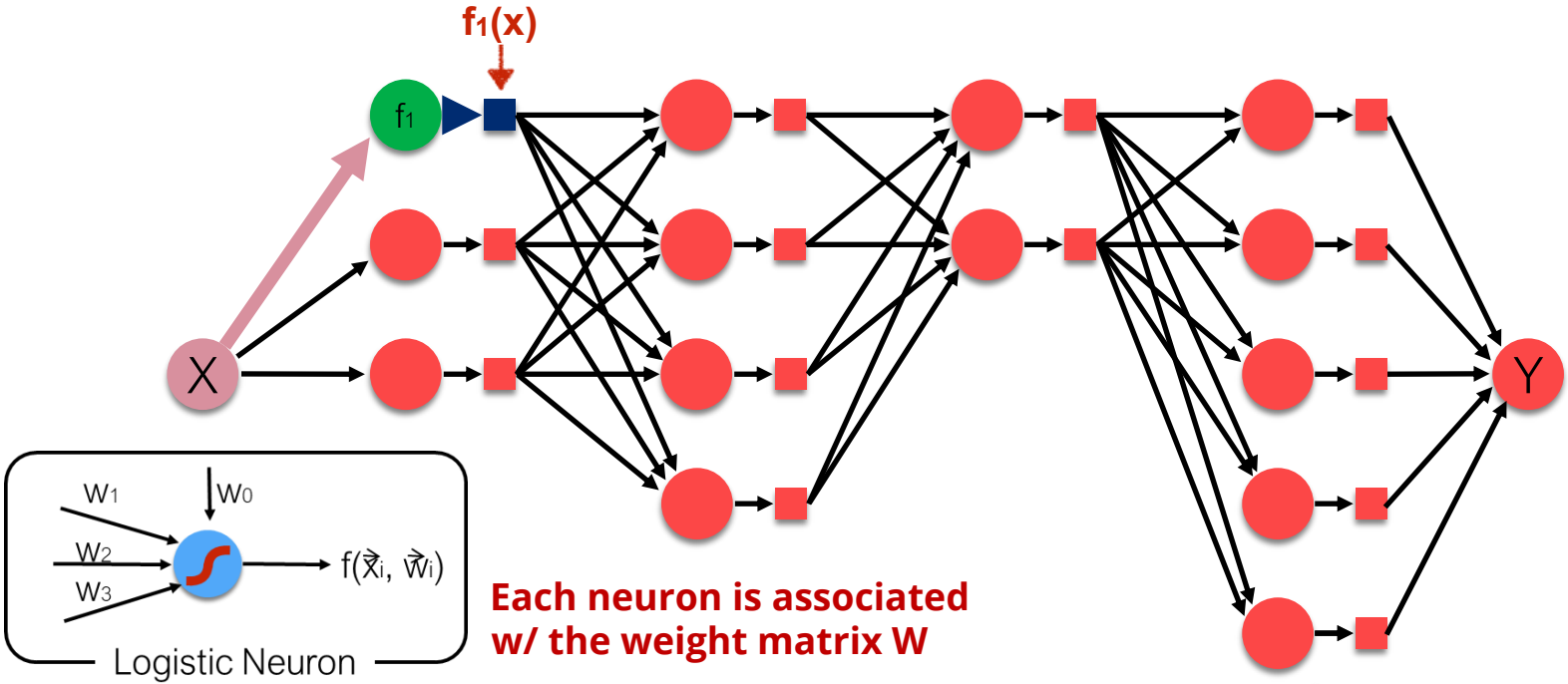
Forward Propagation



Forward Propagation



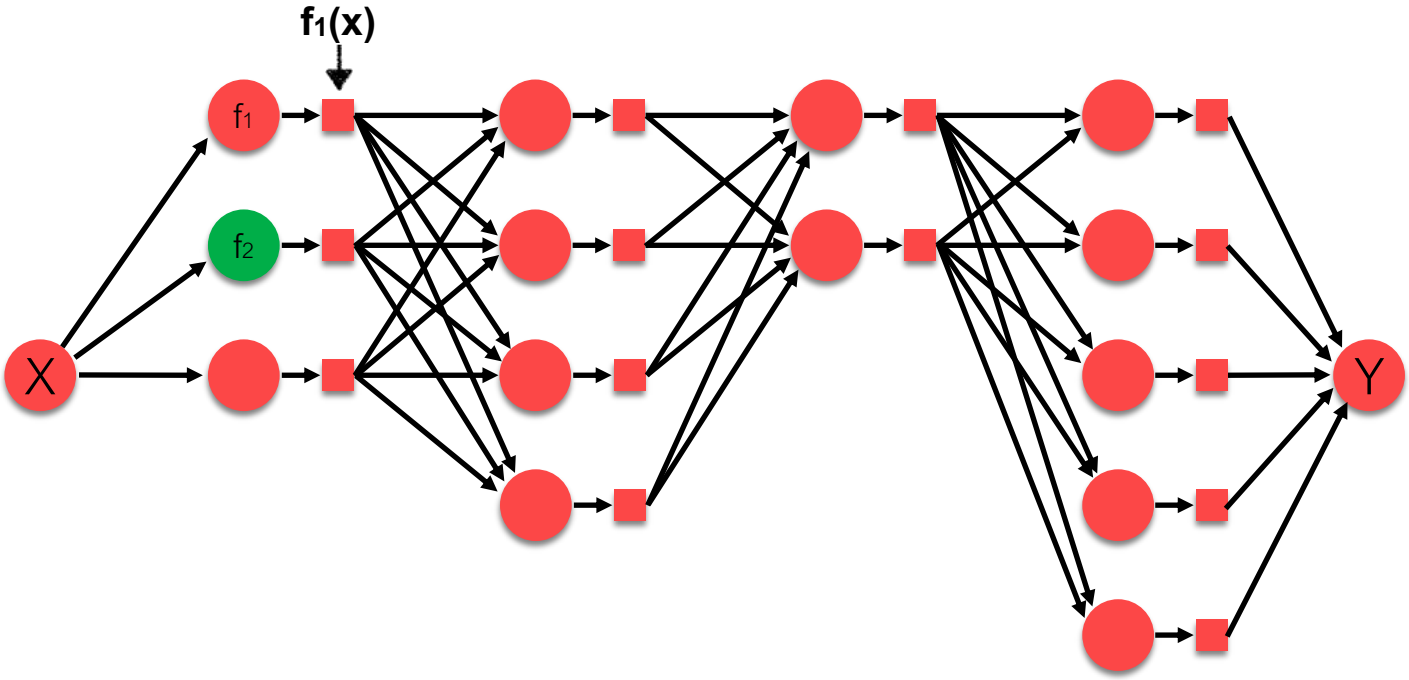
Forward Propagation



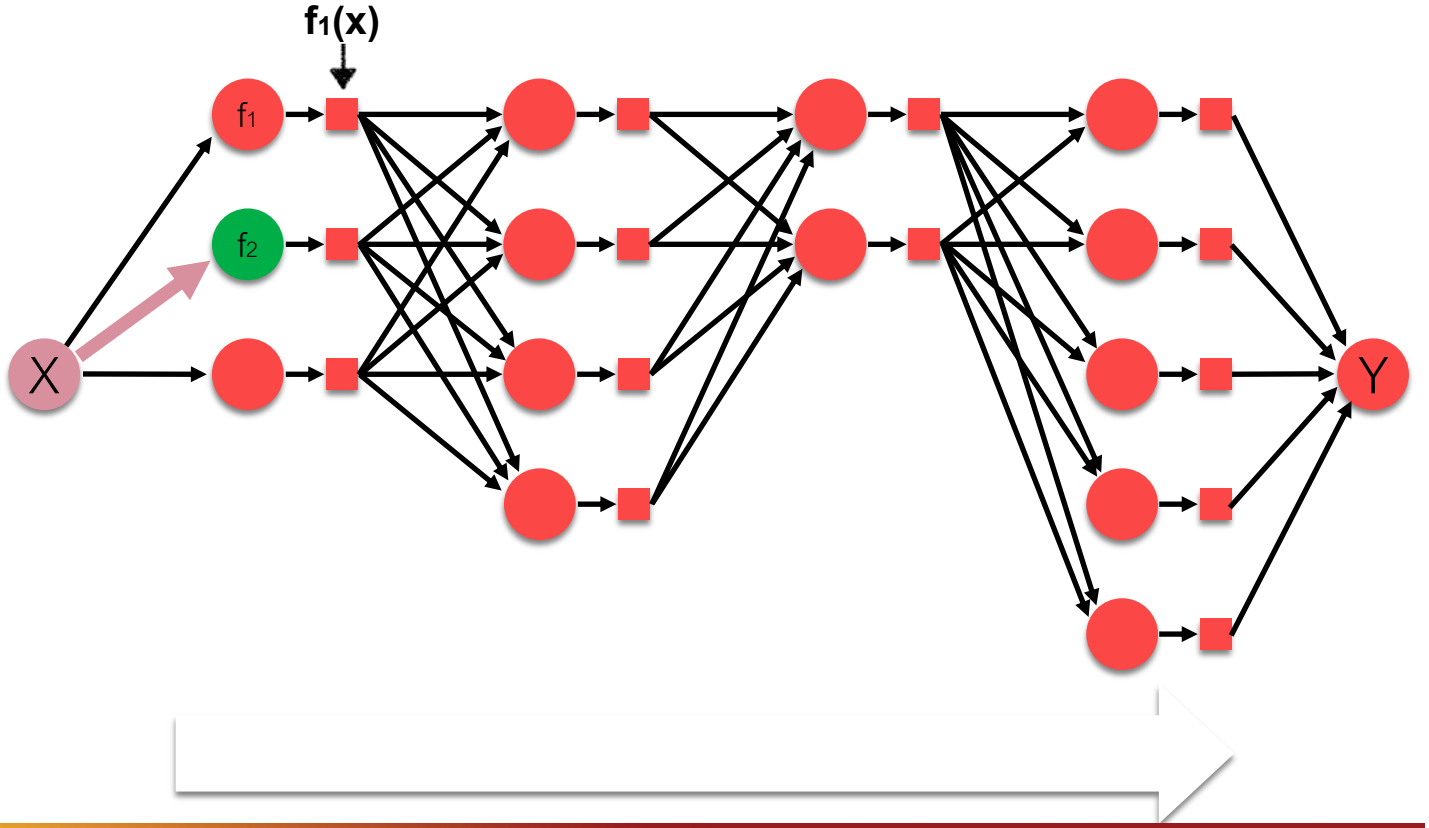
Each neuron is associated w/ the weight matrix W

Sigmod function, can be other non-linear choices

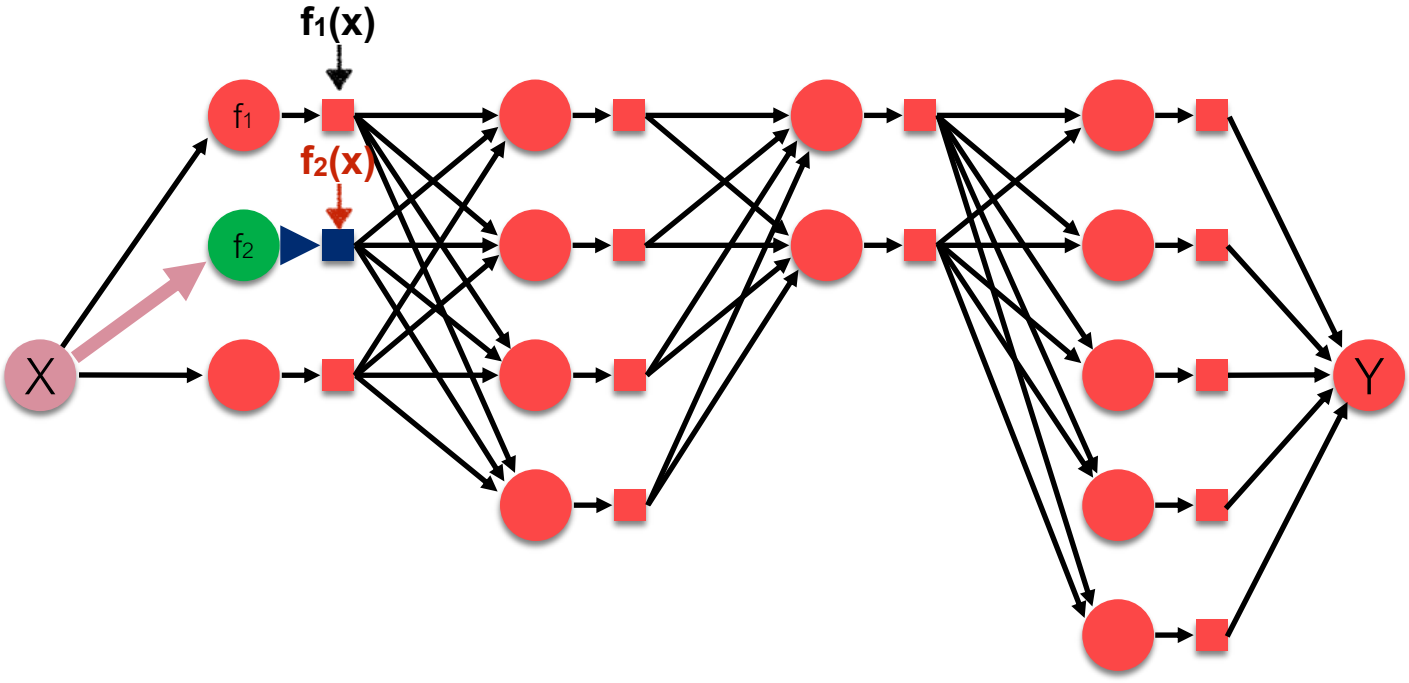
Forward Propagation



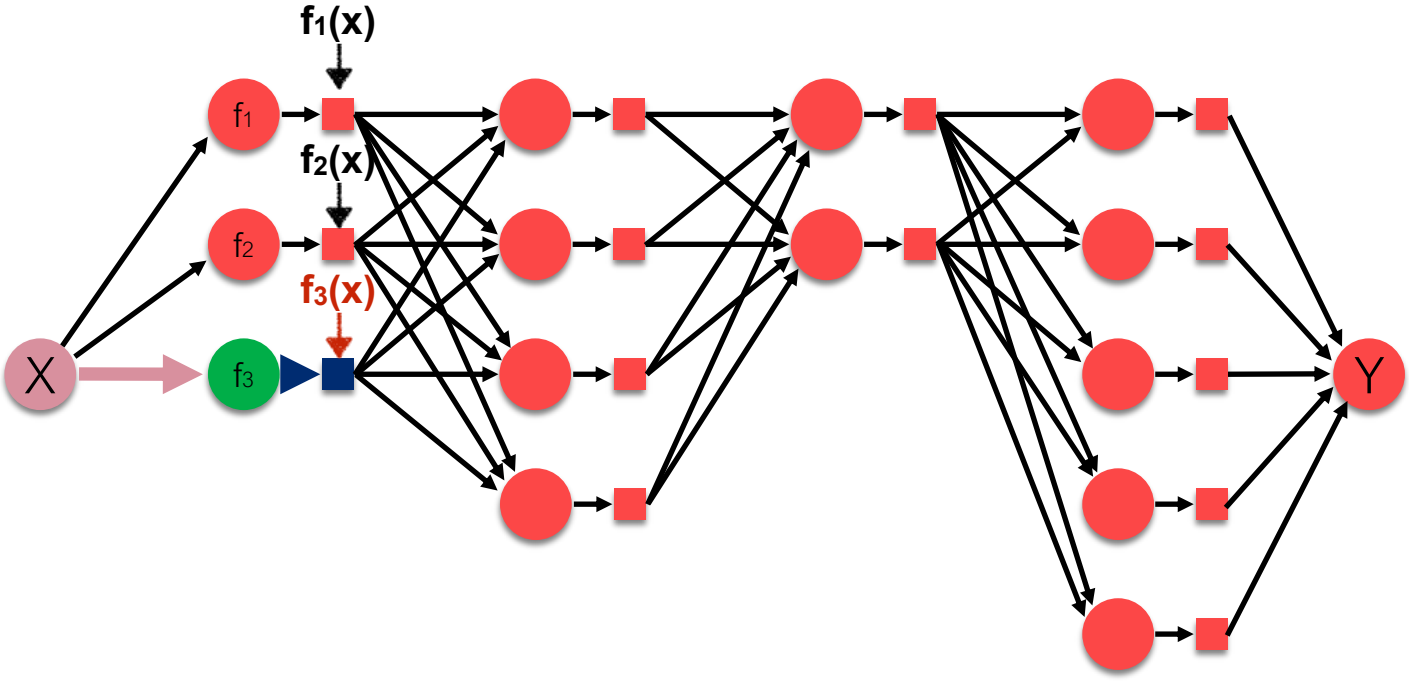
Forward Propagation



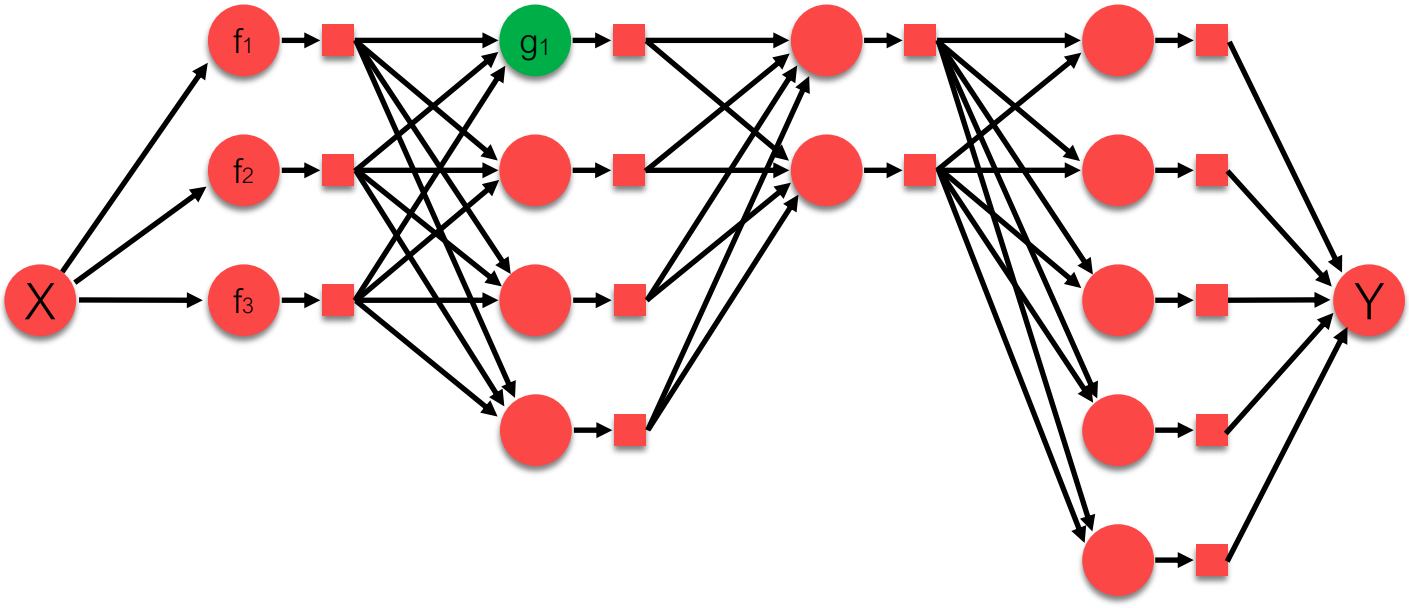
Forward Propagation



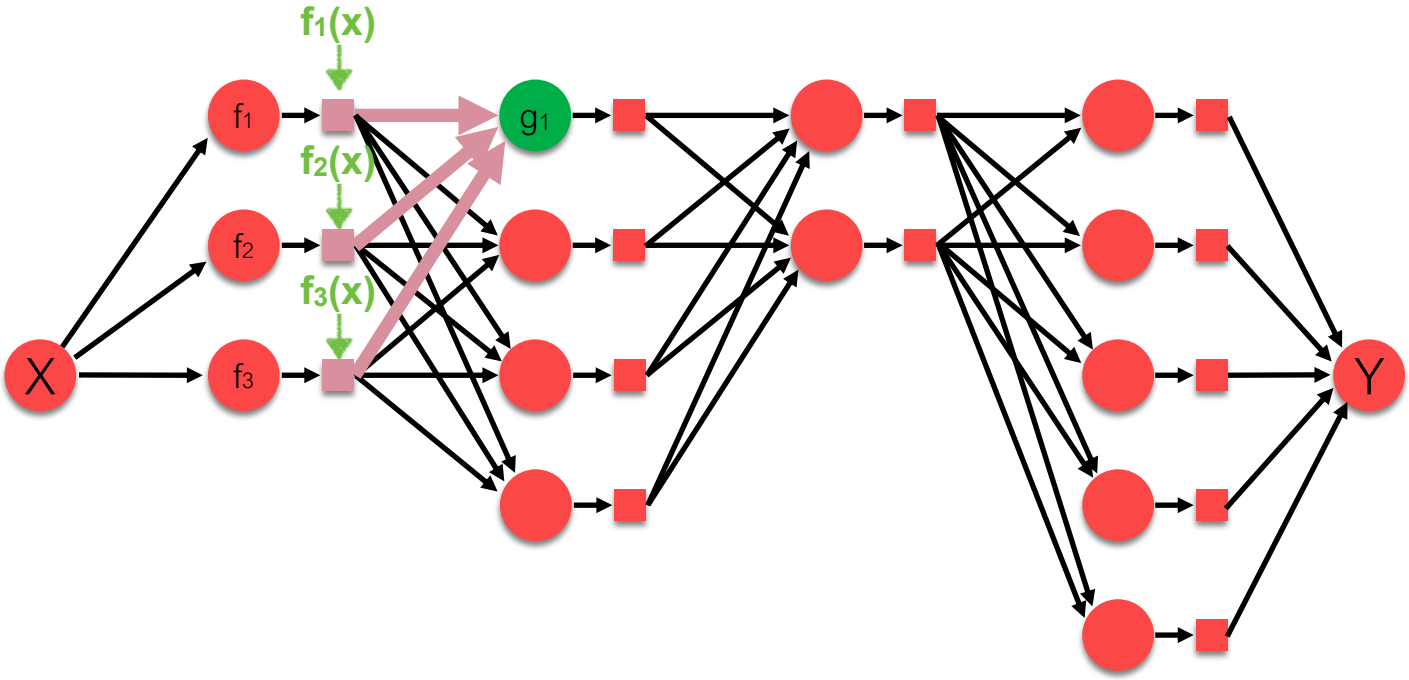
Forward Propagation



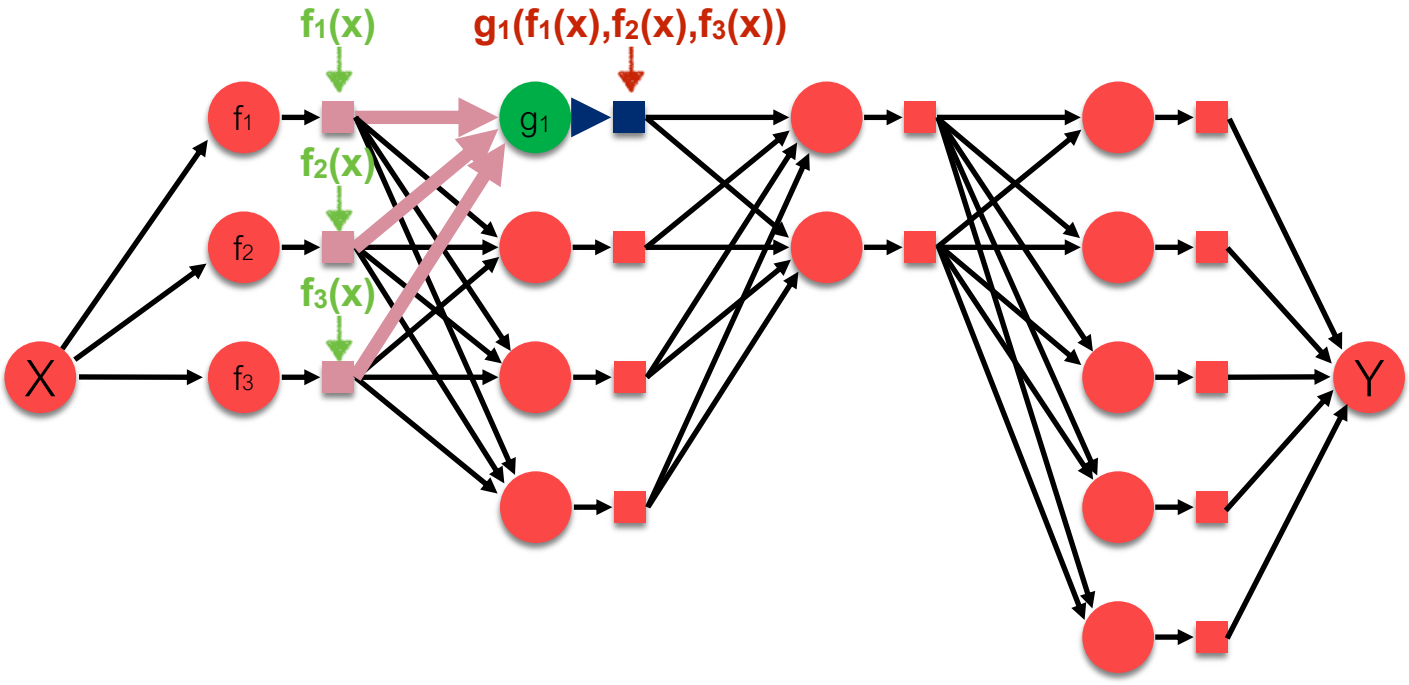
Forward Propagation



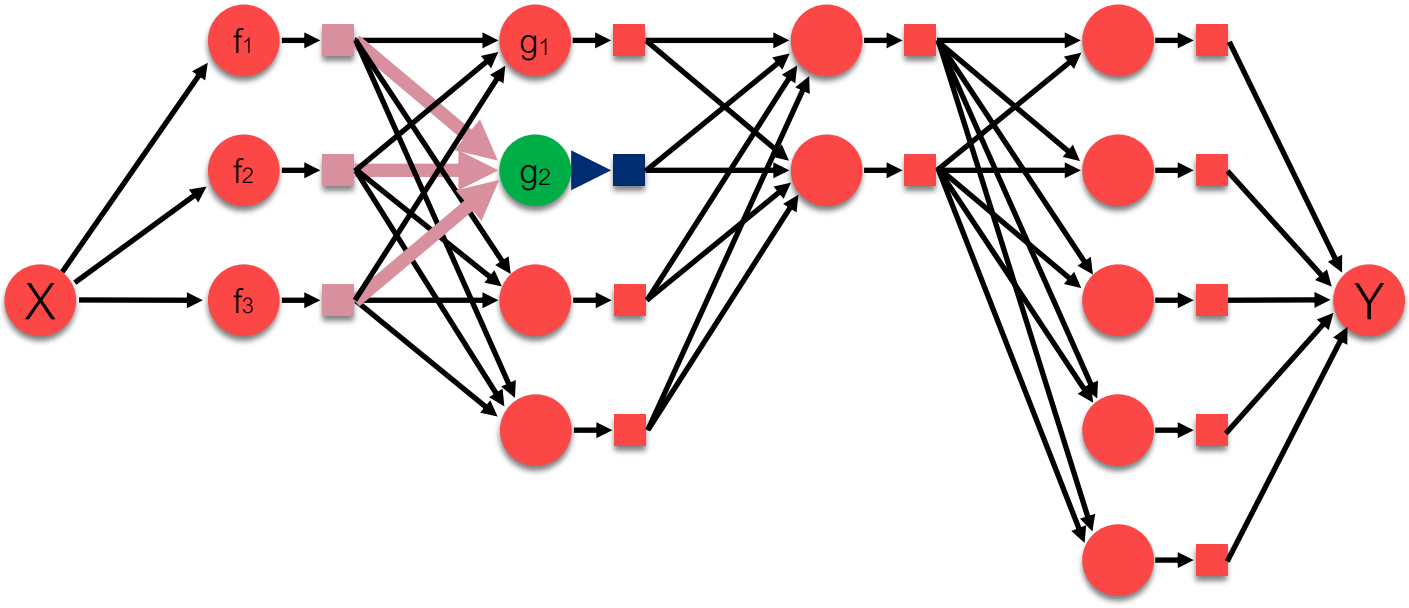
Forward Propagation



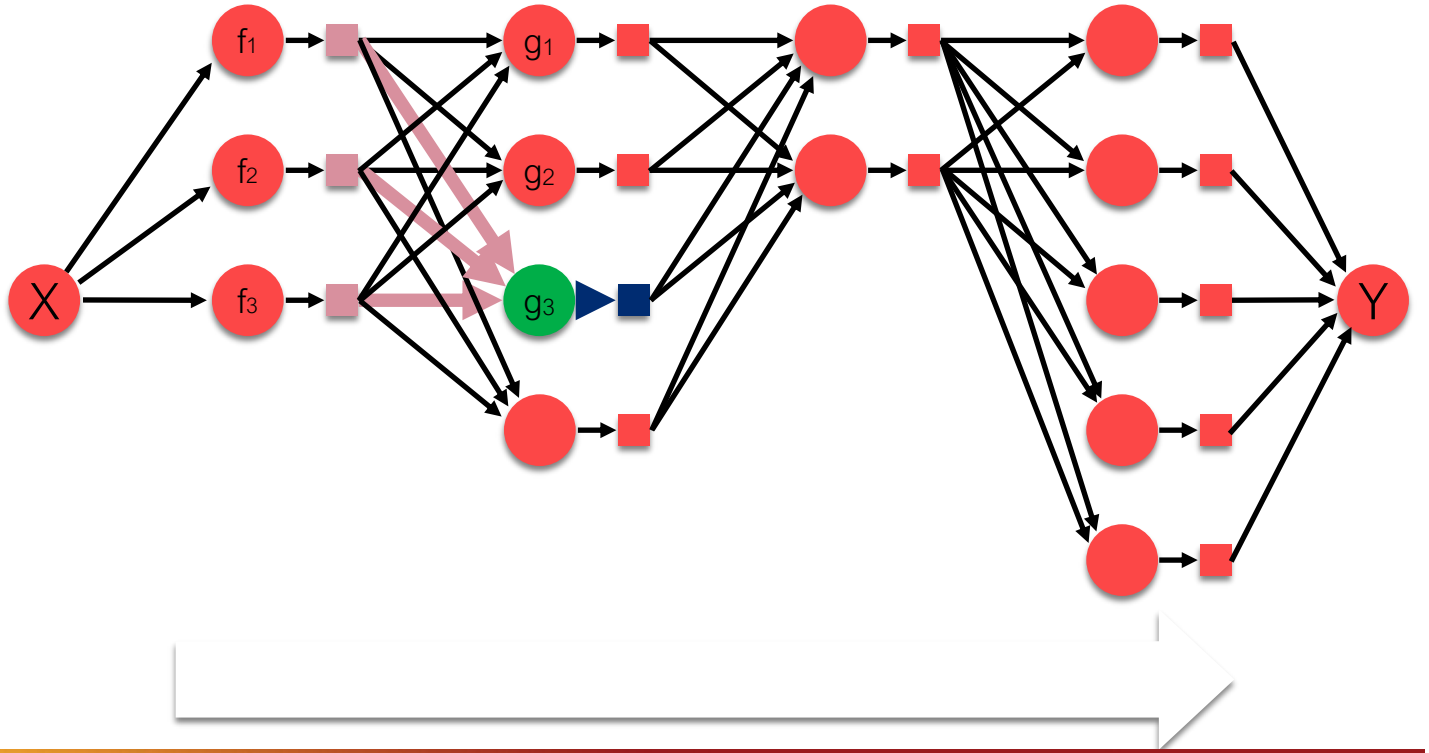
Forward Propagation



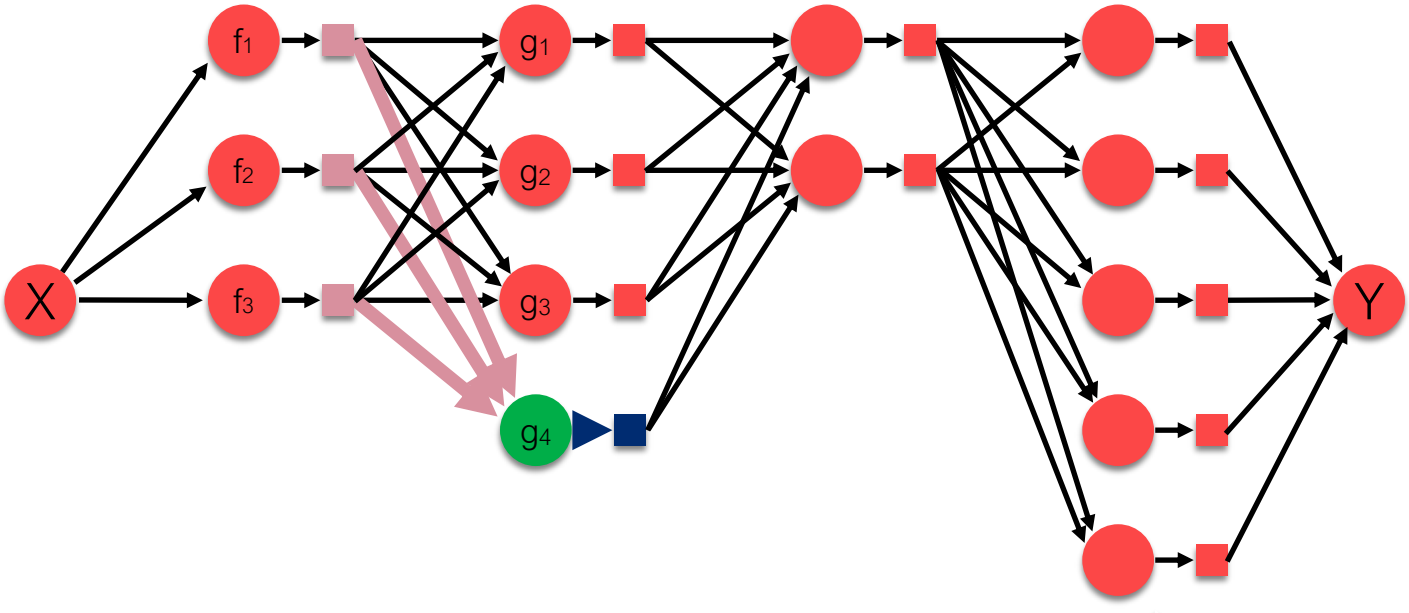
Forward Propagation



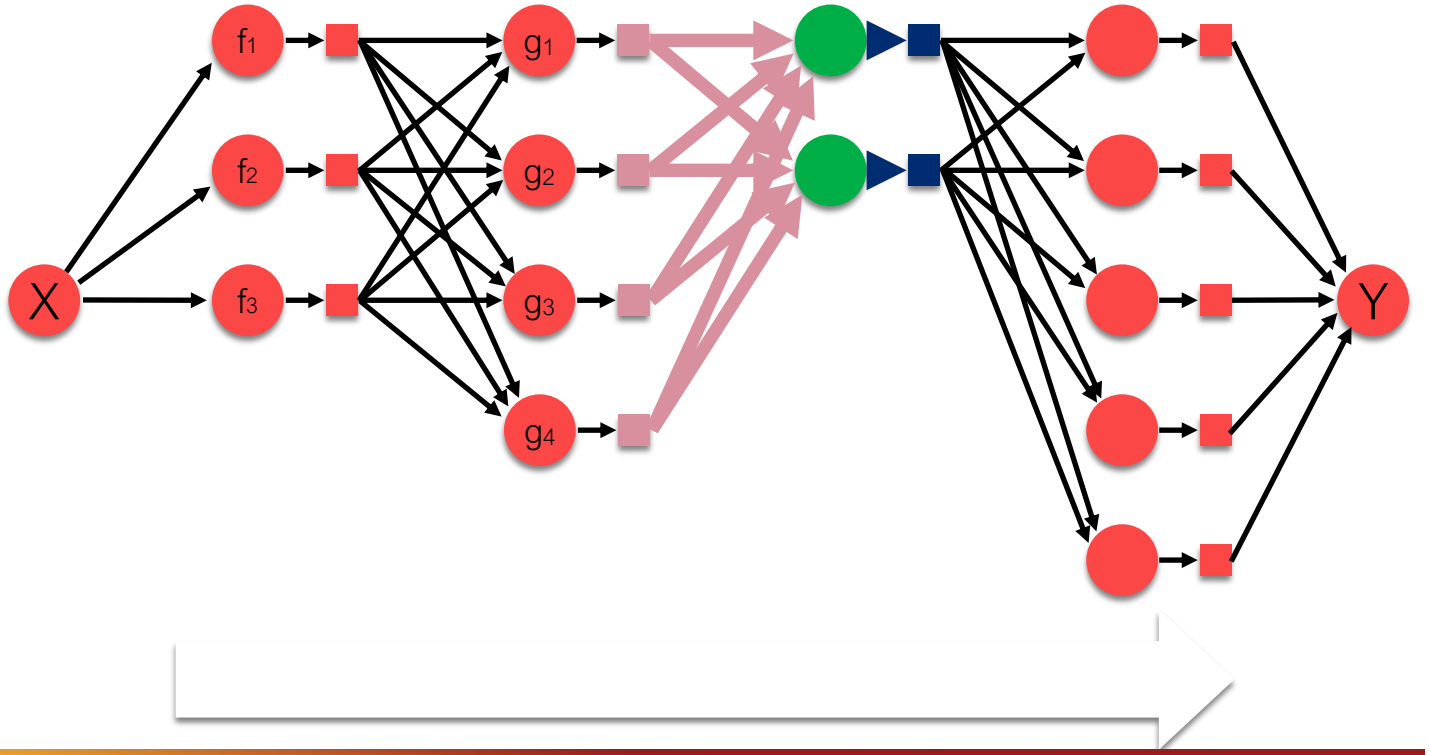
Forward Propagation



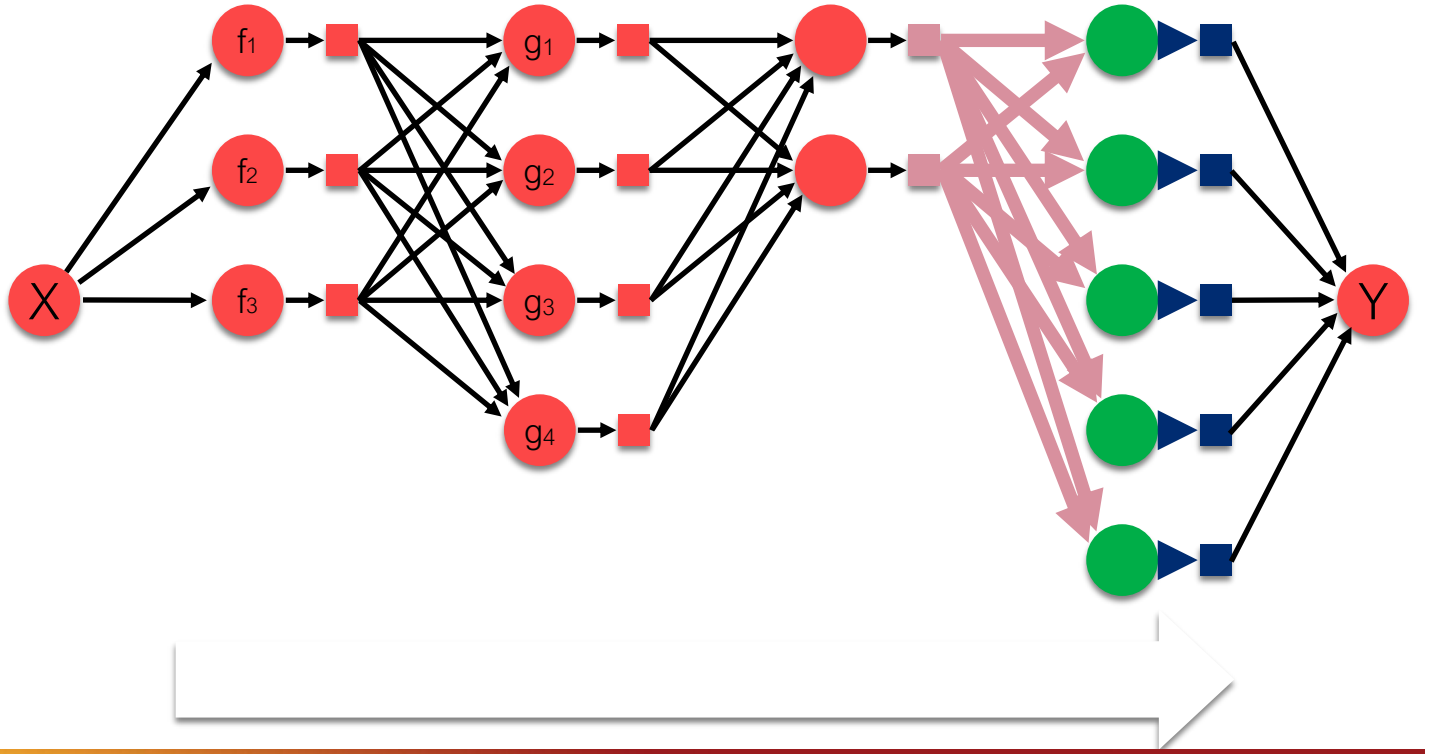
Forward Propagation



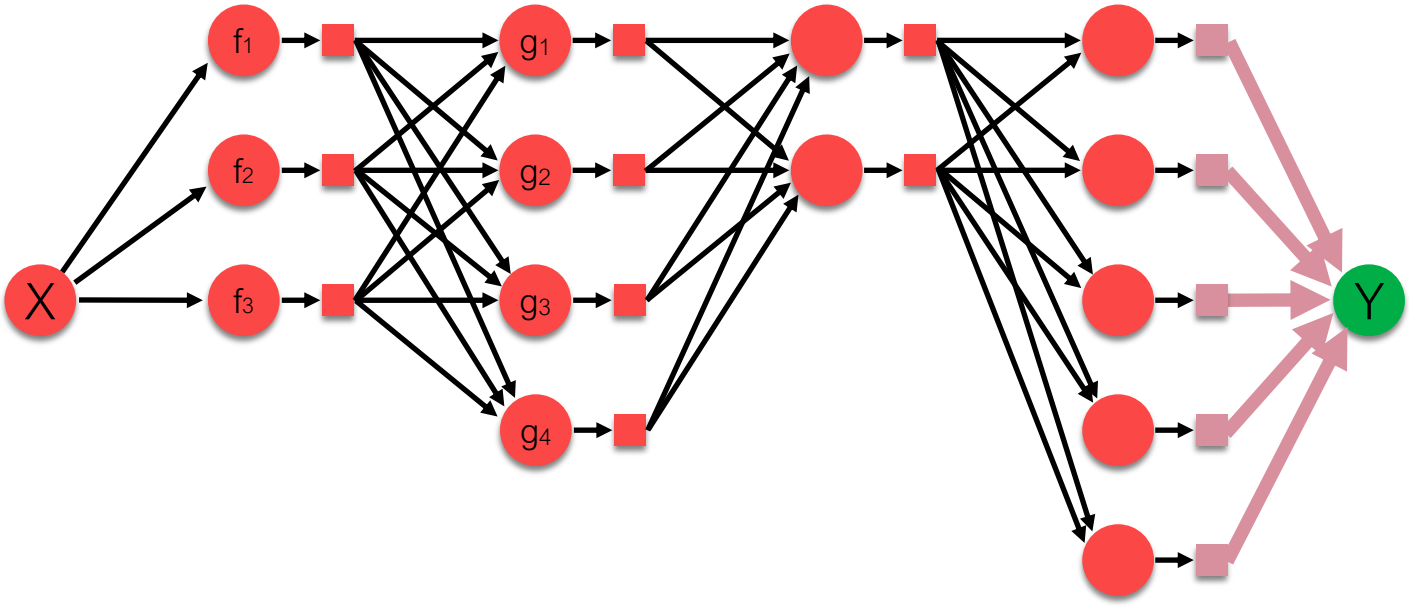
Forward Propagation



Forward Propagation



Forward Propagation



From a Prompt to an Answer

1 / 3

The	light	is	?	?

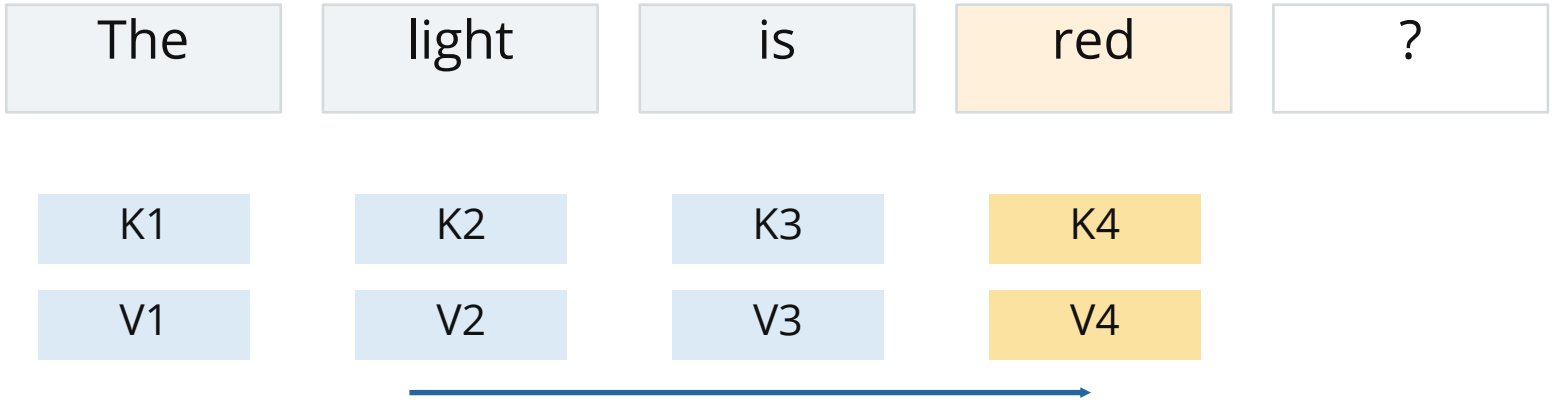
K1	K2	K3
V1	V2	V3

PREFILL: process the prompt

Predict: can the answer tokens all be generated at once?

Hugging Face Transformers, Caching documentation. Schematic tokens and per-layer KV cache.

Reuse the Past Keys and Values



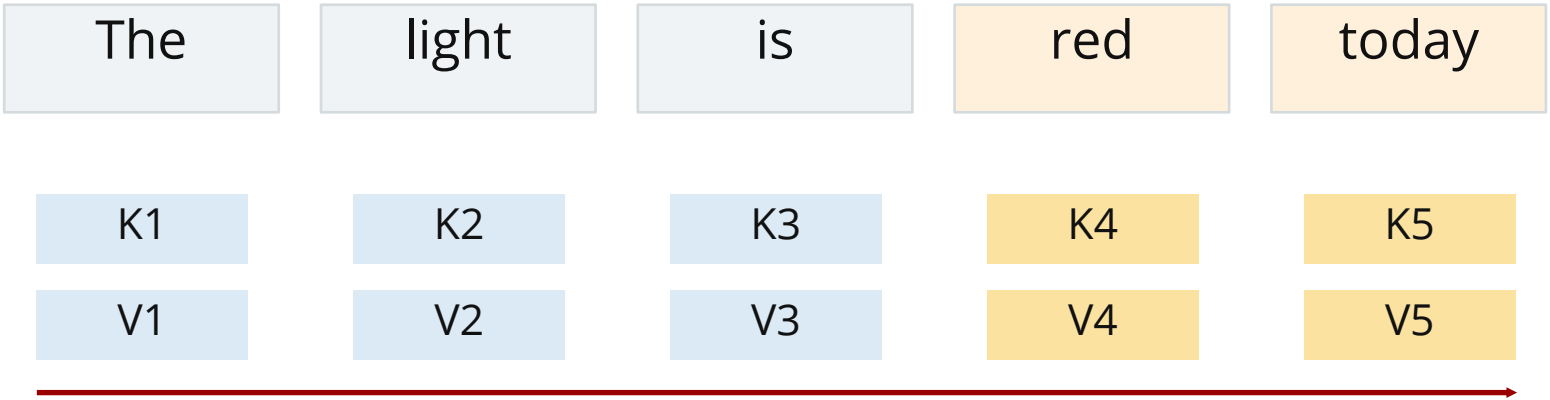
Reuse cached history **+ new K_4, V_4**

DECODE: compute the new position, then append its K and V.

Hugging Face Transformers, Caching documentation. Schematic tokens and per-layer KV cache.

Longer History Still Has a Cost

3 / 3



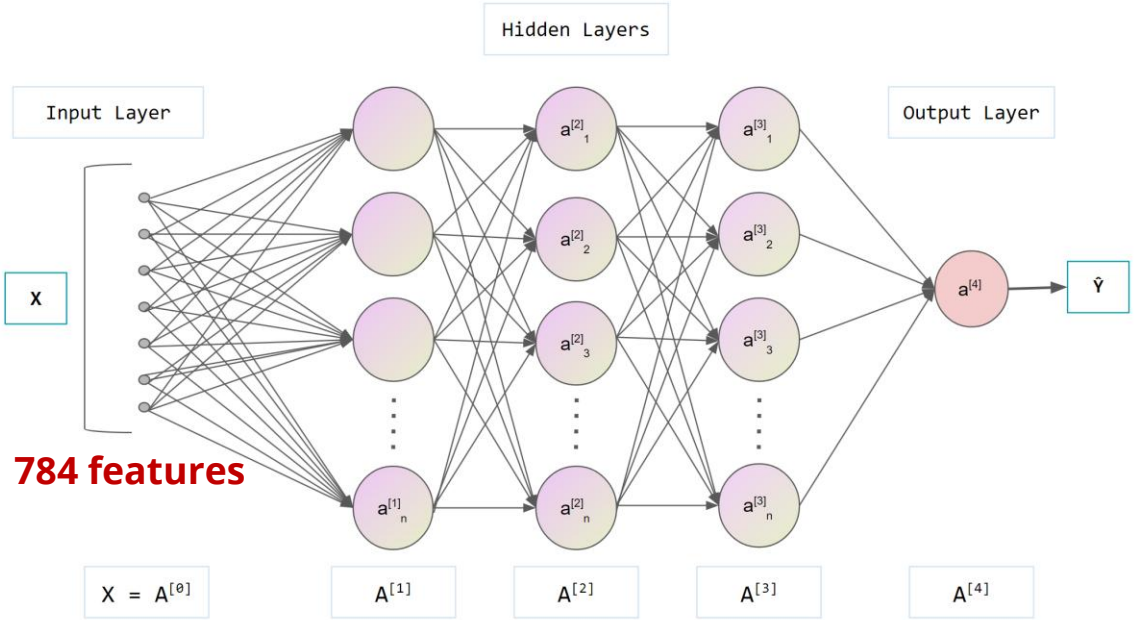
More stored history to attend to

Caching avoids recomputing the past; it does not make history free.

Hugging Face Transformers, Caching documentation. Schematic tokens and per-layer KV cache.

USC Viterbi School of Engineering

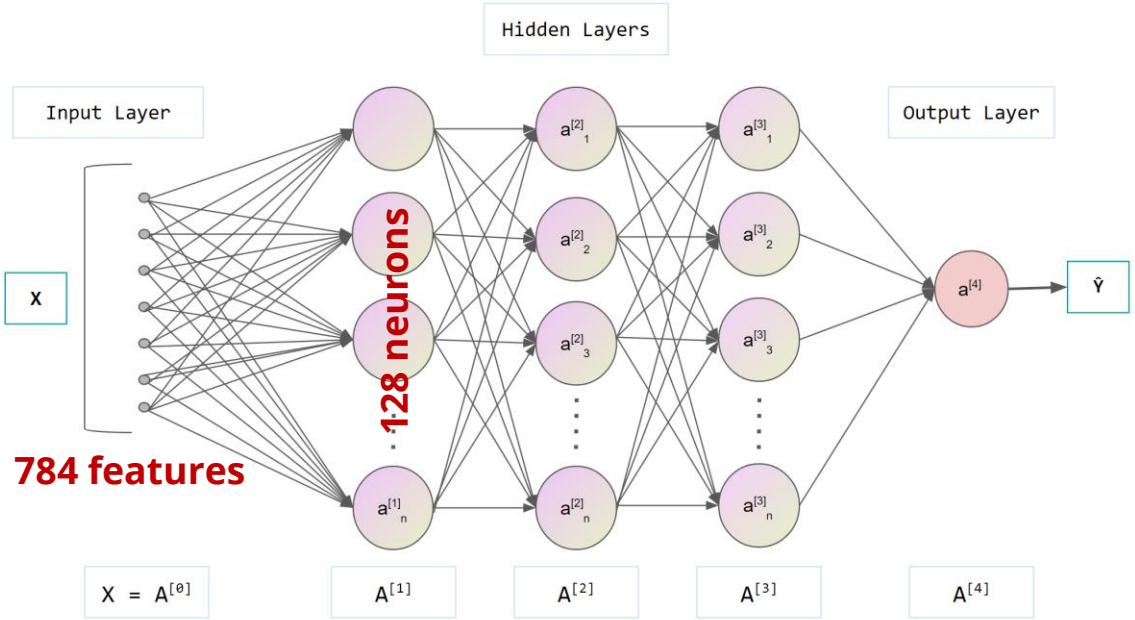
Neural Network Data Shape Explained



Assume each sample is a 28x28 pixels image = 784 features

USC Viterbi School of Engineering

Neural Network Data Shape Explained



The first hidden layer

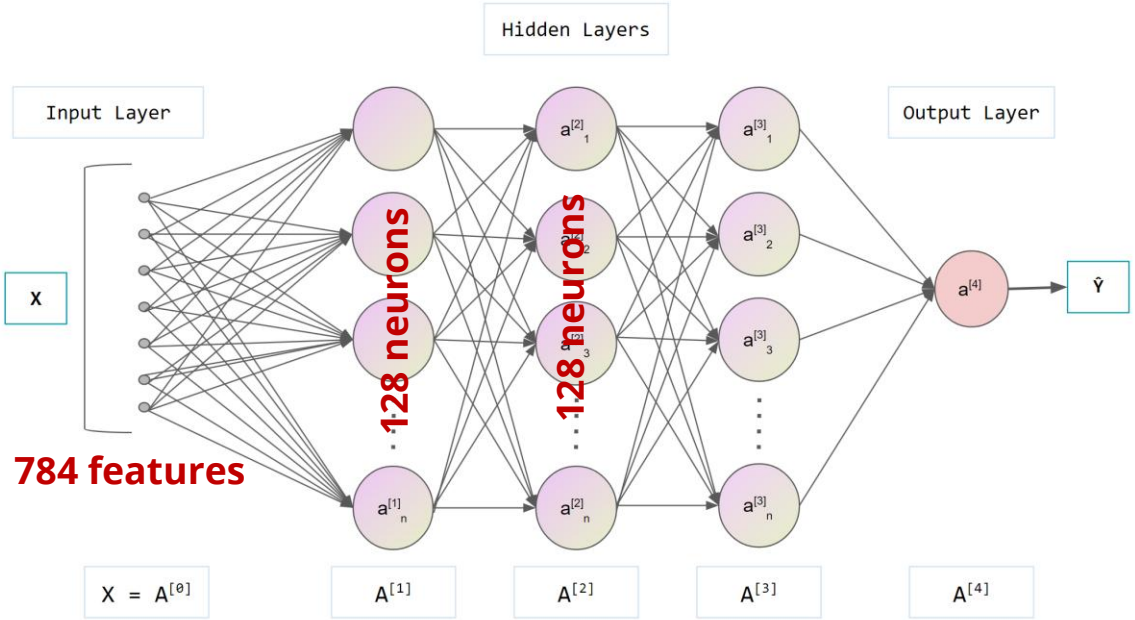
A[1] has 128 neurons

Thus, the weight matrix **W1** for the first layer will be 784x128

After the first layer, it will have

[1x784] · [784x128] = **1x128** outputs

Neural Network Data Shape Explained



After the first layer, it will have

[1x784] · [784x128] = **1x128** outputs

The second layer also gets 128 neurons, and the weight matrix **W2** for the second layer will be 128x128

Neural networks: also called fully connected network

(**Before**) Linear score function: $f = Wx$ This is linear

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

Activation (e.g. sigmoid)

Weight for f2

Weight for f1

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

"Neural Network" is a very broad term; these are more accurately called "fully-connected networks" or sometimes "multi-layer perceptrons" (MLP)

(In practice we will usually add a learnable bias **b** at each layer as well)

Neural networks: 3 layers

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

or 3-layer Neural Network $f = W_3 \max(0, W_2 \max(0, W_1 x))$

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H_1 \times D}, W_2 \in \mathbb{R}^{H_2 \times H_1}, W_3 \in \mathbb{R}^{C \times H_2}$$

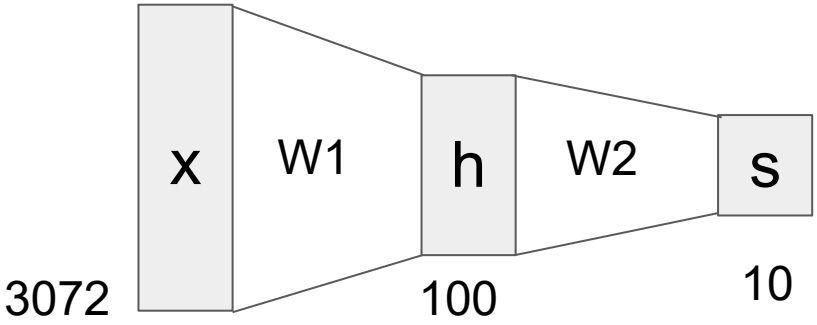
(In practice we will usually add a learnable bias at each layer as well)

Credit to Stanford CS 231n

Neural networks: hierarchical computation

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



Input features compressed from 3072 to 10

$$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$$

Credit to Stanford CS 231n

Neural networks: why is max operator important?

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the **activation function.**

Q: What if we try to build a neural network without it?

$$f = W_2 W_1 x$$

Credit to Stanford CS 231n

Neural networks: why is max operator important?

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

The function $\max(0, z)$ is called the **activation function.**

Q: What if we try to build a neural network without one?

$$f = W_2 W_1 x \qquad W_3 = W_2 W_1 \in \mathbb{R}^{C \times H}, f = W_3 x$$

A: We end up with a linear classifier again!

Neural networks need this non linearity!!

Credit to Stanford CS 231n

Worked Example: One Tiny Network

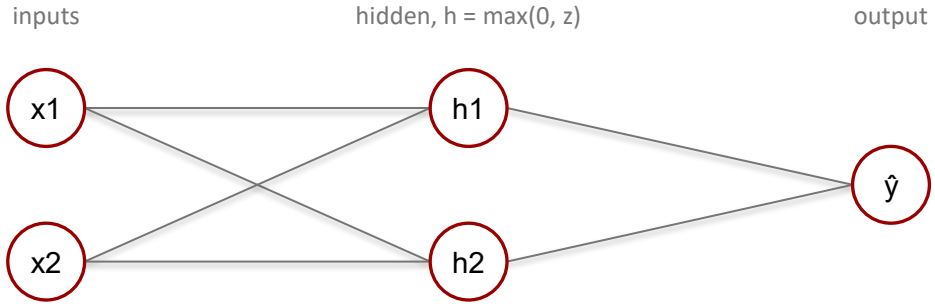
Structure. 2 inputs, 2 hidden units, 1 linear output.

Sample. x = (1, 2), and the label is y = 2.

Hidden rule. h = max(0, z). Both z values here come out positive.

Hidden weights. Unit 1: w = (1, 0.5), b = -0.5. Unit 2: w = (-1, 1), b = 0.

Output weights. v = (1, -0.5), c = 0.



Compute the Hidden Values

Unit 1. z1 = 1(1) + 0.5(2) - 0.5 = 1.5

h1 = max(0, 1.5) = 1.5

Unit 2. z2 = -1(1) + 1(2) + 0 = 1

h2 = max(0, 1) = 1

Note. The bias joins the sum. A weight of -1 subtracts one input, not the whole unit.

Compute the Output

Rule. yhat = v1 h1 + v2 h2 + c

Substitute. yhat = 1(1.5) + (-0.5)(1) + 0

Prediction. yhat = 1.0

Both hidden outputs feed the same output unit, each through its own weight.

Compute the Loss

Label. y = 2

Squared error. L = (yhat - y)^2

Substitute. L = (1 - 2)^2 = 1

Meaning. The prediction sits 1 below the label. Zero is what we want.

Which Way Should v1 Move?

Hold fixed. h1 = 1.5, h2 = 1, v2 = -0.5, c = 0. Only v1 changes.

Try v1 = 1.2. yhat = 1.3, so L = (1.3 - 2)^2 = 0.49

Try v1 = 0.8. yhat = 0.7, so L = (0.7 - 2)^2 = 1.69

Direction. Raising v1 lowers the loss. So v1 should go up.

Differentiate One Small Formula

Freeze. h1 = 1.5, h2 = 1, v2 = -0.5, c = 0 are now constants.

One variable. yhat(v1) = 1.5 v1 - 0.5

Loss. L(v1) = (1.5 v1 - 2.5)^2 = 2.25 v1^2 - 7.5 v1 + 6.25

Derivative. dL/dv1 = 4.5 v1 - 7.5

At v1 = 1. dL/dv1 = -3, and the negative slope agrees with the perturbation.

Take One Gradient Step

Learning rate. eta = 0.2

Update. v1 <- v1 - eta (dL/dv1) = 1 - 0.2(-3) = 1.6

New forward pass. yhat = 1.6(1.5) + (-0.5)(1) + 0 = 1.9

New loss. L = (1.9 - 2)^2 = 0.01

One weight changed, and the loss fell from 1.00 to 0.01.

Your Turn: Update v2

Reset. Back to v1 = 1, v2 = -0.5, h1 = 1.5, h2 = 1, c = 0, y = 2.

Task. Write L as a function of v2, differentiate it at v2 = -0.5, then take one step with eta = 0.2.

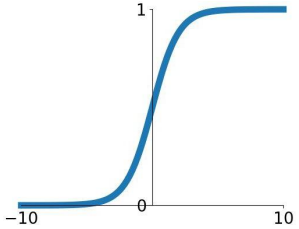
Check. Is your new loss below the original loss of 1?

Activation functions

ReLU is a good default choice for most problems

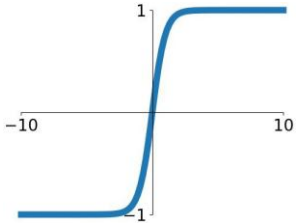
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



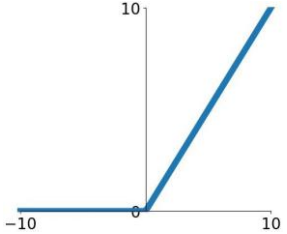
tanh

$$\tanh(x)$$



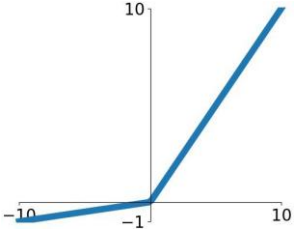
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

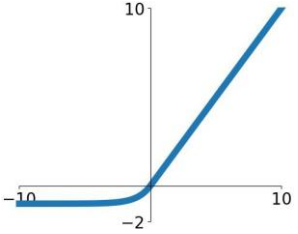


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

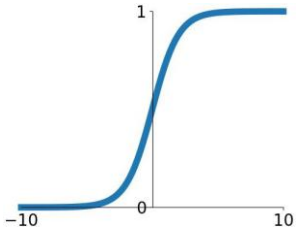
$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation functions

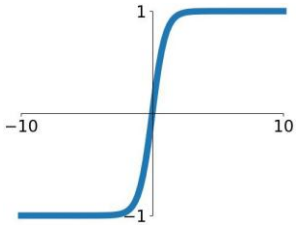
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

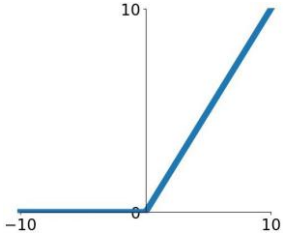


tanh

$$\tanh(x)$$



$$\max(0, x)$$



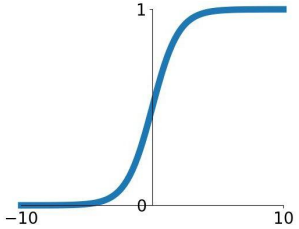
Sigmoid Function

- **Characteristics**: Sigmoid squashes the input values into a range between 0 and 1. It's historically been used for binary classification.
- **When to use**: In the output layer for binary classification tasks. When you need probabilities as outputs, since its output can be interpreted as a probability.
- **Limitations**: Prone to **vanishing gradient problem (discussed later)**. Outputs are not zero-centered.

Activation functions

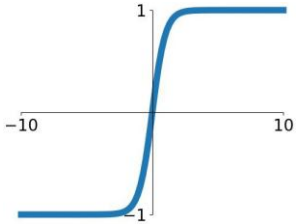
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



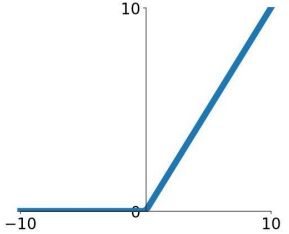
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



ReLU:

Characteristics: ReLU is a piece-wise linear function that outputs the input directly if positive, otherwise, it will output zero.

When to use: In most hidden layers, as it helps with the vanishing gradient problem and speeds up training. Good default choice for deep neural networks.

Limitations:

Can suffer from "dying ReLU" problem, where neurons can become inactive on all current training examples.

Activation functions

Leaky ReLU

Characteristics: A variation of ReLU that allows a small, non-zero gradient when the unit is not active.

When to use:

To combat the dying ReLU problem.

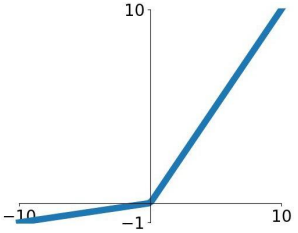
In networks where fast training and convergence are important.

Limitations:

The performance gain over ReLU can be inconsistent.

Leaky ReLU

$$\max(0.1x, x)$$

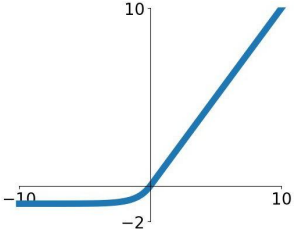


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



🚀 ReLU Strikes Back: Exploiting Activation Sparsity in Large Language Models

Authors:
Iman Mirzadeh, Keivan Alizadeh, Sachin Mehta, Carlo C Del Mundo, Oncel Tuzel, Golnoosh Samei, Mohammad Rastegari, Mehrdad Farajtabar

Conference:
ICLR 2024

Published as a conference paper at ICLR 2024

RELU STRIKES BACK: EXPLOITING ACTIVATION SPARSITY IN LARGE LANGUAGE MODELS

Iman Mirzadeh[†], Keivan Alizadeh, Sachin Mehta, Carlo C Del Mundo[◇]
Oncel Tuzel, Golnoosh Samei[◇], Mohammad Rastegari, Mehrdad Farajtabar[†]
Apple

ABSTRACT

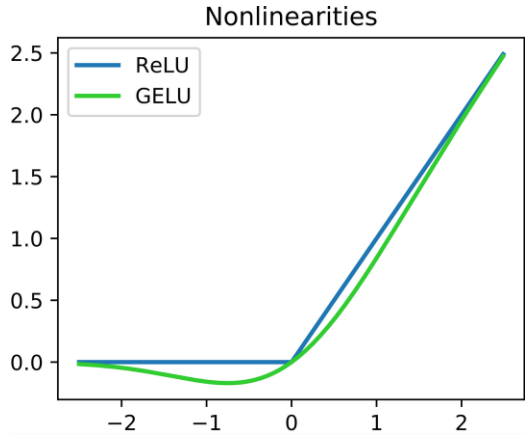
Large Language Models (LLMs) with billions of parameters have drastically transformed AI applications. However, their demanding computation during inference has raised significant challenges for deployment on resource-constrained devices. Despite recent trends favoring alternative activation functions such as GELU or SiLU, known for increased computation, this study strongly advocates for reinstating ReLU activation in LLMs. We demonstrate that using the ReLU activation function has a negligible impact on convergence and performance while significantly reducing computation and weight transfer. This reduction is particularly valuable during the memory-bound inference step, where efficiency is paramount. Exploring sparsity patterns in ReLU-based LLMs, we unveil the reutilization of activated neurons for generating new tokens and leveraging these insights, we propose practical strategies to substantially reduce LLM inference computation up to three times, using ReLU activations with minimal performance trade-offs.

First, Transformers Adopt GELU

GELU Is Standard in Transformers; the Case Is Empirical

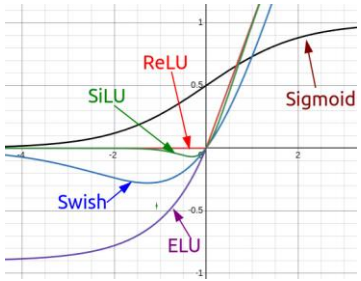
- **Key Issue with ReLU:**

- ReLU is **not differentiable at zero**; it has a **hard cutoff** on the negative side.
- **GELU is smooth everywhere** and has a **Gaussian CDF weighting** of the input.



- **Why This Matters for Transformers:**

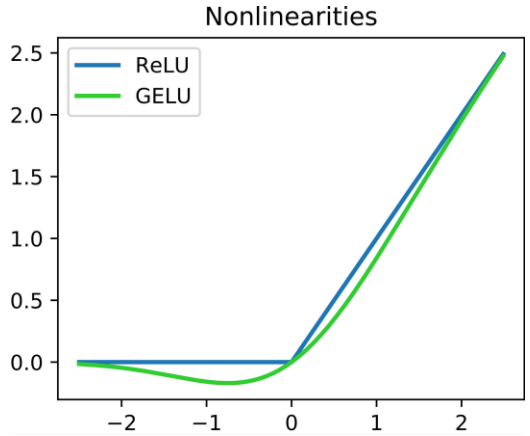
- Transformers, including LLMs, **rely heavily on stable gradient flow**.
- Hendrycks and Gimpel (arXiv 2016) report **gains on the tasks they tested**; smoothness is **not shown to be the cause**.



First, Transformers Adopt GELU

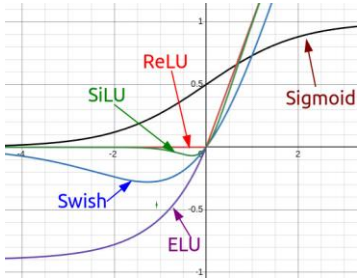
📊 **Evidence:**

- **BERT (Devlin et al., NAACL 2019)** and **GPT-3 (Brown et al., NeurIPS 2020)** both use **GELU**. Neither runs an ablation against ReLU; the comparison is in the **Hendrycks and Gimpel preprint**.



ReLU's Problem: Dead Neurons

- If **many activations fall into the negative range**, ReLU **outputs zero and receives zero gradient**.
- This creates **"dead neurons"**, which **stop learning from that data** until upstream changes revive them.
- This is a significant issue in deep models with large FFN layers, where **some neurons may never activate** after certain weight updates.



USC Viterbi School of Engineering

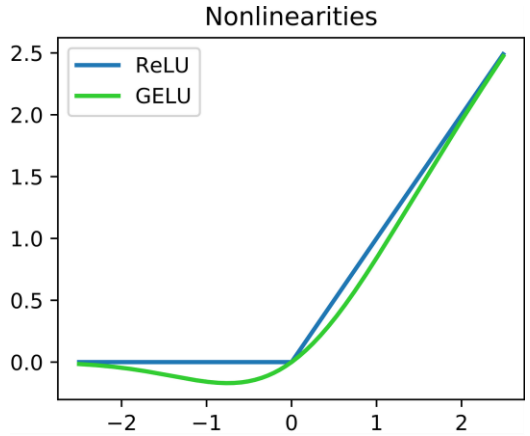
First, Transformers Adopt GELU

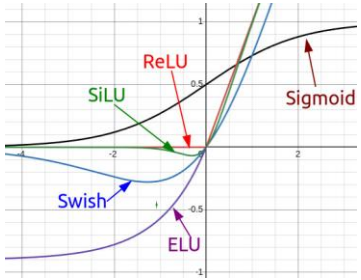
📊 **Evidence:**

- **BERT (Devlin et al., NAACL 2019)** and **GPT-3 (Brown et al., NeurIPS 2020)** both use **GELU**. Neither runs an ablation against ReLU; the comparison is in the **Hendrycks and Gimpel preprint**.

🔹 **Why GELU is Preferred:**

- GELU **has a soft threshold** instead of a hard cutoff.
- Neurons have a **chance of recovering** instead of being stuck at zero output.





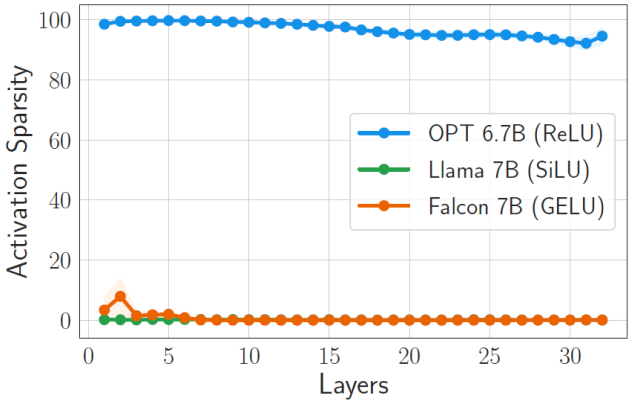
Problem Statements and Why

🧠 **Why Does Activation Function Matter?**

- Large Language Models (LLMs) have **huge inference costs** (compute + memory).
- Common activations like **GELU & SiLU** improve convergence but increase computation.
- **ReLU was largely replaced**, despite its benefits in **activation sparsity**.

⚠️ **Challenge:**

- **Can we bring back ReLU to improve LLM efficiency without losing performance?**



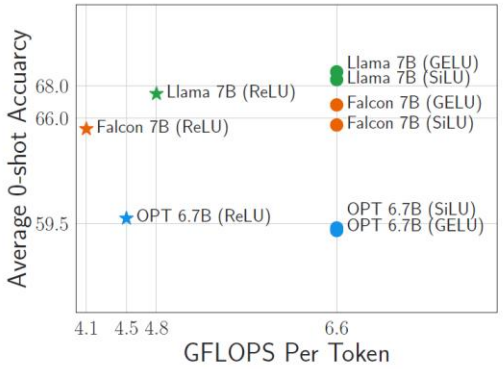
(a) Sparsity of different models

Activation Sparsity of different pretrained models: ReLU-based OPTs show significantly higher sparsity.

Key Contributions & Findings

✅ **What This Paper Proposes:**

- **Reintroducing ReLU in LLMs to exploit activation sparsity.**
- **Minimal performance tradeoff, but significant computational savings.**
- **Relufication:** Converting pretrained LLMs (e.g., Llama, Falcon) to ReLU.
- **Using ReLU sparsity to enhance speculative decoding speed.**
- **Shifted ReLU**: Further improving sparsity with simple modifications.



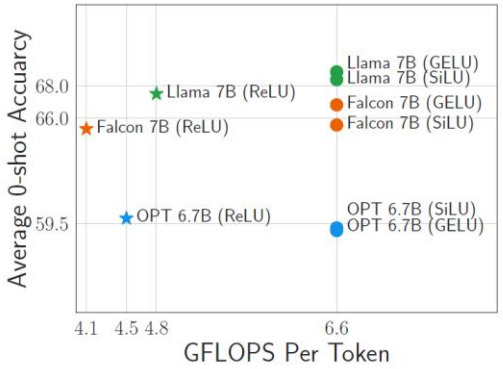
(c) Accuracy vs. Computation

Comparison of inference efficiency and performance of the different models with different activation functions after fine-tuning: The choice of activation function does not significantly impact the accuracy, as any of GELU, SiLU, or ReLU can be used on all three models and achieve the same level of accuracy as the original activation function. However, **using ReLU can provide an additional benefit of leading to activation sparsity and faster inference**

Key Contributions & Findings

🧪 **Experimental Findings:**

- Replacing GELU/SiLU with ReLU costs little accuracy in their results.
- ReLU-based models require up to **3×** fewer inference FLOPs in the models they test.



(c) Accuracy vs. Computation

Comparison of inference efficiency and performance of the different models with different activation functions after fine-tuning: The choice of activation function does not significantly impact the accuracy, as any of GELU, SiLU, or ReLU can be used on all three models and achieve the same level of accuracy as the original activation function. However, **using ReLU can provide an additional benefit of leading to activation sparsity and faster inference**

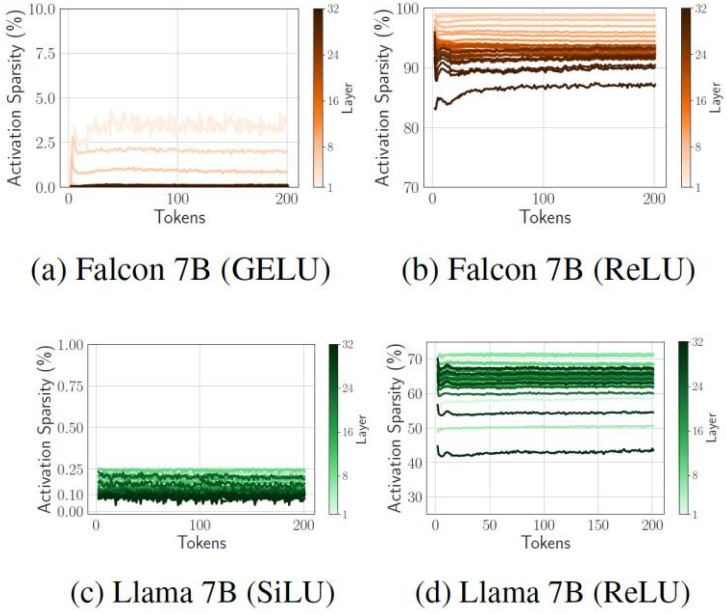
Key Contributions & Findings

📊 **Efficiency Gains from ReLU**
1️⃣ **Higher Activation Sparsity → Fewer Computations**

- Skipping that work needs **sparsity-aware kernels**.
  2️⃣ **Better Memory Efficiency → Less Data Movement**
- Savings depend on the **sparsity pattern**.
  3️⃣ **Easy to Implement**
- **Works in existing transformer architectures** with minimal changes.

🚀 **Impact:**

- **Fewer FLOPs per token → faster with matching kernels**.



Activation sparsity of Falcon and Llama models improves significantly after relufication.

USC Viterbi School of Engineering

Important Activation Interview Questions

Question: What is an activation function in neural networks and why are they used?

Important Activation Interview Questions

Question: What is an activation function in neural networks and why are they used?

Answer: An activation function in a neural network is a mathematical function applied to the output of a neuron or layer of neurons, transforming the input signal into an output signal.

It's used for introducing **non-linear** properties to the network. Without non-linearity, the neural network would essentially become a linear regression model, incapable of handling complex data patterns.

Important Activation Interview Questions

Question: Can you compare and contrast sigmoid, tanh, and ReLU activation functions?

Important Activation Interview Questions

Question: Can you compare and contrast sigmoid, tanh, and ReLU activation functions?

Answer: Sigmoid squashes the input values into a range between 0 and 1. It's often used in the output layer for **binary classification**.

Tanh is similar but outputs values between -1 and 1, making it zero-centered and thus, in some cases, more efficient than sigmoid.

ReLU is piece-wise linear, outputting the input directly if it is positive, otherwise zero. It's widely used in hidden layers due to its computational efficiency and ability to mitigate the **vanishing gradient** problem. However, ReLU can suffer from the "dying ReLU" problem, where units output zero on current data.

Important Activation Interview Questions

Question: What is the 'dying ReLU' problem and how can it be addressed?

Important Activation Interview Questions

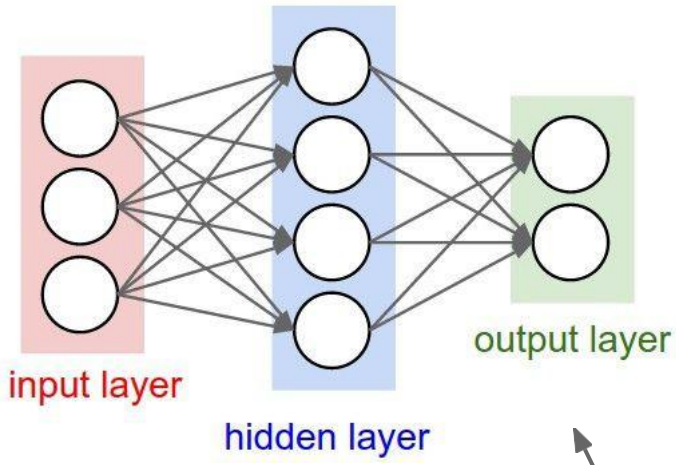
Question: What is the 'dying ReLU' problem and how can it be addressed?

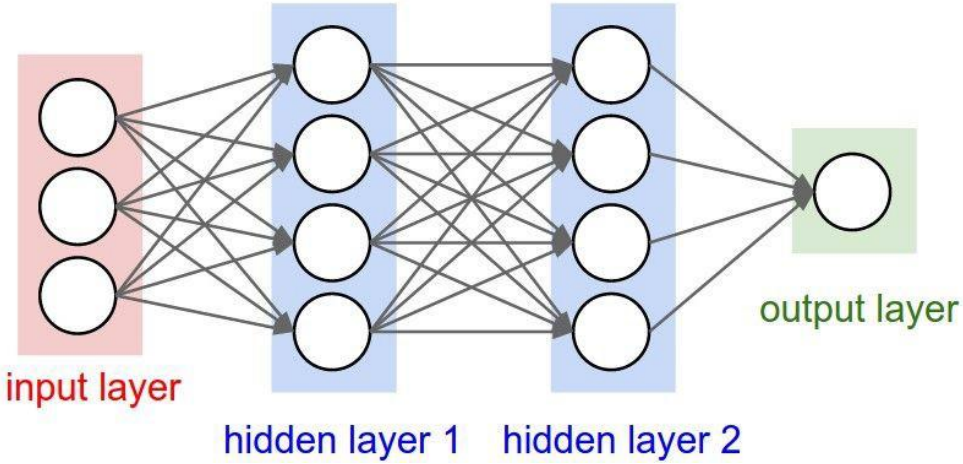
Answer: The 'dying ReLU' problem occurs when **ReLU** neurons become inactive and **output zero on every current training example**.

This happens when the weights shift so that the neuron's pre-activation is negative on every training example, which zeroes its gradient and stalls its learning.

To address this, variants of ReLU like **Leaky ReLU** or Parametric ReLU (PReLU) are used. These functions allow a small, positive gradient when the unit is not active, thereby keeping the neurons alive.

Neural networks: Architectures (Revisited)





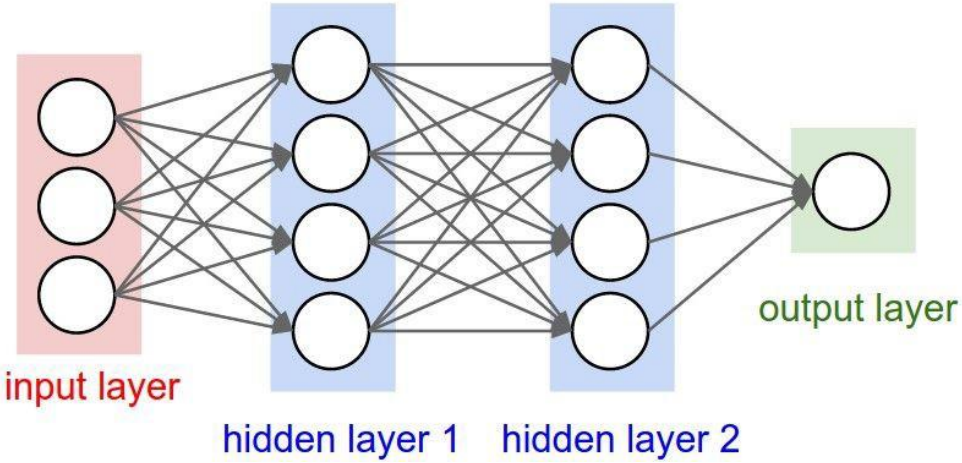
"2-layer Neural Net", or
"1-hidden-layer Neural Net"

"3-layer Neural Net", or
"2-hidden-layer Neural Net"

"Fully-connected" layers

Credit to Stanford CS 231n

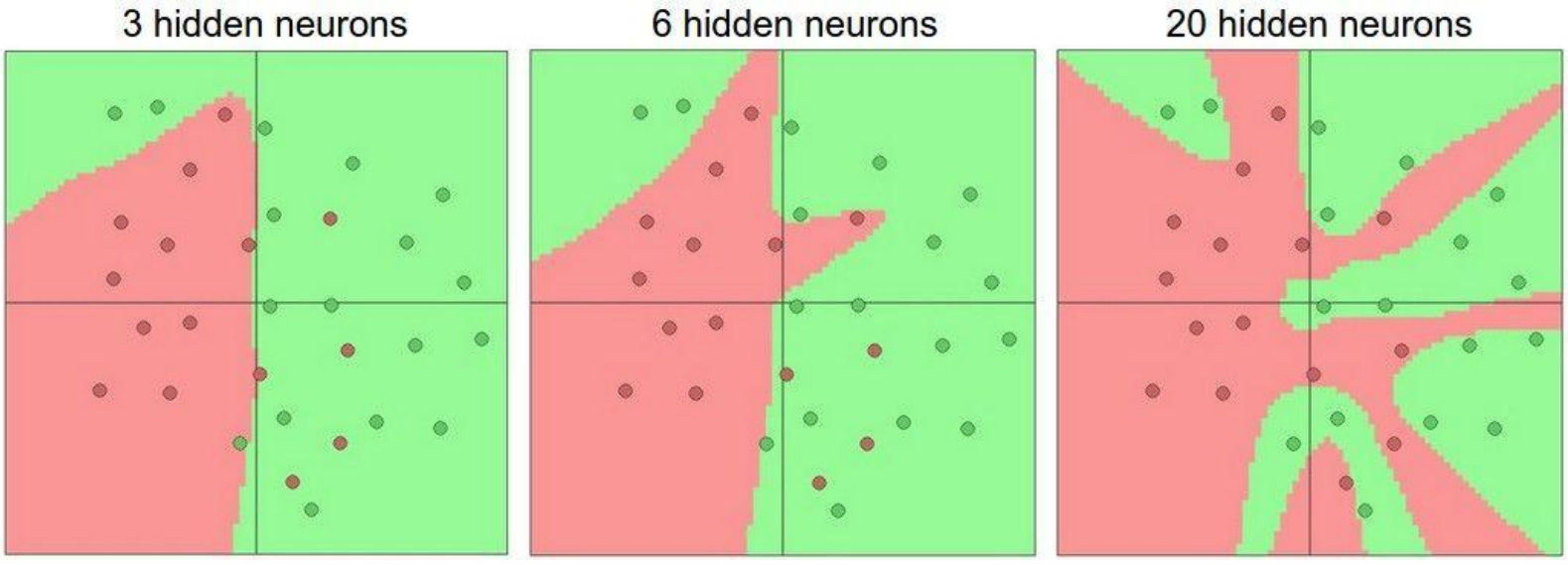
Example feed-forward computation of a neural network




```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

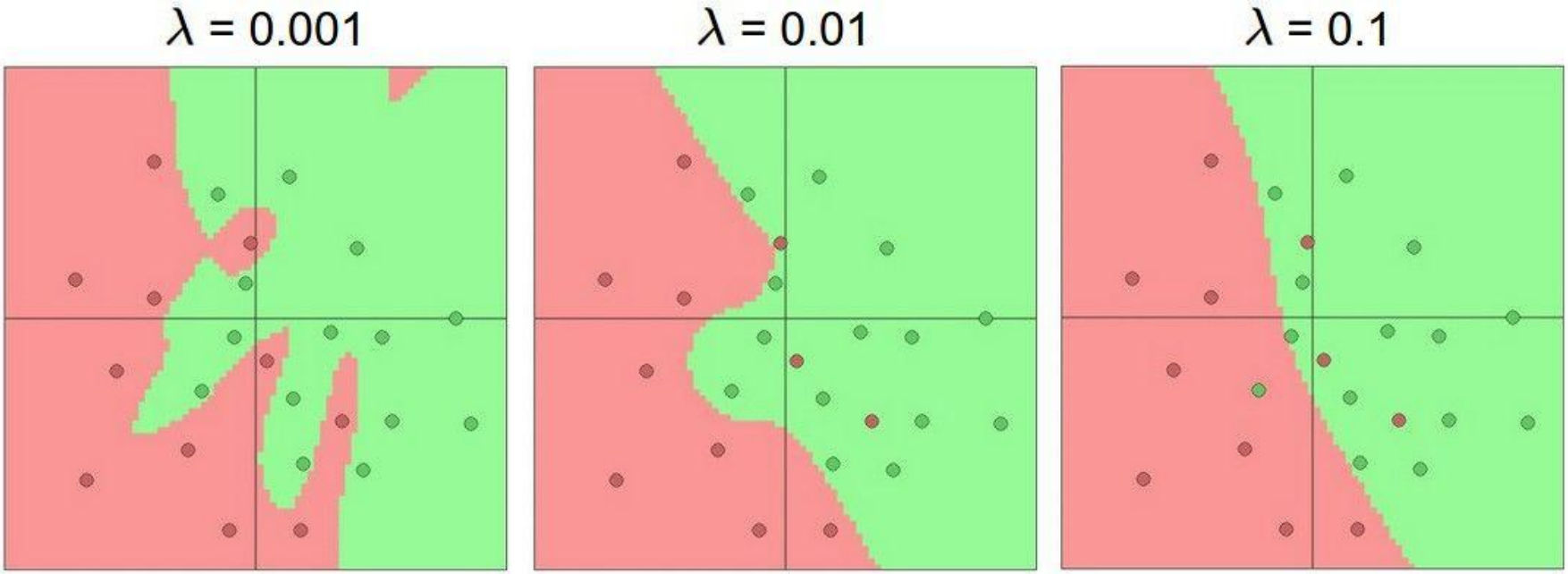
Credit to Stanford CS 231n

More Layers Larger Model Capacity



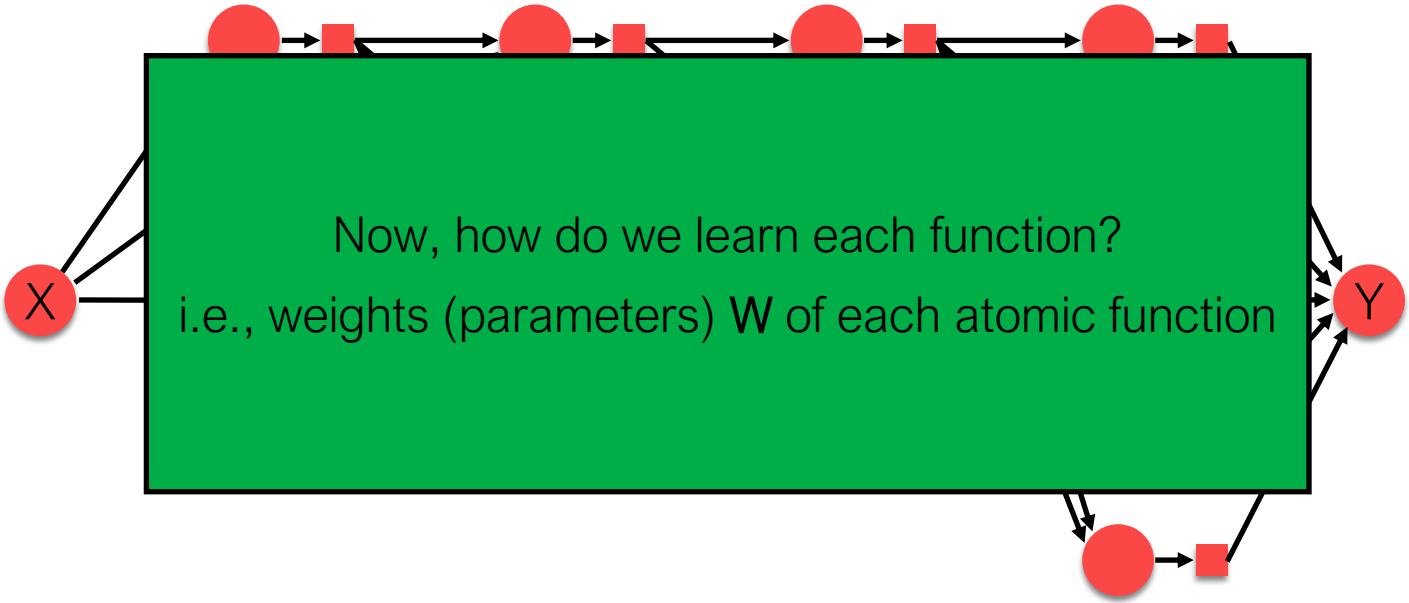
more neurons = more capacity

Larger Model Capacity May Overfit – Need to Regularize them



$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i) + \lambda R(W)$$

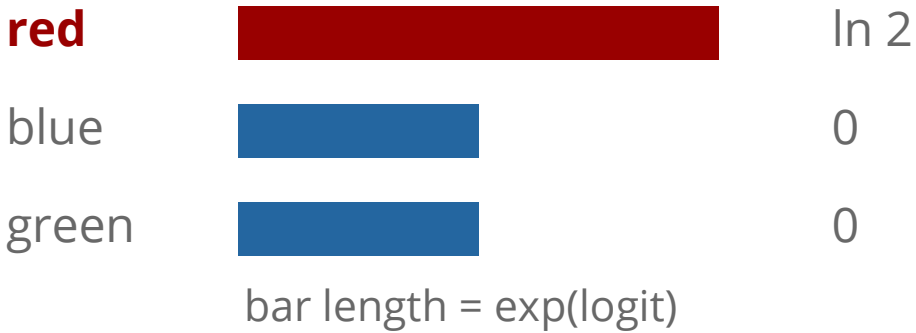
Neural Network Examples



The Next Token Is a Classification Problem

The | light | is | ?

Same context



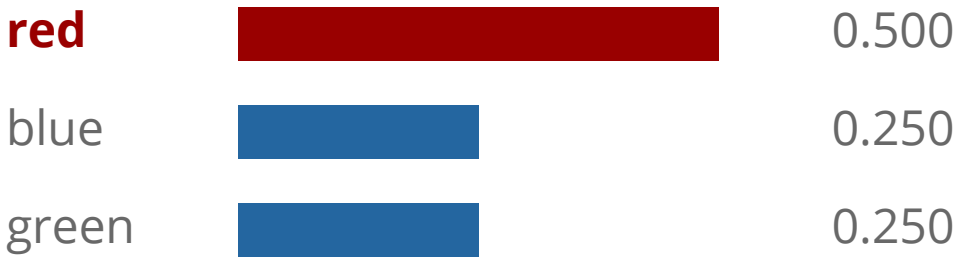
Linear head
$z = Wh + b$

Labels show logits.

Turn Scores Into Probabilities

The	light	is	?

Same context



$\exp(z)$: 2, 1, 1
Sum: 4

$p_i = \exp(z_i) / 4$

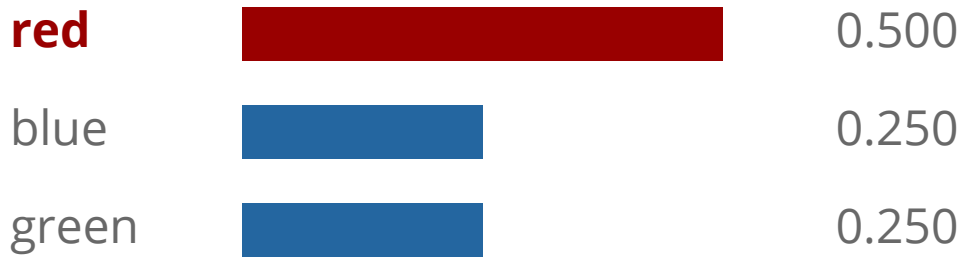
These are probabilities over the toy vocabulary.

USC Viterbi School of Engineering

The Observed Token Supplies the Target

The	light	is	?

Same context



Observed next token
red

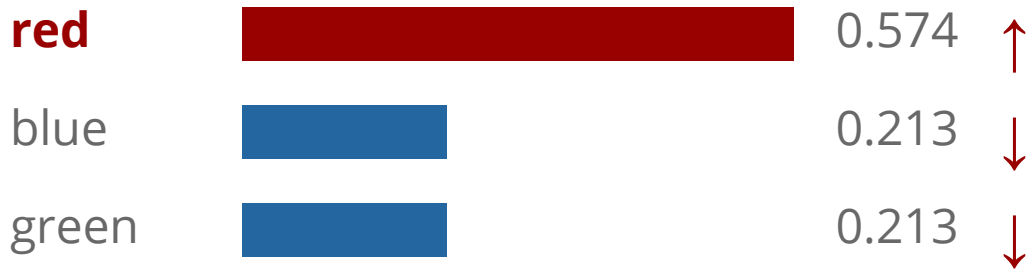
$L = -\ln 0.5 = 0.693$

Predict: which score should go up? Which should go down?

A Gradient Moves the Distribution

The	light	is	?

Same context



$\partial L/\partial z = p - y$

$(-.5, +.25, +.25)$

One step, $\eta = 0.4$:

$z \leftarrow z - 0.4(p - y)$

Loss: $.693 \rightarrow .554$

Direct logit step shown; training updates network parameters.

Plugging in neural networks with loss functions

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$ Nonlinear score function

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$ Loss on predictions

$$R(W) = \sum_k W_k^2$$ Regularization

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda R(W_1) + \lambda R(W_2)$$ Total loss: data loss + regularization

Credit to Stanford CS 231n

Problem: How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$ Nonlinear score function

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$ SVM Loss on predictions

$$R(W) = \sum_k W_k^2$$ Regularization

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda R(W_1) + \lambda R(W_2)$$ Total loss: data loss + regularization

If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

Credit to Stanford CS 231n

(Bad) Idea: Derive $\nabla_W L$ on paper

$$s = f(x; W) = Wx$$

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$= \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda \sum_k W_k^2$$

$$= \frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2$$

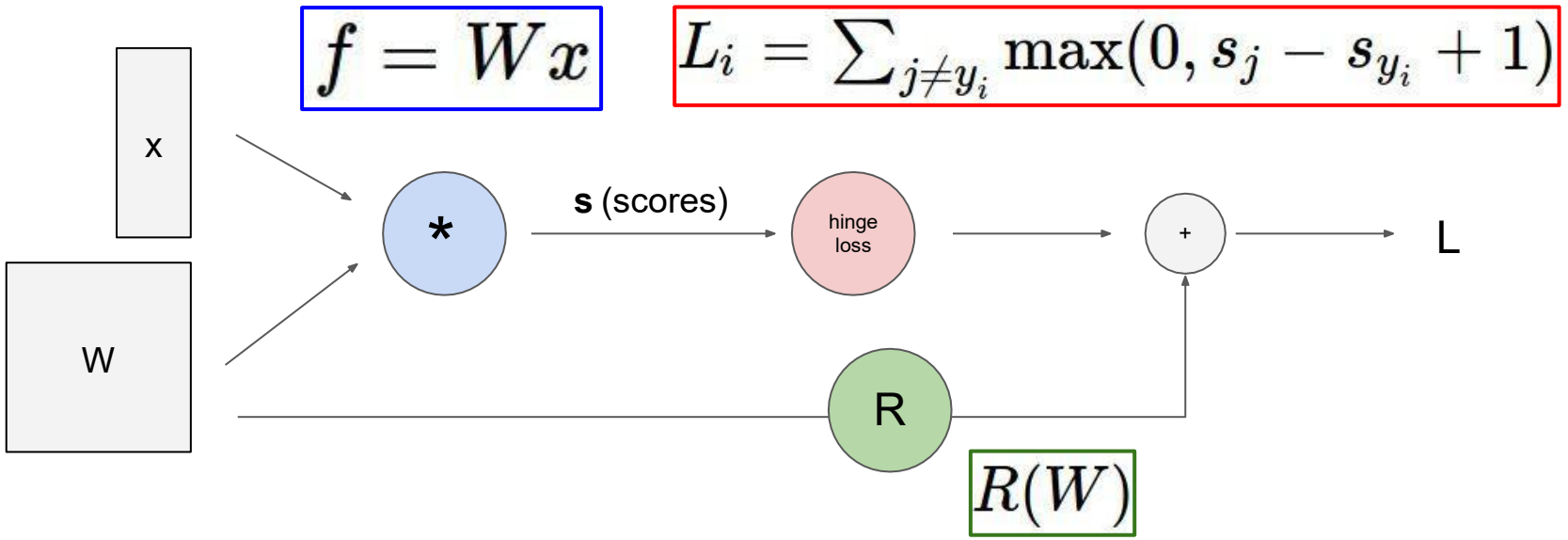
$$\nabla_W L = \nabla_W \left(\frac{1}{N} \sum_{i=1}^{N} \sum_{j \neq y_i} \max(0, W_{j,:} \cdot x + W_{y_i,:} \cdot x + 1) + \lambda \sum_k W_k^2 \right)$$

Problem: Very tedious: Lots of matrix calculus, need lots of paper

Problem: What if we want to change loss? E.g. L2 other than L1? Need to re-derive from scratch =(

Problem: Not feasible for complex models!

Better Idea: Computational graphs + Backpropagation



Credit to Stanford CS 231n

Neural Networks

Back propagation

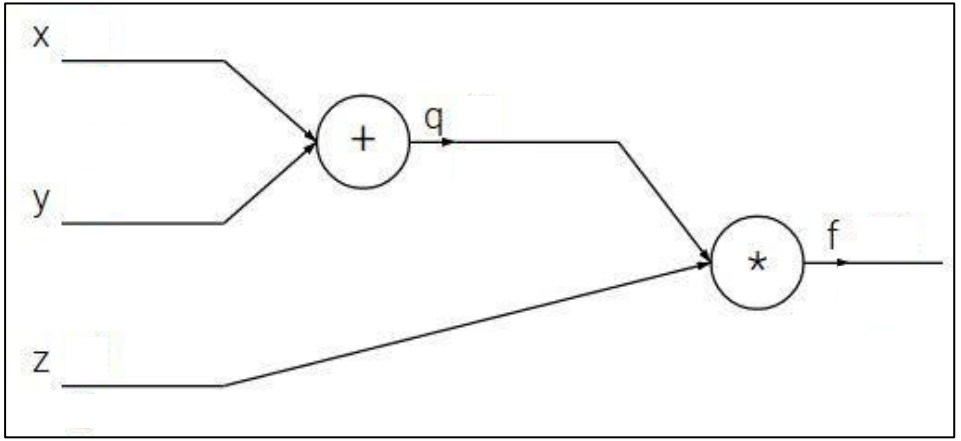
Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

Credit to Stanford CS 231n

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

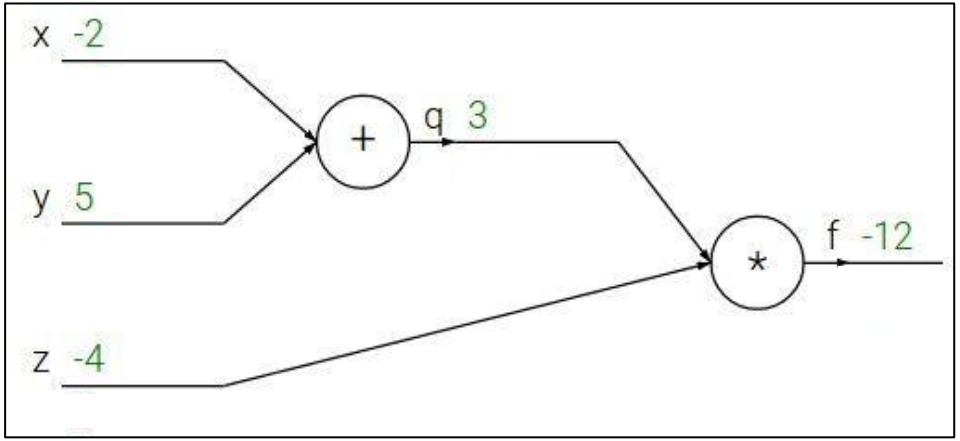


Credit to Stanford CS 231n

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4



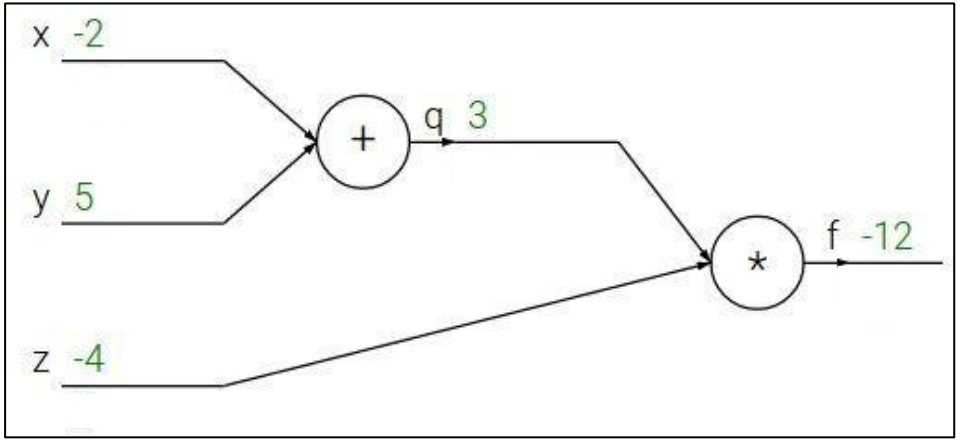
Credit to Stanford CS 231n

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

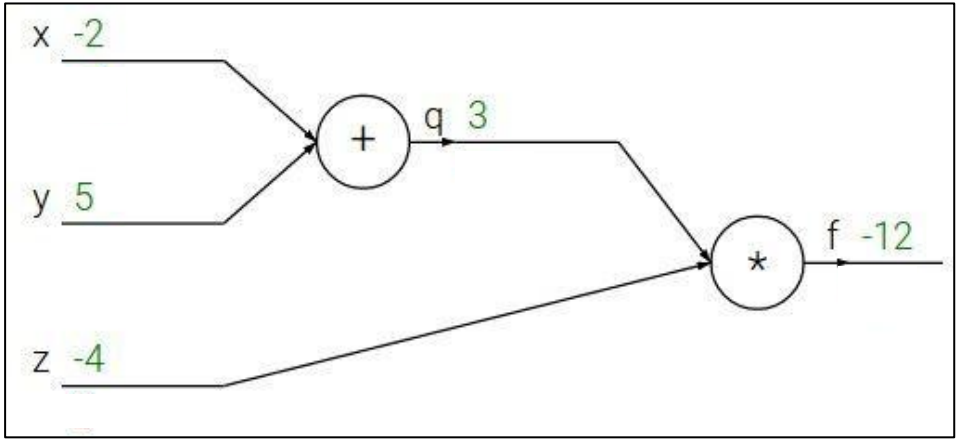


Credit to Stanford CS 231n

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4



$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

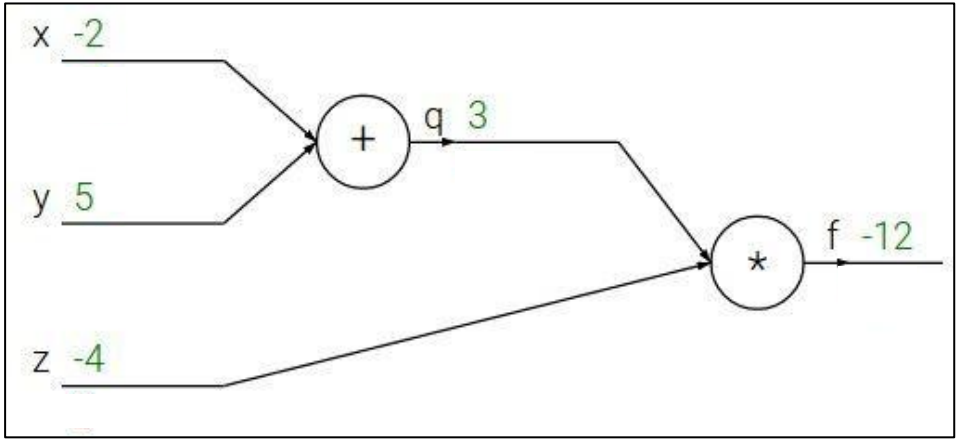
$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Credit to Stanford CS 231n

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4



$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n

Backpropagation: a simple example

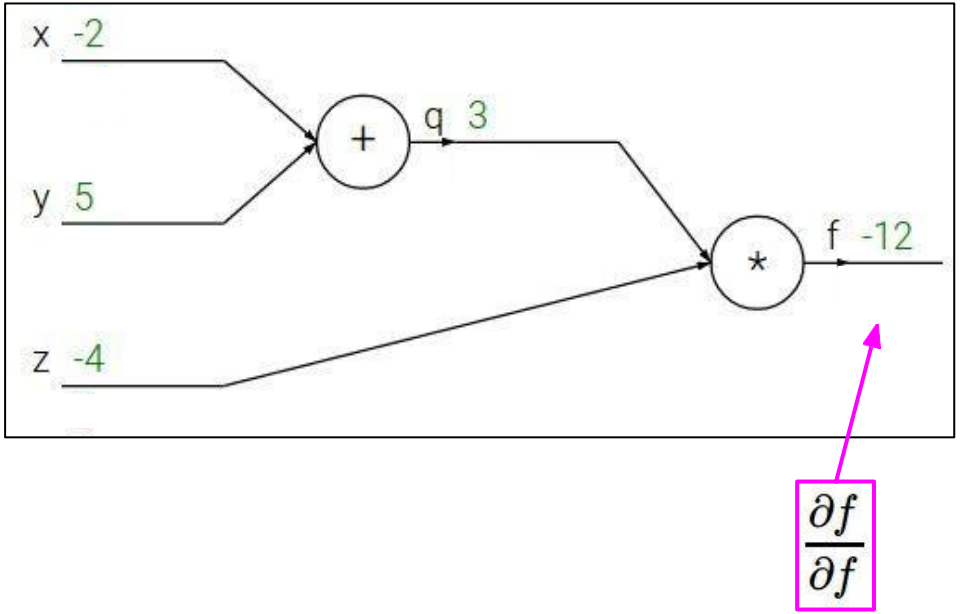
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Credit to Stanford CS 231n

Backpropagation: a simple example

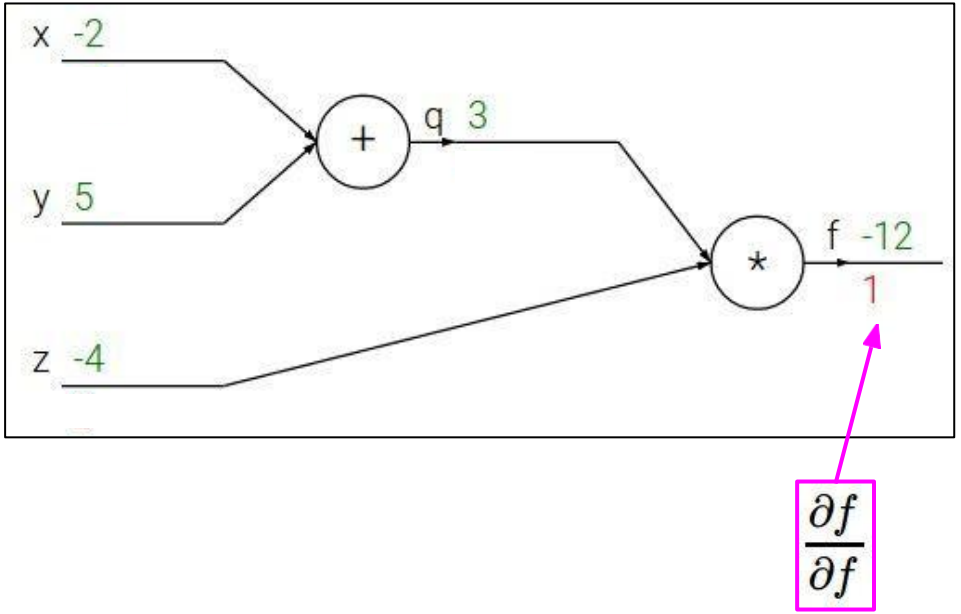
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

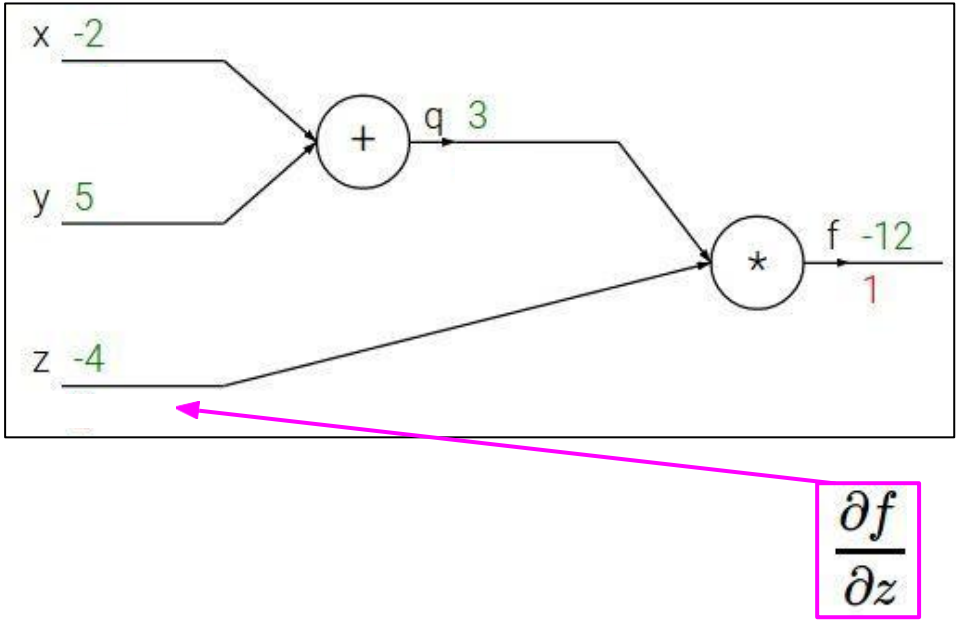


Credit to Stanford CS 231n

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4



$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n

Backpropagation: a simple example

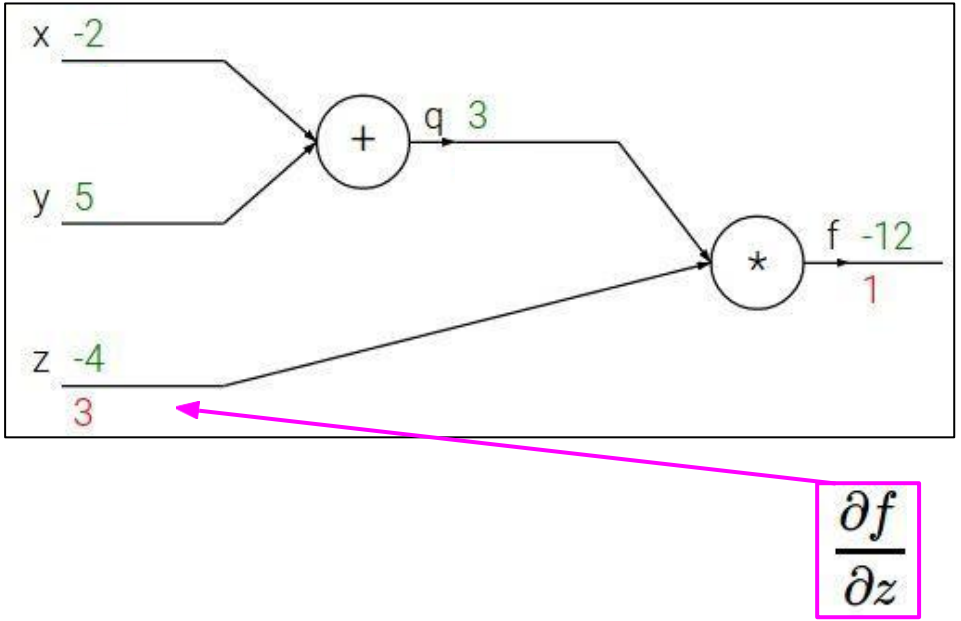
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Credit to Stanford CS 231n

Backpropagation: a simple example

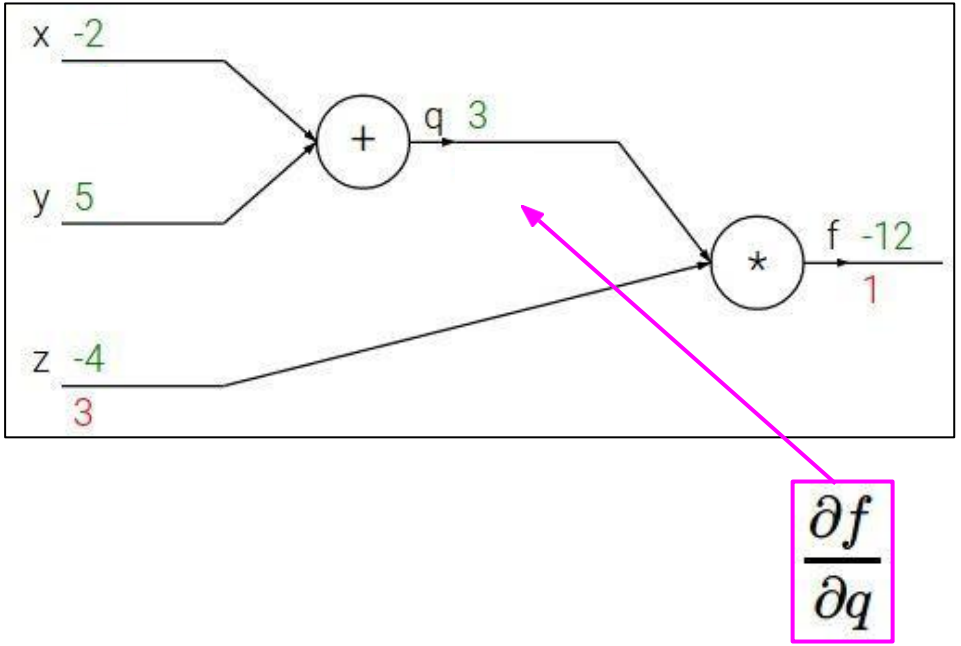
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Credit to Stanford CS 231n

Backpropagation: a simple example

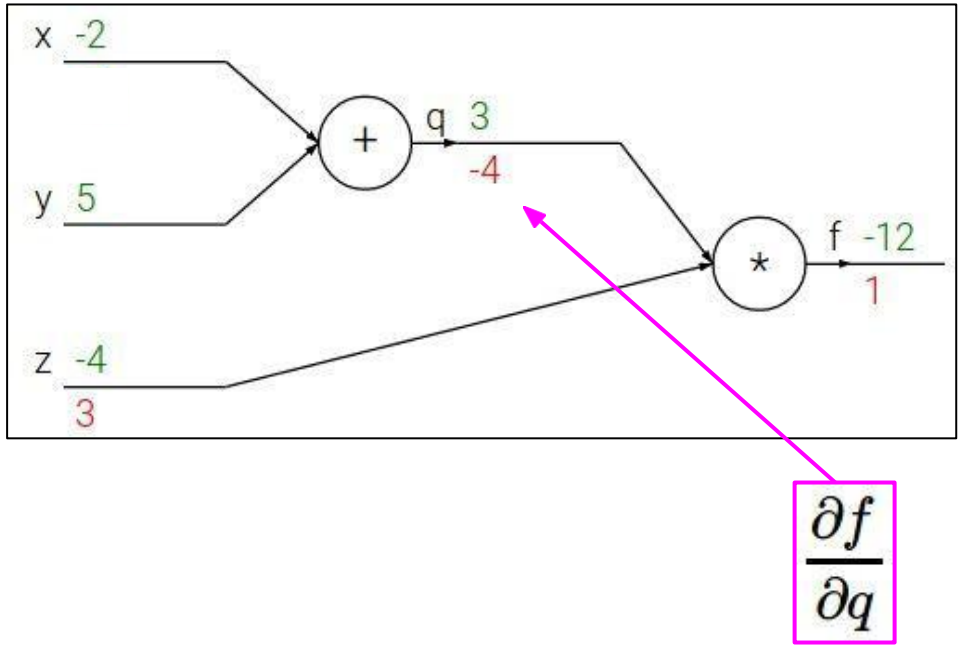
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

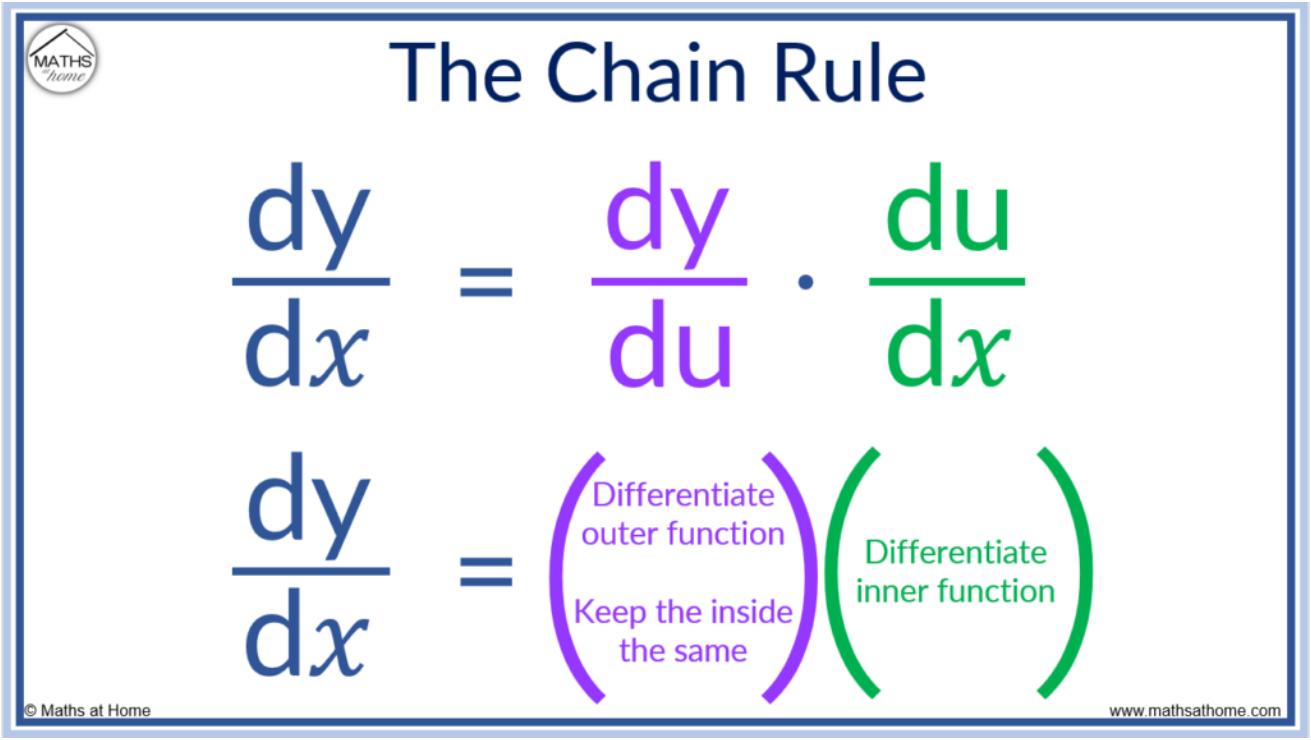
$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Credit to Stanford CS 231n

Chain Rule Revisited



5.1.2 Differentiation Rules

In the following, we briefly state basic differentiation rules, where we denote the derivative of f by f'.

$$\text{Product rule:} \quad (f(x)g(x))' = f'(x)g(x) + f(x)g'(x) \tag{5.29}$$

$$\text{Quotient rule:} \quad \left(\frac{f(x)}{g(x)}\right)' = \frac{f'(x)g(x) - f(x)g'(x)}{(g(x))^2} \tag{5.30}$$

$$\text{Sum rule:} \quad (f(x) + g(x))' = f'(x) + g'(x) \tag{5.31}$$

$$\text{Chain rule:} \quad \big(g(f(x))\big)' = (g \circ f)'(x) = g'(f(x))f'(x) \tag{5.32}$$

Here, $g \circ f$ denotes function composition $x \mapsto f(x) \mapsto g(f(x))$.

Example 5.5 (Chain Rule)
Let us compute the derivative of the function $h(x) = (2x + 1)^4$ using the chain rule. With

$$h(x) = (2x + 1)^4 = g(f(x)), \tag{5.33}$$

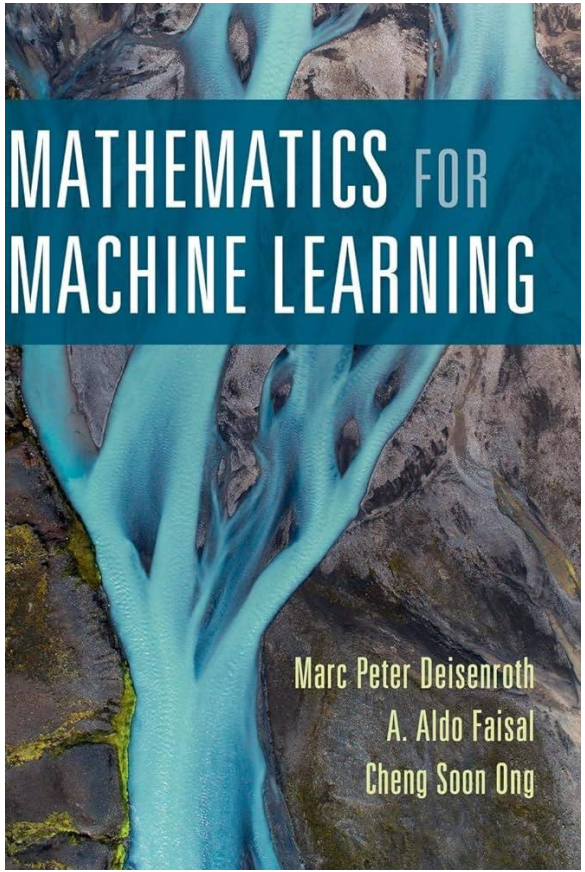
$$f(x) = 2x + 1, \tag{5.34}$$

$$g(f) = f^4, \tag{5.35}$$

we obtain the derivatives of f and g as

$$f'(x) = 2, \tag{5.36}$$

$$g'(f) = 4f^3, \tag{5.37}$$



5.2 Partial Differentiation and Gradients

Differentiation as discussed in Section 5.1 applies to functions f of a scalar variable $x \in \mathbb{R}$. In the following, we consider the general case where the function f depends on one or more variables $\boldsymbol{x} \in \mathbb{R}^n$, e.g., $f(\boldsymbol{x}) = f(x_1, x_2)$. The generalization of the derivative to functions of several variables is the *gradient*.

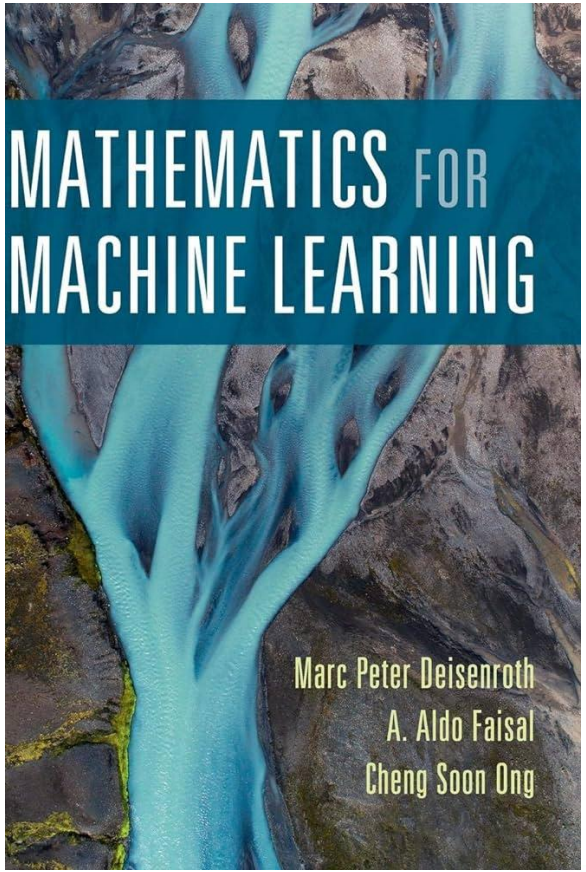
We find the gradient of the function f with respect to $\boldsymbol{x}$ by *varying one variable at a time* and keeping the others constant. The gradient is then the collection of these *partial derivatives*.

Definition 5.5 (Partial Derivative)**.** For a function $f : \mathbb{R}^n \to \mathbb{R}$, $\boldsymbol{x} \mapsto f(\boldsymbol{x})$, $\boldsymbol{x} \in \mathbb{R}^n$ of n variables $x_1, \ldots, x_n$ we define the *partial derivatives* as

$$\begin{aligned} \frac{\partial f}{\partial x_1} &= \lim_{h \to 0} \frac{f(x_1 + h, x_2, \ldots, x_n) - f(\boldsymbol{x})}{h} \\ &\vdots \\ \frac{\partial f}{\partial x_n} &= \lim_{h \to 0} \frac{f(x_1, \ldots, x_{n-1}, x_n + h) - f(\boldsymbol{x})}{h} \end{aligned} \tag{5.39}$$

and collect them in the row vector

$$\nabla_{\boldsymbol{x}} f = \operatorname{grad} f = \frac{\mathrm{d}f}{\mathrm{d}\boldsymbol{x}} = \left[\frac{\partial f(\boldsymbol{x})}{\partial x_1} \quad \frac{\partial f(\boldsymbol{x})}{\partial x_2} \quad \cdots \quad \frac{\partial f(\boldsymbol{x})}{\partial x_n} \right] \in \mathbb{R}^{1 \times n}, \tag{5.40}$$



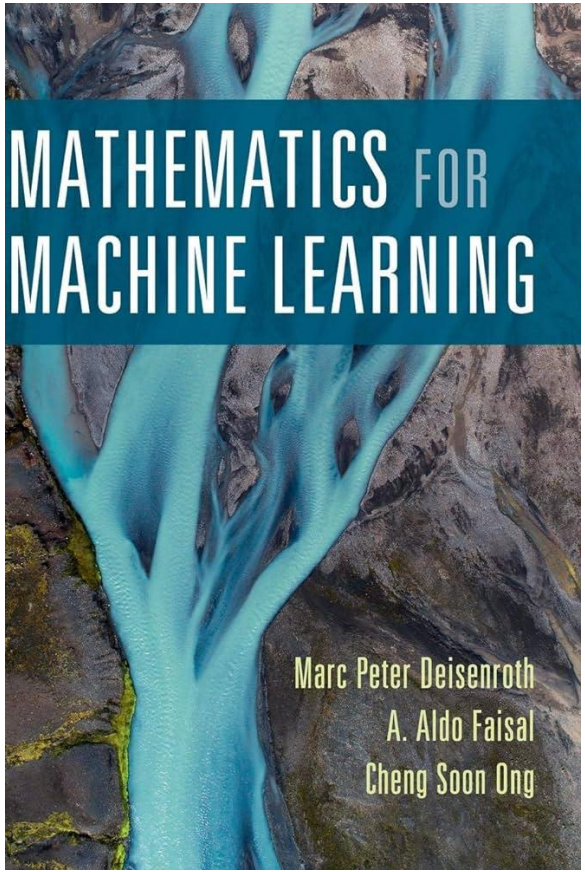
Example 5.7 (Gradient)
For $f(x_1, x_2) = x_1^2 x_2 + x_1 x_2^3 \in \mathbb{R}$, the partial derivatives (i.e., the derivatives of f with respect to x_1 and x_2) are

$$\frac{\partial f(x_1, x_2)}{\partial x_1} = 2x_1 x_2 + x_2^3 \tag{5.43}$$

$$\frac{\partial f(x_1, x_2)}{\partial x_2} = x_1^2 + 3x_1 x_2^2 \tag{5.44}$$

and the gradient is then

$$\frac{\mathrm{d}f}{\mathrm{d}\boldsymbol{x}} = \begin{bmatrix} \frac{\partial f(x_1, x_2)}{\partial x_1} & \frac{\partial f(x_1, x_2)}{\partial x_2} \end{bmatrix} = \begin{bmatrix} 2x_1 x_2 + x_2^3 & x_1^2 + 3x_1 x_2^2 \end{bmatrix} \in \mathbb{R}^{1\times 2} . \tag{5.45}$$



5.2.2 Chain Rule

Consider a function $f : \mathbb{R}^2 \to \mathbb{R}$ of two variables x_1, x_2. Furthermore, $x_1(t)$ and $x_2(t)$ are themselves functions of t. To compute the gradient of f with respect to t, we need to apply the chain rule (5.48) for multivariate functions as

$$\frac{\mathrm{d}f}{\mathrm{d}t} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial x_1(t)}{\partial t} \\ \frac{\partial x_2(t)}{\partial t} \end{bmatrix} = \frac{\partial f}{\partial x_1}\frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2}\frac{\partial x_2}{\partial t}, \tag{5.49}$$

where d denotes the gradient and ∂ partial derivatives.

Example 5.8

Consider $f(x_1, x_2) = x_1^2 + 2x_2$, where $x_1 = \sin t$ and $x_2 = \cos t$, then

$$\frac{\mathrm{d}f}{\mathrm{d}t} = \frac{\partial f}{\partial x_1}\frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2}\frac{\partial x_2}{\partial t} \tag{5.50a}$$

$$= 2\sin t \frac{\partial \sin t}{\partial t} + 2\frac{\partial \cos t}{\partial t} \tag{5.50b}$$

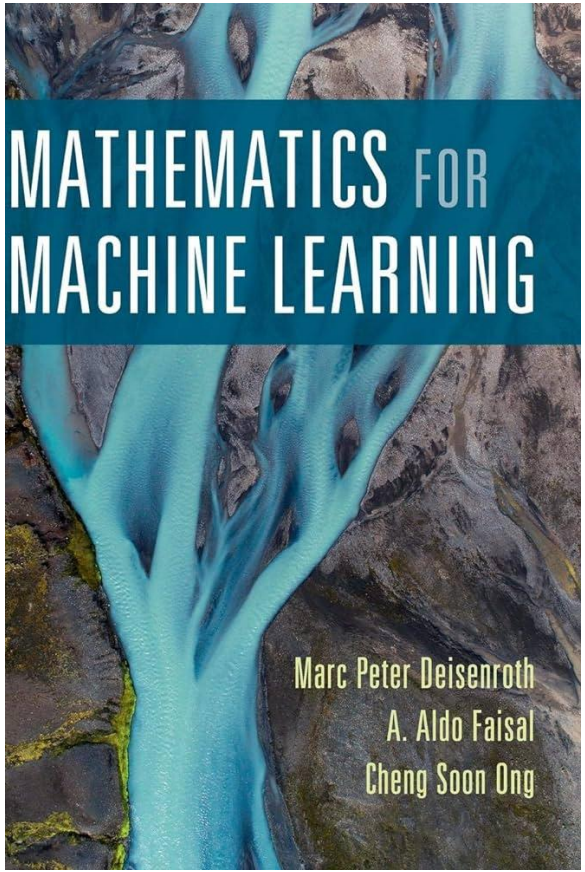
$$= 2\sin t \cos t - 2\sin t = 2\sin t(\cos t - 1) \tag{5.50c}$$

is the corresponding derivative of f with respect to t.

If $f(x_1, x_2)$ is a function of x_1 and x_2, where $x_1(s, t)$ and $x_2(s, t)$ are themselves functions of two variables s and t, the chain rule yields the partial derivatives

$$\frac{\partial f}{\partial s} = \frac{\partial f}{\partial x_1}\frac{\partial x_1}{\partial s} + \frac{\partial f}{\partial x_2}\frac{\partial x_2}{\partial s}, \tag{5.51}$$

$$\frac{\partial f}{\partial t} = \frac{\partial f}{\partial x_1}\frac{\partial x_1}{\partial t} + \frac{\partial f}{\partial x_2}\frac{\partial x_2}{\partial t}, \tag{5.52}$$



Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

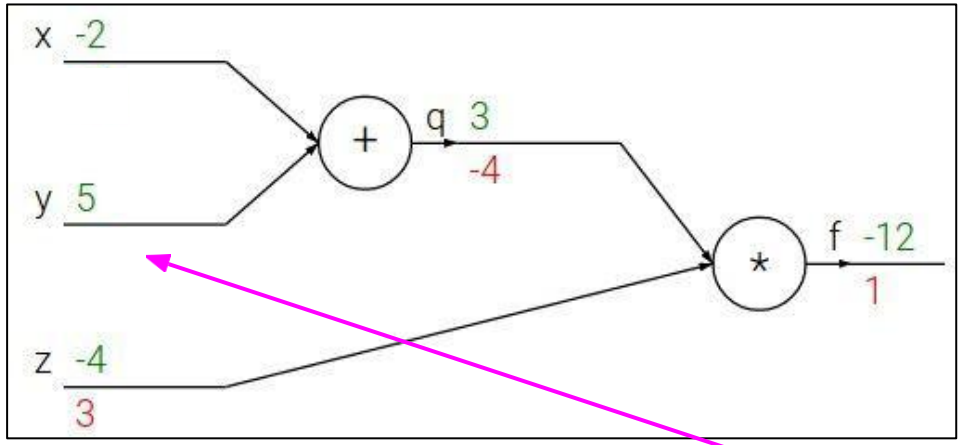
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

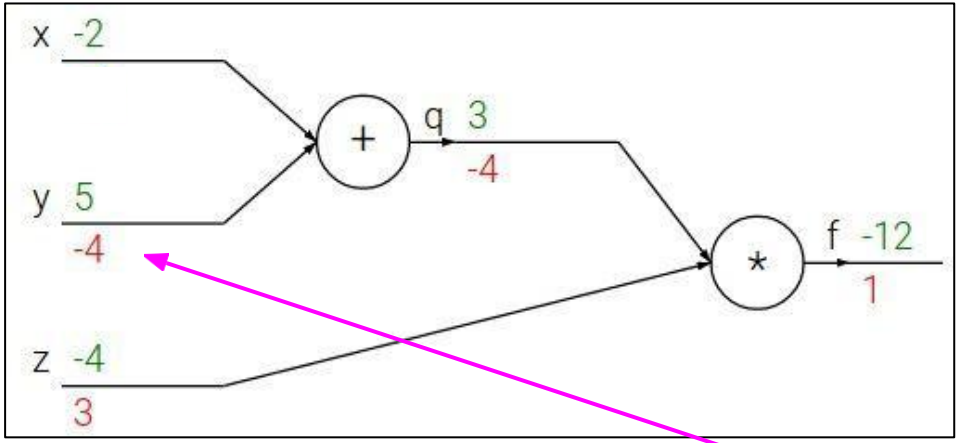
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial y}$$

Upstream gradient

Local gradient

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

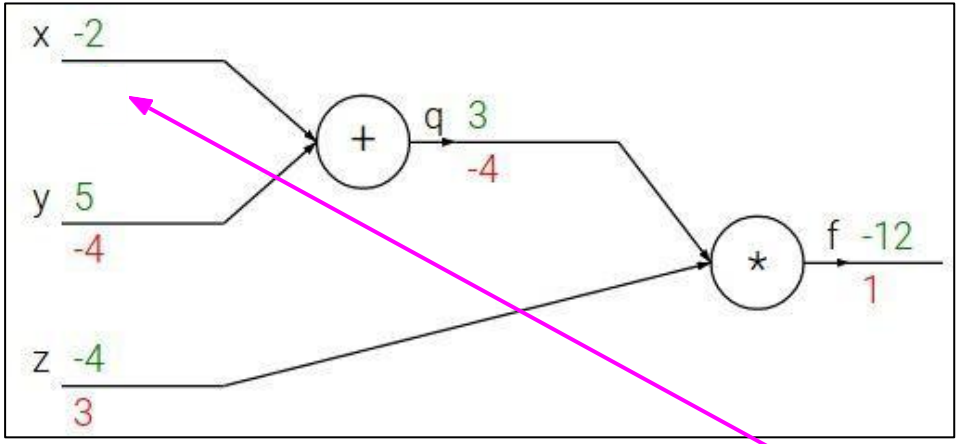
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n



Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

Local gradient

USC Viterbi School of Engineering

Backpropagation: a simple example

$$f(x, y, z) = (x + y)z$$

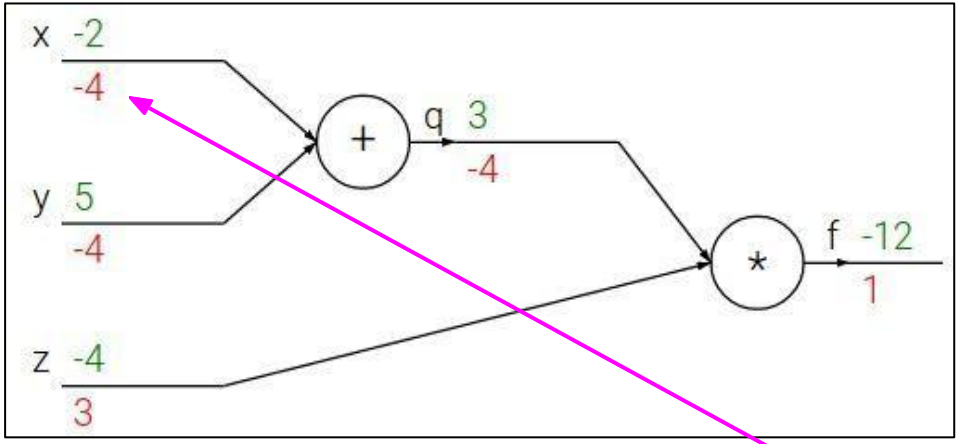
e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$

Credit to Stanford CS 231n

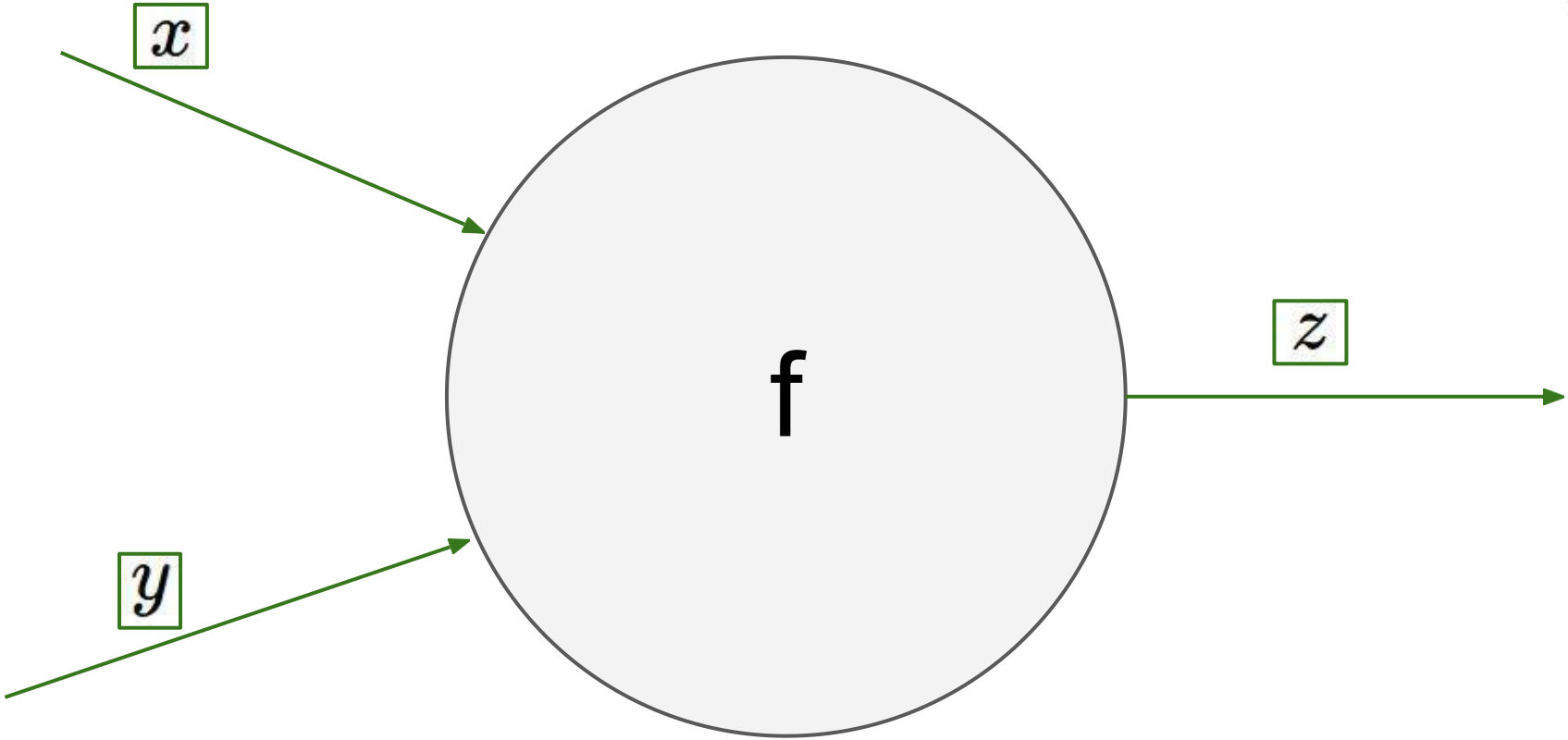


Chain rule:

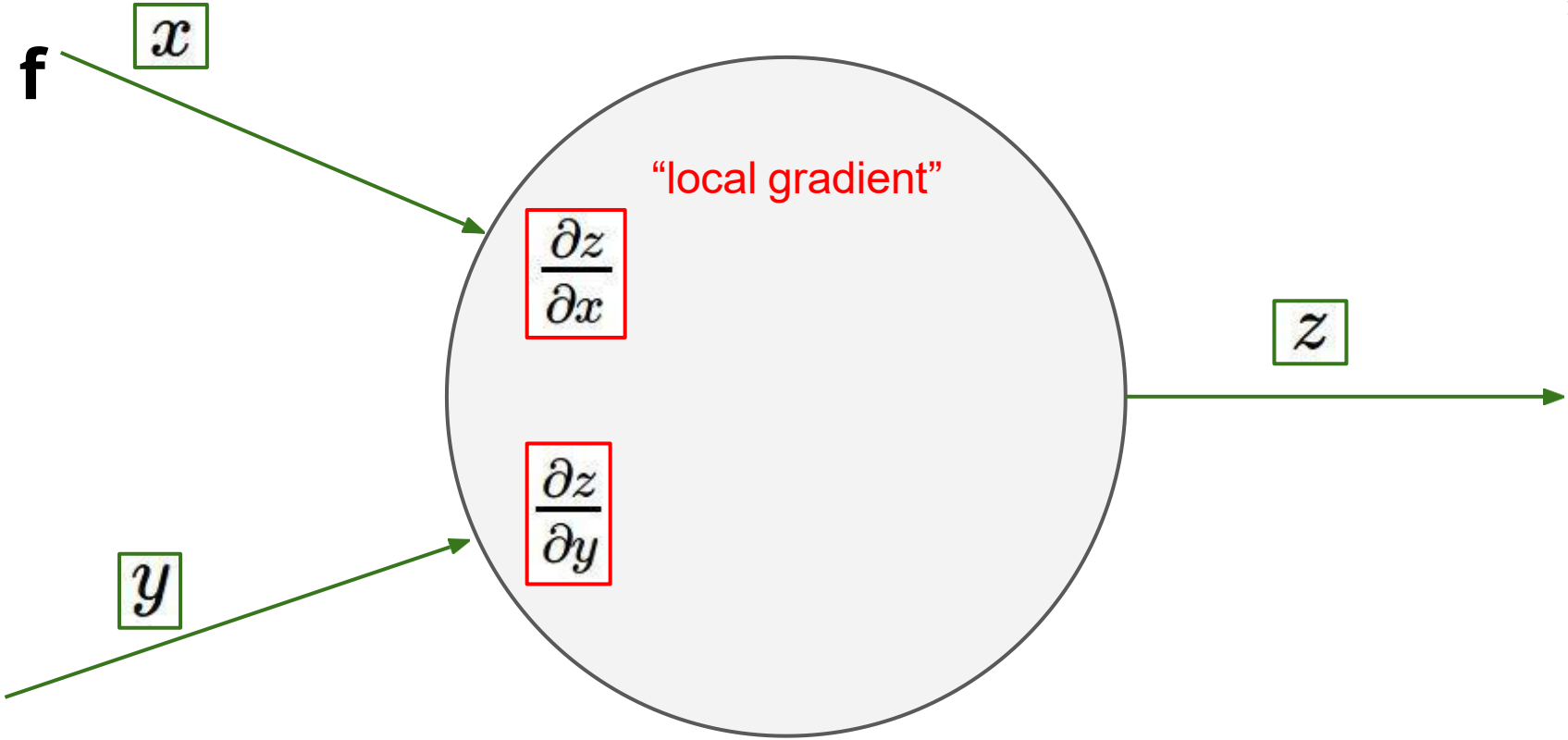
$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

Upstream gradient

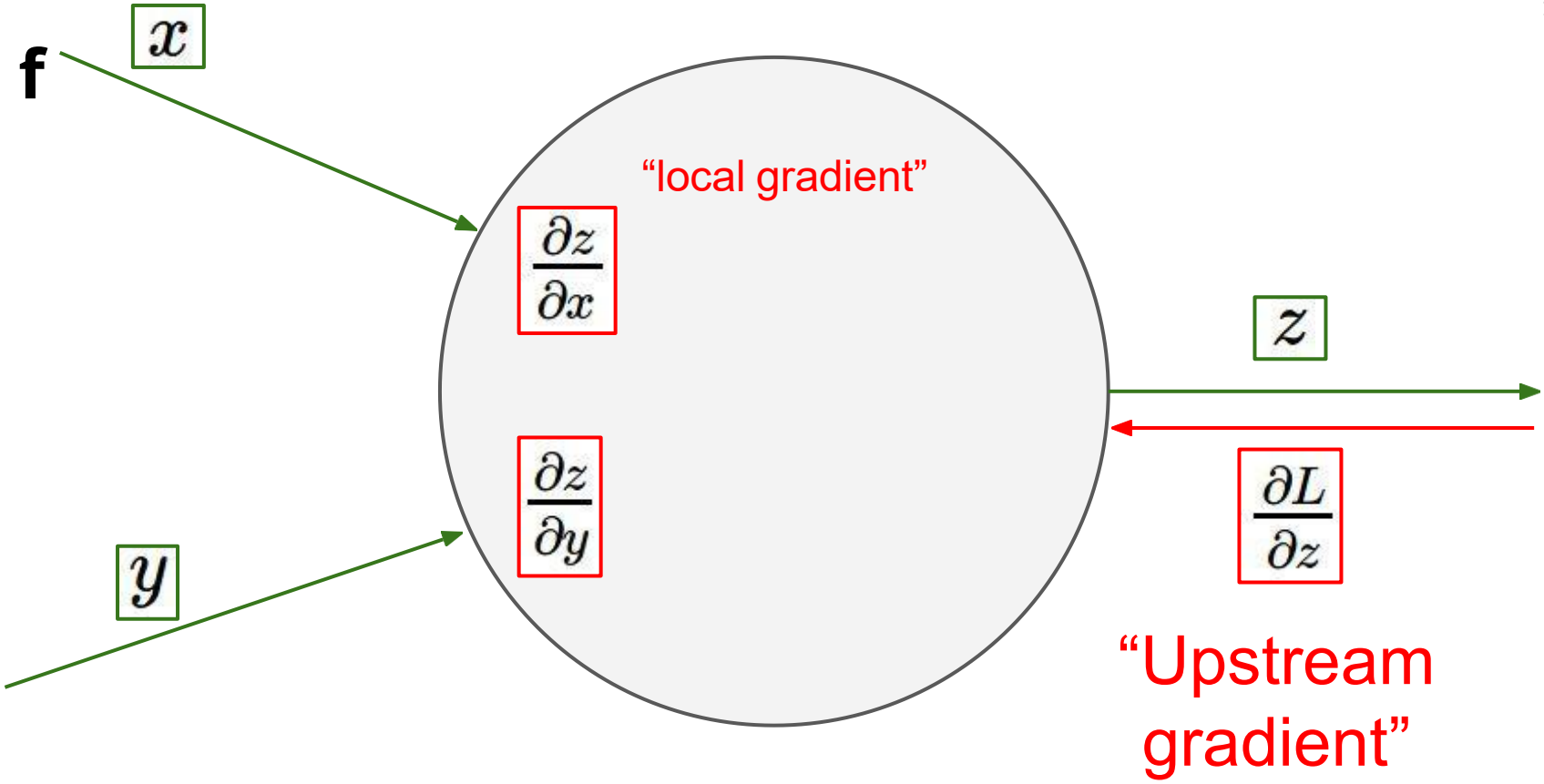
Local gradient



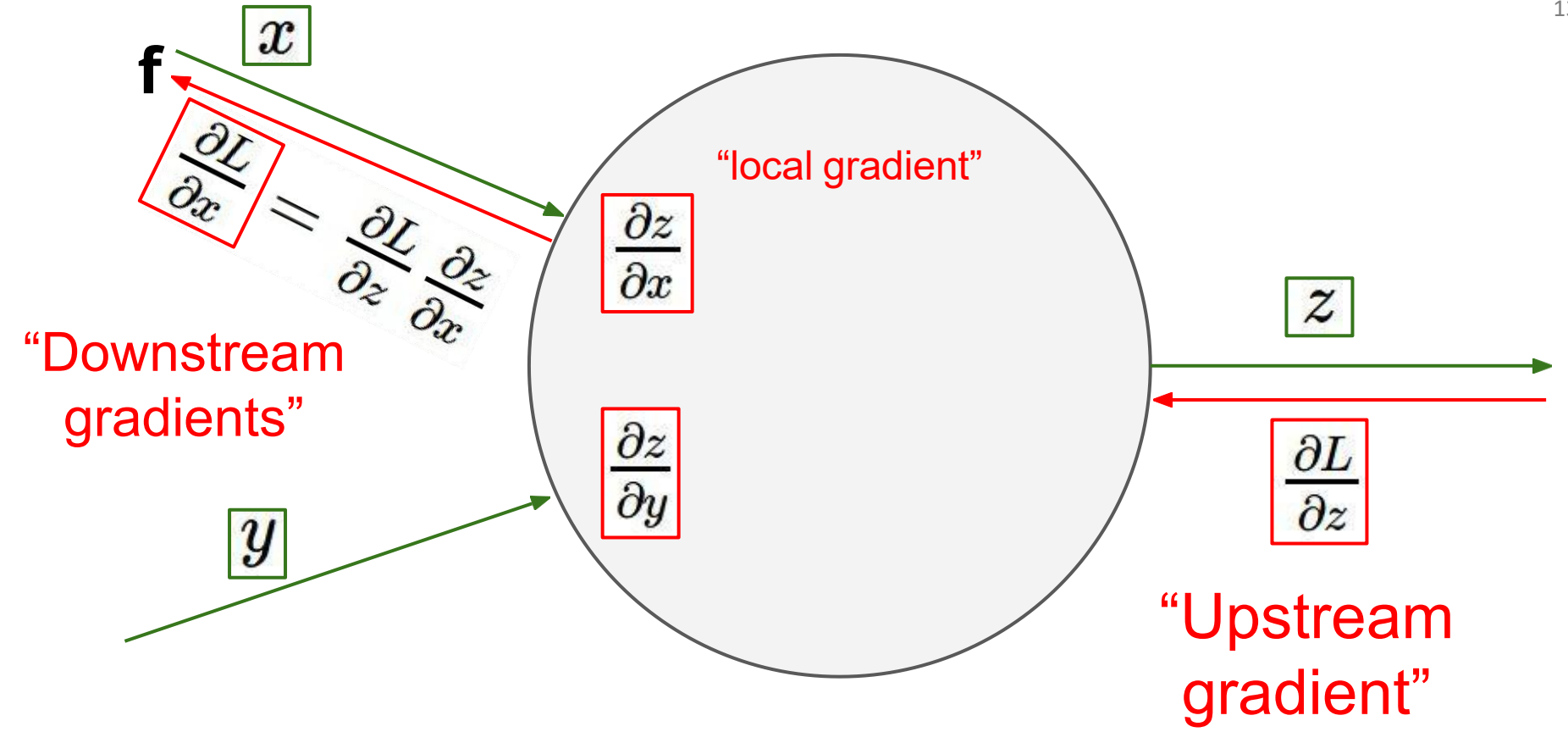
Credit to Stanford CS 231n



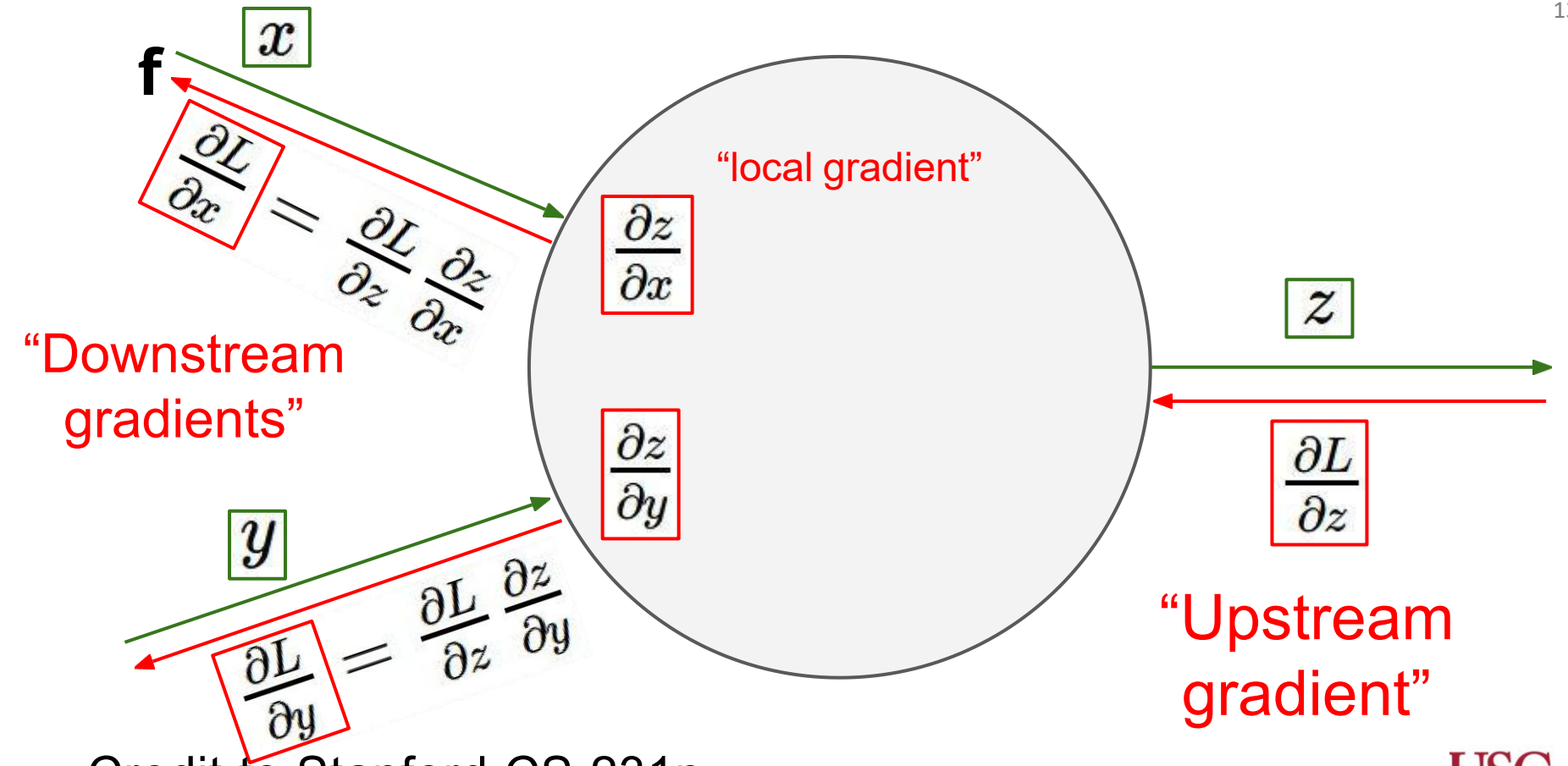
Credit to Stanford CS 231n



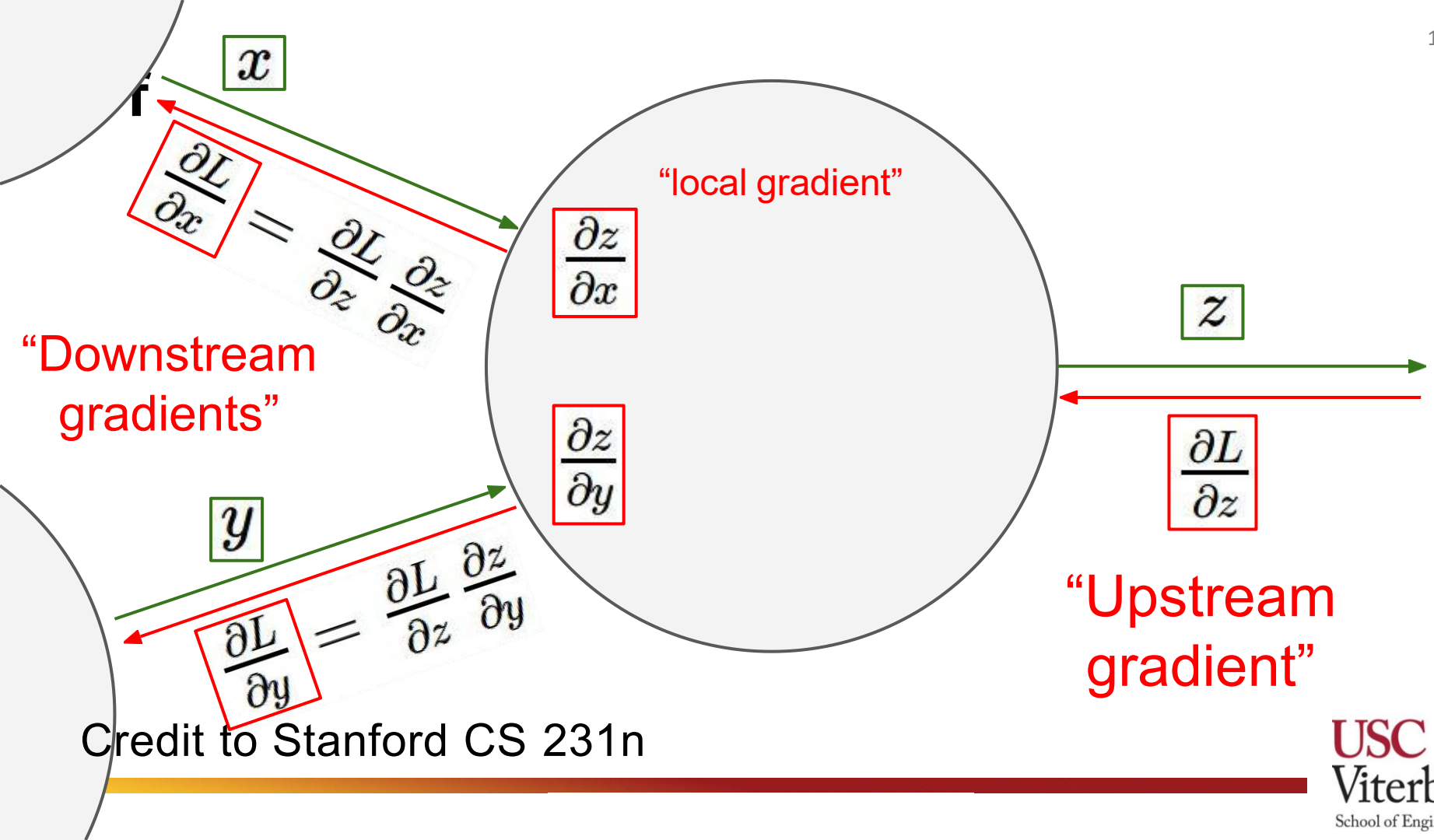
Credit to Stanford CS 231n



Credit to Stanford CS 231n



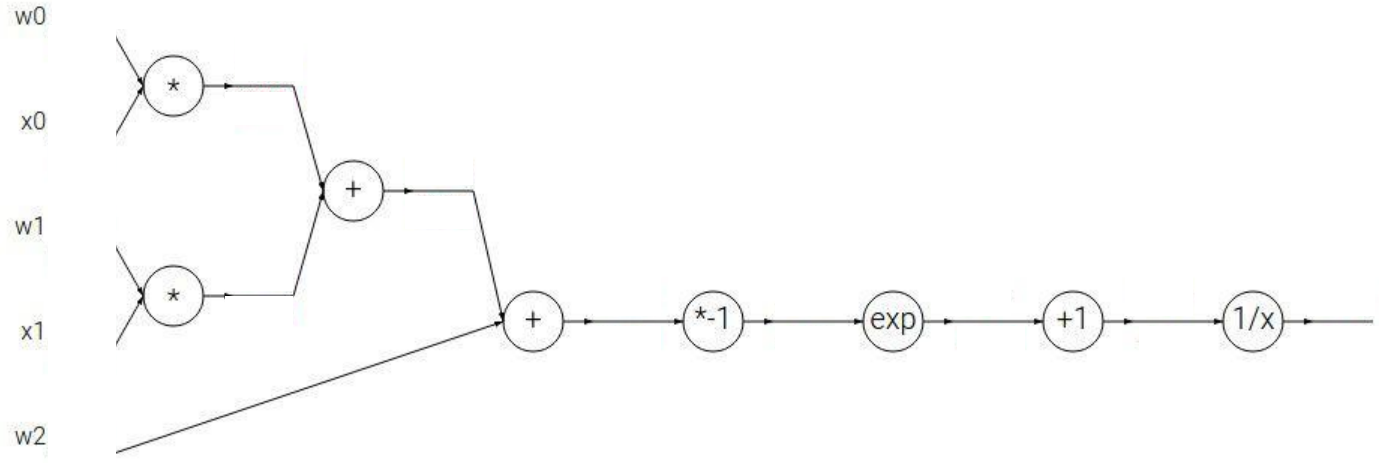
Credit to Stanford CS 231n



Credit to Stanford CS 231n

Another example:

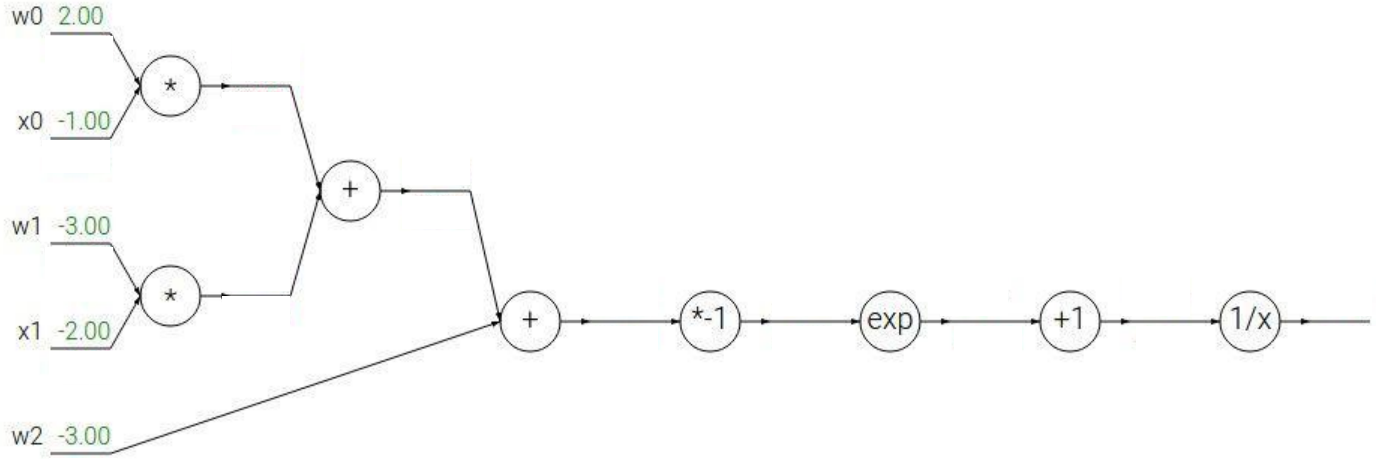
$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



Credit to Stanford CS 231n

Another example:

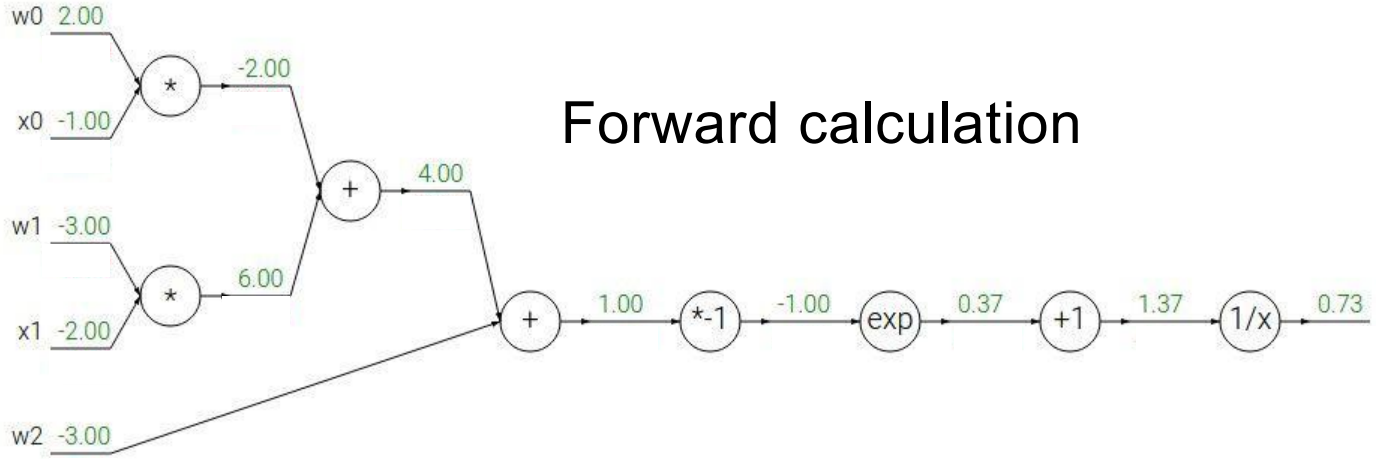
$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

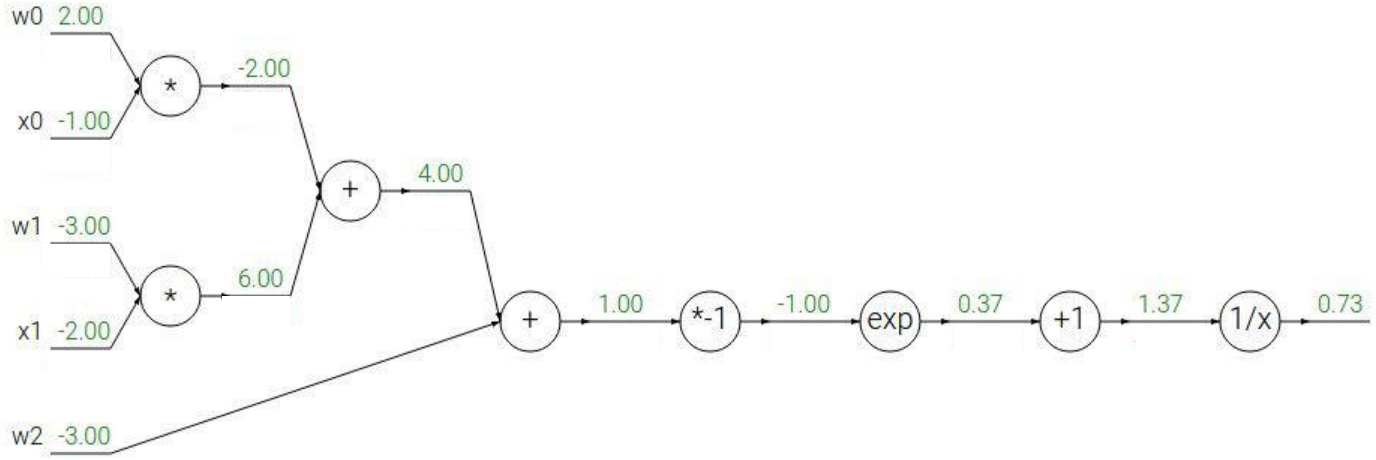


Forward calculation

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

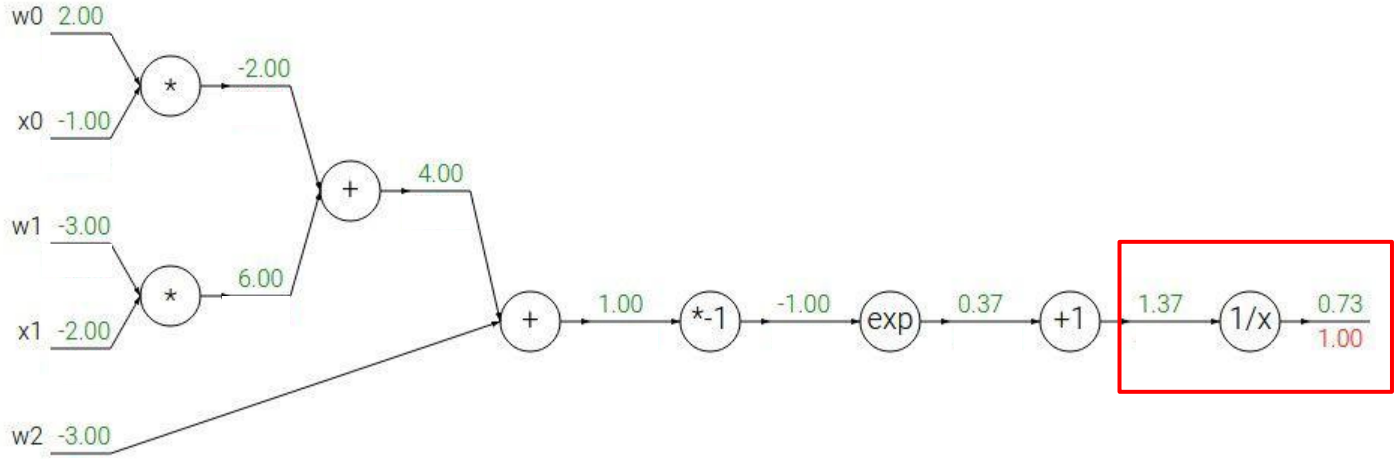
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

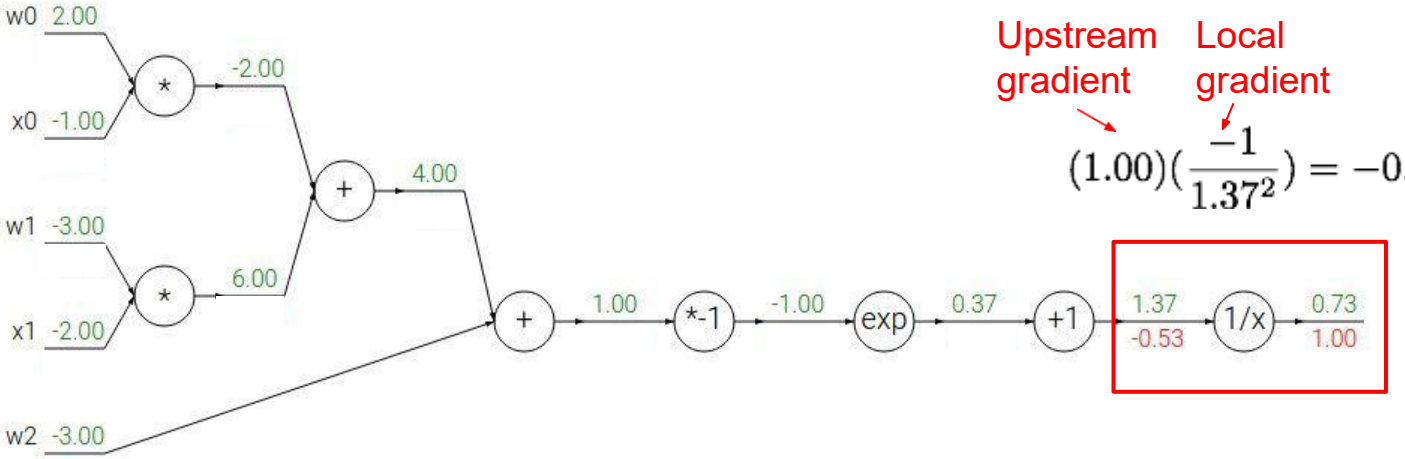
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

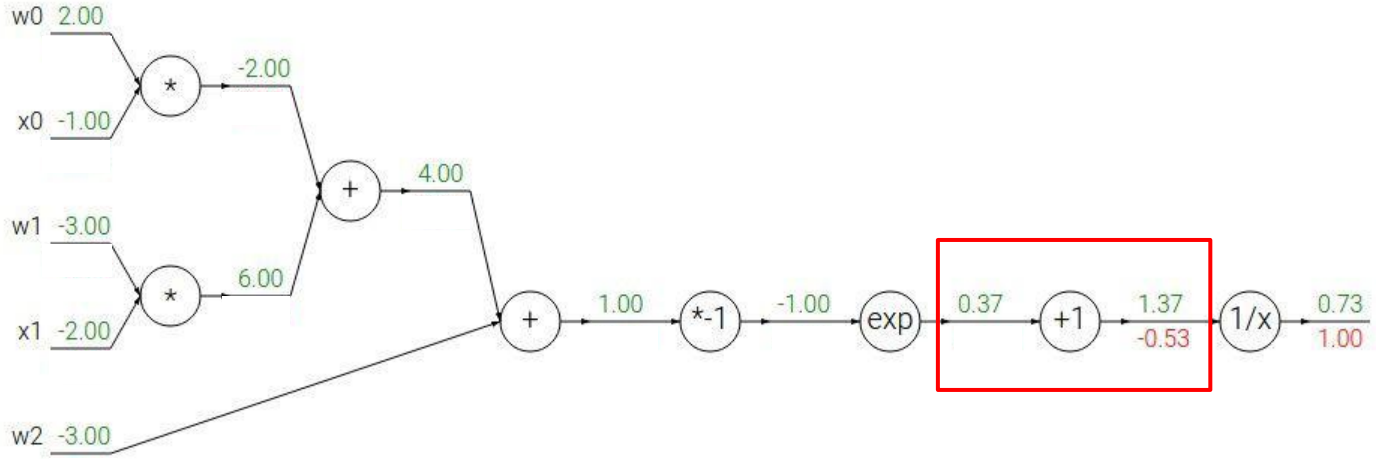
$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

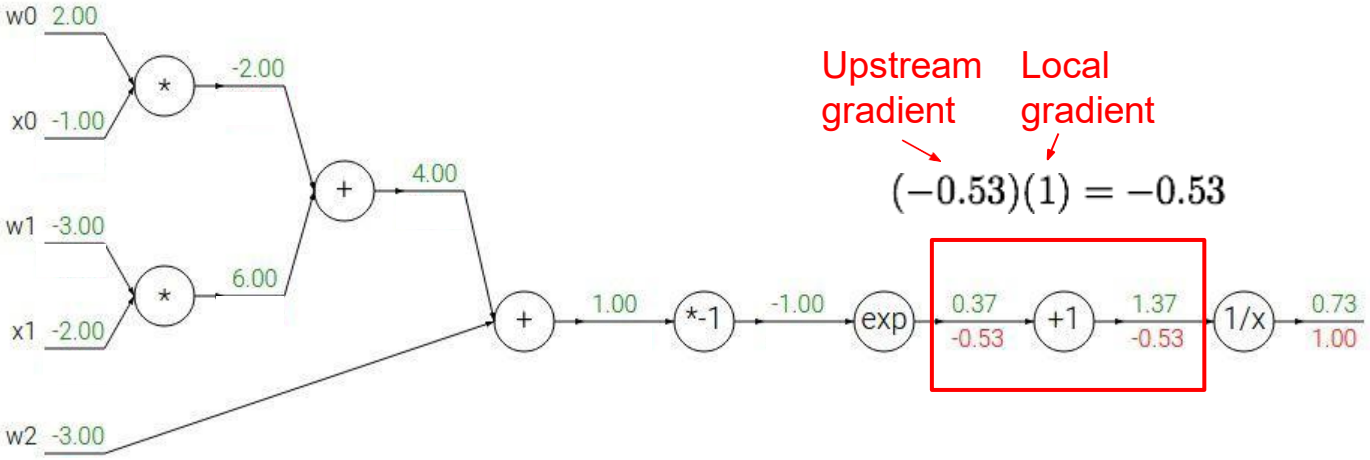
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

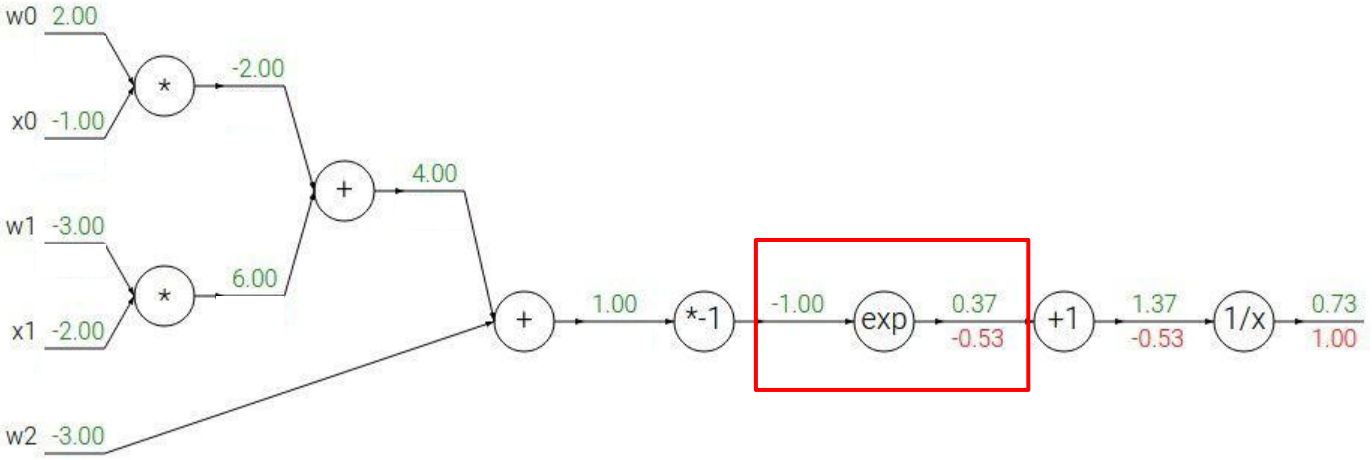
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

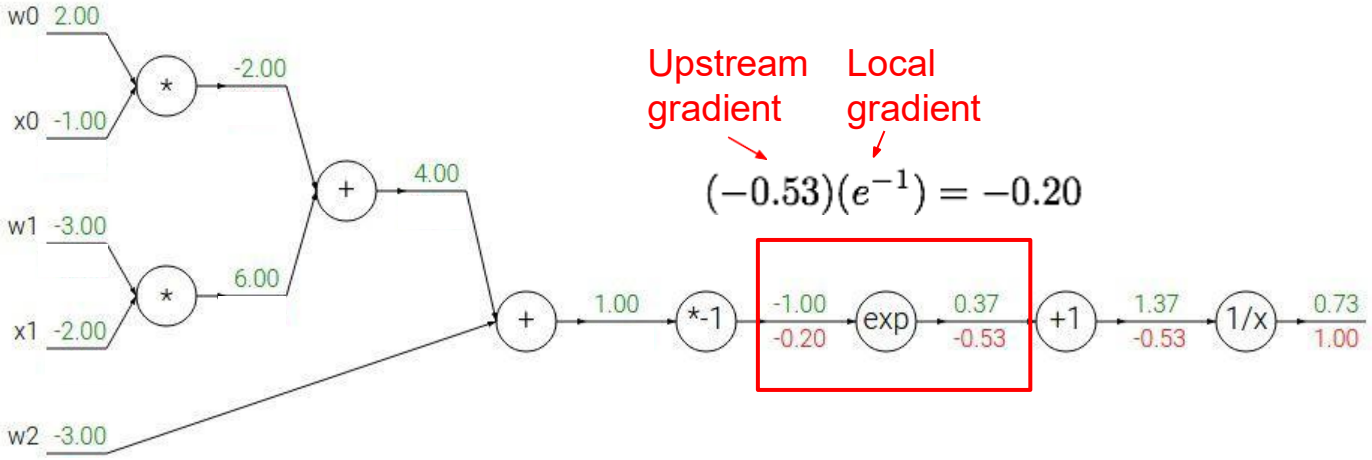
$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

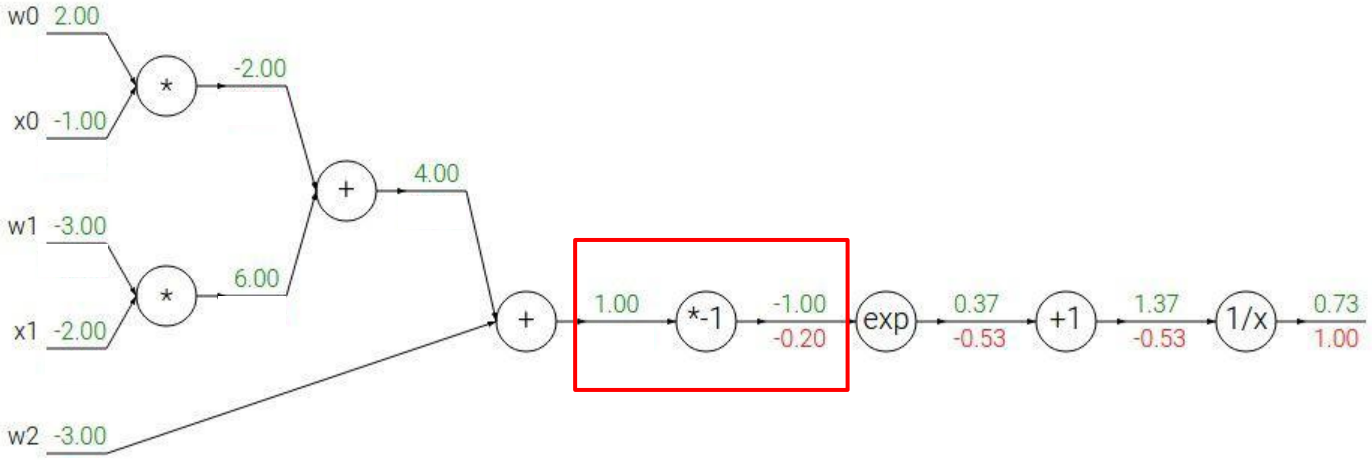
$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example: $f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

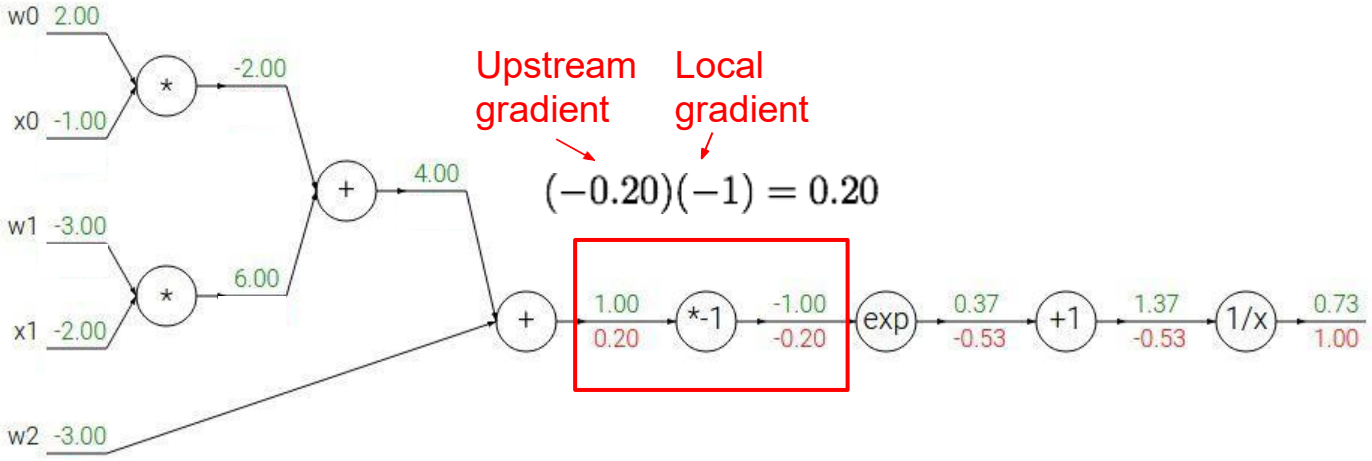
$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$$

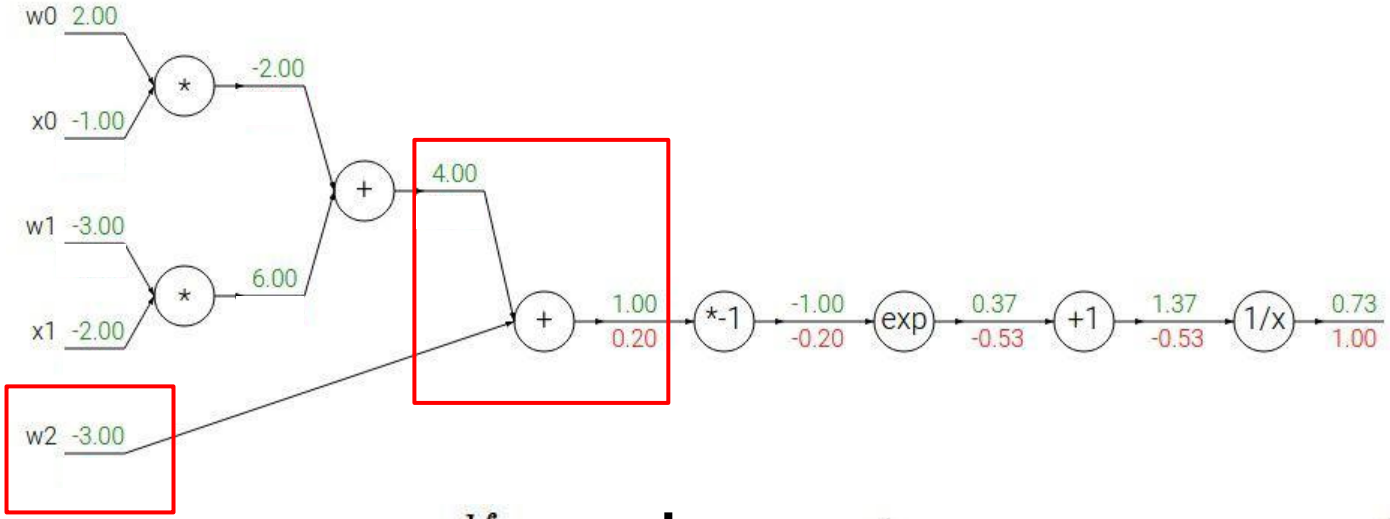
$$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example: $f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$



$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

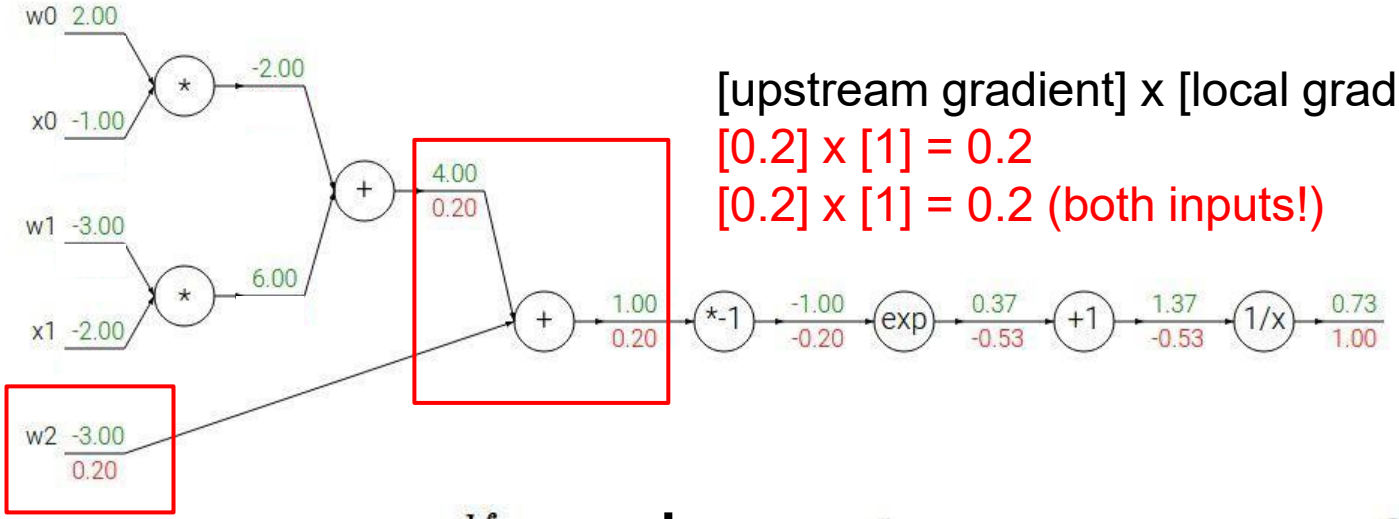
$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[upstream gradient] x [local gradient]

[0.2] x [1] = 0.2

[0.2] x [1] = 0.2 (both inputs!)

$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$

$f_a(x) = ax \rightarrow \frac{df}{dx} = a$

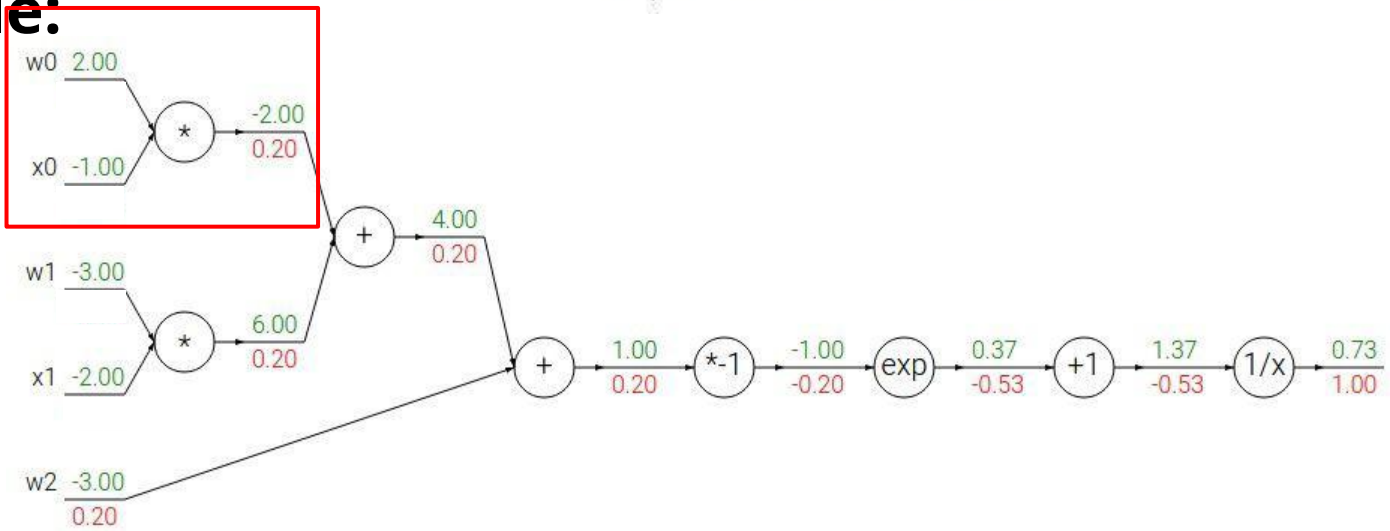
$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$

$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

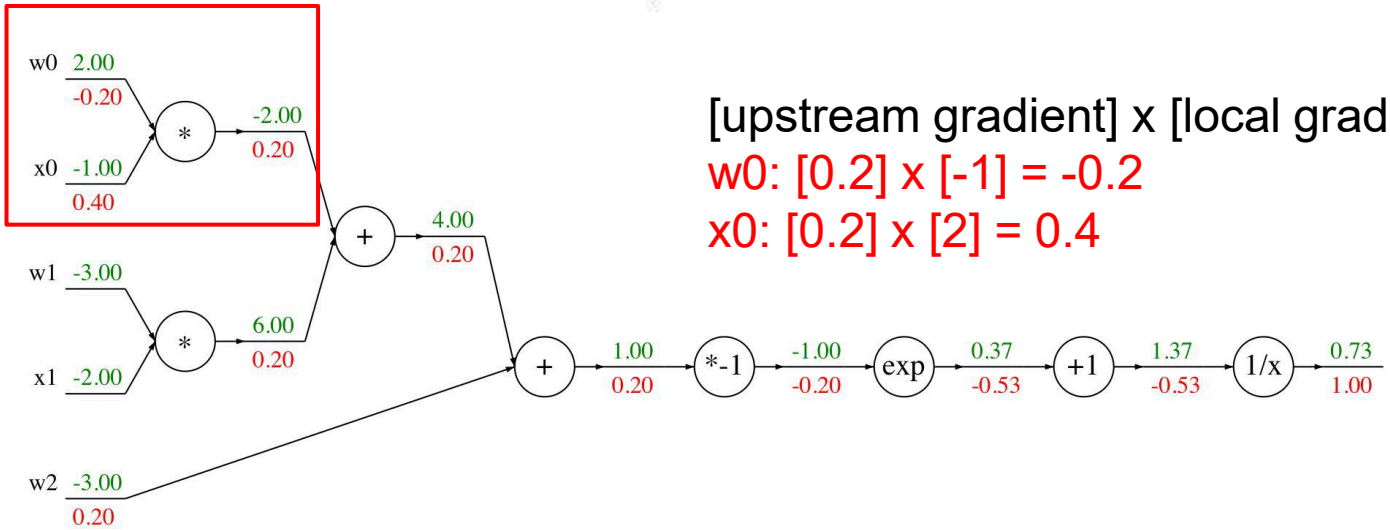
$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$



[upstream gradient] x [local gradient]

w0: [0.2] x [-1] = -0.2

x0: [0.2] x [2] = 0.4

$$f(x) = e^x \rightarrow \frac{df}{dx} = e^x$$

$$f_a(x) = ax \rightarrow \frac{df}{dx} = a$$

$$f(x) = \frac{1}{x} \rightarrow \frac{df}{dx} = -1/x^2$$

$$f_c(x) = c + x \rightarrow \frac{df}{dx} = 1$$

Credit to Stanford CS 231n

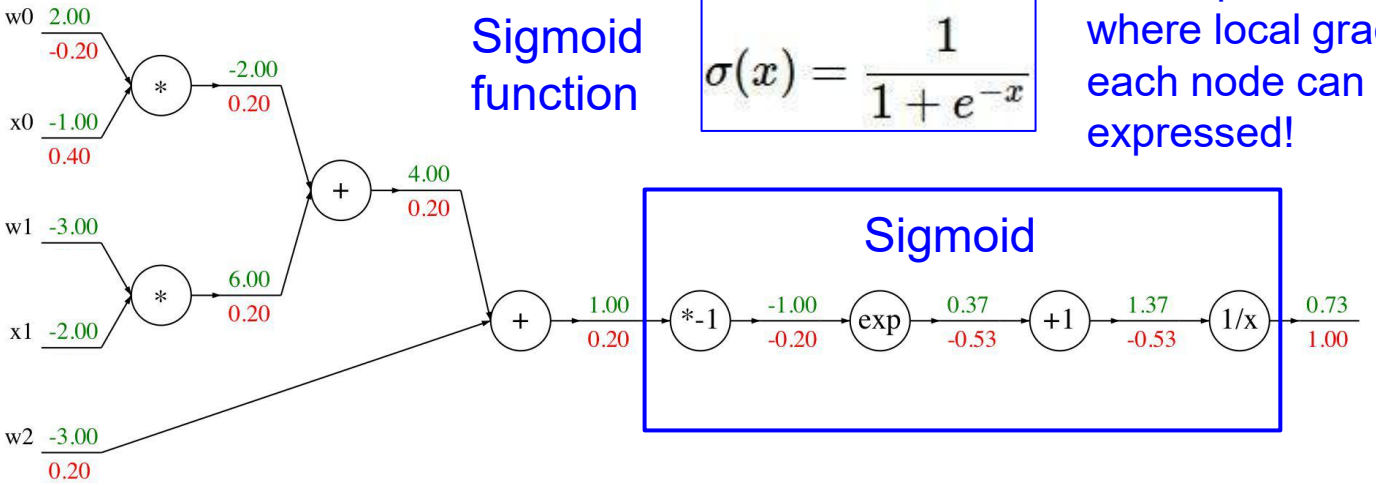
Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!

Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

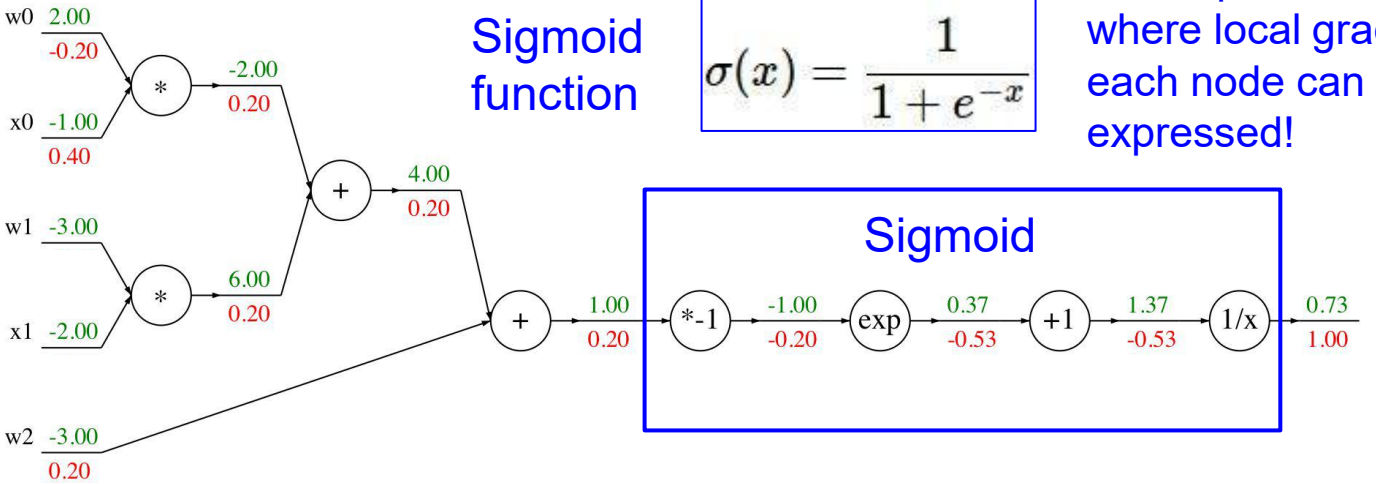


Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Sigmoid local gradient:

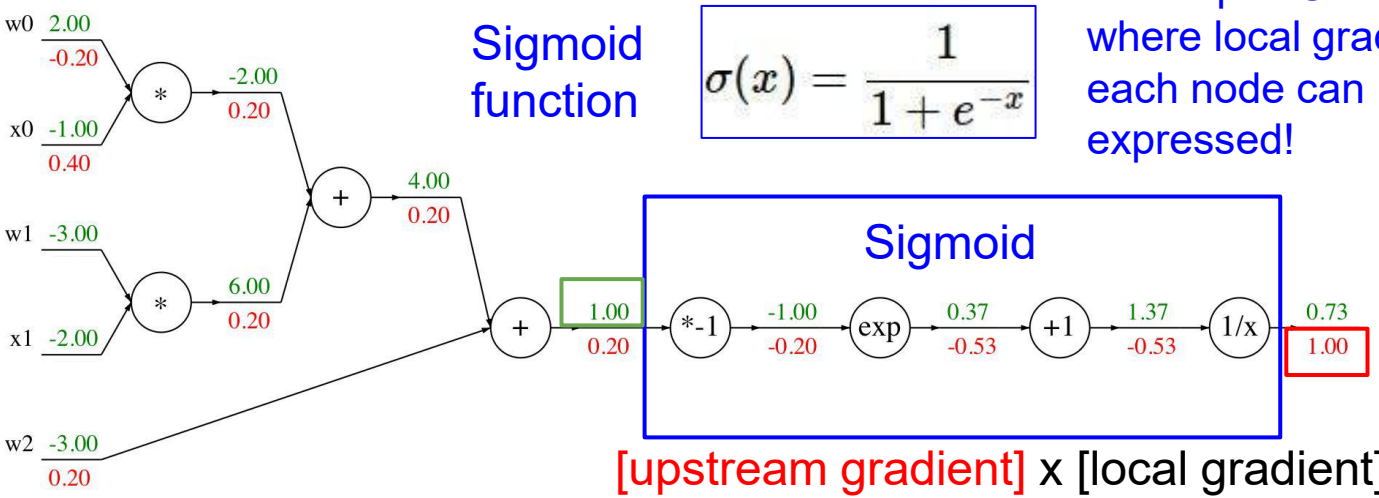
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

[upstream gradient] x [local gradient]

[1.00] x [(1 - 1/(1+e^{-1})) (1/(1+e^{-1}))] = 0.2

Sigmoid local gradient:

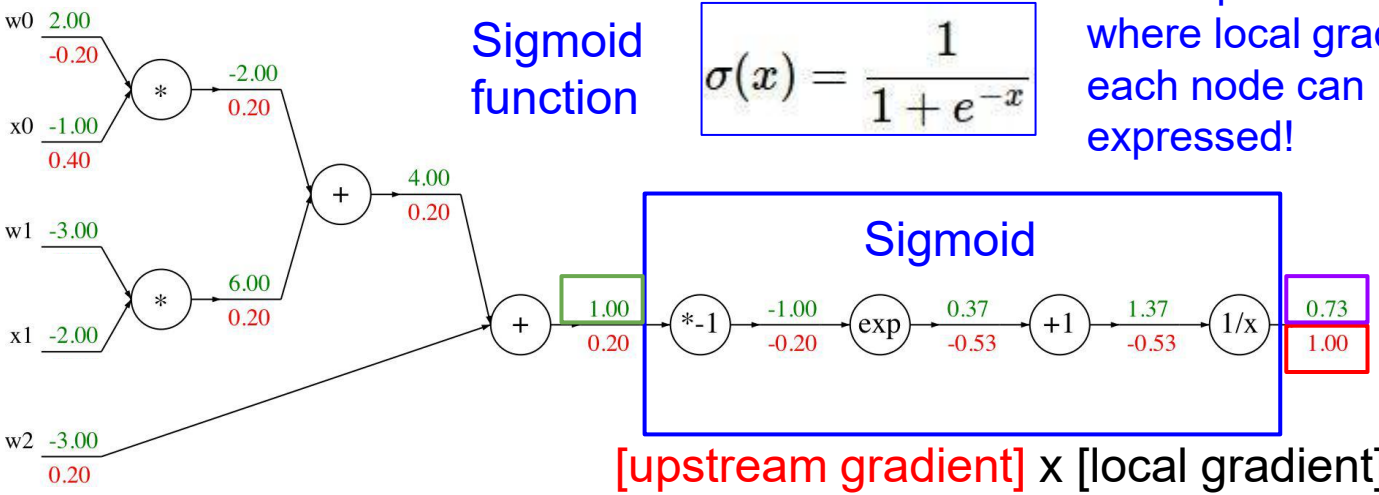
$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$

Credit to Stanford CS 231n

Another example:

$$f(w,x) = \frac{1}{1+e^{-(w_0x_0+w_1x_1+w_2)}}$$

Computational graph representation may not be unique. Choose one where local gradients at each node can be easily expressed!



Sigmoid function

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

[upstream gradient] x [local gradient]

[1.00] x [(1 - 0.73) (0.73)] = 0.2

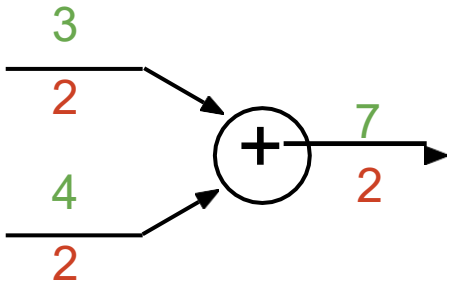
Sigmoid local gradient:

$$\frac{d\sigma(x)}{dx} = \frac{e^{-x}}{(1+e^{-x})^2} = \left(\frac{1+e^{-x}-1}{1+e^{-x}}\right)\left(\frac{1}{1+e^{-x}}\right) = (1-\sigma(x))\,\sigma(x)$$

Credit to Stanford CS 231n

Patterns in gradient flow

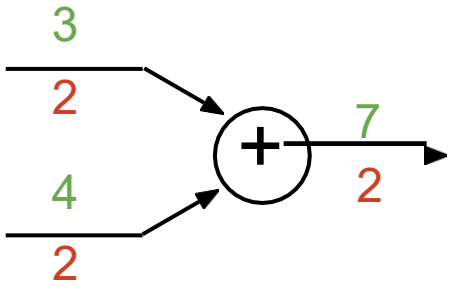
add gate: gradient distributor



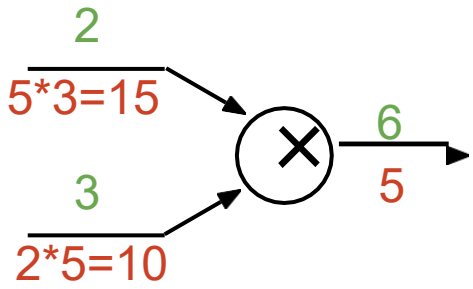
Credit to Stanford CS 231n

Patterns in gradient flow

add gate: gradient distributor



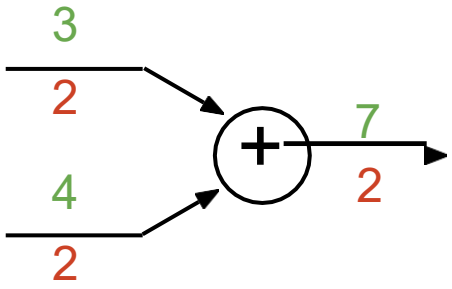
mul gate: “swap multiplier”



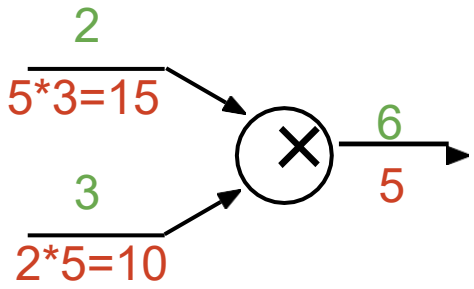
Credit to Stanford CS 231n

Patterns in gradient flow

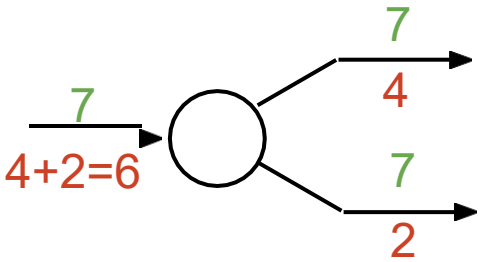
add gate: gradient distributor



mul gate: “swap multiplier”



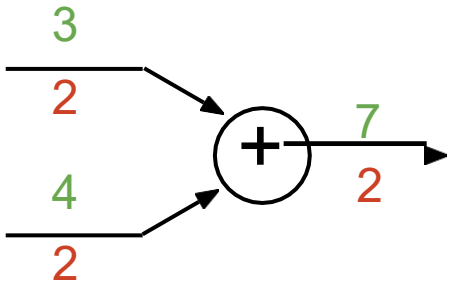
copy gate: gradient adder



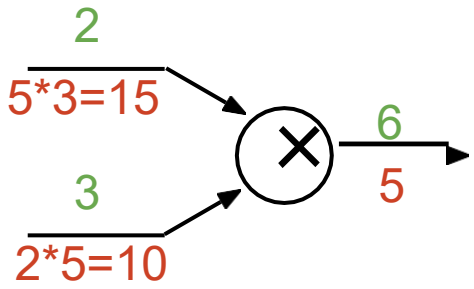
Credit to Stanford CS 231n

Patterns in gradient flow

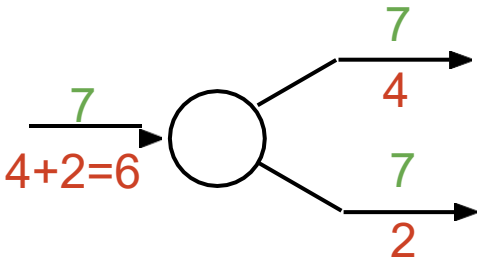
add gate: gradient distributor



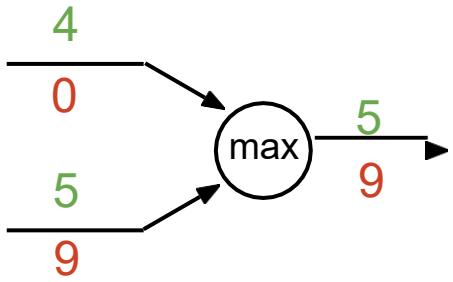
mul gate: “swap multiplier”



copy gate: gradient adder



max gate: gradient router

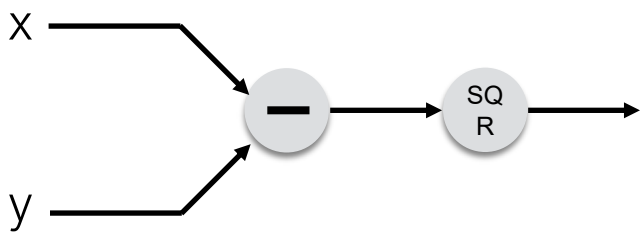


Credit to Stanford CS 231n

Computational Graphs

L2-loss

$$L_2(x, y) = (x - y)^2$$

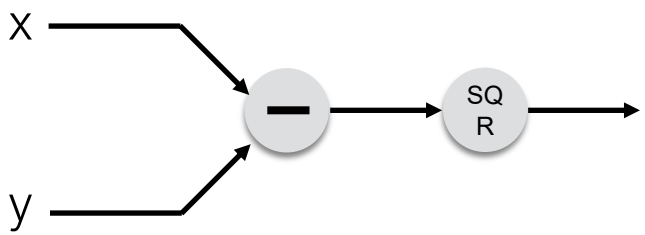


Credit to Stanford CS 231n

Computational Graphs

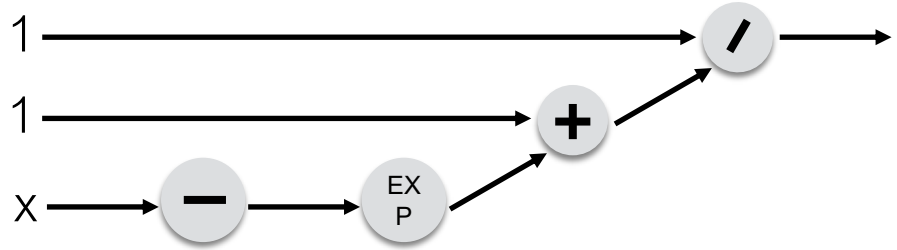
L2-loss

$$L_2(x, y) = (x - y)^2$$



Sigmoid

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

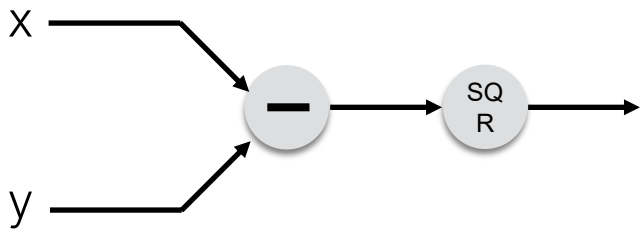


Credit to Stanford CS 231n

Computational Graphs

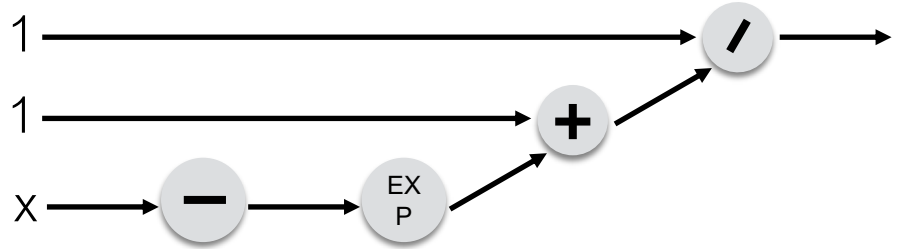
L2-loss

$$L_2(x, y) = (x - y)^2$$



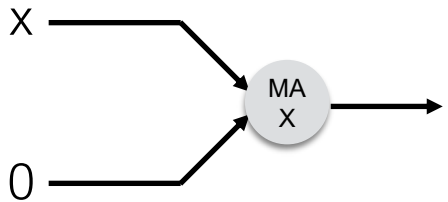
Sigmoid

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



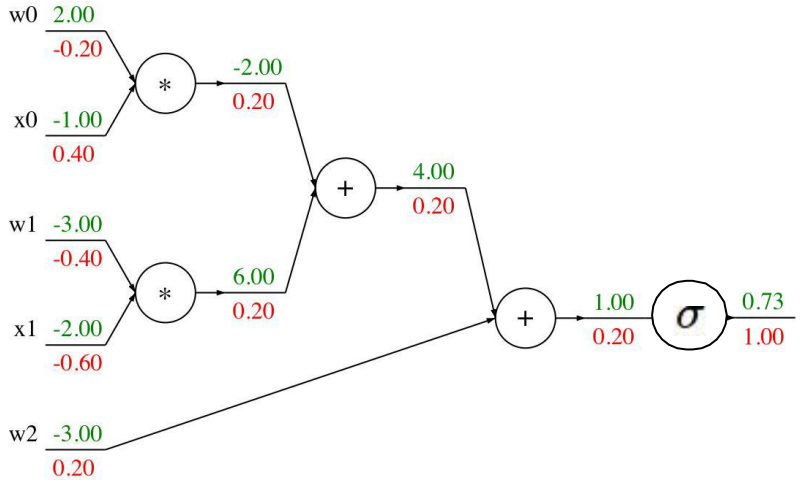
ReLU

$$\text{ReLU}(x) = \max(x, 0)$$



Credit to Stanford CS 231n

Backprop Implementation: "Flat" code



Forward pass:
Compute output

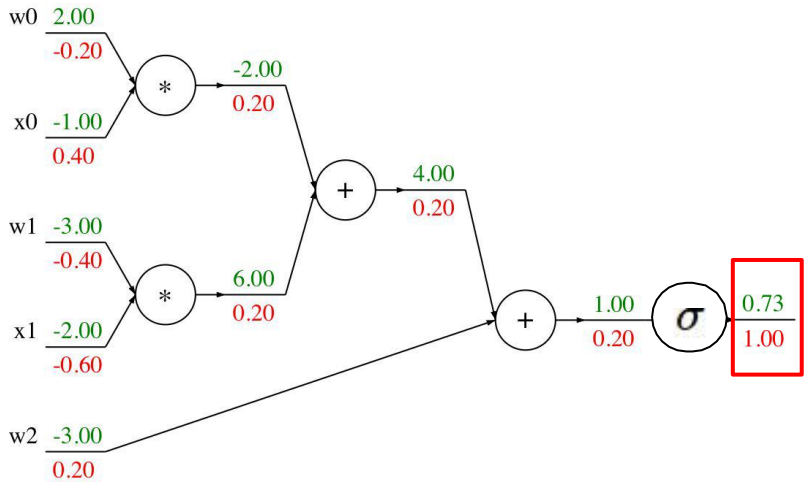
```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Backward pass:
Compute grads

```
grad_L = 1.0
grad_s3 = grad_L * (1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

Credit to Stanford CS 231n

Backprop Implementation: "Flat" code



Forward pass: Compute output

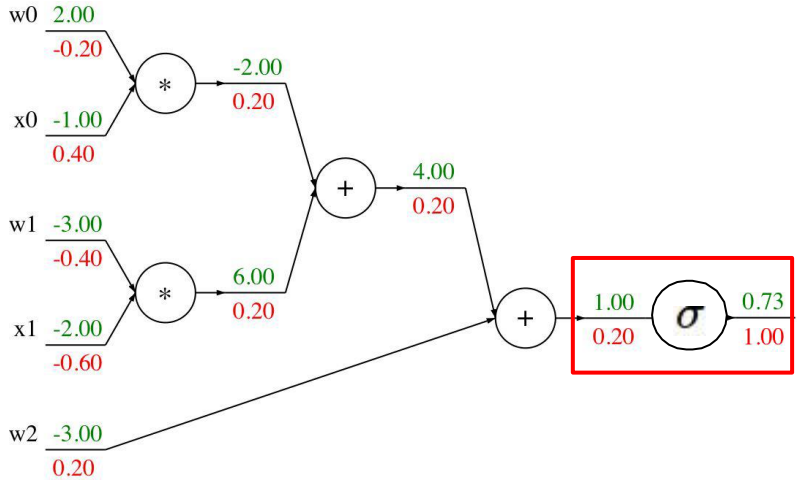
```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Base case

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Credit to Stanford CS 231n

Backprop Implementation: "Flat" code



Forward pass:
Compute output

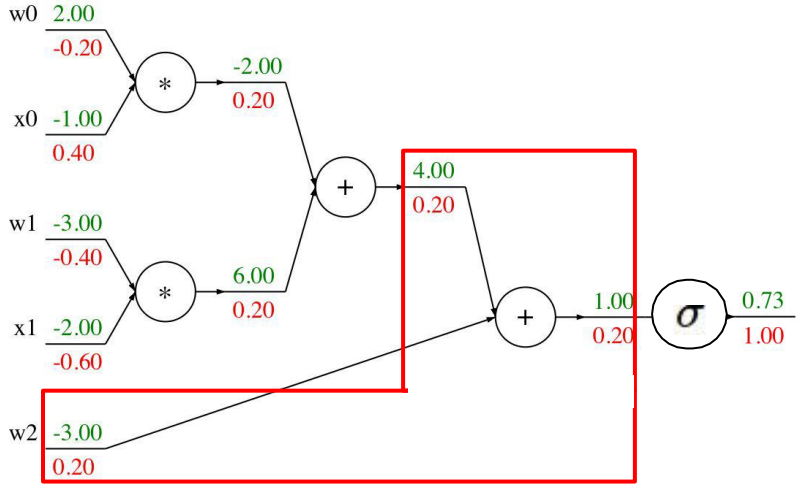
```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Sigmoid

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Credit to Stanford CS 231n

Backprop Implementation: "Flat" code



Forward pass:
Compute output

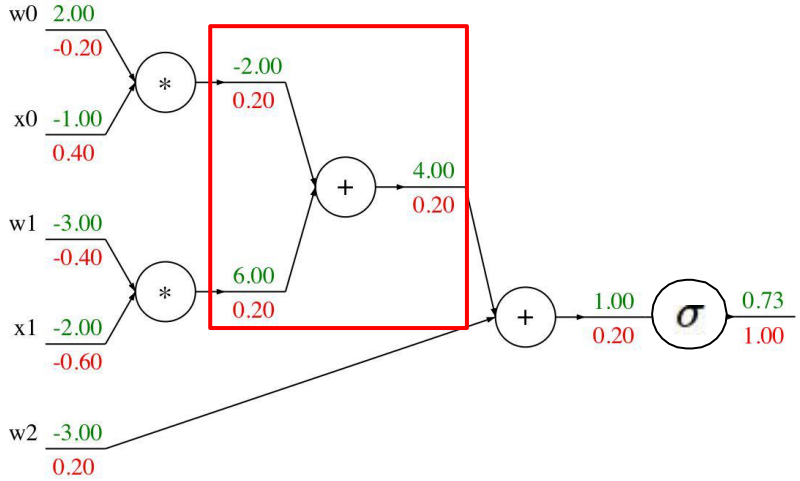
```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Add gate

```
grad_L = 1.0
grad_s3 = grad_L * (1 - L) * L
grad_w2 = grad_s3
grad_s2 = grad_s3
grad_s0 = grad_s2
grad_s1 = grad_s2
grad_w1 = grad_s1 * x1
grad_x1 = grad_s1 * w1
grad_w0 = grad_s0 * x0
grad_x0 = grad_s0 * w0
```

Credit to Stanford CS 231n

Backprop Implementation: "Flat" code



Forward pass:
Compute output

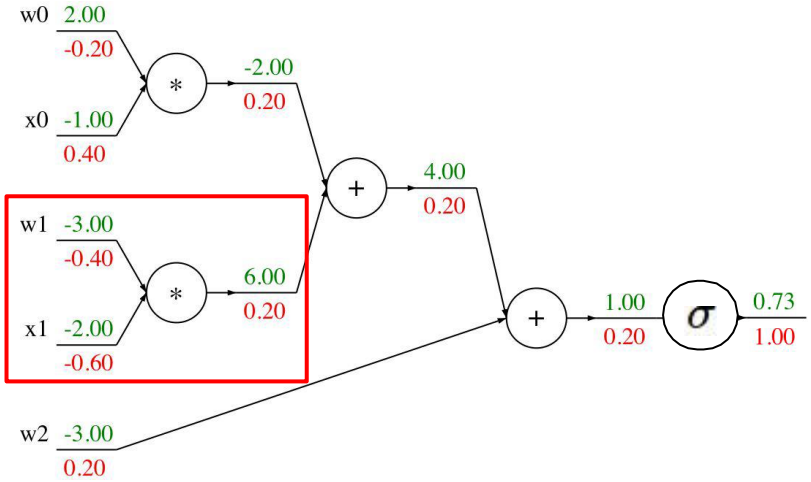
```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

Add gate

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Credit to Stanford CS 231n

Backprop Implementation: "Flat" code



Forward pass:
Compute output

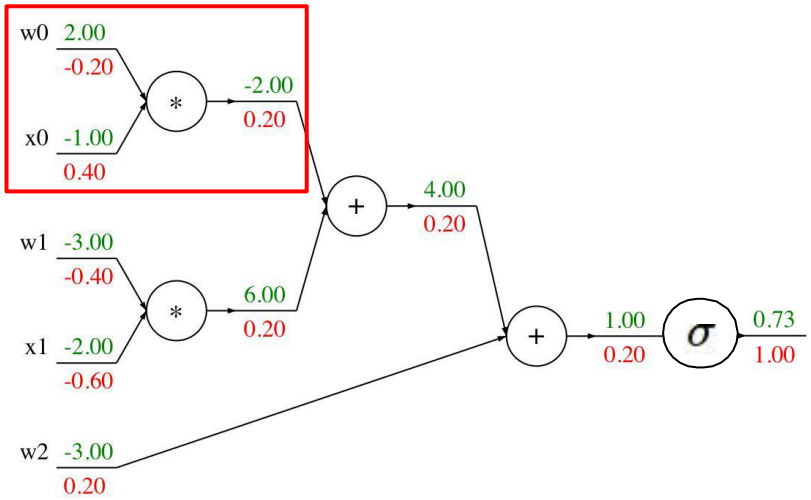
```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Multiply gate

Credit to Stanford CS 231n

Backprop Implementation: "Flat" code



Forward pass: Compute output

```
def f(w0, x0, w1, x1, w2):
  s0 = w0 * x0
  s1 = w1 * x1
  s2 = s0 + s1
  s3 = s2 + w2
  L = sigmoid(s3)
```

```
  grad_L = 1.0
  grad_s3 = grad_L * (1 - L) * L
  grad_w2 = grad_s3
  grad_s2 = grad_s3
  grad_s0 = grad_s2
  grad_s1 = grad_s2
  grad_w1 = grad_s1 * x1
  grad_x1 = grad_s1 * w1
  grad_w0 = grad_s0 * x0
  grad_x0 = grad_s0 * w0
```

Multiply gate

Credit to Stanford CS 231n

Vector derivatives

Scalar to Scalar

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If x changes by a small amount, how much will y change?

Credit to Stanford CS 231n

Vector derivatives

Scalar to Scalar

$$x \in \mathbb{R}, y \in \mathbb{R}$$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If x changes by a small amount, how much will y change?

Vector to Scalar

$$x \in \mathbb{R}^N, y \in \mathbb{R}$$

Derivative is **Gradient**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x}\right)_n = \frac{\partial y}{\partial x_n}$$

For each element of x, if it changes by a small amount then how much will y change?

Credit to Stanford CS 231n

Vector derivatives

Scalar to Scalar

$x \in \mathbb{R}, y \in \mathbb{R}$

Regular derivative:

$$\frac{\partial y}{\partial x} \in \mathbb{R}$$

If x changes by a small amount, how much will y change?

Vector to Scalar

$x \in \mathbb{R}^N, y \in \mathbb{R}$

Derivative is **Gradient**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^N \quad \left(\frac{\partial y}{\partial x}\right)_n = \frac{\partial y}{\partial x_n}$$

For each element of x, if it changes by a small amount then how much will y change?

Vector to Vector

$x \in \mathbb{R}^N, y \in \mathbb{R}^M$

Derivative is **Jacobian**:

$$\frac{\partial y}{\partial x} \in \mathbb{R}^{N \times M} \quad \left(\frac{\partial y}{\partial x}\right)_{n,m} = \frac{\partial y_m}{\partial x_n}$$

For each element of x, if it changes by a small amount then how much will each element of y change?

Credit to Stanford CS 231n

Example 5.9 (Gradient of a Vector-Valued Function)
We are given

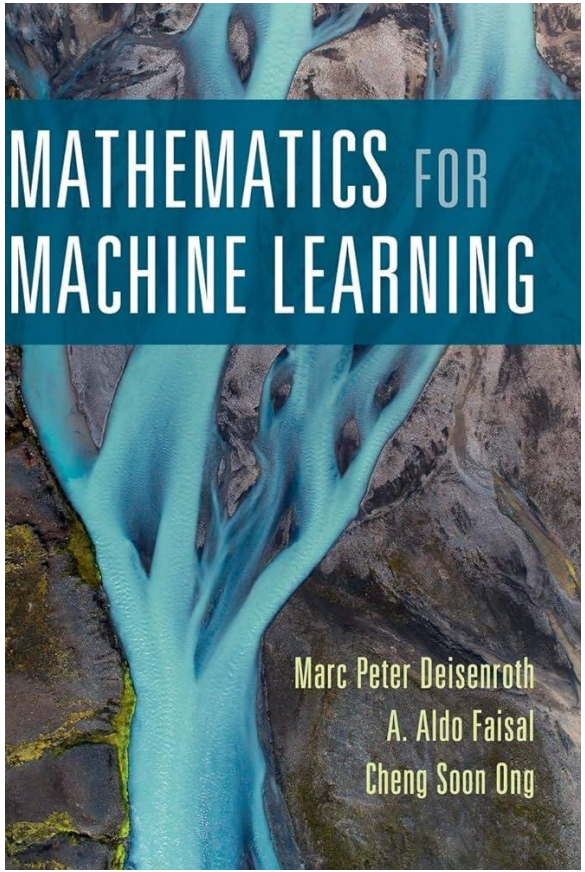
$$\boldsymbol{f}(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}\,, \qquad \boldsymbol{f}(\boldsymbol{x}) \in \mathbb{R}^M, \quad \boldsymbol{A} \in \mathbb{R}^{M\times N}, \quad \boldsymbol{x} \in \mathbb{R}^N\,.$$

To compute the gradient $\mathrm{d}\boldsymbol{f}/\mathrm{d}\boldsymbol{x}$ we first determine the dimension of $\mathrm{d}\boldsymbol{f}/\mathrm{d}\boldsymbol{x}$: Since $\boldsymbol{f} : \mathbb{R}^N \to \mathbb{R}^M$, it follows that $\mathrm{d}\boldsymbol{f}/\mathrm{d}\boldsymbol{x} \in \mathbb{R}^{M\times N}$. Second, to compute the gradient we determine the partial derivatives of f with respect to every x_j:

$$f_i(\boldsymbol{x}) = \sum_{j=1}^{N} A_{ij}x_j \implies \frac{\partial f_i}{\partial x_j} = A_{ij} \tag{5.67}$$

We collect the partial derivatives in the Jacobian and obtain the gradient

$$\frac{\mathrm{d}\boldsymbol{f}}{\mathrm{d}\boldsymbol{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_N} \\ \vdots & & \vdots \\ \frac{\partial f_M}{\partial x_1} & \cdots & \frac{\partial f_M}{\partial x_N} \end{bmatrix} = \begin{bmatrix} A_{11} & \cdots & A_{1N} \\ \vdots & & \vdots \\ A_{M1} & \cdots & A_{MN} \end{bmatrix} = \boldsymbol{A} \in \mathbb{R}^{M\times N}\,. \tag{5.68}$$



Example 5.10 (Chain Rule)
Consider the function $h : \mathbb{R} \to \mathbb{R}$, $h(t) = (f \circ g)(t)$ with

$$f : \mathbb{R}^2 \to \mathbb{R} \tag{5.69}$$

$$g : \mathbb{R} \to \mathbb{R}^2 \tag{5.70}$$

$$f(\boldsymbol{x}) = \exp(x_1 x_2^2)\,, \tag{5.71}$$

$$\boldsymbol{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = g(t) = \begin{bmatrix} t\cos t \\ t \sin t \end{bmatrix} \tag{5.72}$$

and compute the gradient of h with respect to t. Since $f : \mathbb{R}^2 \to \mathbb{R}$ and $g : \mathbb{R} \to \mathbb{R}^2$ we note that

$$\frac{\partial f}{\partial \boldsymbol{x}} \in \mathbb{R}^{1\times 2}\,, \quad \frac{\partial g}{\partial t} \in \mathbb{R}^{2\times 1}\,. \tag{5.73}$$

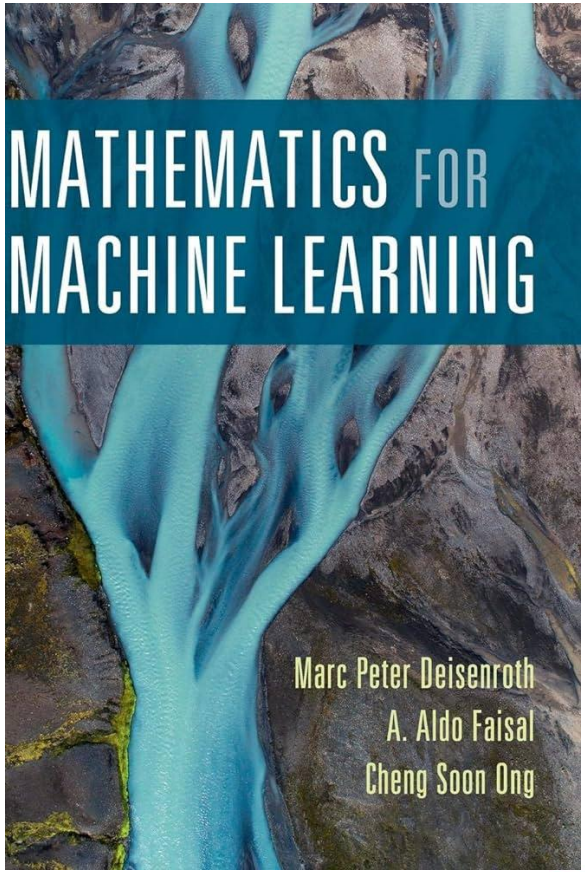
The desired gradient is computed by applying the chain rule:

$$\frac{\mathrm{d}h}{\mathrm{d}t} = \frac{\partial f}{\partial \boldsymbol{x}}\frac{\partial \boldsymbol{x}}{\partial t} = \begin{bmatrix} \frac{\partial f}{\partial x_1} & \frac{\partial f}{\partial x_2} \end{bmatrix} \begin{bmatrix} \frac{\partial x_1}{\partial t} \\ \frac{\partial x_2}{\partial t} \end{bmatrix} \tag{5.74a}$$

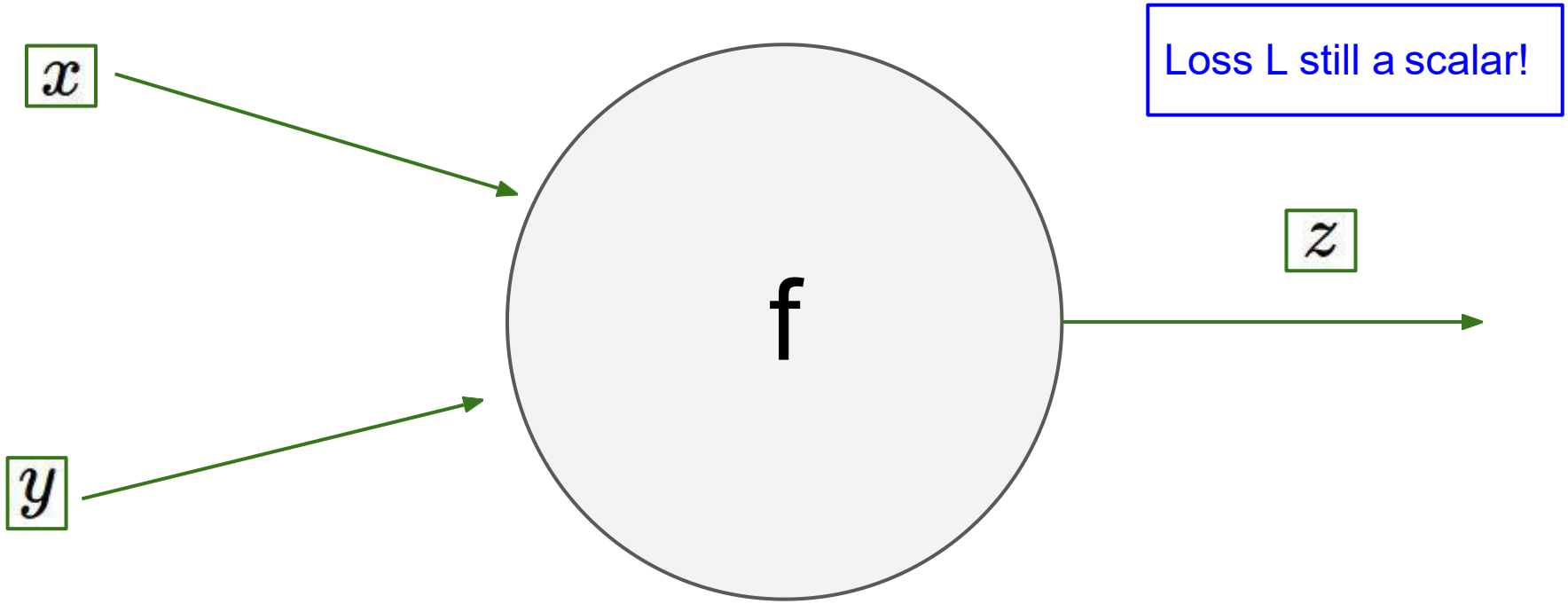
$$= \begin{bmatrix} \exp(x_1 x_2^2) x_2^2 & 2\exp(x_1 x_2^2) x_1 x_2 \end{bmatrix} \begin{bmatrix} \cos t - t \sin t \\ \sin t + t \cos t \end{bmatrix} \tag{5.74b}$$

$$= \exp(x_1 x_2^2)\big(x_2^2(\cos t - t \sin t) + 2x_1 x_2(\sin t + t \cos t)\big)\,, \tag{5.74c}$$

where $x_1 = t\cos t$ and $x_2 = t \sin t$; see (5.72).

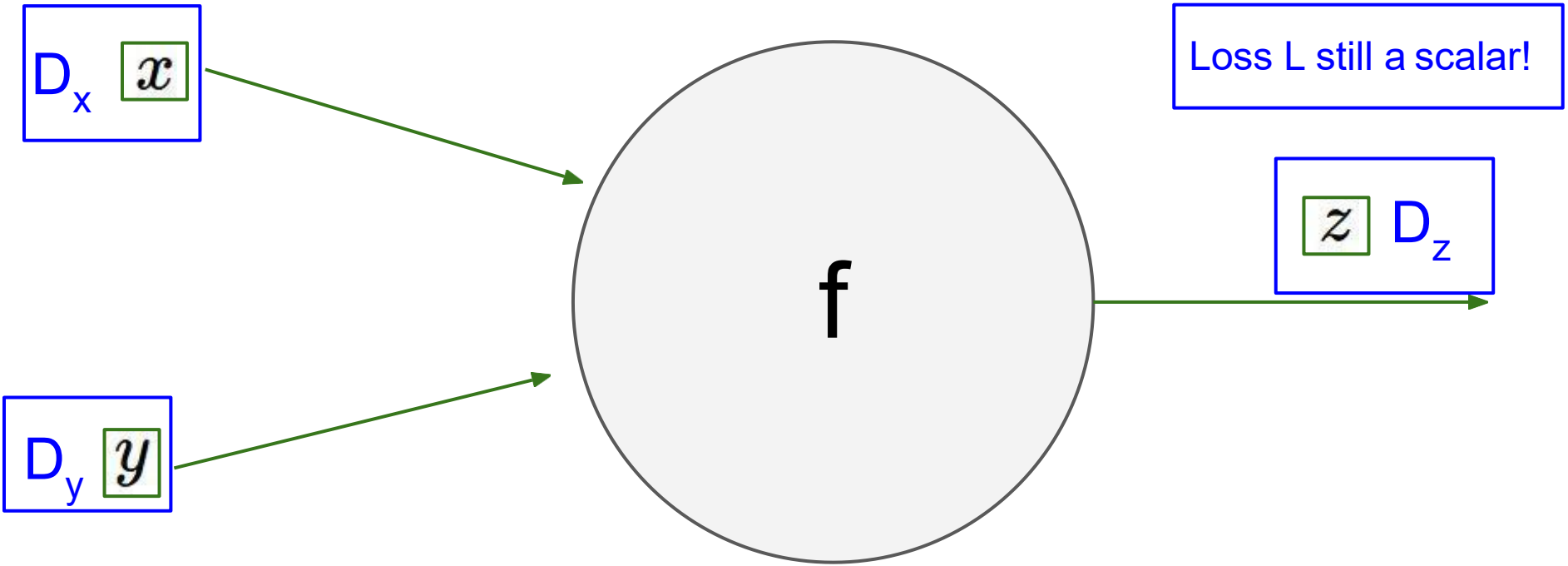


Backprop with Vectors



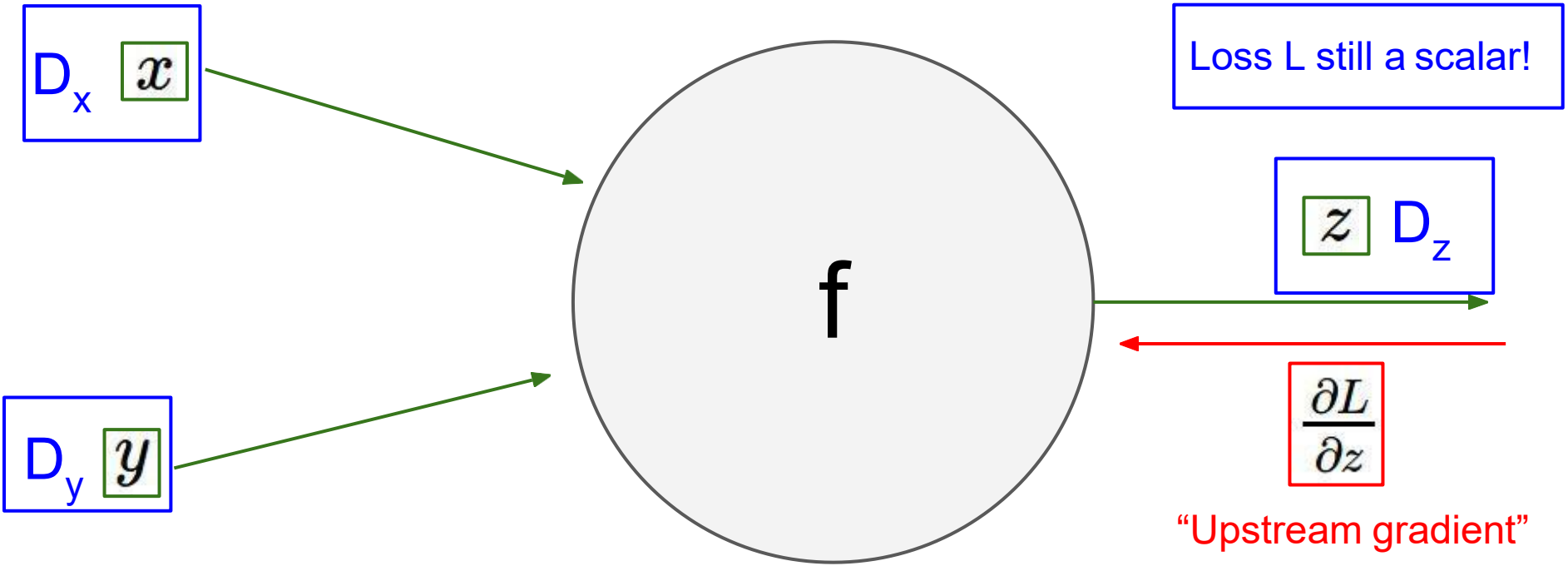
Credit to Stanford CS 231n

Backprop with Vectors



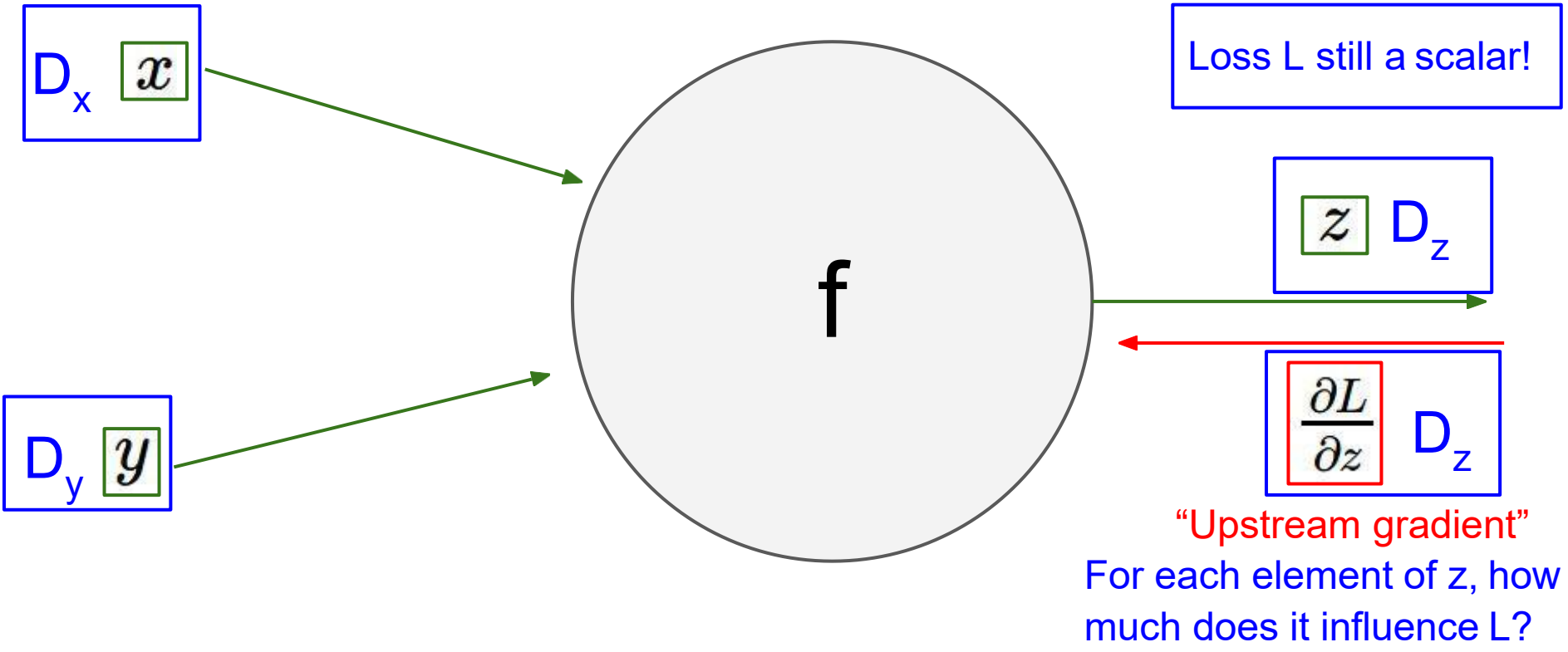
Credit to Stanford CS 231n

Backprop with Vectors



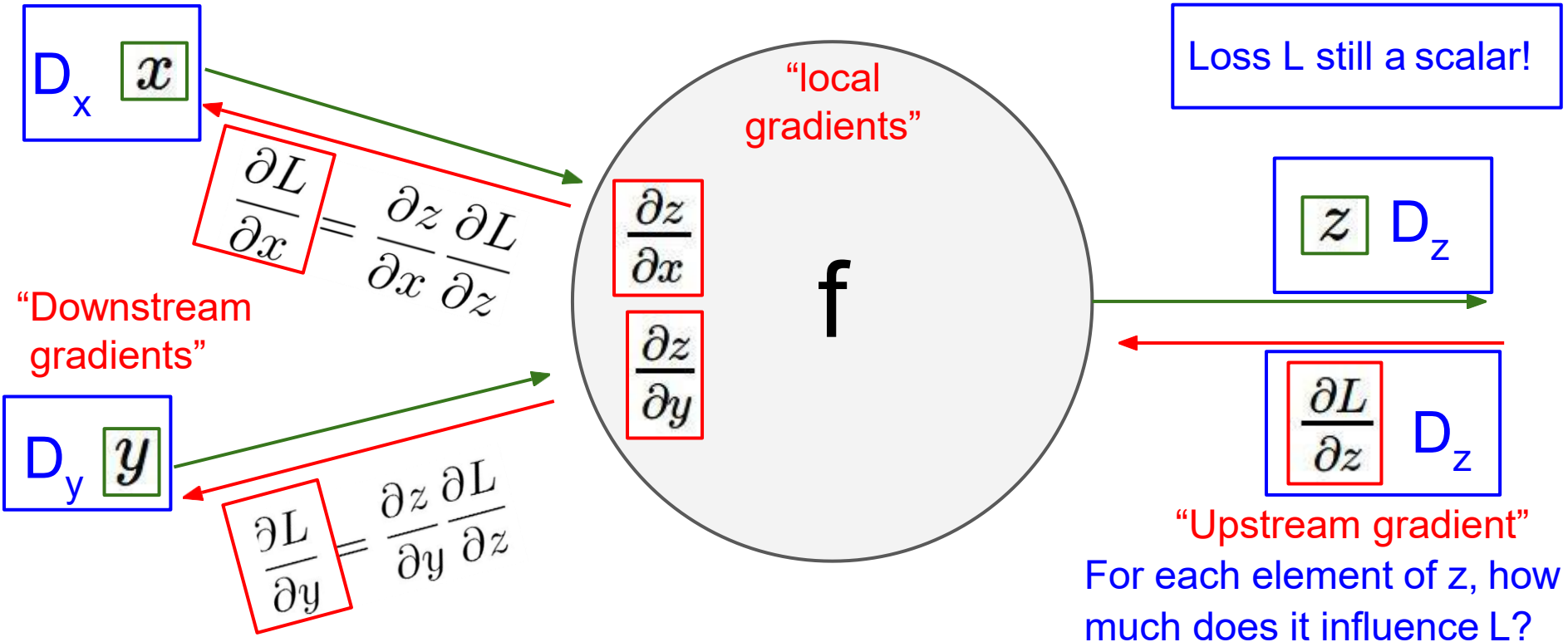
Credit to Stanford CS 231n

Backprop with Vectors



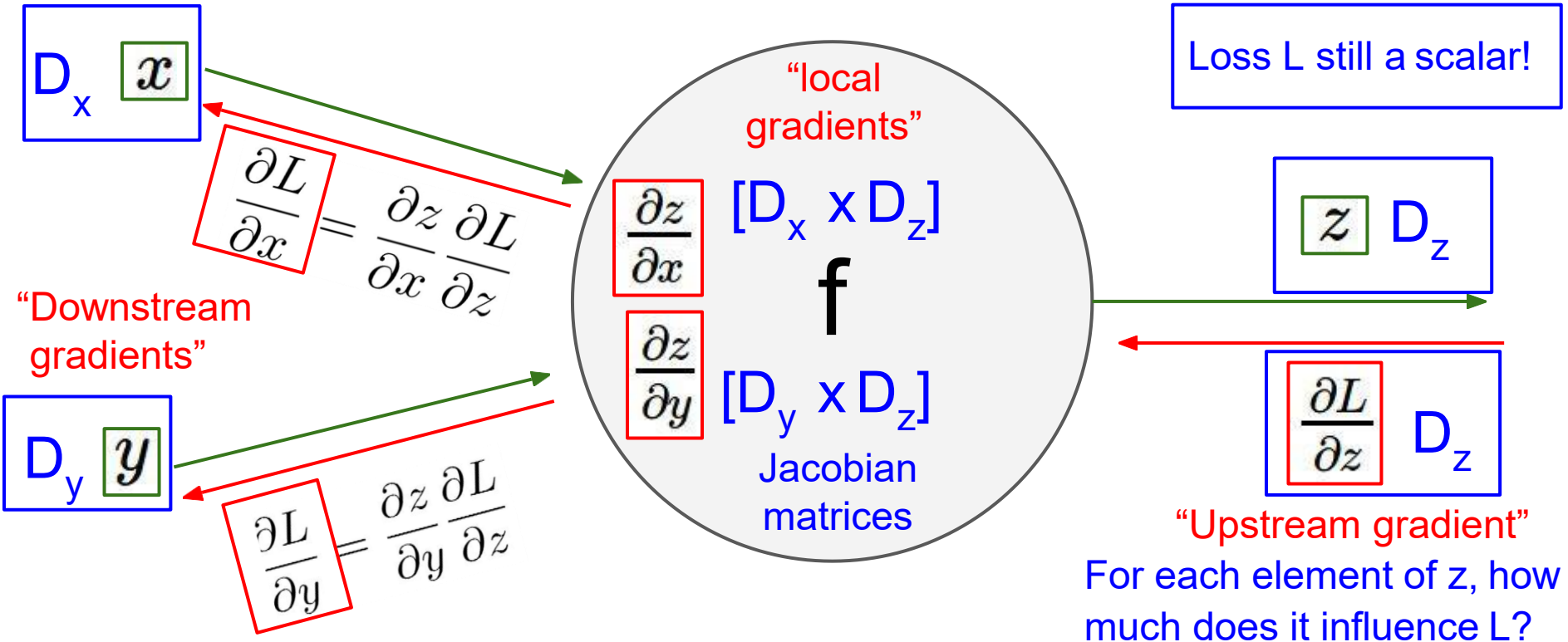
Credit to Stanford CS 231n

Backprop with Vectors



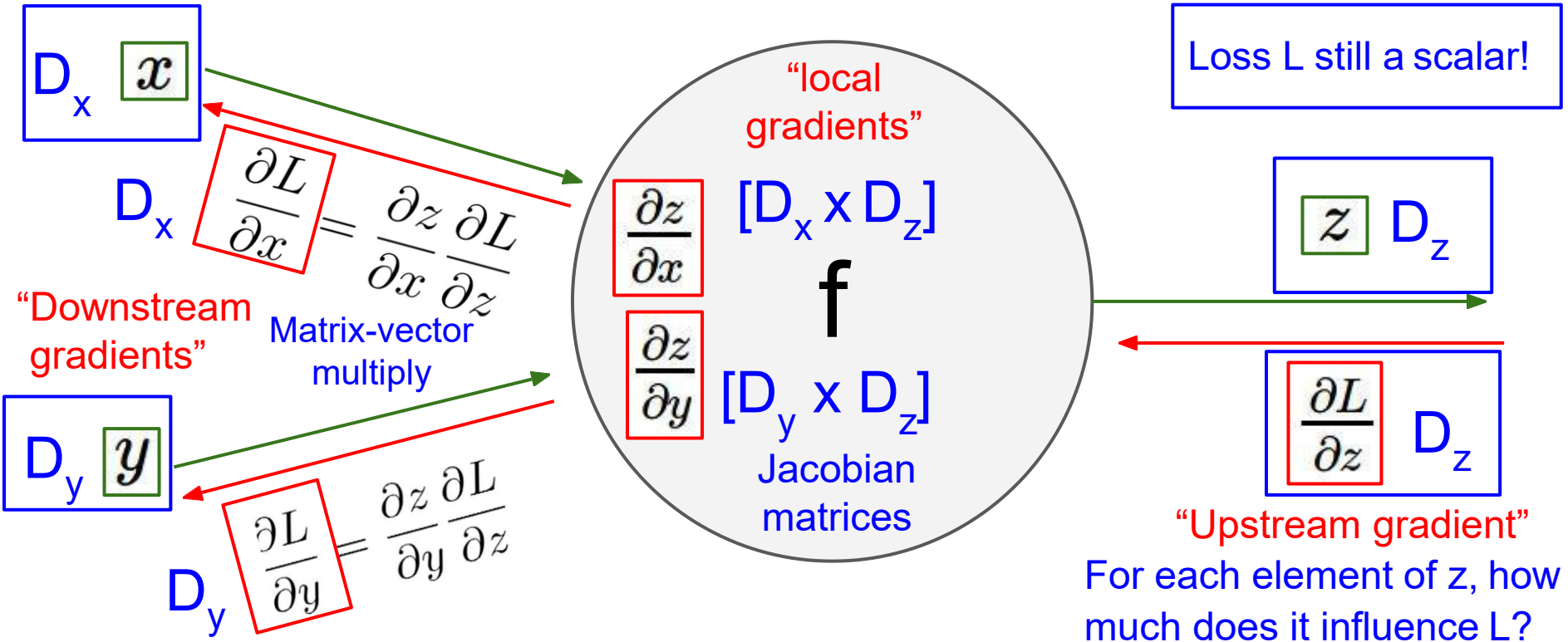
Credit to Stanford CS 231n

Backprop with Vectors



Credit to Stanford CS 231n

Backprop with Vectors



Credit to Stanford CS 231n

Backprop Skips the Full Jacobian

Signal. For z=f(x), backprop maps g_z to g_x through a vector-Jacobian product (VJP).

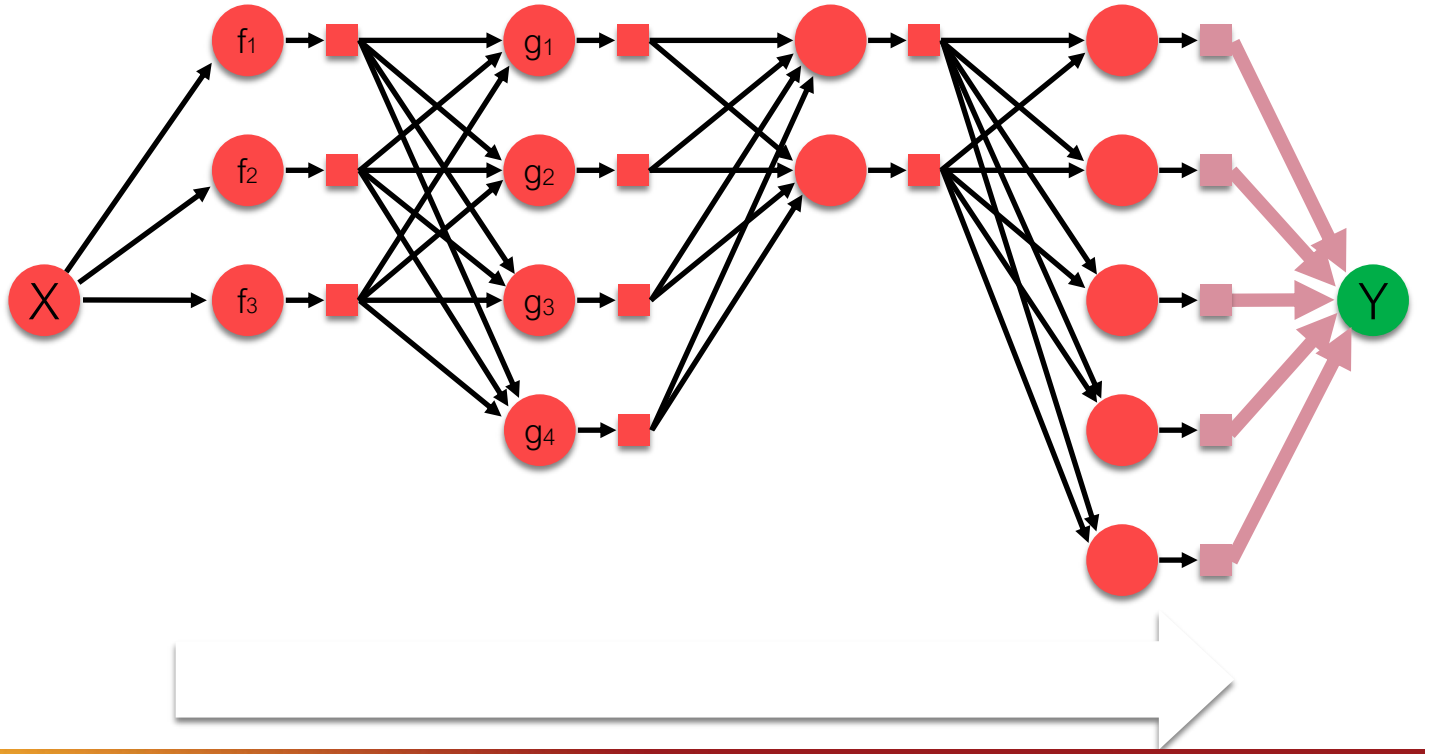
Memory. No full Jacobian is materialized; each operation applies a local VJP.

Scale. For scalar L, one reverse sweep gets all parameter gradients. Baydin et al. (JMLR 2018) show this has the same asymptotic order as evaluation.

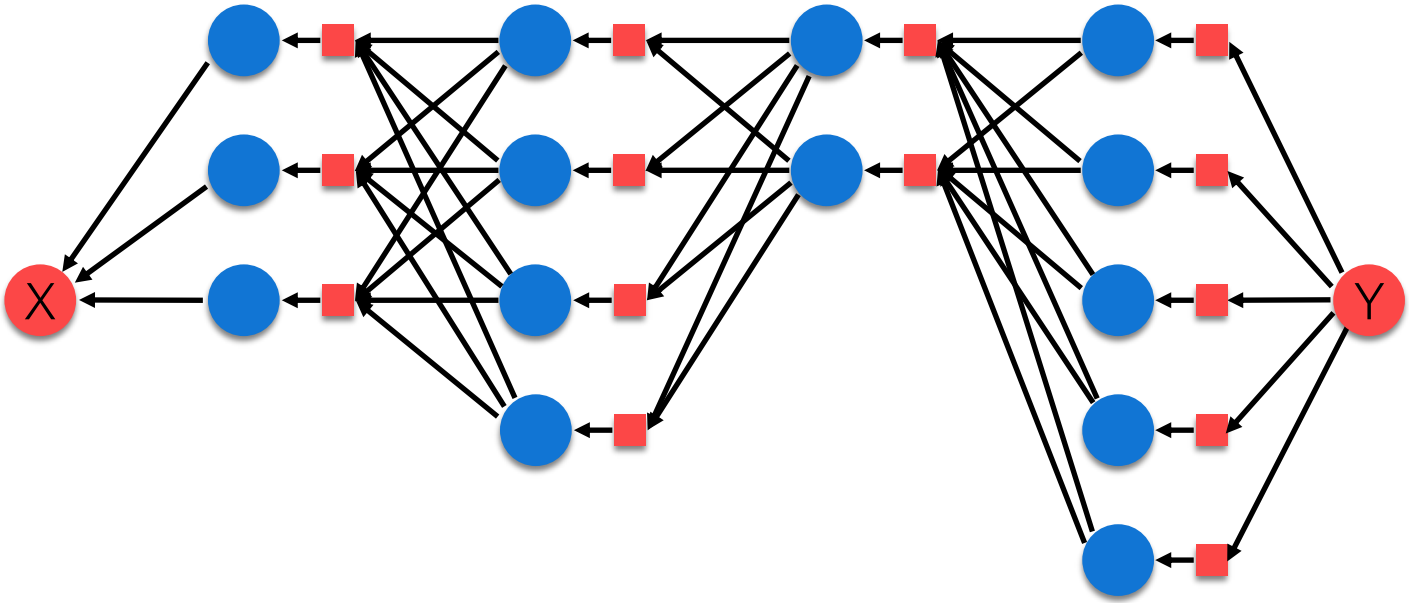
Caveat. Same order is not equal time: backward reads saved activations and accumulates gradients.

Backprop scales by moving gradient vectors, not full Jacobian matrices.

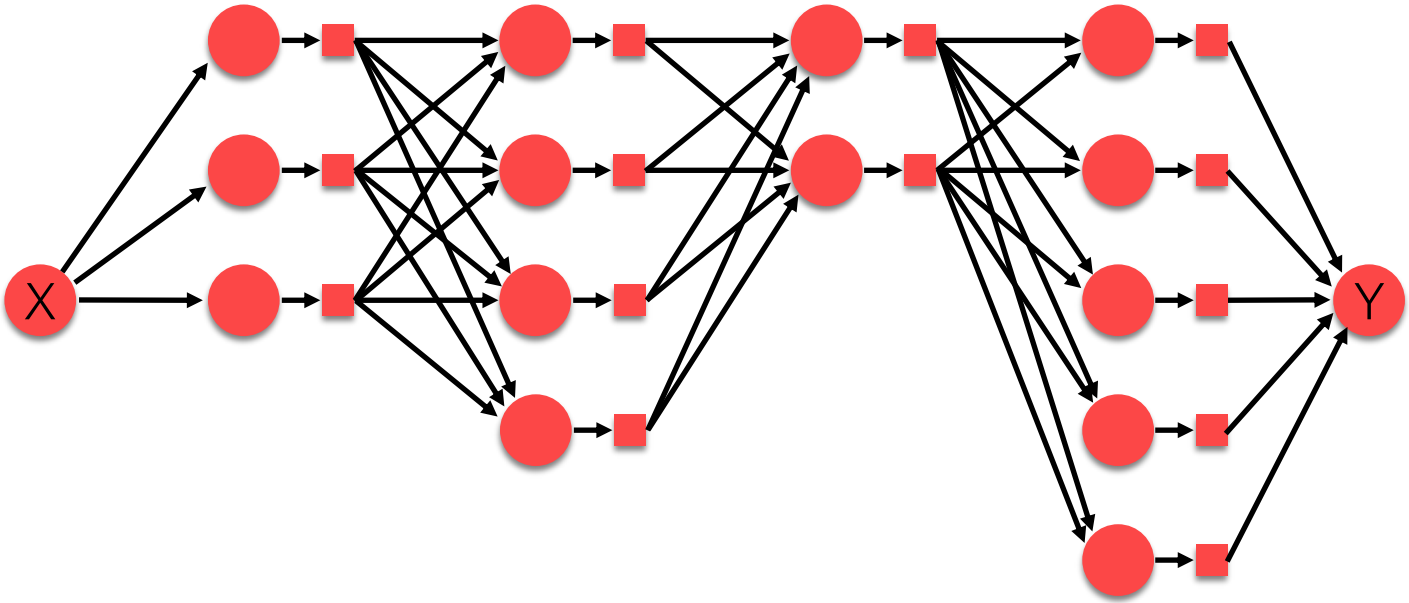
Forward Propagation



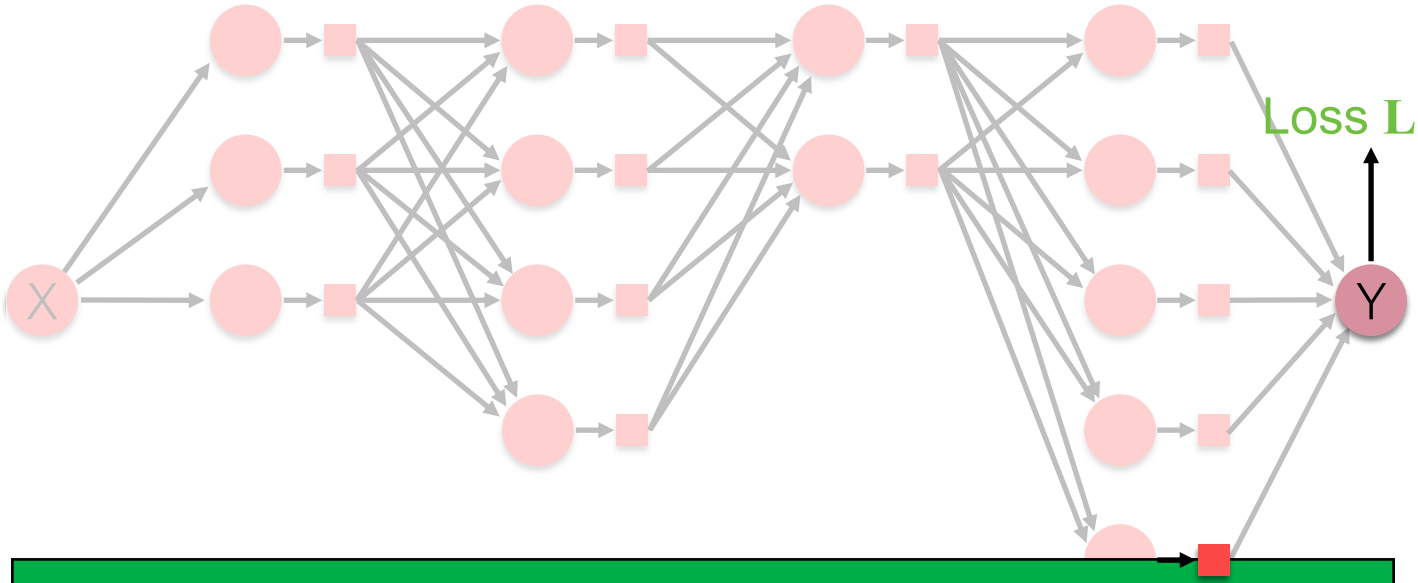
Back Propagation



Back Propagation



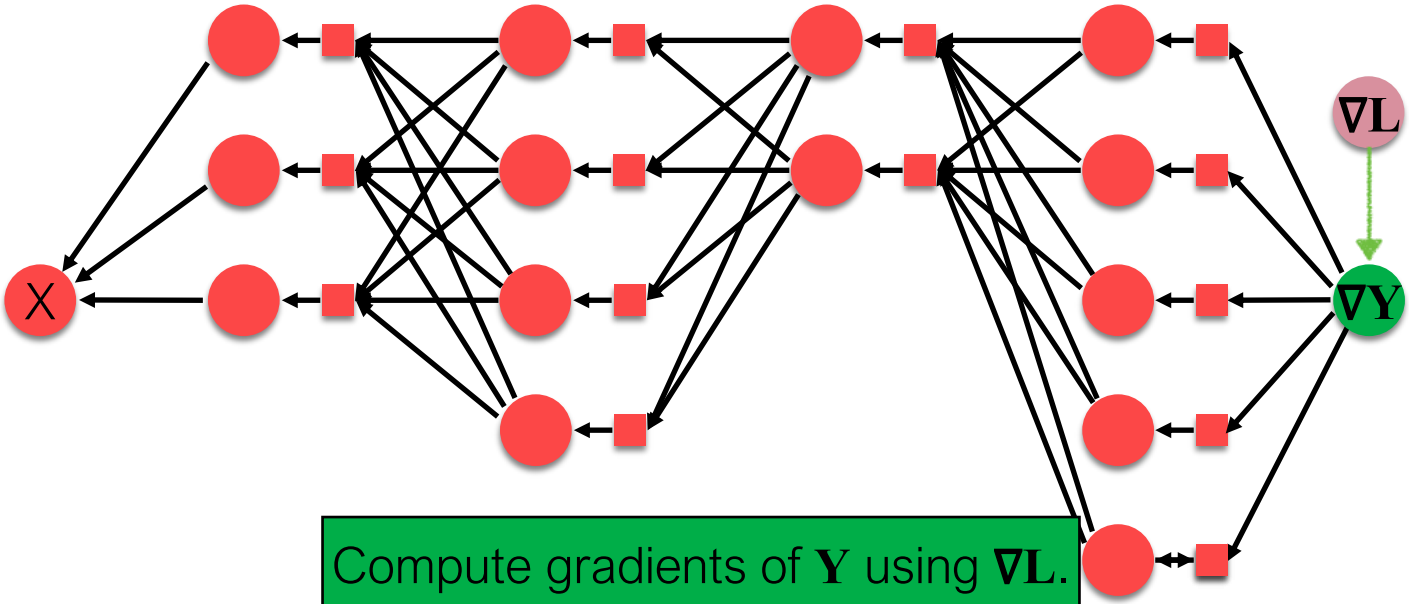
Back Propagation



Compute a loss using an estimated Y and the ground truth Y*.

e.g., $L(W) = \| f(X; W) - Y^* \|_2$

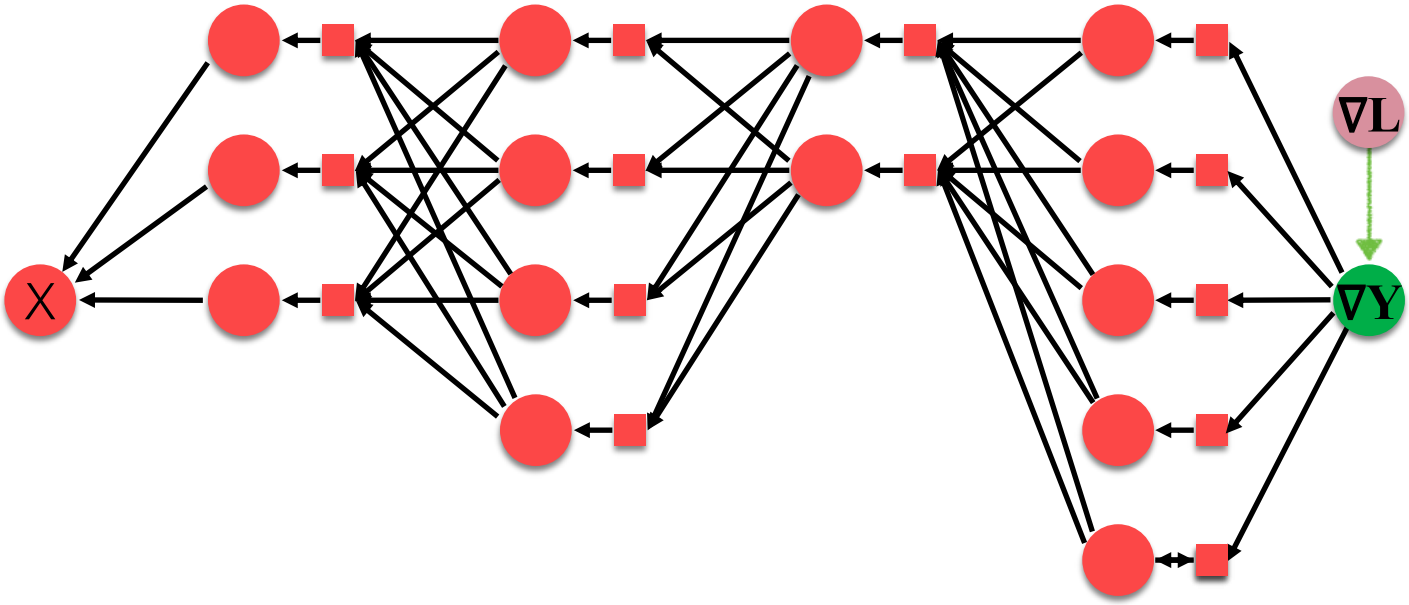
Back Propagation



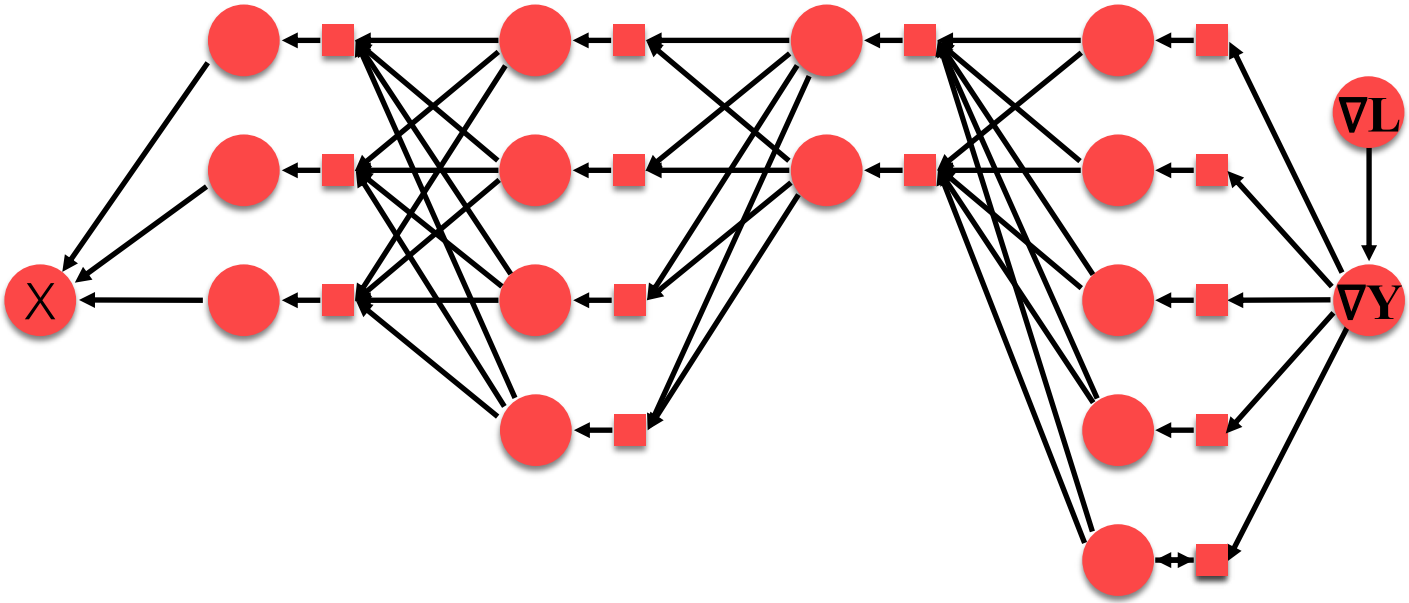
Compute gradients of $\mathbf{Y}$ using $\nabla\mathbf{L}$.

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \cdot \frac{\partial Y}{\partial X}$$

Back Propagation

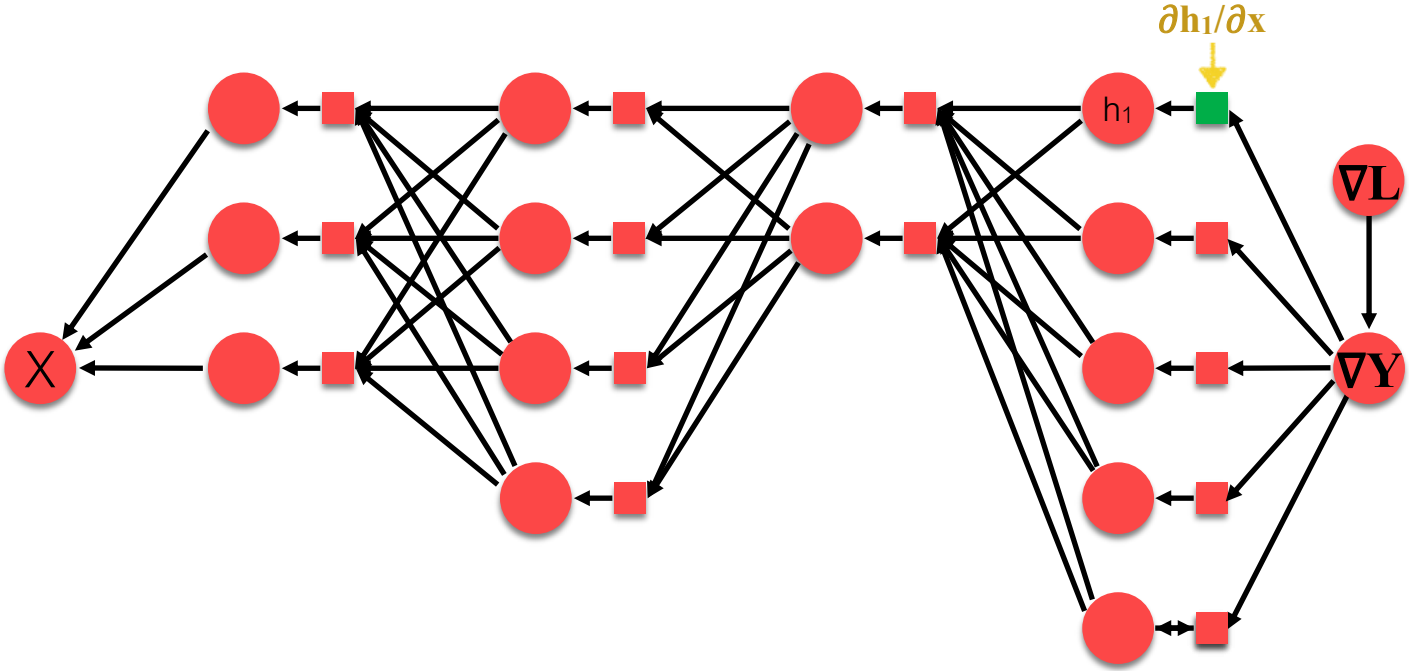


Back Propagation



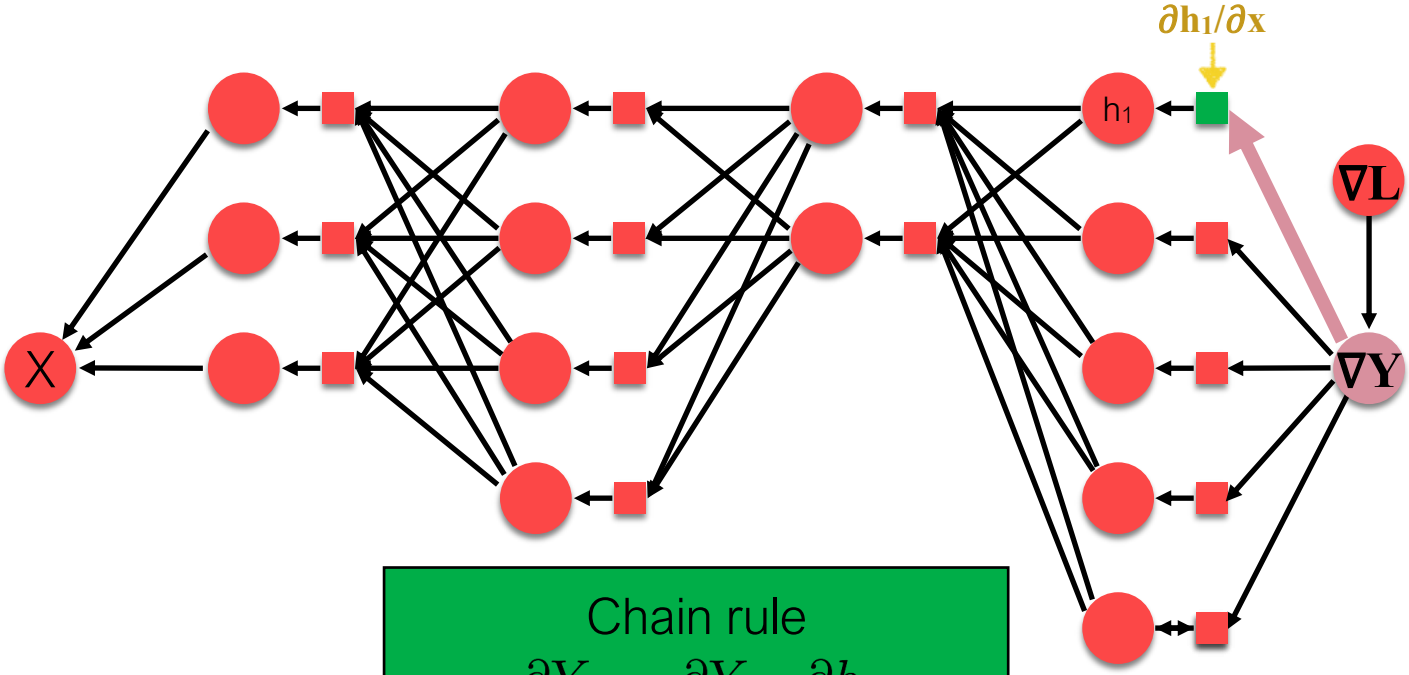
USC Viterbi School of Engineering

Back Propagation



USC Viterbi School of Engineering

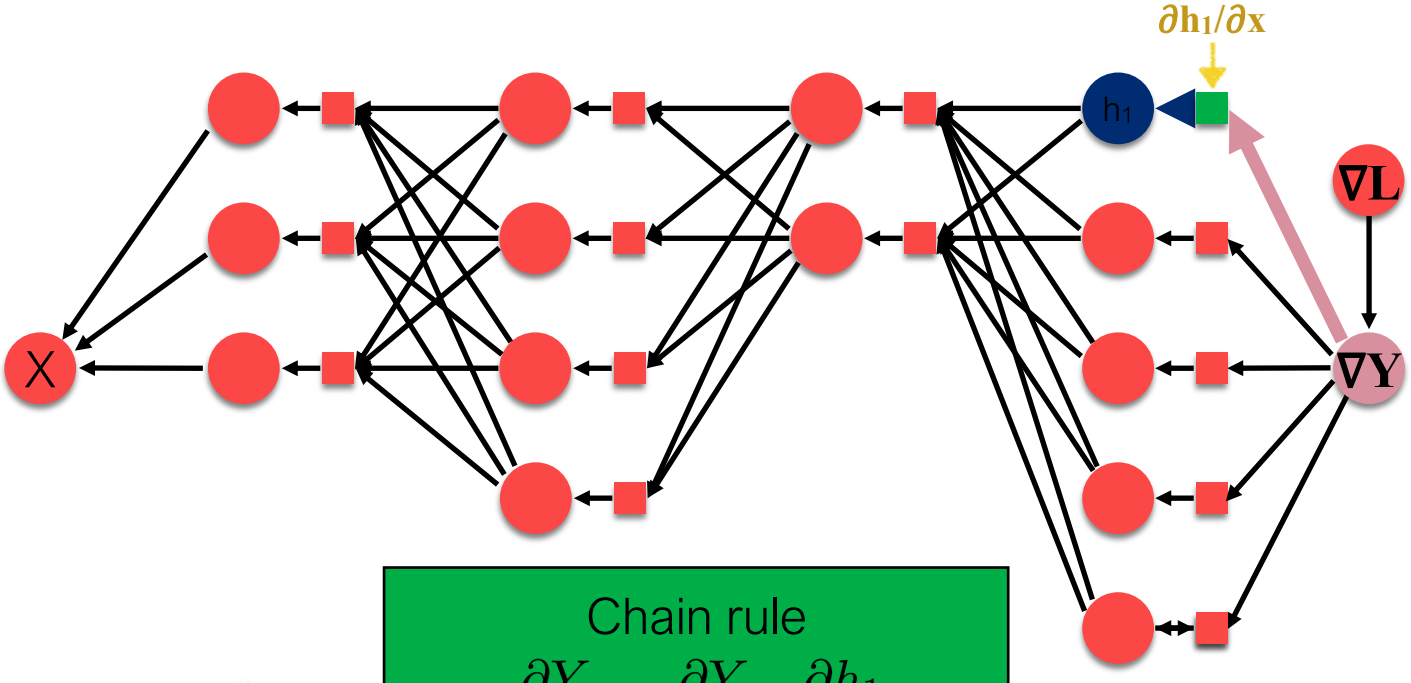
Back Propagation



Chain rule

$$\frac{\partial Y}{\partial X} = \frac{\partial Y}{\partial h_1} \cdot \frac{\partial h_1}{\partial X}$$

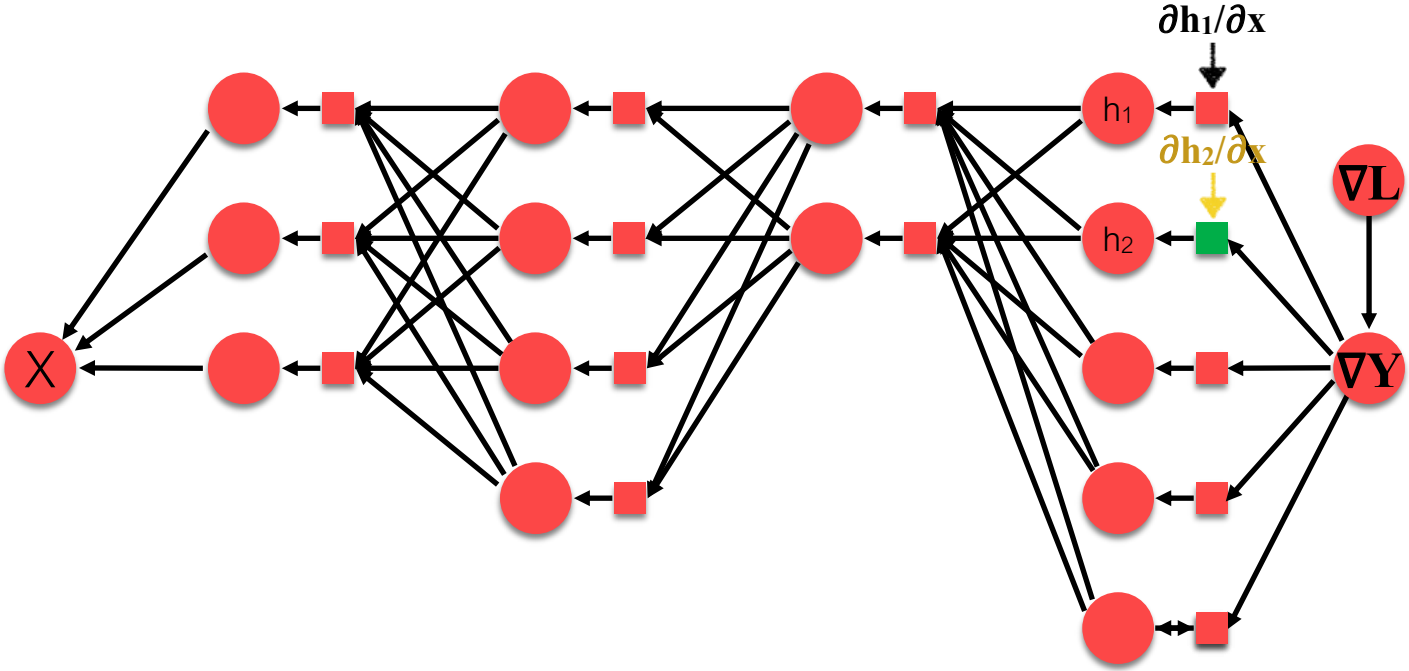
Back Propagation



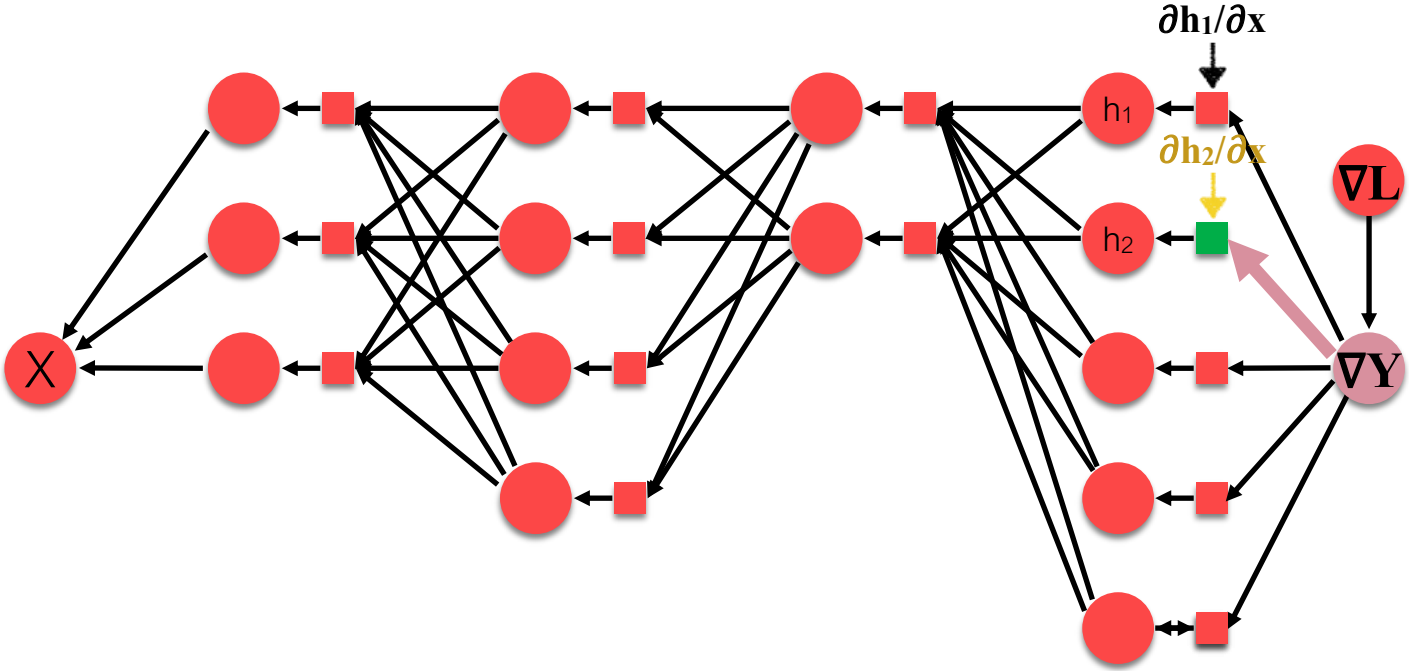
Chain rule

$$\frac{\partial Y}{\partial X} = \frac{\partial Y}{\partial h_1} \cdot \frac{\partial h_1}{\partial X}$$

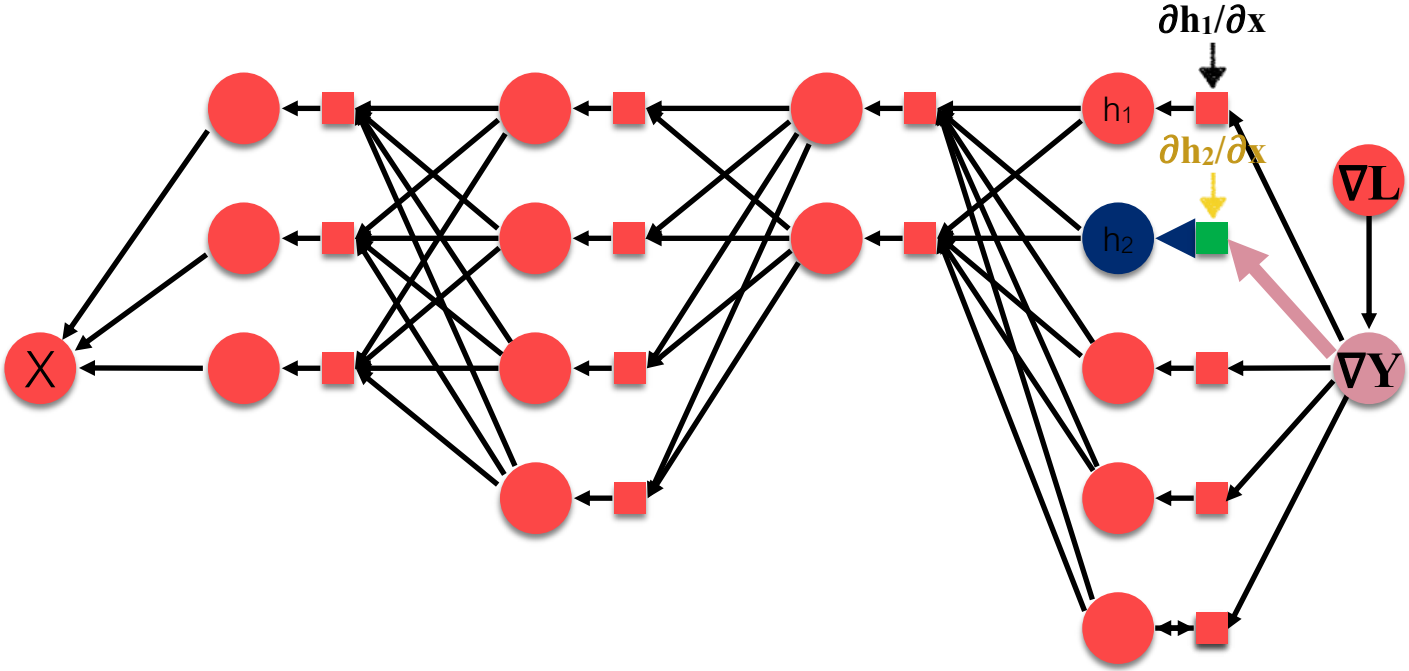
Back Propagation



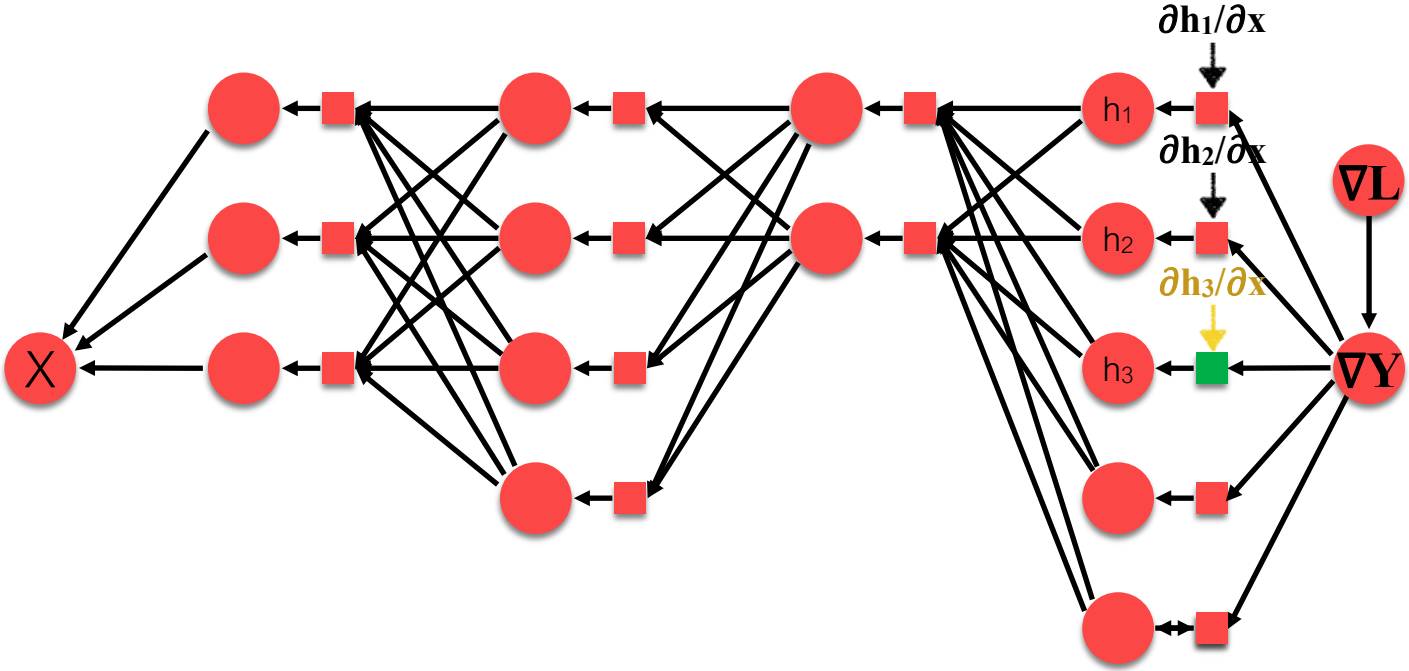
Back Propagation



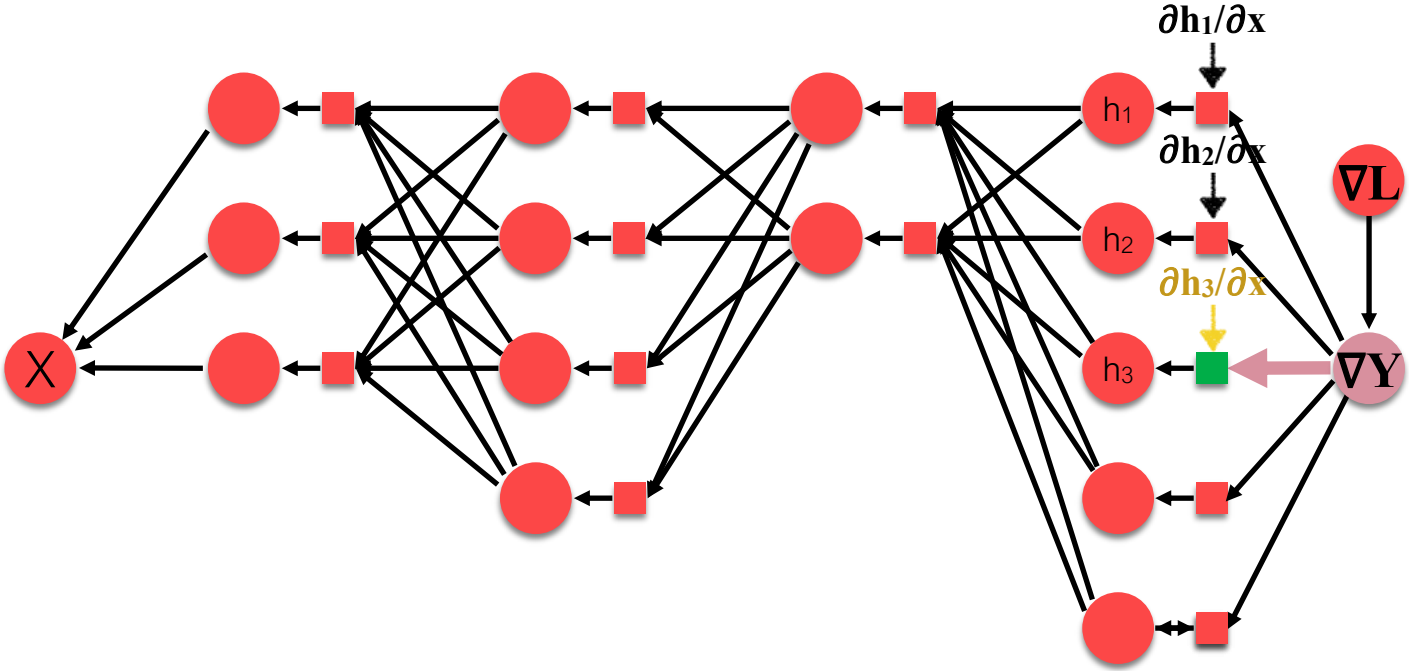
Back Propagation



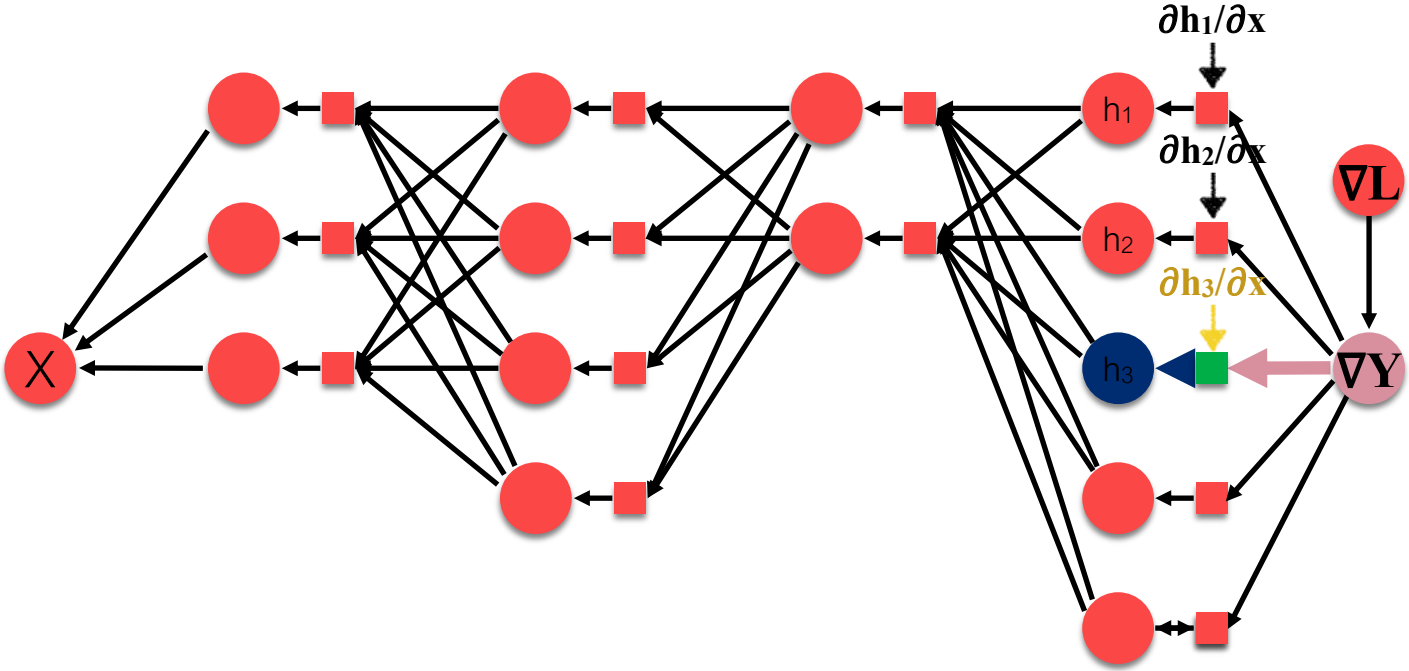
Back Propagation



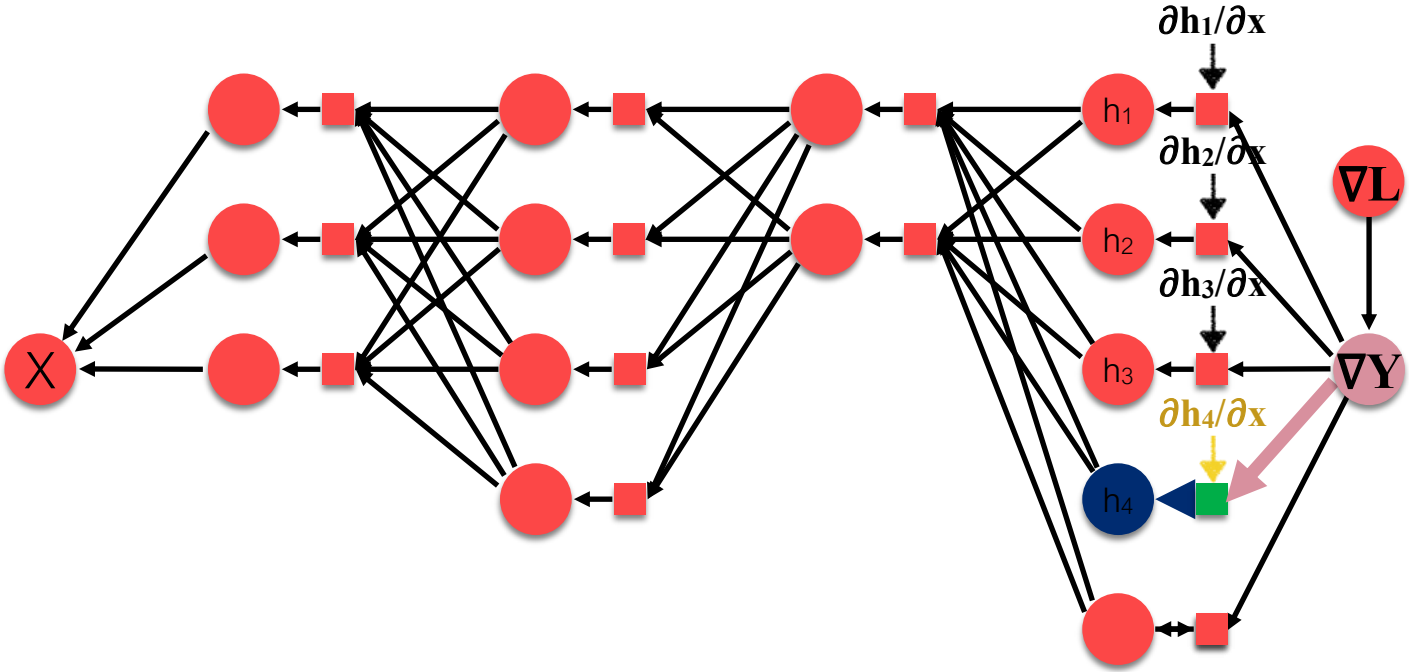
Back Propagation



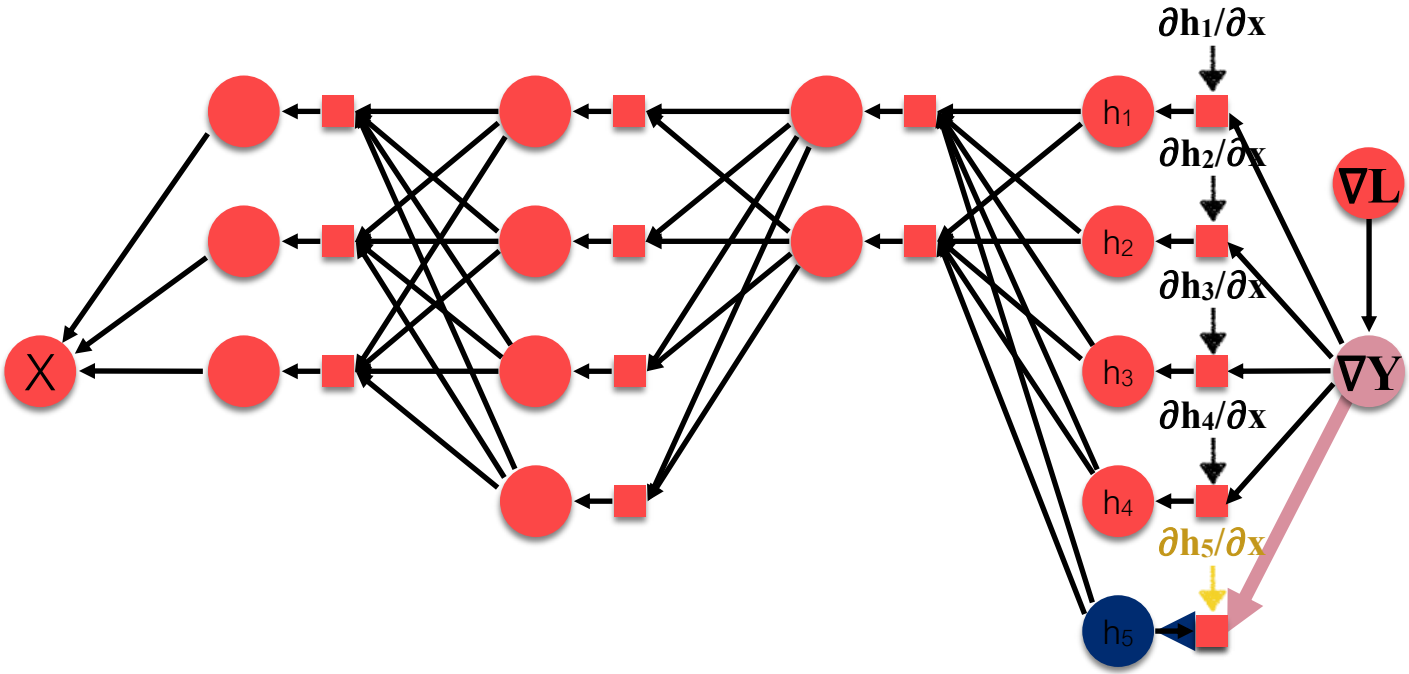
Back Propagation



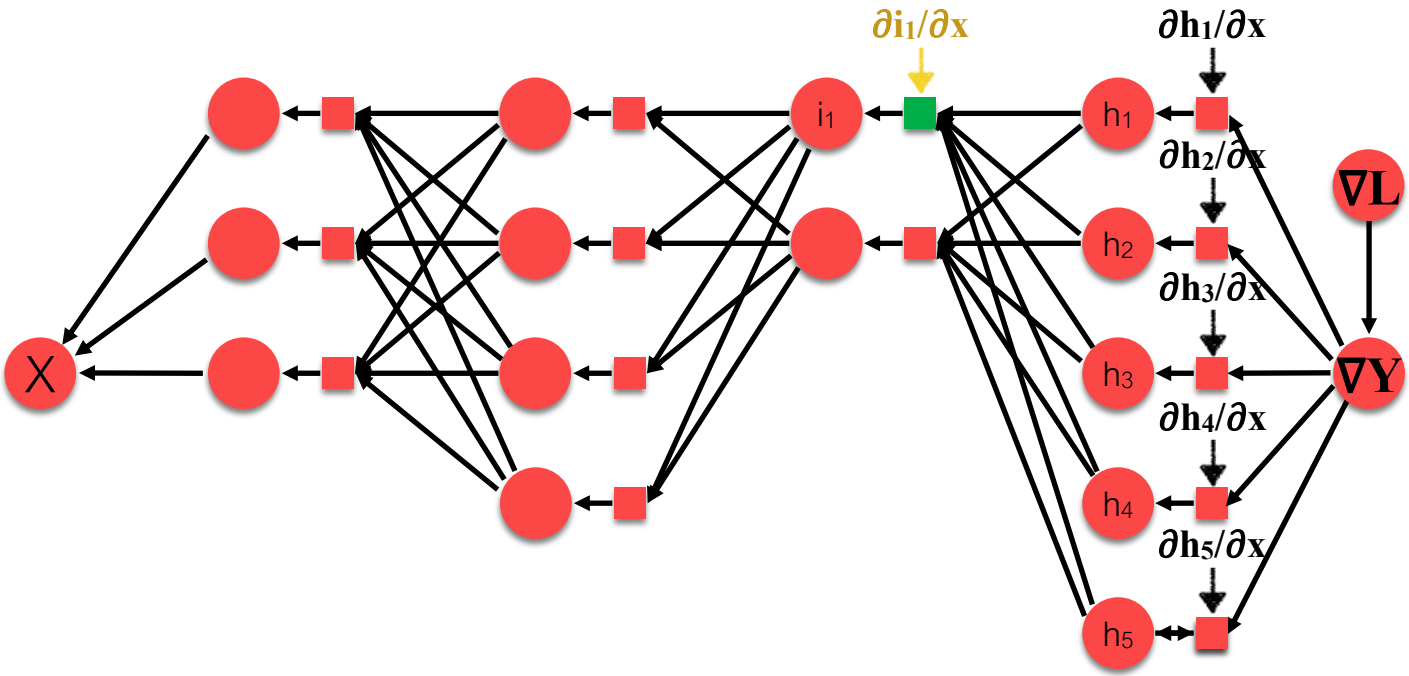
Back Propagation



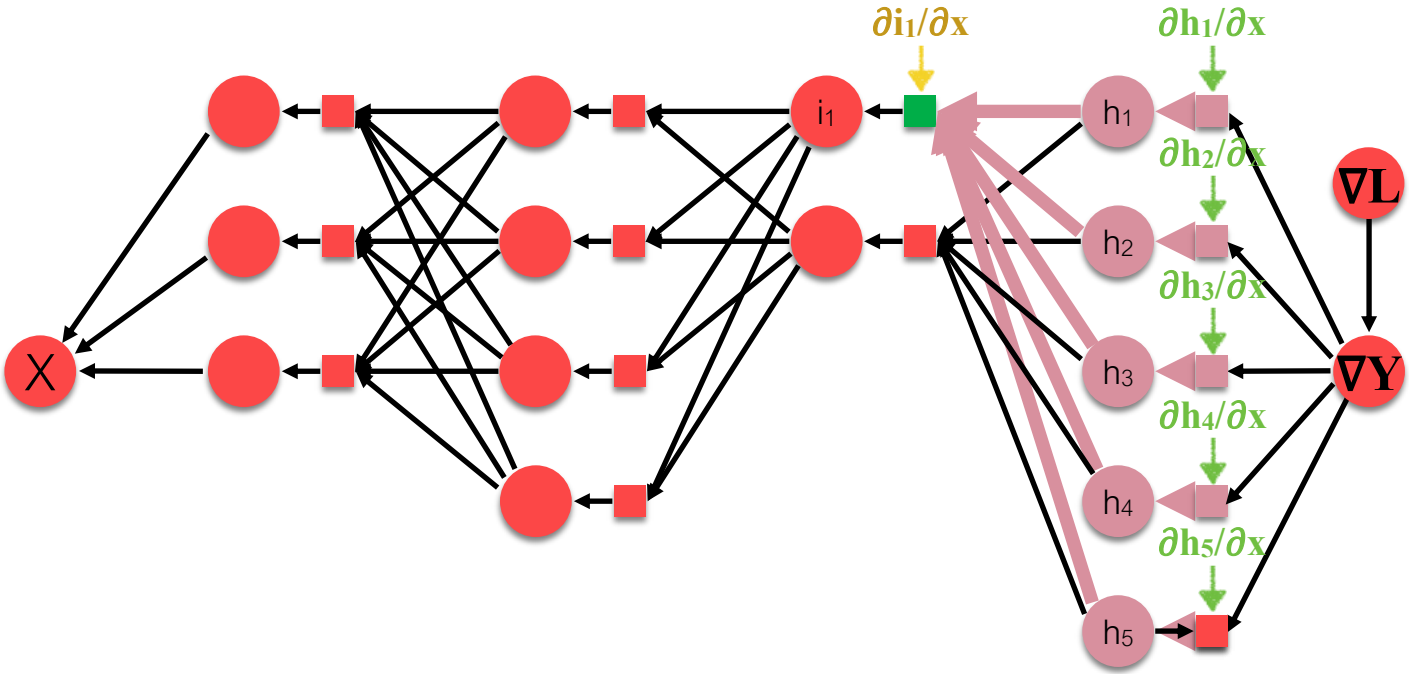
Back Propagation



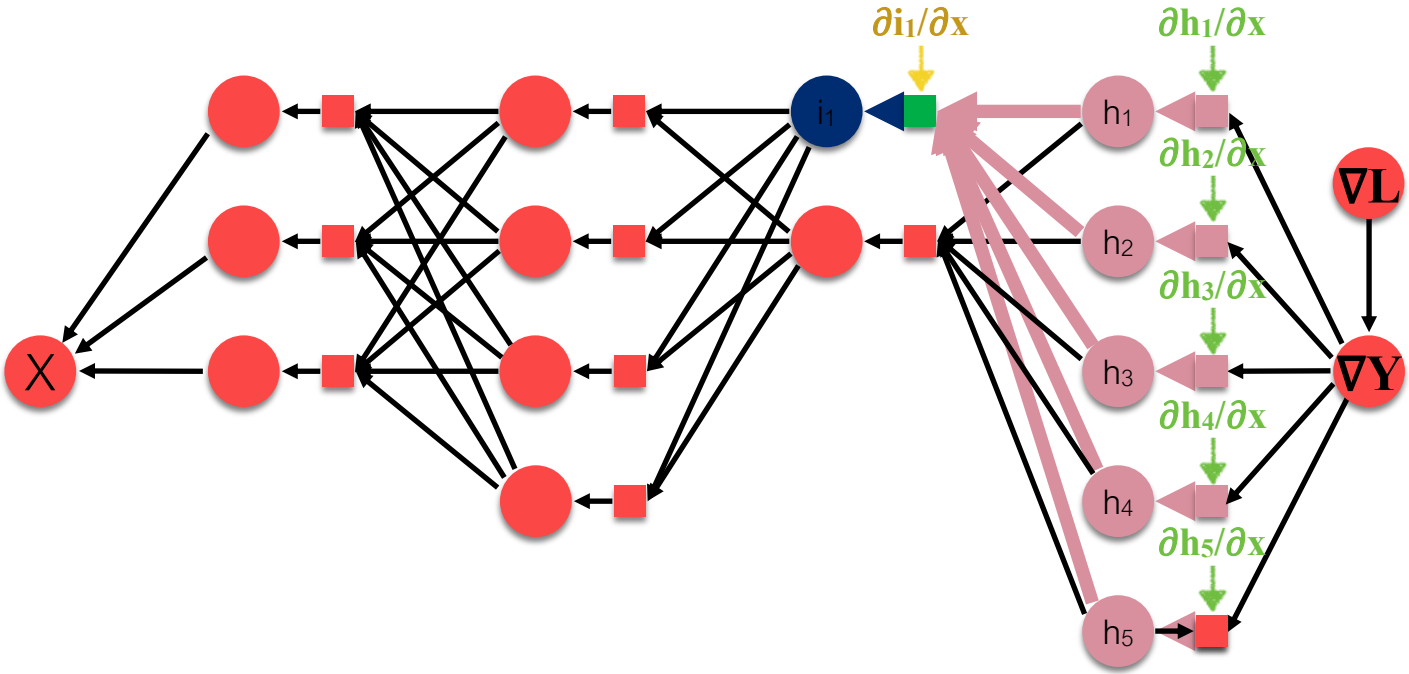
Back Propagation



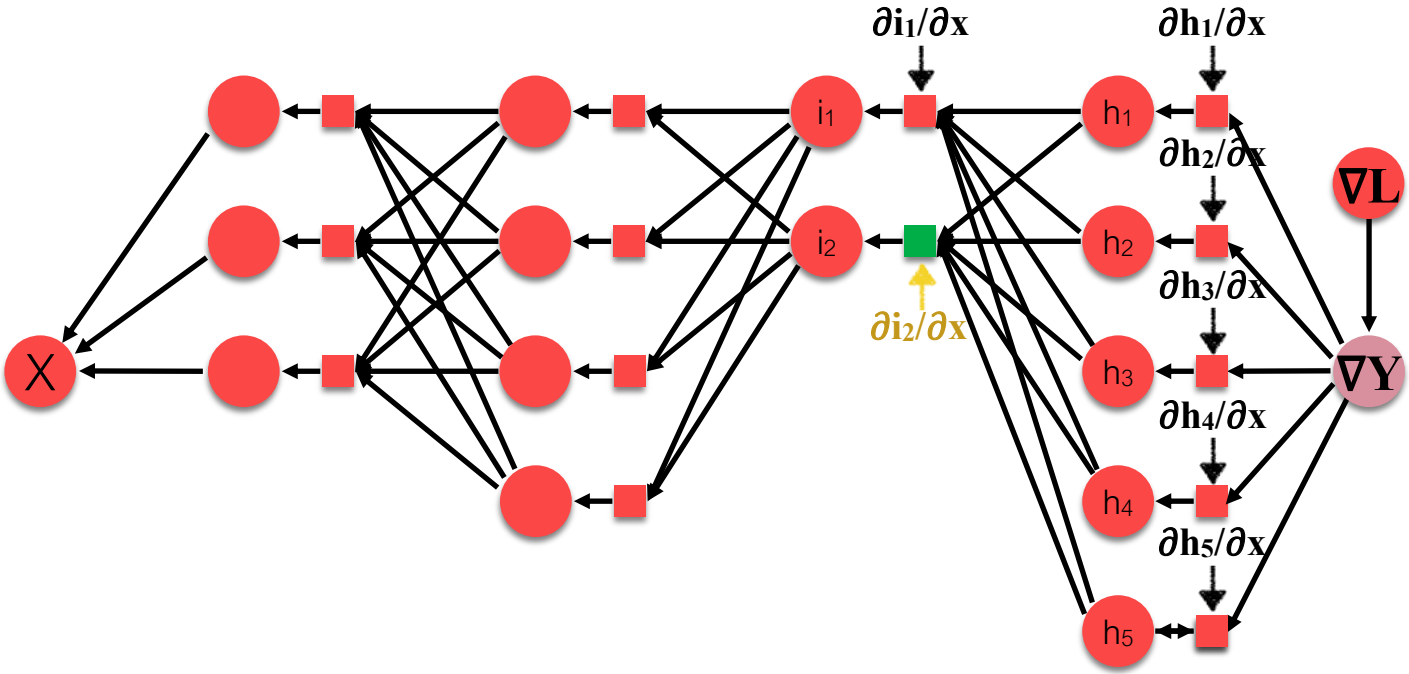
Back Propagation



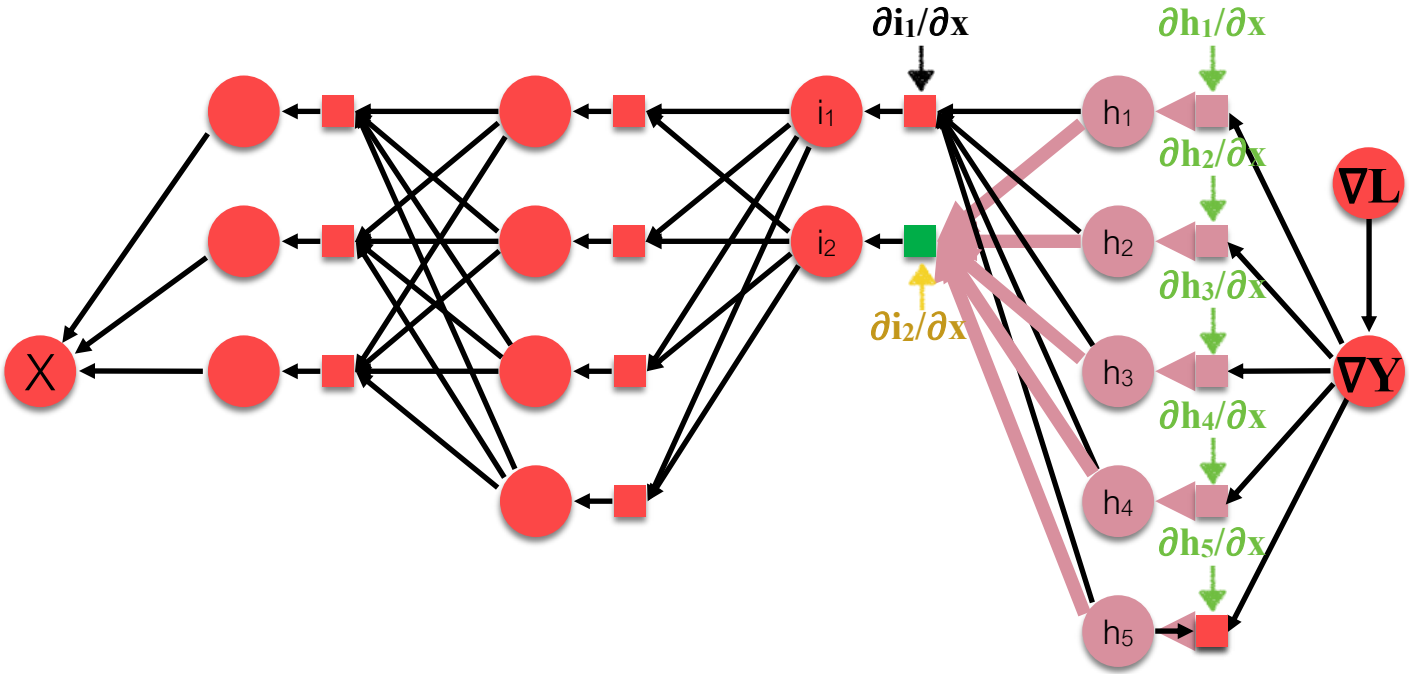
Back Propagation



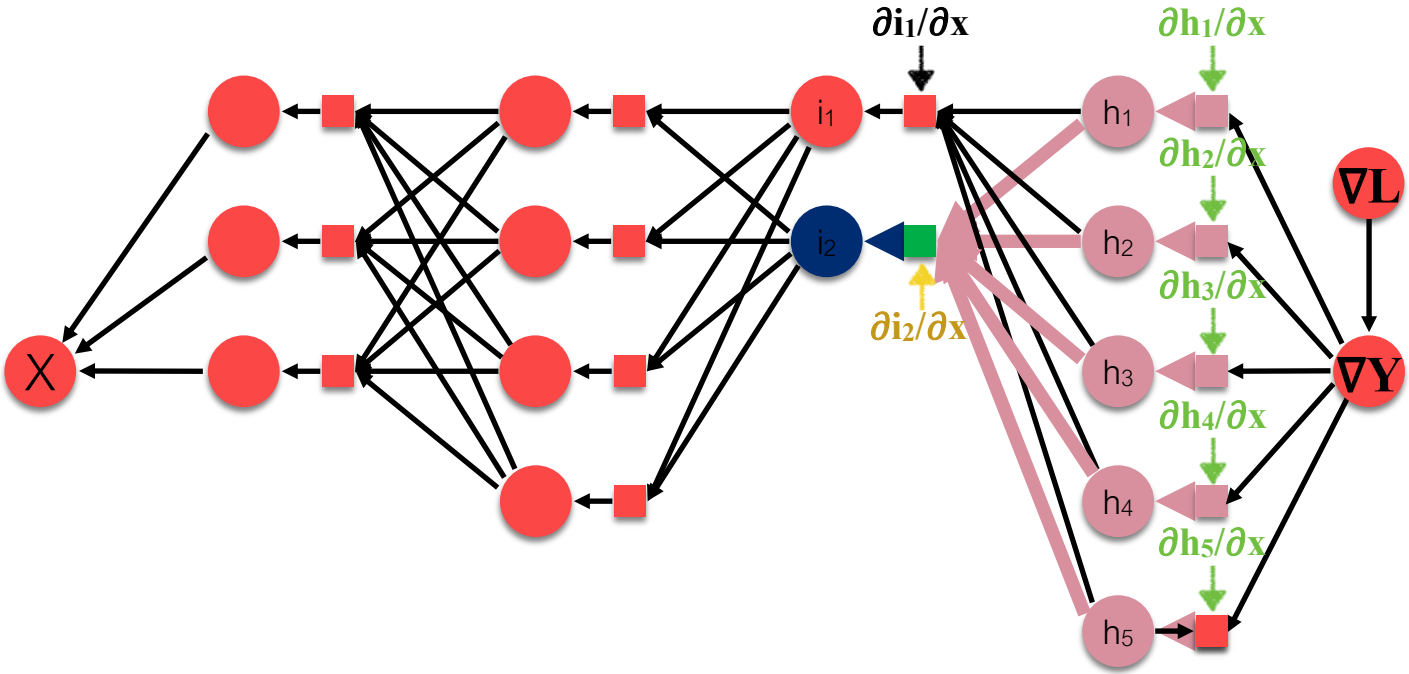
Back Propagation



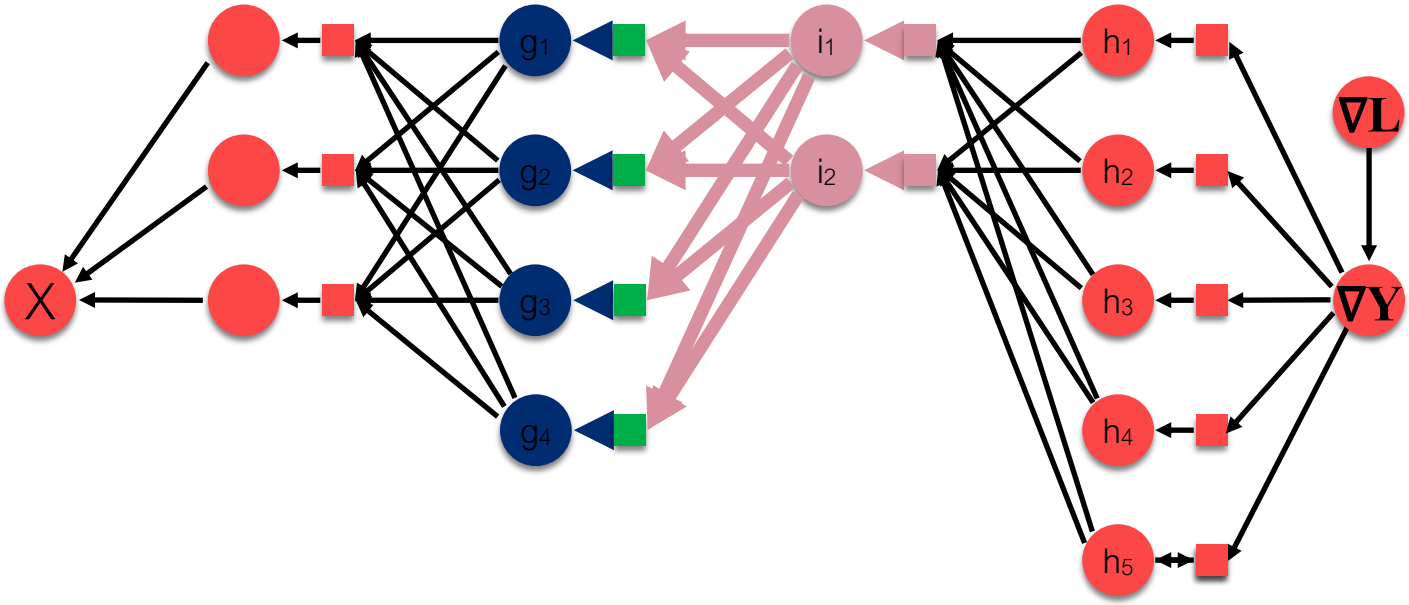
Back Propagation



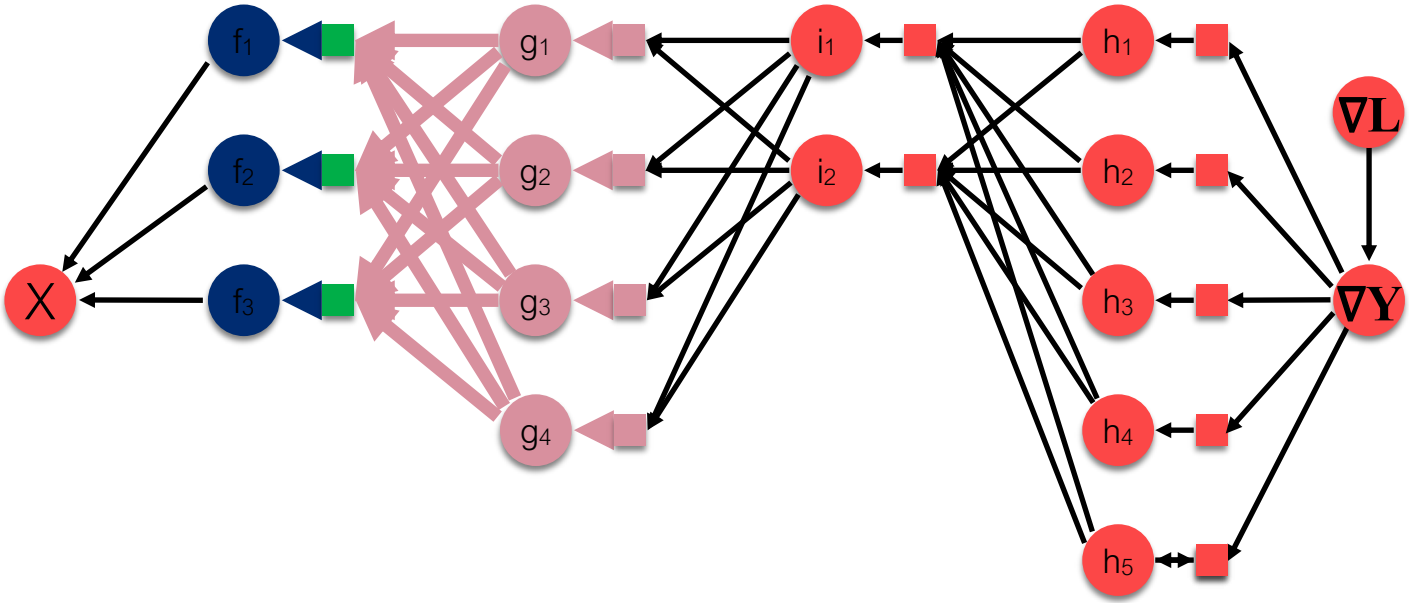
Back Propagation



Back Propagation



Back Propagation



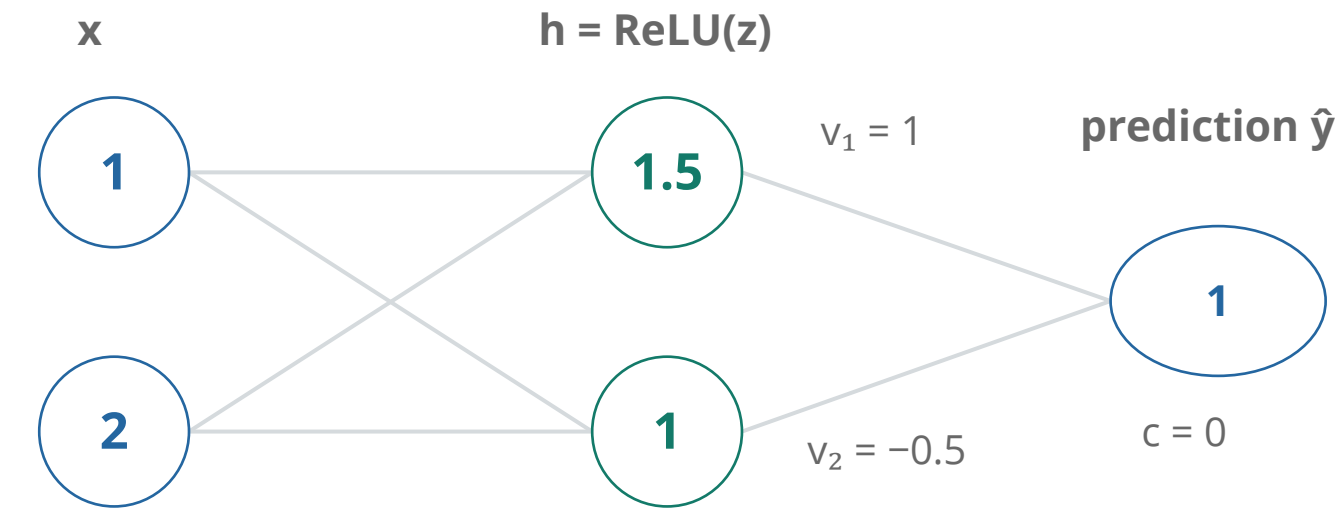
Back Propagation

- **(Fully-connected) Neural Networks** are stacks of linear functions and nonlinear activation functions; they have much more representational power than linear classifiers

- **backpropagation** = recursive application of the chain rule along a computational graph to compute the gradients of all inputs/ parameters/ intermediates

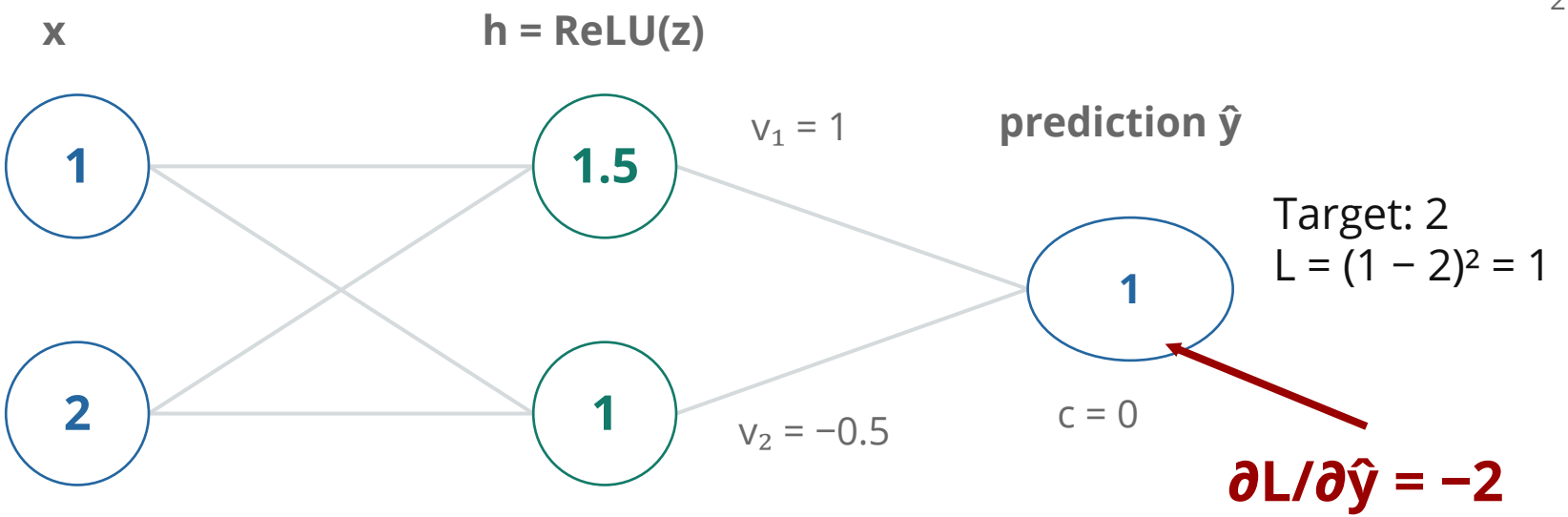
- **forward**: compute the result of an operation and save any intermediates needed for gradient computation in memory
- **backward**: apply the chain rule to compute the gradient of the loss function with respect to the inputs

Reset the Same Network



USC Viterbi School of Engineering

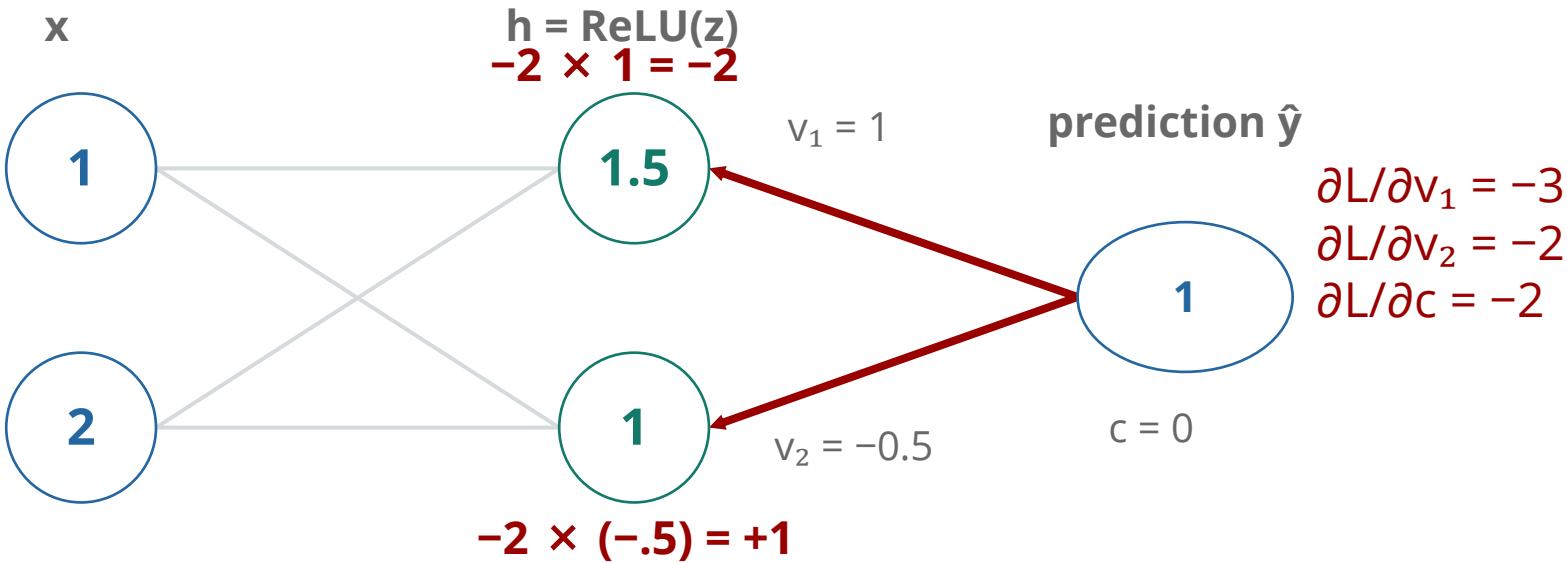
Start at the Prediction Error



Predict: should gradient descent raise or lower the output?

USC Viterbi School of Engineering

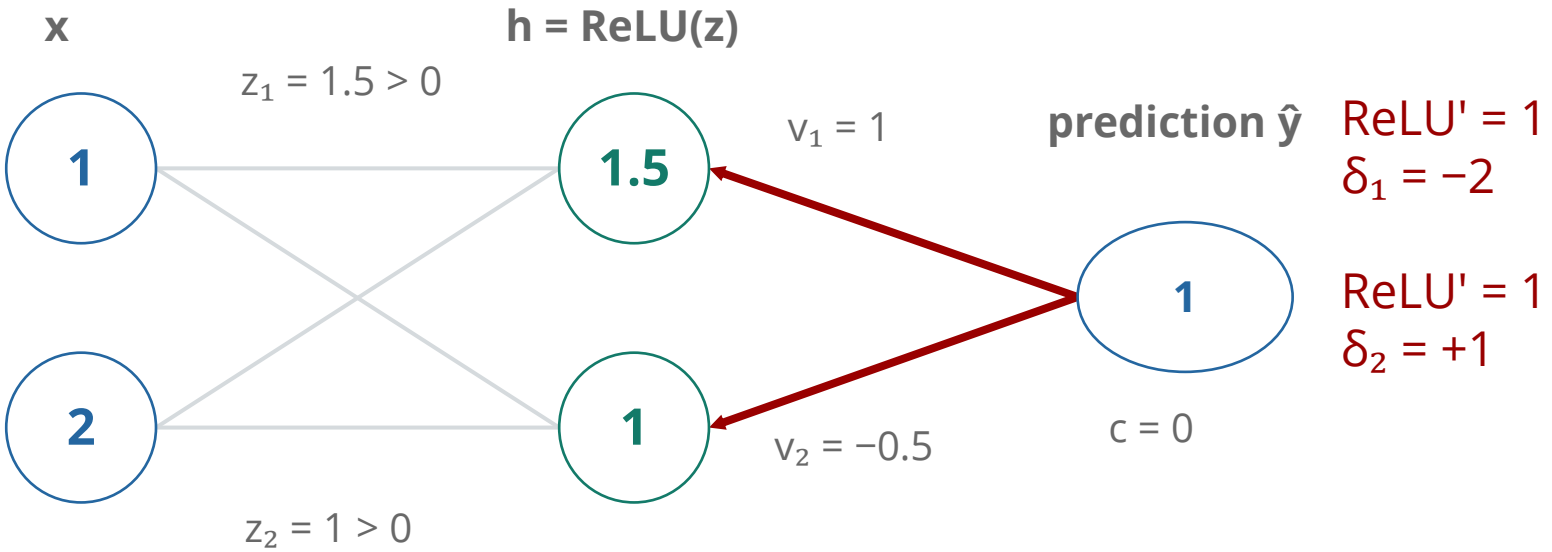
Backprop Through Both Output Edges



The negative output weight flips one hidden gradient.

USC Viterbi School of Engineering

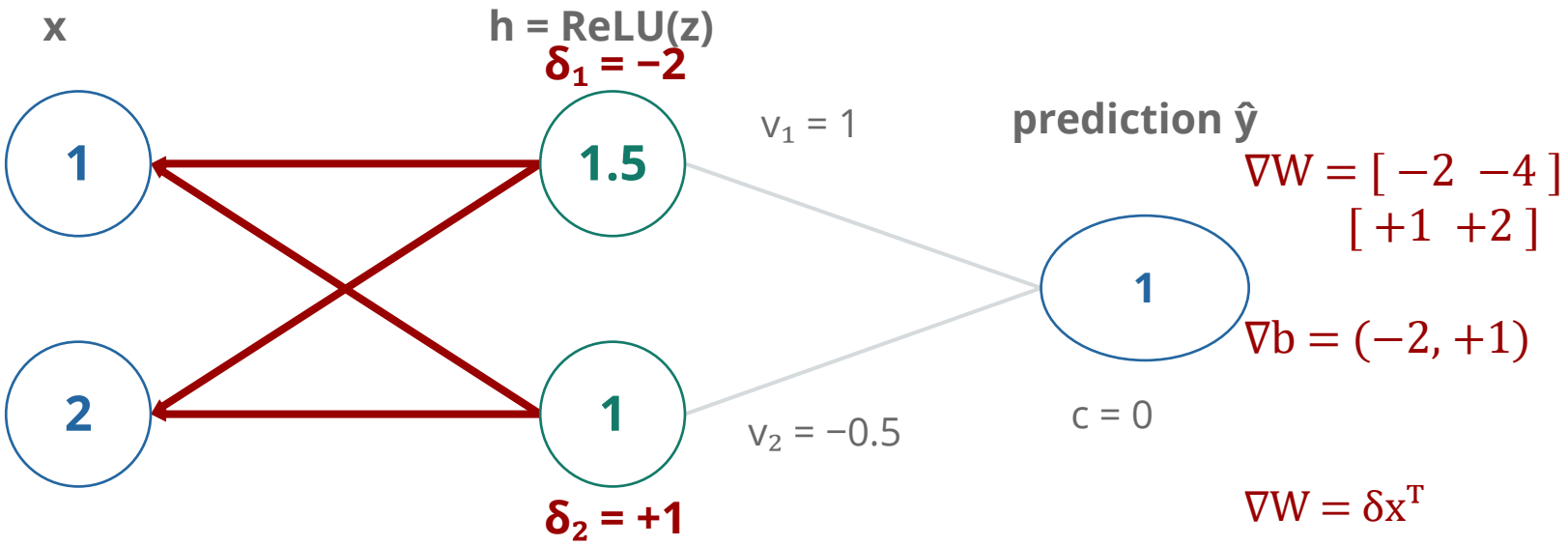
Pass Through the Active ReLUs



Predict: what changes if a pre-activation is negative?

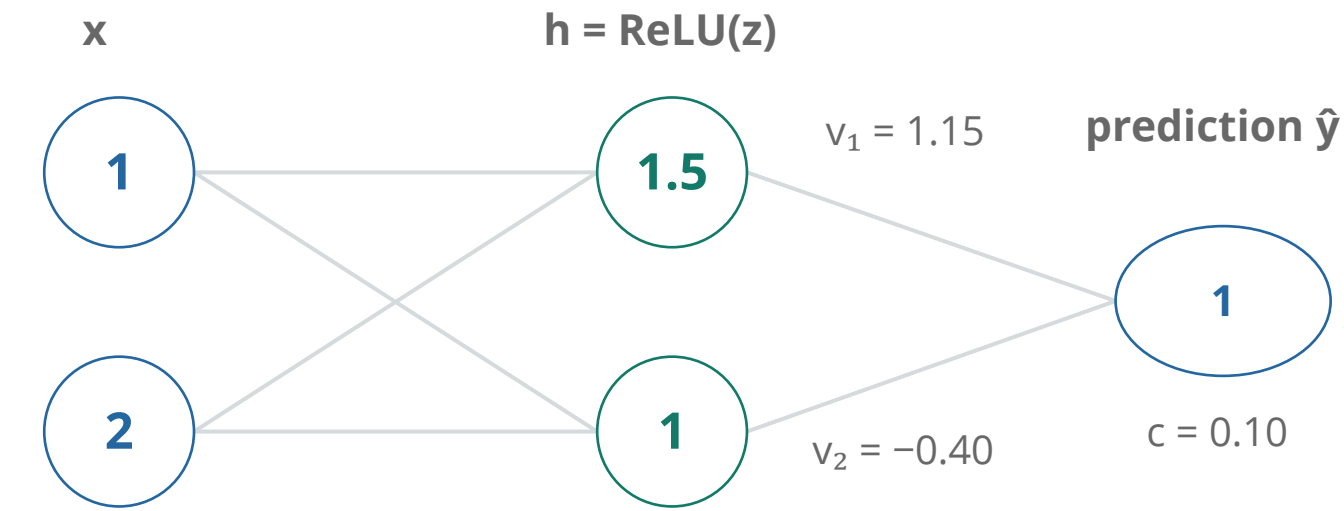
USC Viterbi School of Engineering

Each Weight Uses Its Own Input



Why is the second gradient column twice the first?

Update All Nine Parameters Together



$$W = \begin{bmatrix} 1.10 & 0.70 \\ -1.05 & 0.90 \end{bmatrix}$$

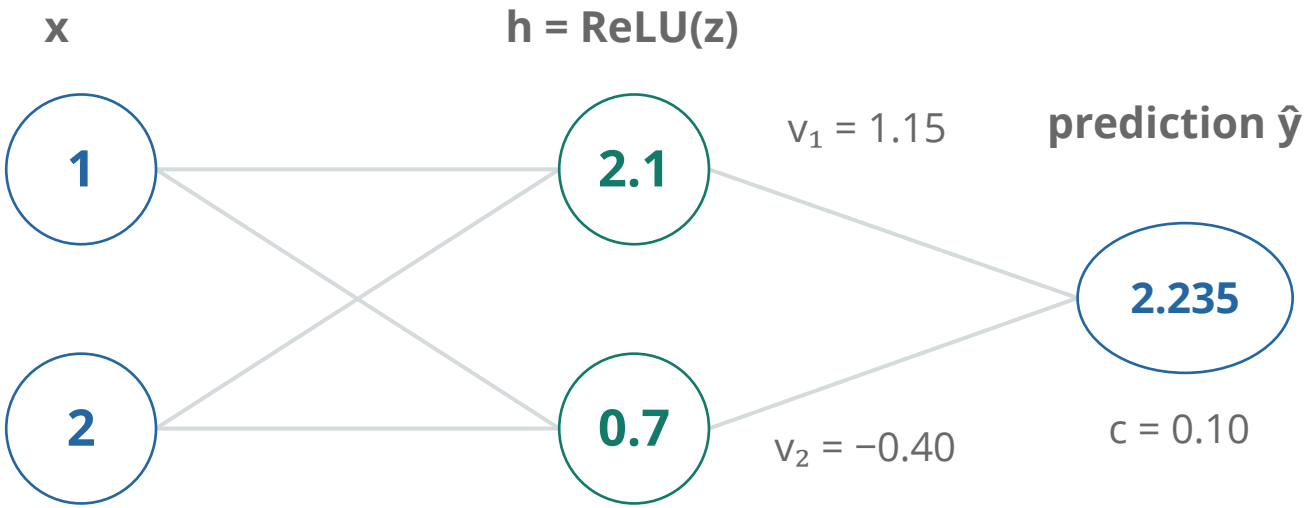
$$b = (-.40, -.05)$$

θnew = θold − η∇L

η = 0.05

Activations shown are still the old ones.

Run the Network Again



Target: 2
Loss before: 1
Loss after: 0.055225

Overshooting the target can still reduce the loss.

USC Viterbi School of Engineering

Why Backprop Fills GPU Memory

Forward. Each operation stores activations that the backward pass will later need.

Cost. That memory is held until the gradient arrives, so it grows with depth and batch size.

Checkpointing. Keep a few boundary activations and recompute the rest during the backward pass.

Evidence. Korthikanti et al. (MLSys 2023) measure this at scale; Adacc (arXiv:2508.00806) tunes it per tensor.

Less memory, more compute. Exact recomputation leaves the gradient unchanged.

Agent Context Can Change Without Backprop

Training. A loss sends gradients backward, and the optimizer step persists until a later update.

Agent run. Model calls in one run normally reuse the same fixed weights.

Adaptation. Instructions, tool results, and retrieved notes enter the next call as input. The ACE paper (ICLR 2026) draws this same line.

Caveat. A separate fine-tuning job can still update weights. That is a different loop.

Behavior can change from step to step while the parameters stay the same.

Frozen Base, Backprop Still Runs

Base. Freeze a learned weight matrix W. None of its entries receive an update.

Adapter. Replace Wx with (W + BA)x, where B and A are small. Only A and B are trained.

Backward. Backprop follows paths from the loss to A and B, including intervening frozen layers.

Memory. Fewer trainable weights shrink the optimizer state, but activation memory can stay large.

LoRA (ICLR 2022) narrows what you update. The backward pass runs either way.

Can Context Supply a Training Signal?

The	code	is	731	?

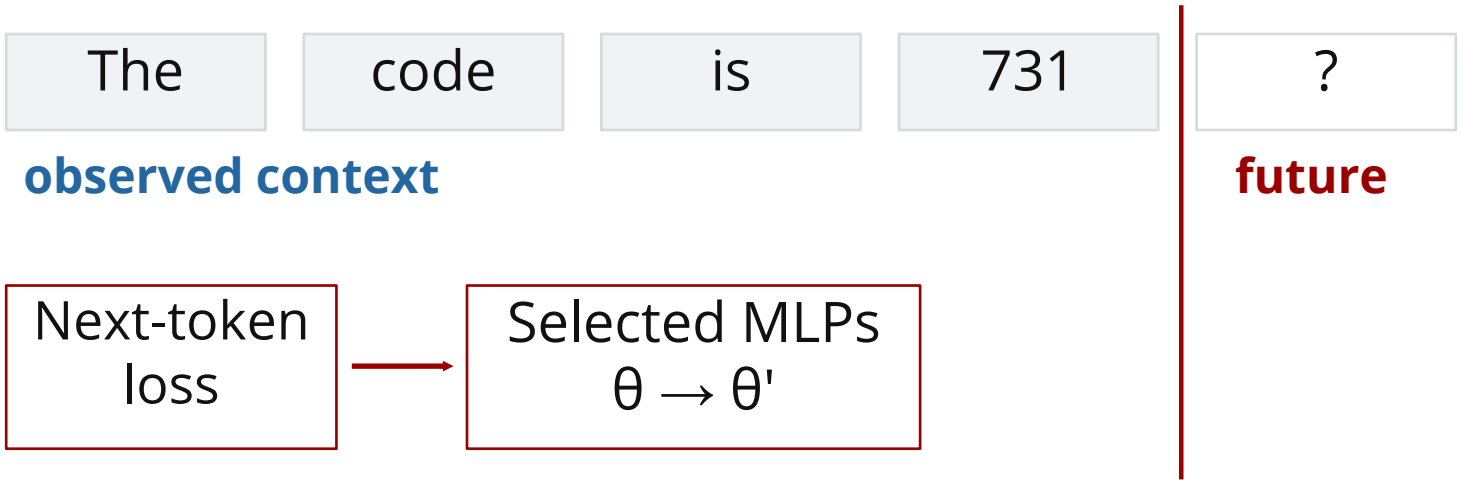
observed context **future**

Predict “731” from the past.
Target: already observed.

The past can provide next-token training examples.

Tandon et al. (2025), End-to-End Test-Time Training for Long Context, arXiv:2512.23675.

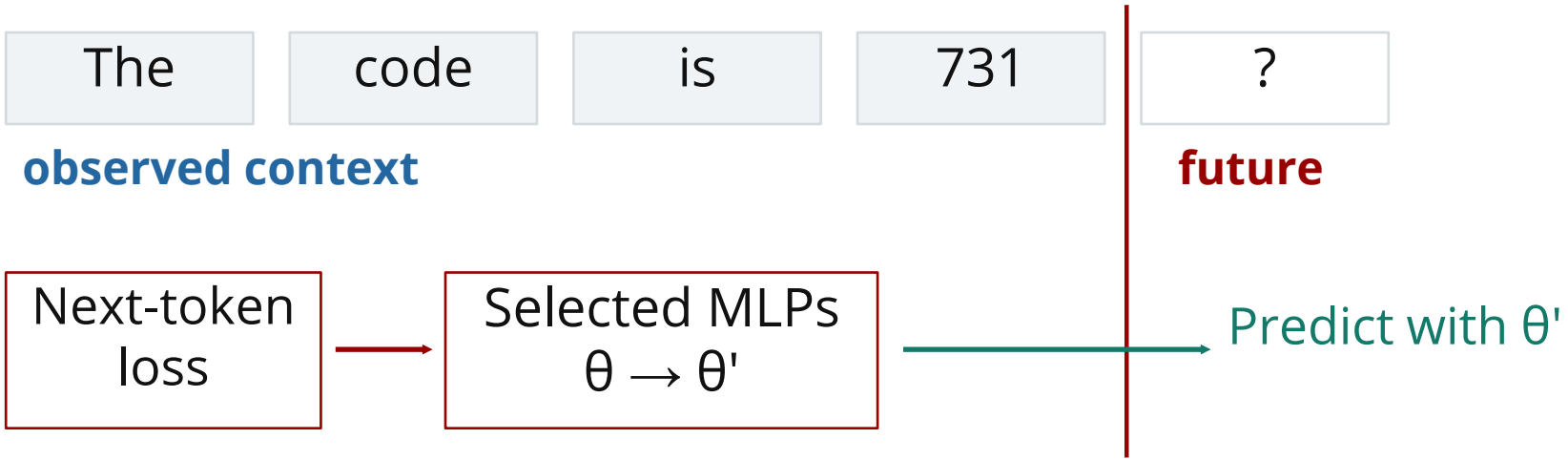
Backprop at Test Time



Outer training prepares an adaptable initialization.

Tandon et al. (2025), End-to-End Test-Time Training for Long Context, arXiv:2512.23675.

Predict Without Reading Future Targets



No future ground truth enters the adaptation loss.

Tandon et al. (2025), End-to-End Test-Time Training for Long Context, arXiv:2512.23675.

An Agent Can Remember Without Training

Tool result
length: 200 cm

→ Fixed model θ →

200 m ?

User correction
→ save feedback

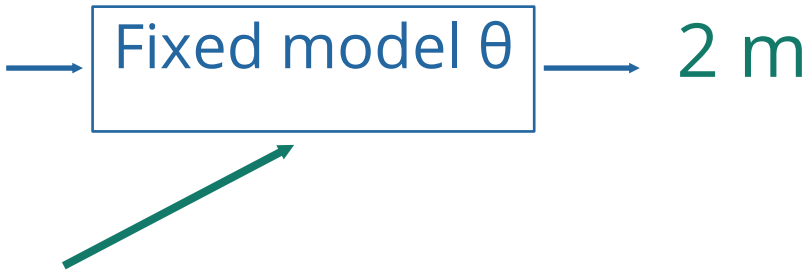
Memory record
Tool returns cm. Divide by 100 for m.

A file was written. Did any weight receive a gradient?

USC Viterbi School of Engineering

Retrieve the Record on the Next Attempt

Tool result
length: 200 cm



Memory record
Tool returns cm. Divide by 100 for m.

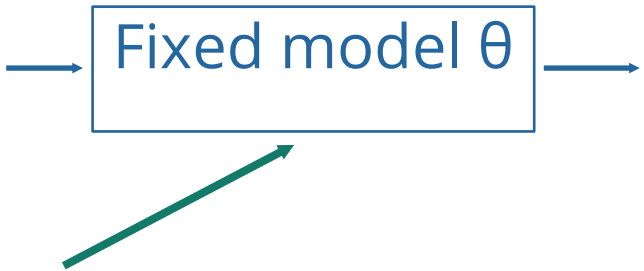
Retrieve into context

Context changed; θ did not.

USC Viterbi School of Engineering

Locate the State That Actually Changed

Tool result
length: 200 cm



Memory record
Tool returns cm. Divide by 100 for m.

This memory example keeps θ fixed.

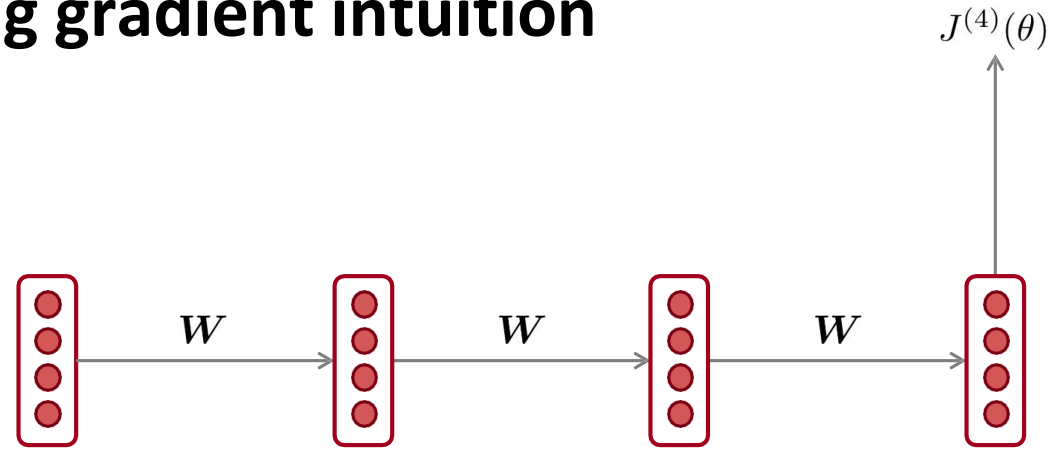
What changes?

Here: context tokens
θ stays fixed

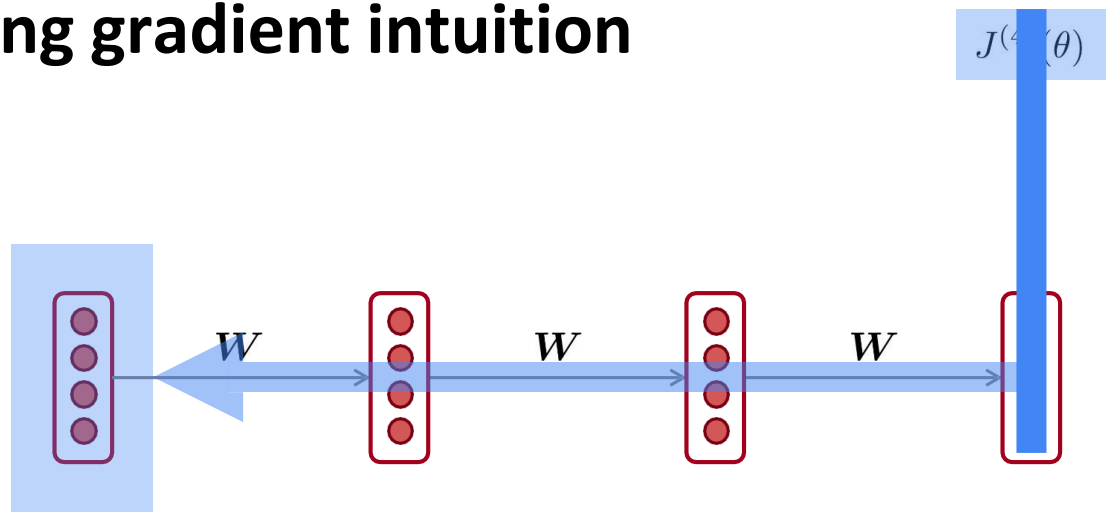
LoRA: adapter A, B
base W stays fixed

TTT: selected weights
$\theta \rightarrow \theta'$

Vanishing gradient intuition

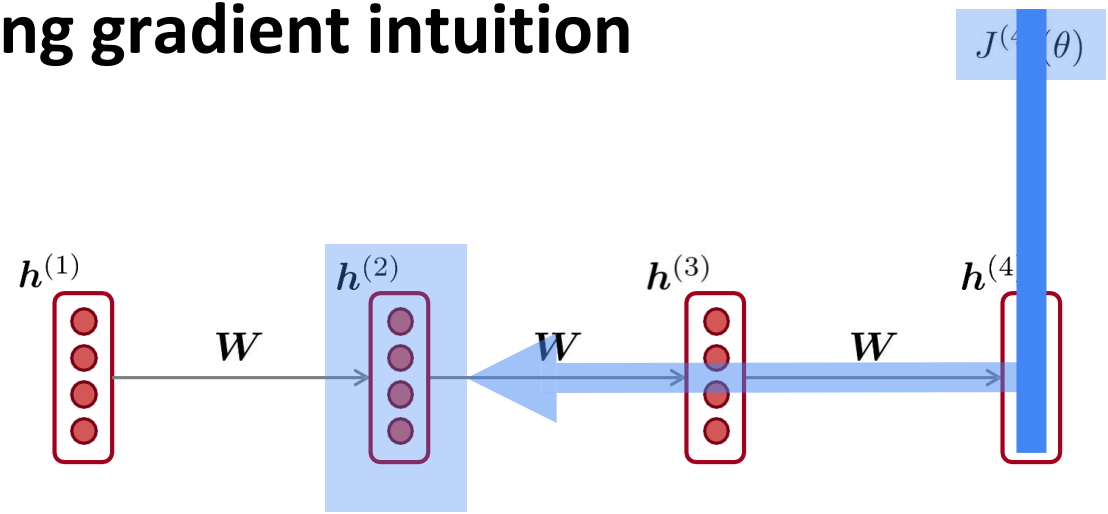


Vanishing gradient intuition



The vanishing gradient can occur in any type of deep neural network. However, it is more prominently discussed in the architecture with sequential data.

Vanishing gradient intuition

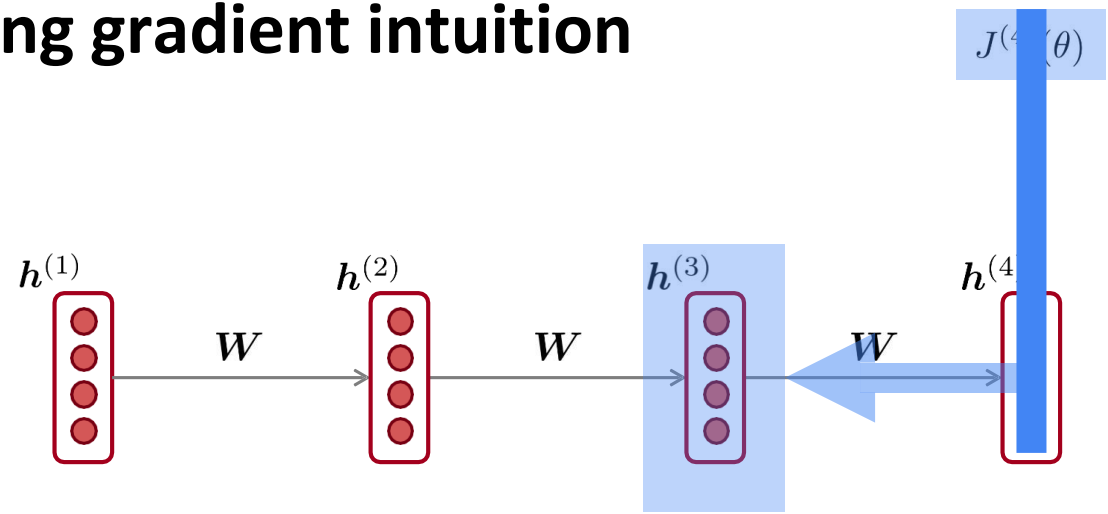


$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial J^{(4)}}{\partial h^{(2)}}$$

chain rule!

USC Viterbi School of Engineering

Vanishing gradient intuition

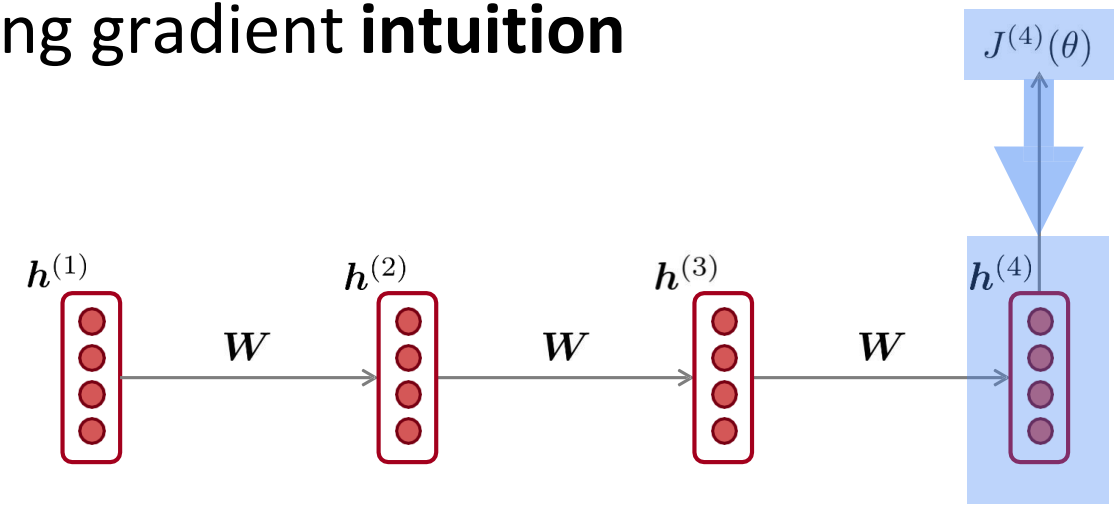


$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial J^{(4)}}{\partial h^{(3)}}$$

chain rule!

USC Viterbi School of Engineering

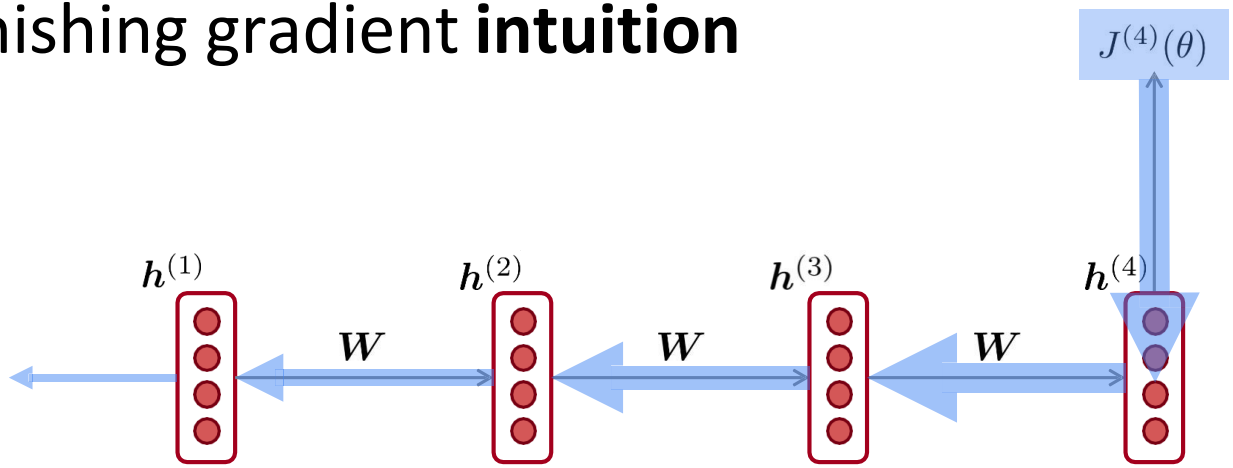
Vanishing gradient **intuition**



chain rule!

USC Viterbi School of Engineering

Vanishing gradient **intuition**



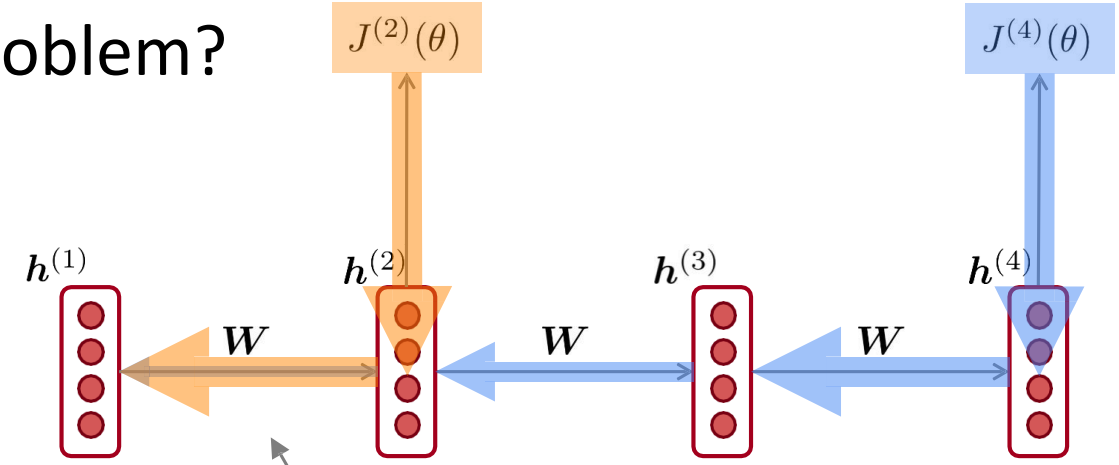
$$\frac{\partial J^{(4)}}{\partial h^{(1)}} = \frac{\partial h^{(2)}}{\partial h^{(1)}} \times \frac{\partial h^{(3)}}{\partial h^{(2)}} \times \frac{\partial h^{(4)}}{\partial h^{(3)}} \times \frac{\partial J^{(4)}}{\partial h^{(4)}}$$

What happens if these are small?

Vanishing gradient problem: when these are small, the gradient shrinks at every step.

USC Viterbi School of Engineering

Why is vanishing gradient **a** problem?



Gradient signal from faraway is lost because it's much smaller than gradient signal from close-by.

So model weights are only updated only with respect to near effects, not long-term effects.

Why is vanishing gradient **a** problem?

- Another explanation: Gradient can be viewed as a measure of *the effect of the past on the future*

- If the gradient becomes vanishingly small over longer distances (step *t* to step *t+n*), then we can't tell whether:
  1. There's no dependency between step *t* and *t+n* in the data
  2. We have wrong parameters to capture the true dependency between *t* and *t+n*

Fixing Vanishing Gradients

Use of Appropriate Activation Functions:

- ReLU and its Variants: ReLU (Rectified Linear Unit) is less susceptible to the vanishing gradient problem because it does not saturate in the positive region. Variants like Leaky ReLU, Parametric ReLU (PReLU), or Exponential Linear Unit (ELU) can also be used to avoid dead neurons in ReLU.

Batch Normalization (will discuss later – the idea is to fix the range):

- During training this standardizes each channel using mini-batch statistics, then applies a learned scale and shift, so the output need not have mean 0 and variance 1. It often stabilizes optimization; reducing internal covariate shift was the original motivation, not an established explanation.

Residual Paths Add an Identity Route

Activation. ReLU variants avoid positive-side saturation.

Normalization. Keeping signals in range helps gradients stay usable.

Residual. He et al. (CVPR 2016) use $y=x+f(x)$: $dy/dx=I+J_f$. They target degradation; the identity route for gradients follows.

2026. Residual paths plus normalization are common in deep models; Hyper-Connections (ICLR 2025) shows the path is being redesigned. Week 5 opens f.

Three fixes: avoid saturation, control scale, and preserve an identity path for the gradient.

Why is exploding gradient a problem?

- If the gradient becomes too big, then the SGD update step becomes too big:

$$\theta^{new} = \theta^{old} - \underbrace{\overbrace{\alpha}^{\text{learning rate}} \nabla_\theta J(\theta)}_{\text{gradient}}$$

- This can cause bad updates: we take too large a step and reach a bad parameter configuration (with large loss)

- In the worst case, this will result in Inf or NaN in your network (then you have to restart training from an earlier checkpoint)

Gradient clipping: **solution** for exploding gradient

- Gradient clipping: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

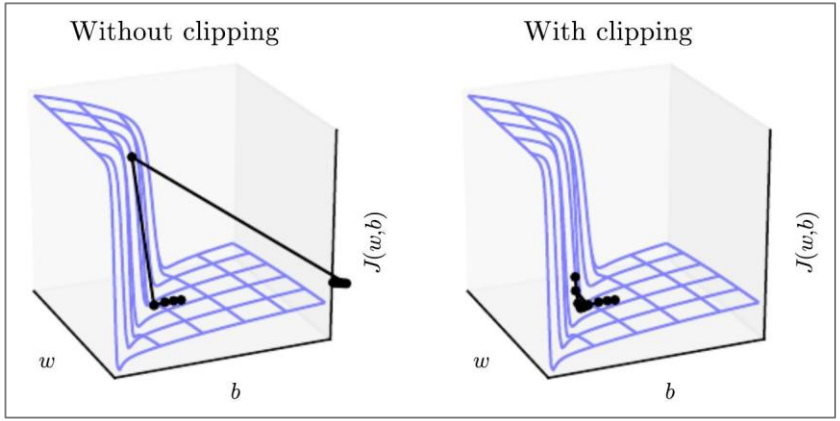
Algorithm 1 Pseudo-code for norm clipping

$\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$
if $\|\hat{\mathbf{g}}\| \geq threshold$ **then**
  $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$
end if

- Intuition: take a step in the same direction, but a smaller step

Source: "On the difficulty of training recurrent neural networks", Pascanu et al, 2013. http://proceedings.mlr.press/v28/pascanu13.pdf

Gradient clipping: **solution** for exploding gradient



- This shows the loss surface of a simple RNN (hidden state is a scalar not a vector)
- The "cliff" is dangerous because it has steep gradient
- On the left, gradient descent takes two very big steps due to steep gradient, resulting in climbing the cliff then shooting off to the right (both bad updates)
- On the right, gradient clipping reduces the size of those steps, so effect is less drastic

Source: "Deep Learning", Goodfellow, Bengio and Courville, 2016. Chapter 10.11.1. https://www.deeplearningbook.org/contents/rnn.html

Clipping Is Training Triage

Signal. Log the gradient norm beside the loss. A norm spike often precedes a loss jump.

Clip. Rescale the gradient before the optimizer step. The direction survives; the step length does not.

Escalate. Depending on the system: skip a batch, restore a checkpoint, or check data and learning rate.

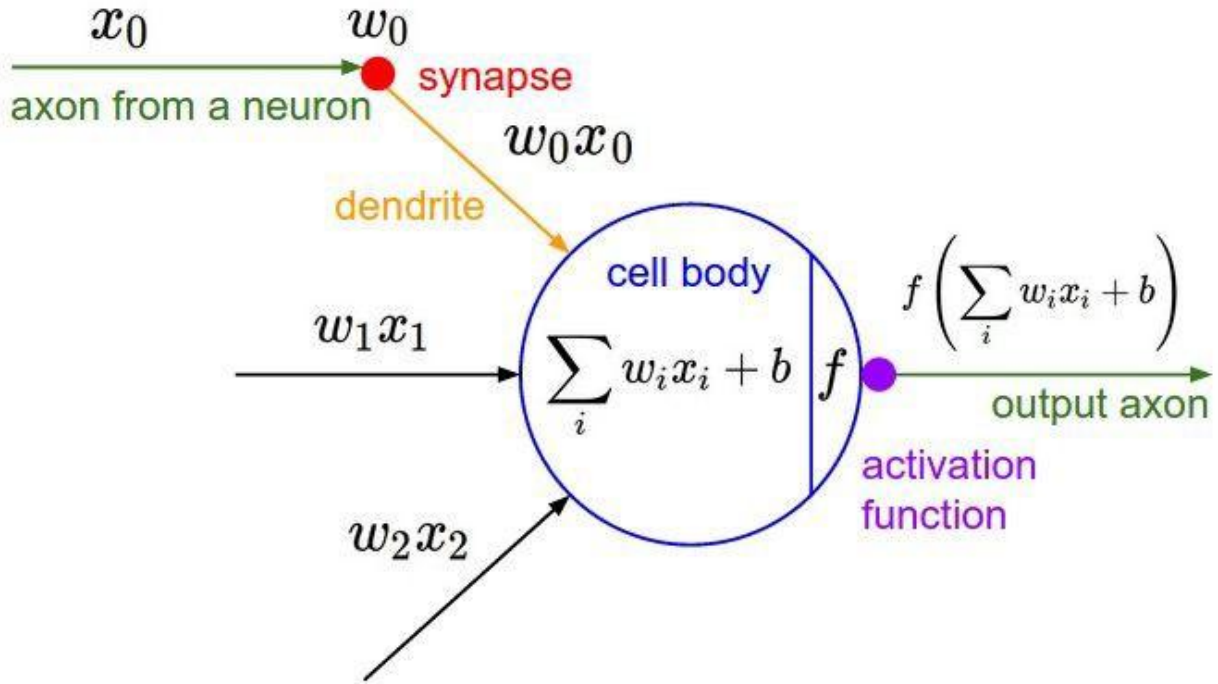
Practice. OLMo (ACL 2024) publishes its clipping setting; ZClip (arXiv:2504.02507) treats spikes as routine.

Clipping limits the damage of one step. Finding the cause is a separate job.

Training Neural Networks

Activation Functions

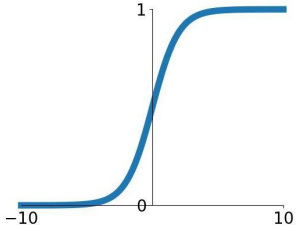
Activation Functions



Activation Functions

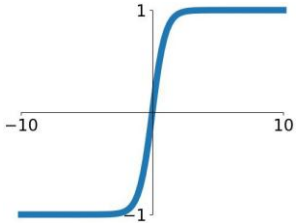
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



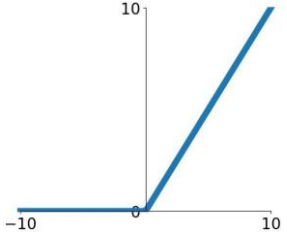
tanh

$$\tanh(x)$$



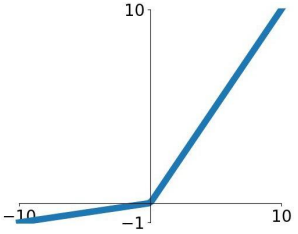
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

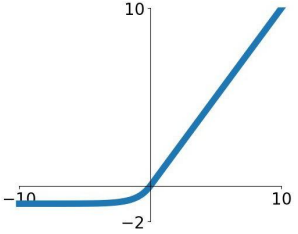


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

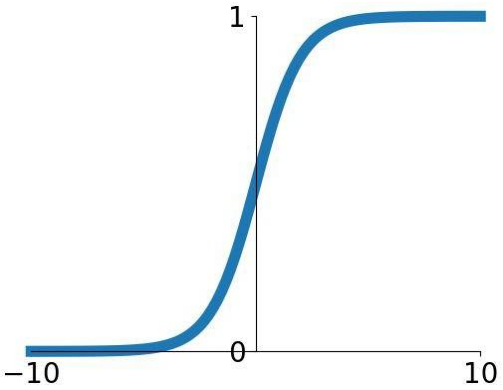
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$

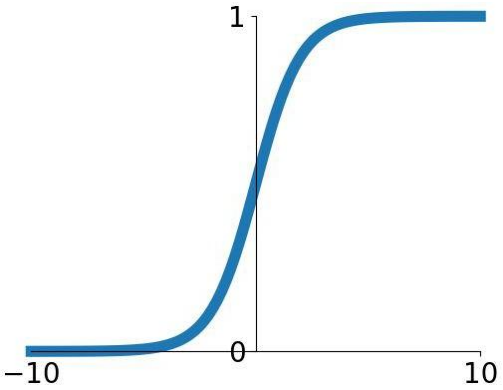


Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating "firing rate" of a neuron

Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$

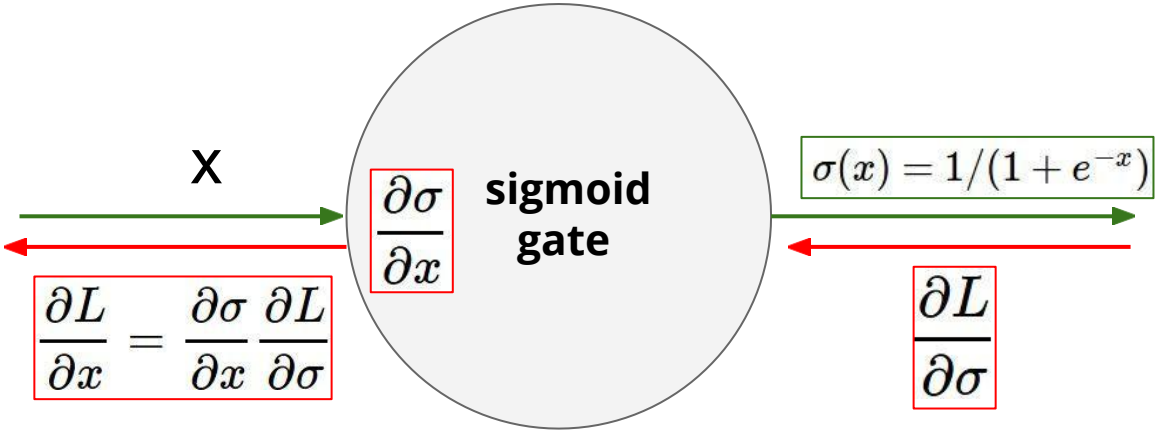


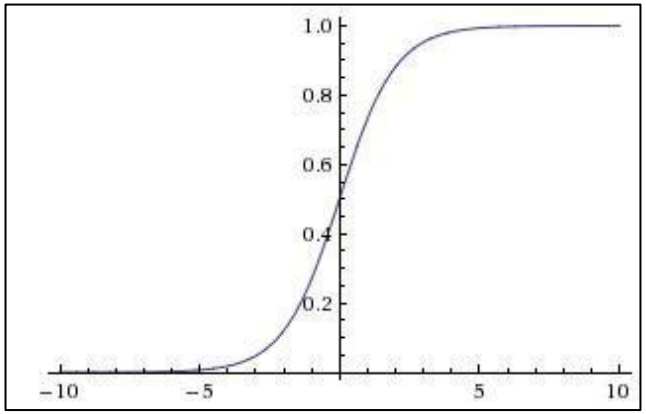
Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating "firing rate" of a neuron

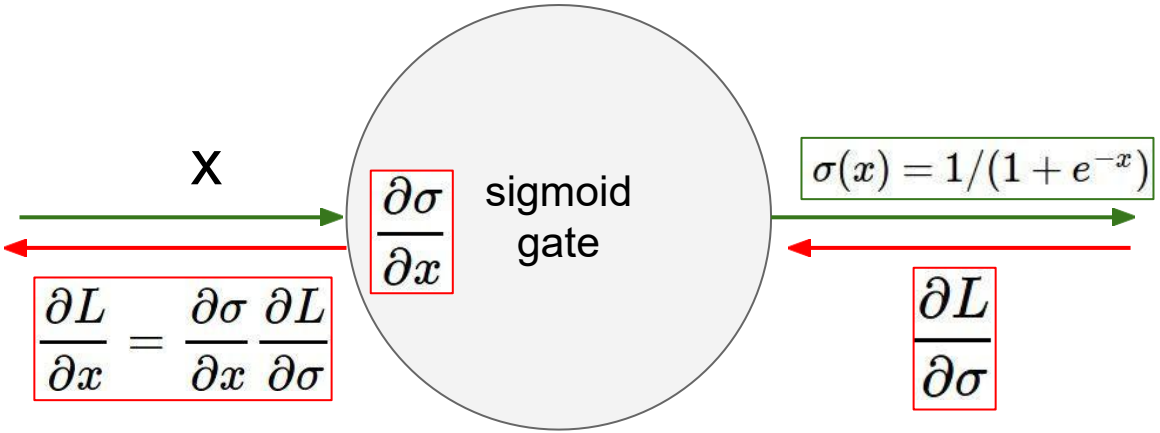
3 problems:

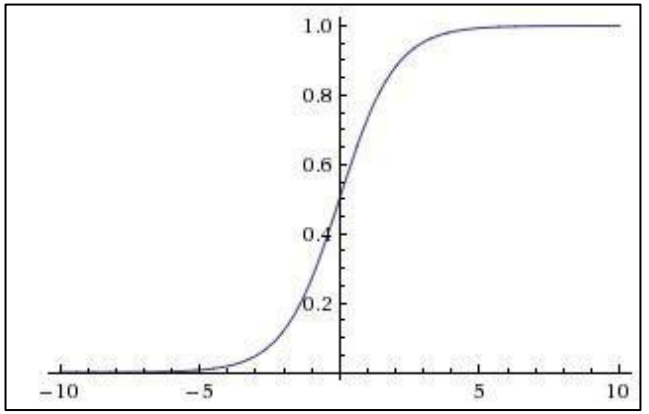
1. Saturated neurons shrink the gradients





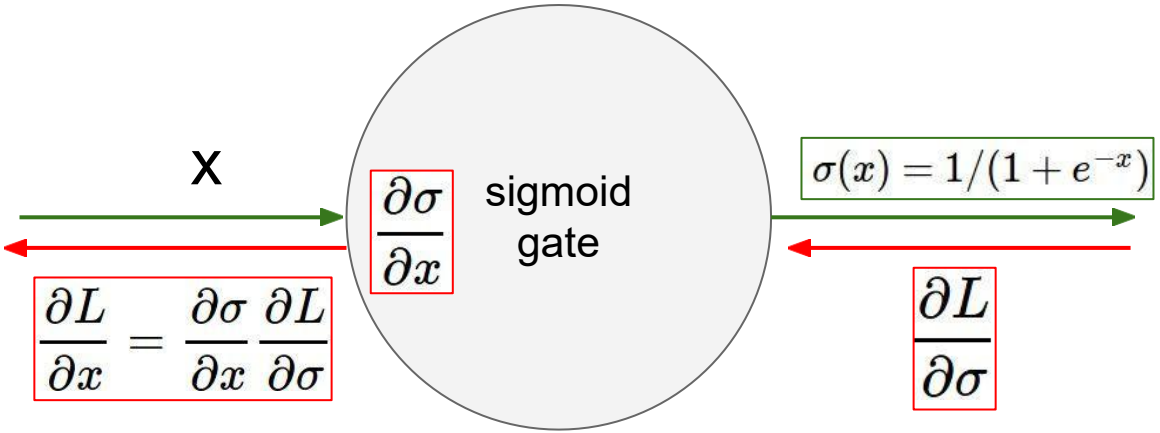
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

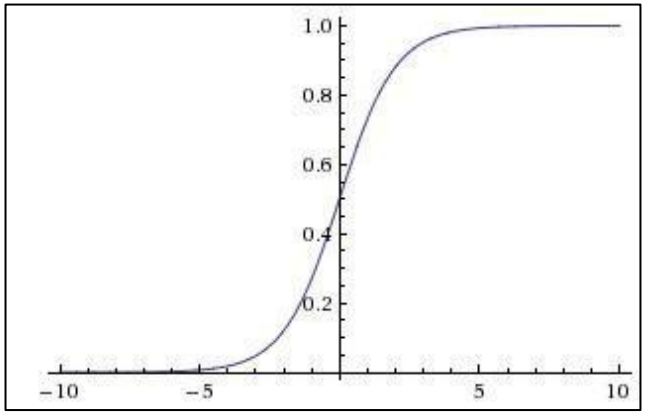




What happens when x = -10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$



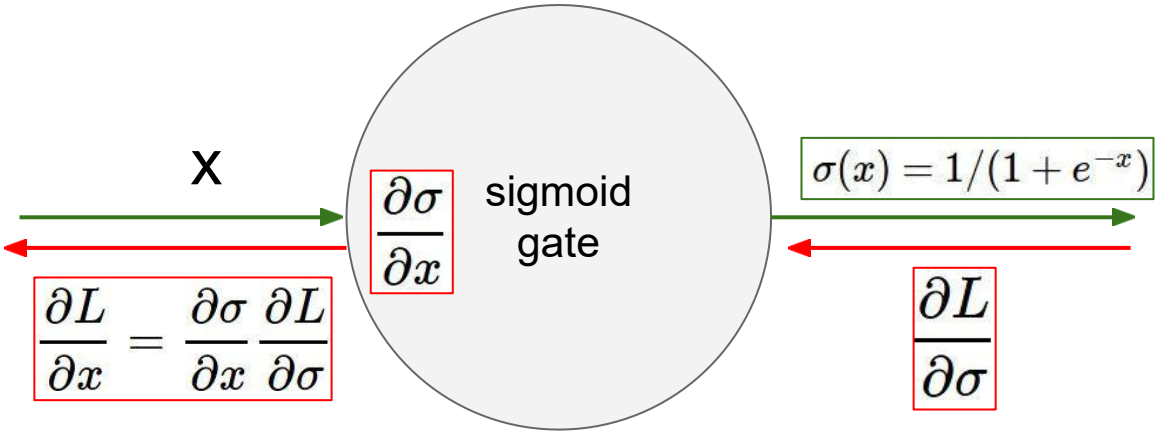


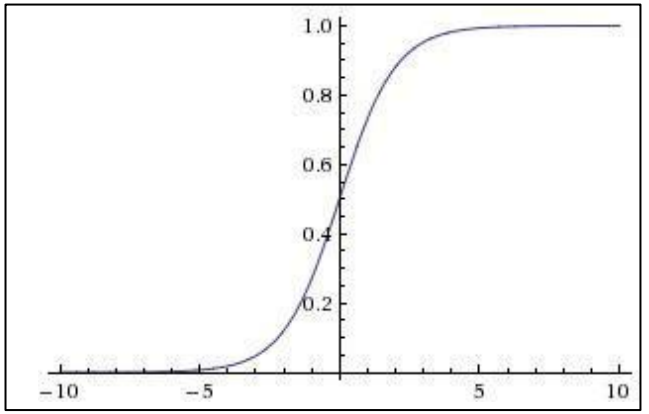
What happens when x = -10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

$$\sigma(x) = \sim 0$$

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x)) = 0(1-0) = 0$$

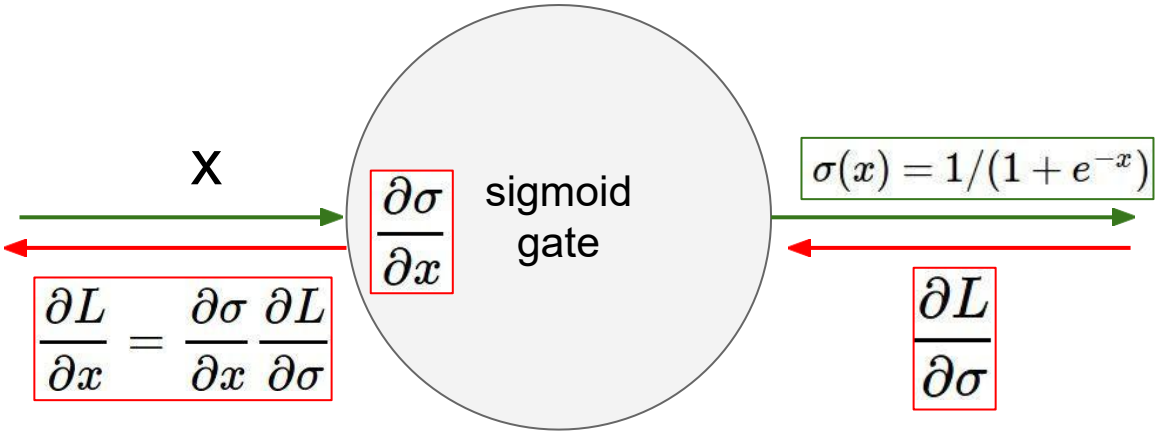


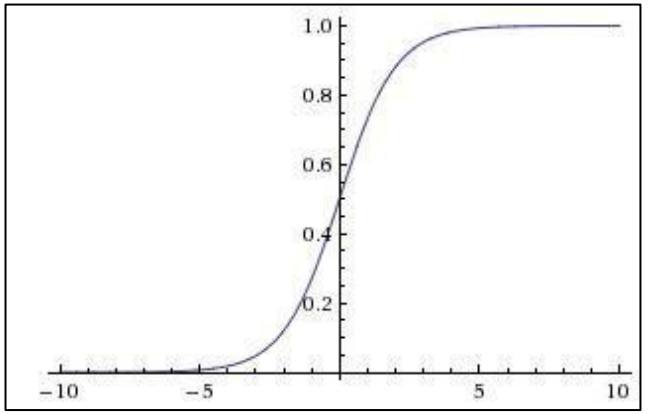


What happens when x = -10?

What happens when x = 0?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$



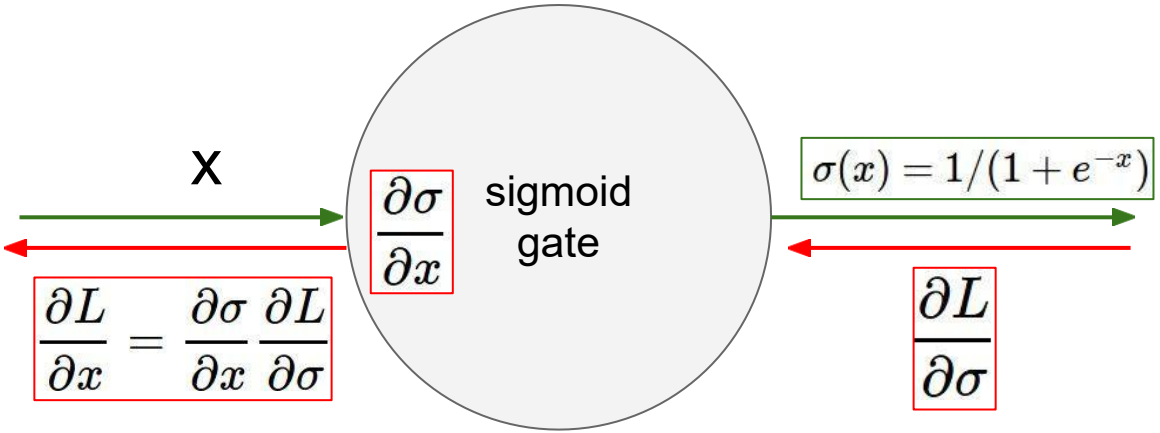


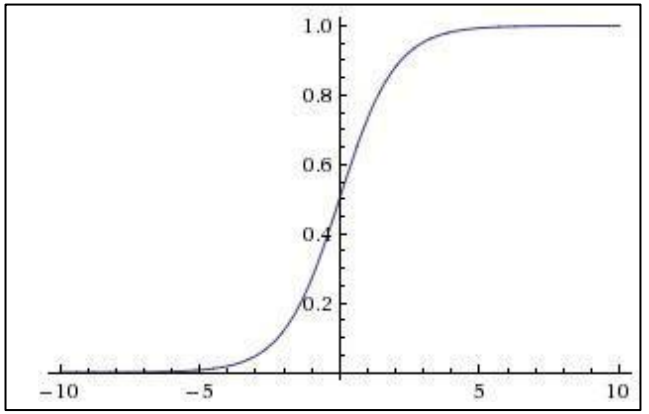
What happens when x = -10?
What happens when x = 0?
What happens when x = 10?

$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

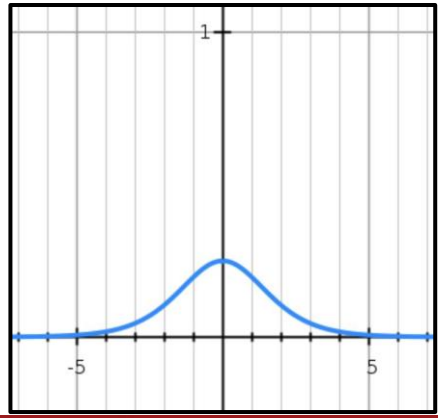
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x)) = 1(1-1) = 0$$

USC Viterbi School of Engineering





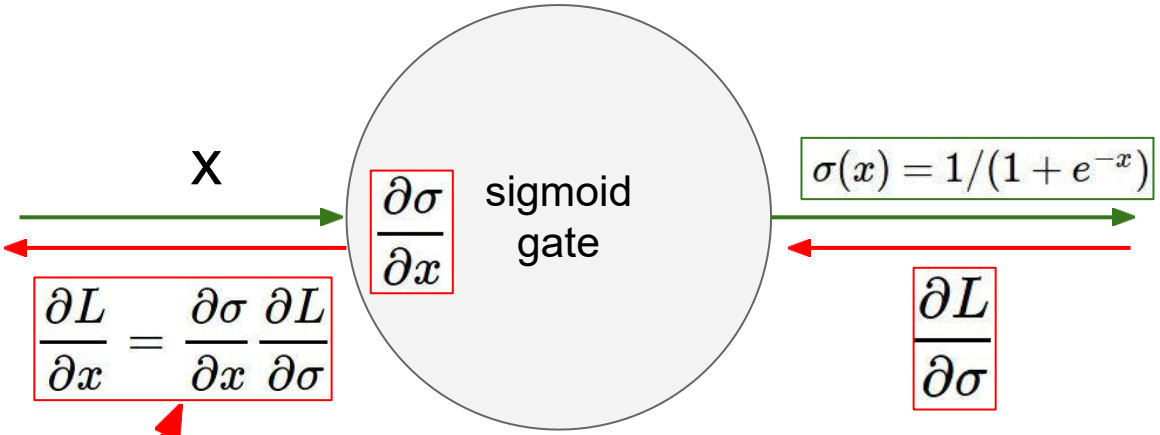
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

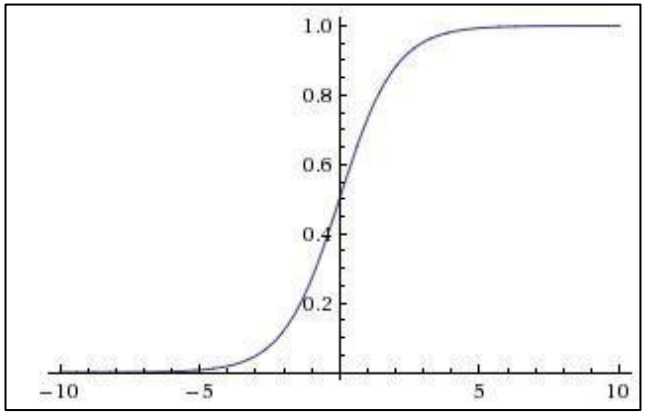


What happens when x = -10?

What happens when x = 0?

What happens when x = 10?





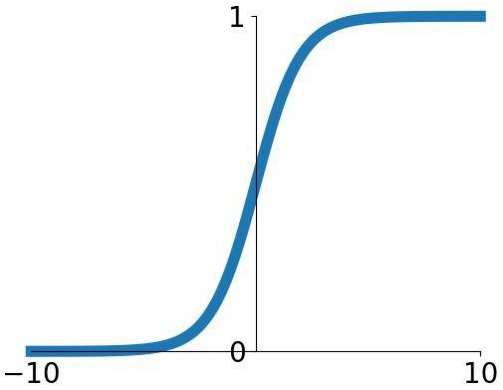
$$\frac{\partial \sigma(x)}{\partial x} = \sigma(x)\,(1 - \sigma(x))$$

Why is this a problem?

Then the gradients flowing back are tiny (about 4.5e-5 here) and the weights barely move

Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$



Sigmoid

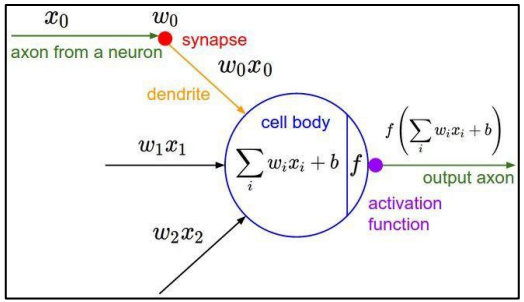
- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating "firing rate" of a neuron

3 problems:

1. Saturated neurons shrink the gradients
2. Sigmoid outputs are not zero-centered

Consider what happens when the input to a neuron is always positive...

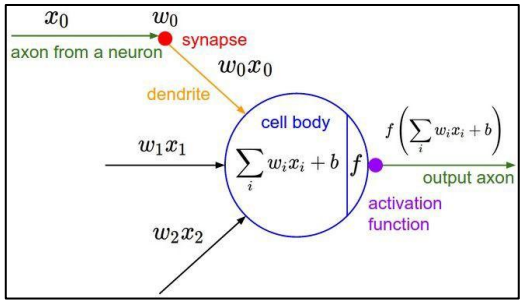
$$f\left(\sum_i w_i x_i + b\right)$$



What can we say about the gradients on **w**?

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$

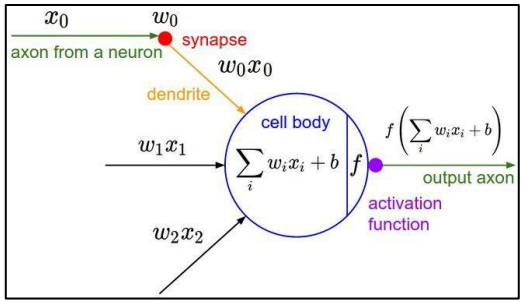


What can we say about the gradients on **w**?

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



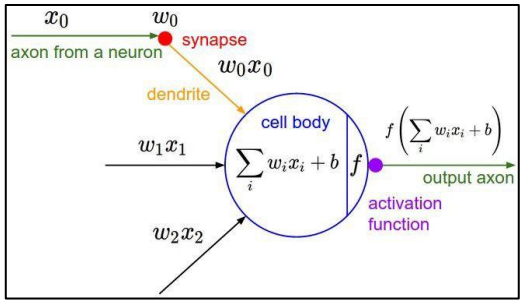
What can we say about the gradients on **w**?

We know that local gradient of sigmoid is always positive

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



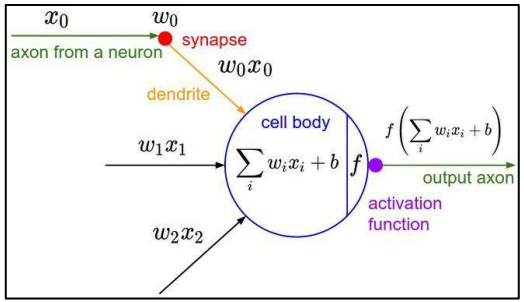
What can we say about the gradients on **w**?

We know that local gradient of sigmoid is always positive
We are assuming x is always positive

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right) x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$



What can we say about the gradients on **w**?

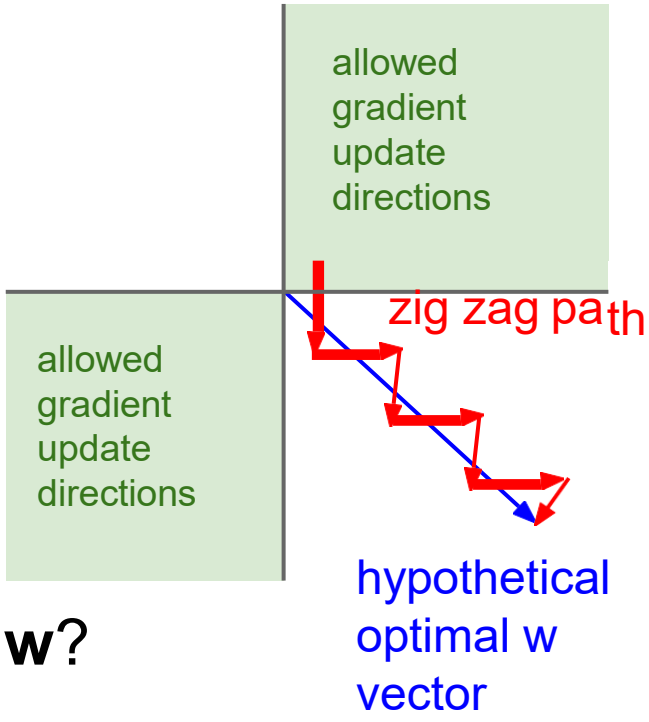
We know that local gradient of sigmoid is always positive
We are assuming x is always positive

So!! Sign of gradient **for all w_i** is the same as the sign of upstream scalar gradient!

$$\frac{\partial L}{\partial w} = \sigma\left(\sum_i w_i x_i + b\right)\left(1 - \sigma\left(\sum_i w_i x_i + b\right)\right)x \times upstream_gradient$$

Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$

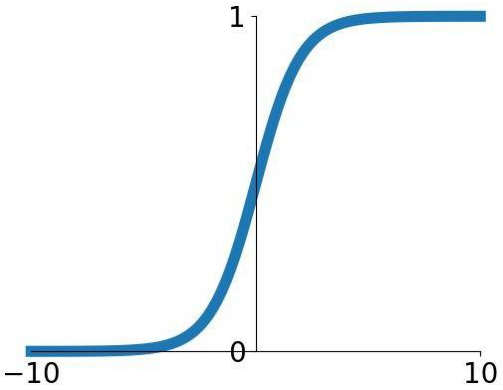


What can we say about the gradients on **w**?

For one example, all same sign :(

Activation Functions

$$\sigma(x) = 1/(1 + e^{-x})$$



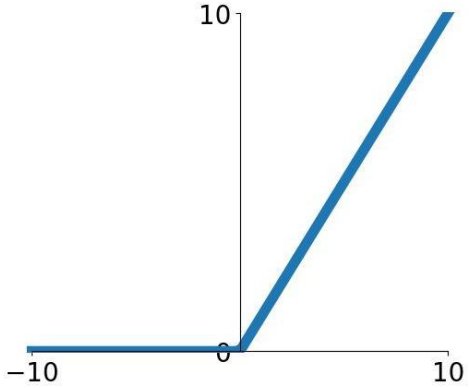
Sigmoid

- Squashes numbers to range [0,1]
- Historically popular since they have nice interpretation as a saturating “firing rate” of a neuron

3 problems:

1. Saturated neurons shrink the gradients
2. Sigmoid outputs are not zero-centered
3. exp() is a bit compute expensive

Activation Functions



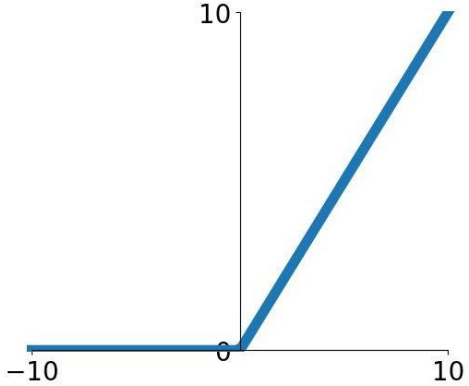
ReLU
(Rectified Linear Unit)

- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Reached 25% train error in ~1/6 the iterations of tanh (CIFAR-10)

[Krizhevsky et al., 2012]

Activation Functions



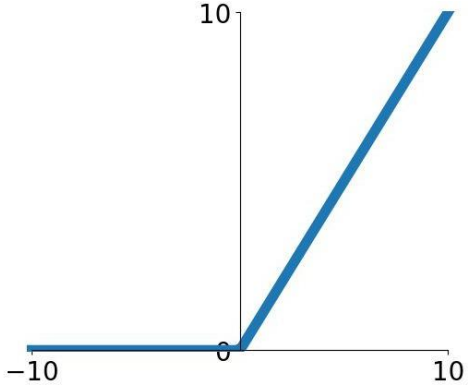
ReLU
(Rectified Linear Unit)

- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Reached 25% train error in ~1/6 the iterations of tanh (CIFAR-10)

- Not zero-centered output

Activation Functions



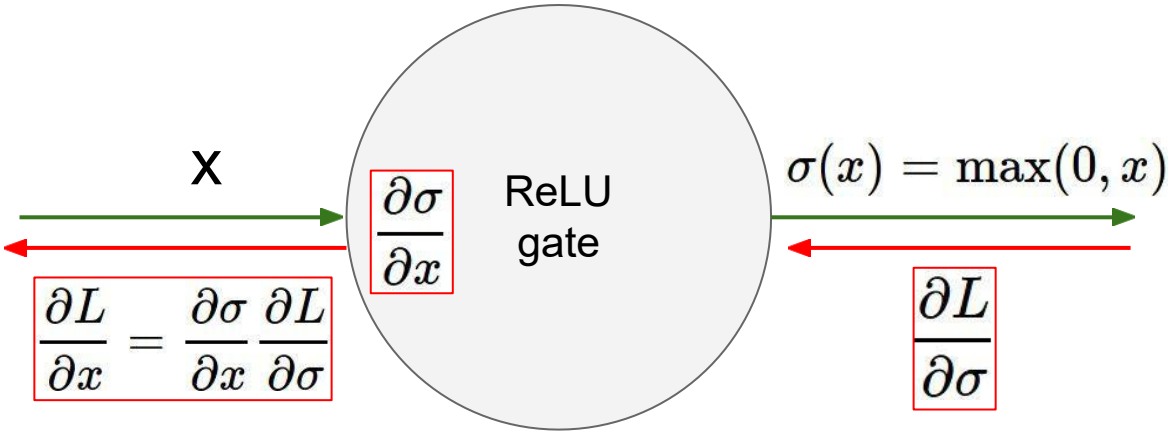
ReLU
(Rectified Linear Unit)

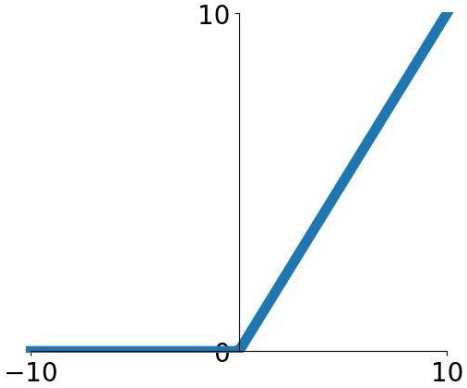
- Computes **f(x) = max(0,x)**

- Does not saturate (in +region)
- Very computationally efficient
- Reached 25% train error in ~1/6 the iterations of tanh (CIFAR-10)

- Not zero-centered output
- An annoyance:

hint: what is the gradient when x < 0?



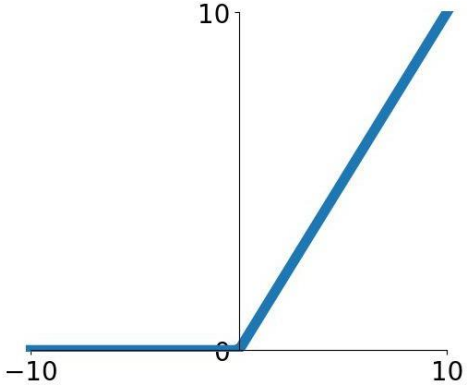


What happens when x = -10?
What happens when x = 0?
What happens when x = 10?

If the input is always negative (bad learning rate, inappropriate data preprocessing) – they are “died” .

Why does ReLU can still learn without the negative response?

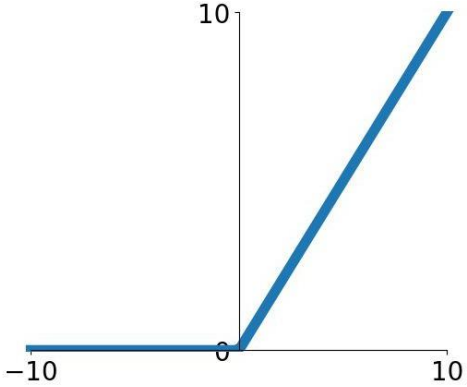
Sparse Activation: ReLU outputs zero for any negative input, leading to a portion of neurons in the network not activating. Realizing a speedup from that sparsity requires kernels that exploit it; effects on regularization are empirical.



Non-saturation: The ReLU function is linear for positive inputs, meaning it does not saturate (or flatline) as the input gets large, unlike the sigmoid or tanh functions.

Why does ReLU can still learn without the negative response?

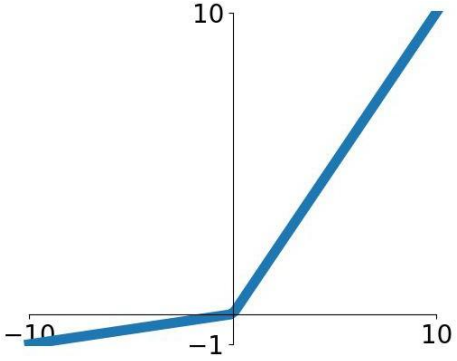
Enhancement of Positive Features:
Although ReLU does not directly process negative inputs (setting them to zero), it can emphasize positive features through the adjustment of weights learned by the network.



Collaboration in Multilayer Networks: even though individual ReLU units cannot process negative inputs, multiple layers of ReLU units can interact through their weights and structure to abstract useful information and features from complex inputs.

Activation Functions

[Maas et al., 2013]
[He et al., 2015]



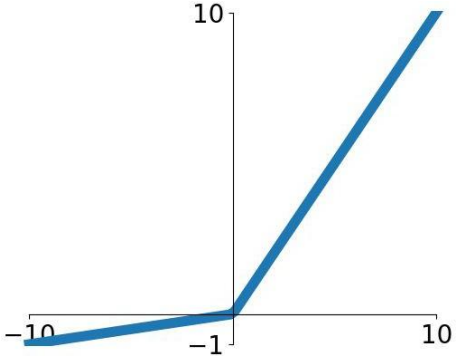
- Does not saturate
- Computationally efficient
- Performs about the same as ReLU in the cited experiments
- **less prone to "dying".**

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

Activation Functions

[Maas et al., 2013]
[He et al., 2015]



- Does not saturate
- Computationally efficient
- Performs about the same as ReLU in the cited experiments
- **less prone to "dying".**

Leaky ReLU

$$f(x) = \max(0.01x, x)$$

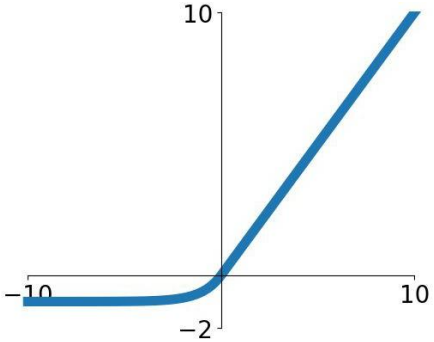
Parametric Rectifier (PReLU)

$$f(x) = \max(\alpha x, x)$$

backprop into α (parameter)

[Clevert et al., 2015]

Exponential Linear Units (ELU)



- Non-saturating for x > 0, like ReLU
- Closer to zero mean outputs
- Negative saturation regime compared with Leaky ReLU adds some robustness to noise

$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha\,(\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

α = 1 in this plot

- Computation requires exp()

TLDR: In practice:

- Use ReLU. Be careful with your learning rates
- Try out Leaky ReLU / Maxout / ELU
  - To squeeze out some marginal gains
- Don't default to sigmoid or tanh in hidden layers

We will cover more about neural network training once we finish CNN, RNN, and GNN

Important Activation Interview Questions

Question: What limits gradient-based network training?

Important Activation Interview Questions

Question: What limits gradient-based network training?

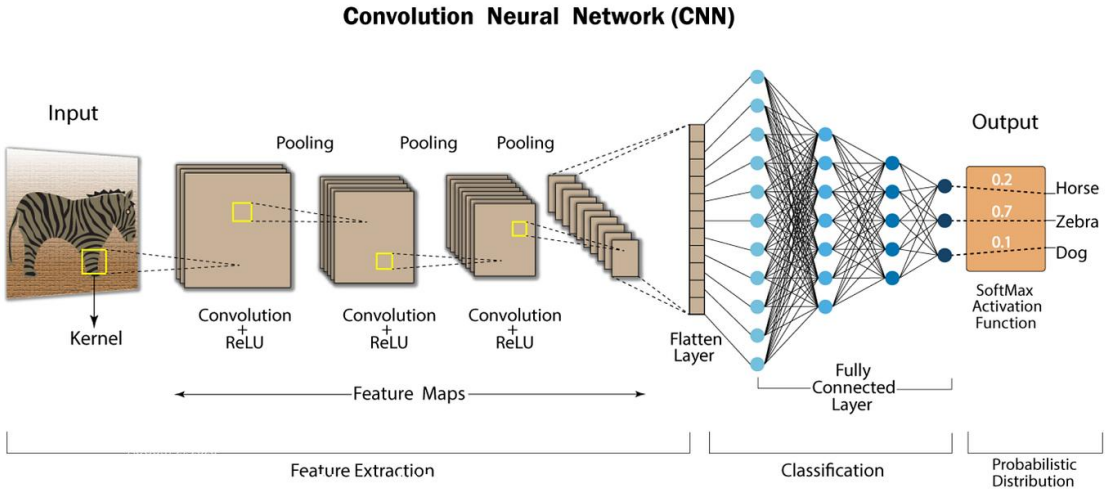
Answer:

- **Vanishing Gradient Problem**: In deep networks, gradients can become very small, so the weights update extremely slowly.
- **Exploding Gradient Problem**: Conversely, gradients can become excessively large, leading to divergent behavior.
- **Dependency on Data and Initialization**: Training quality relies on the data and the initial weights, not on the gradient computation itself.
- **Non-convex Optimization**: Gradient descent can settle at a poor stationary point rather than the global minimum.
- **Computational Intensity**: For large networks, backpropagation can be computationally expensive.

Next Blocks – From Simple NN to CNN, RNN, GNN

Real-world data is more complex than simple features:

- For a 1024x1080 picture – how many features does it have?
- For stock data with time dependency, how to consider the time dependency?
- For social networks with strong connectivity, how to capture it?

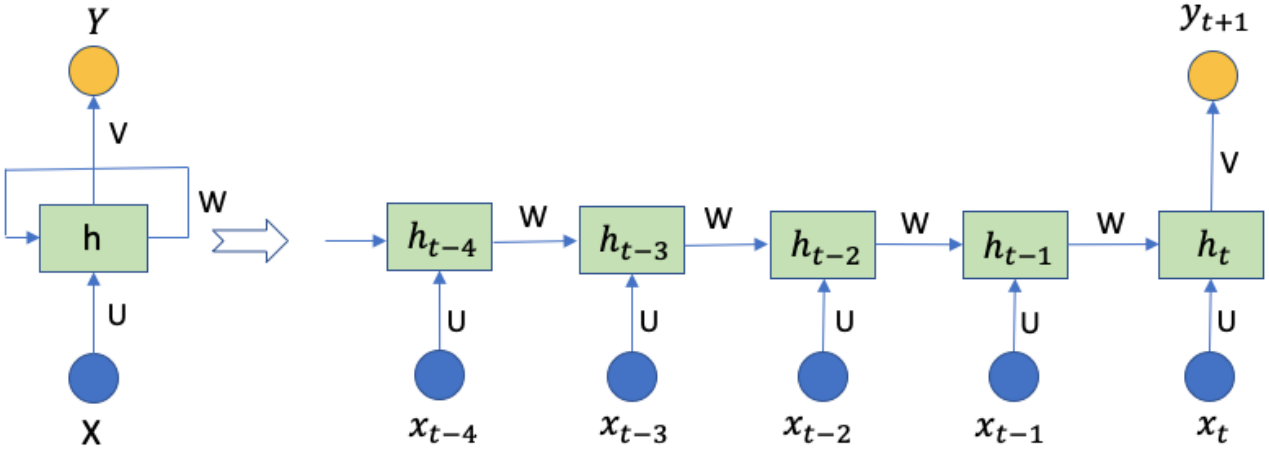


Convolution Neural Networks (CNN) for images and videos

Next Blocks – From Simple NN to CNN, RNN, GNN

Real-world data is more complex than simple features:

- For a 1024x1080 picture – how many features does it have?
- For stock data with time dependency, how to consider the time dependency?
- For social networks with strong connectivity, how to capture it?

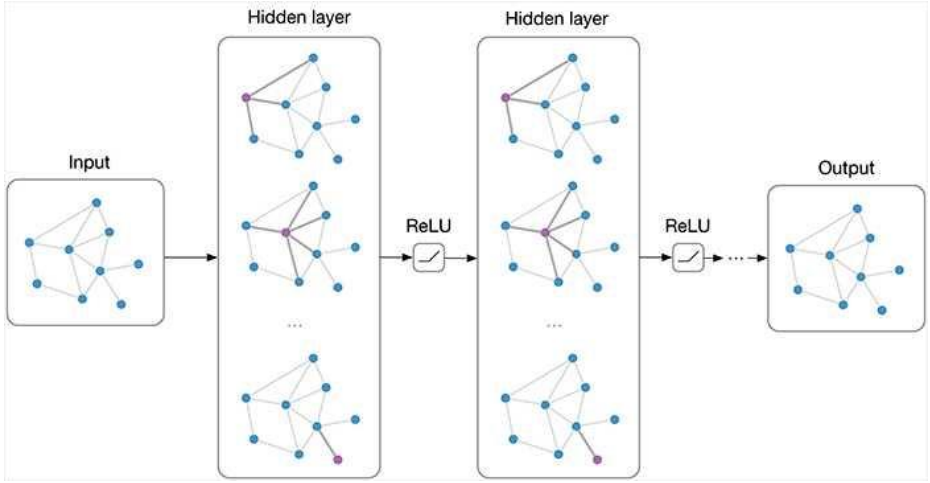


Recurrent neural networks (RNN) for sequences

Next Blocks – From Simple NN to CNN, RNN, GNN

Real-world data is more complex than simple features:

- For a 1024x1080 picture – how many features does it have?
- For stock data with time dependency, how to consider the time dependency?
- For social networks with strong connectivity, how to capture it?



Graph neural networks (GNN) for graphs